天筹求解器
数学规划求解器手册

发布 *3.6.0*

天筹求解器团队

2026 年 08 月 10 日

Contents

1 天筹 (OptVerse) 求解器简介 **1**
- 1.1 数学规划求解器概述 . . . 1
- 1.2 约束规划求解器概述 . . . 2
- 1.3 联系信息 . . . 2

2 安装说明 **3**
- 2.1 支持平台 . . . 3
- 2.2 获取渠道 . . . 3
- 2.3 安装指南 . . . 3

3 命令行工具 **7**
- 3.1 使用方法 . . . 7
- 3.2 参数说明 . . . 8
- 3.3 使用示例 . . . 9
- 3.4 日志解析 . . . 10

4 天筹求解器快速入门 **15**
- 4.1 天筹数学规划求解器快速入门 . . . 15
- 4.2 天筹约束规划求解器快速入门 . . . 49

5 文件格式 **61**
- 5.1 MPS 格式文件说明 . . . 61
- 5.2 LP 格式文件说明 . . . 69
- 5.3 FZN 文件格式说明 . . . 77

6 属性介绍 **81**
- 6.1 数学规划求解器属性介绍 . . . 81
- 6.2 属性使用示例 . . . 98

7 参数参考 **99**
- 7.1 数学规划求解器参数说明 . . . 99
- 7.2 参数示例 . . . 110
- 7.3 约束规划求解器参数说明 . . . 111

8 API 参考手册 **115**
- 8.1 环境 . . . 115
- 8.2 数学规划求解器 API 参考手册 . . . 124

8.3 枚举类 384
8.4 约束规划求解器 API 参考手册 386

9 SDK 回放功能 **469**
9.1 Record 记录 469
9.2 Replay 回放 470

10 网络流规划问题 **471**
10.1 问题定义 471
10.2 网络流规划样例 472

CHAPTER 1

天筹 (OptVerse) 求解器简介

天筹求解器是一款由华为云推出的高效通用数求解器套件。

本用户手册简要介绍了数学规划求解器与约束规划求解器的功能概述，包括安装说明 和使用帮助。

求解器可在*Linux* 环境下使用，并支持以下调用方式：

- 命令行方式: 读取标准格式模型文件进行求解, 支持常用参数配置。
  - 数学规划求解器提供*C++ API* , *Python API* 和*Java API* 接口。
  - 约束规划求解器提供*C++ API* 和*Python API* 接口。

1.1 数学规划求解器概述

现阶段数学规划求解器提供线性规划问题 (linear programming, LP)，混合整数线性规划问题 (mixed integer linear programming, MILP) 问题和二次约束二次规划 (quadratically constrained quadratic program, QCQP) 问题的求解能力，并致力于支持更多类型的优化问题求解。

考虑（混合整数）线性规划的一般形式：

$$\begin{aligned} \text{minimize} \quad & c_0 + c^T x \\ \text{subject to} \quad & L \leq Ax \leq U \\ & l \leq x \leq u \\ & x_i \in \mathbb{Z}, i \in \mathrm{I} \end{aligned}$$

这里

$$\begin{aligned} c_0 &\in \mathbb{R} \\ c, x, l, u &\in \mathbb{R}^n \\ L, U &\in \mathbb{R}^m \\ A &\in \mathbb{R}^{m \times n} \end{aligned}$$

而且 I 为指定变量 x 取整的下标集。

数学规划求解器提供如下求解算法：

- 对于 LP 问题，数学规划求解器提供原始/对偶单纯形 (primal/dual simplex method) 和内点算法 (interior point method，包括确定性和非确定性方法)，及并发算法。
- 对于满足特定结构特征的网络流规划问题，数学规划求解器会自动选择网络流单纯形法 (network simplex method) 进行求解。
- 对于凸二次规划及二次约束二次规划问题 (目标或约束中包含二次项 $x'Qx$), 数学规划求解器会识别并调用内点算法 (interior point method) 进行求解。
- 对于混合整数规划问题 (集合 I 非空)，数学规划求解器主要通过分支定界方法 (branch and bound) 来求解，并融合了业界多种启发式策略用以提高求解效率。
- 对于二阶锥规划问题 (约束中包含二阶锥不等式 $\|A_i x + b_i\| \leq c_i^T x + d_i$ ， i 为任意整数), 数学规划求解器会识别并调用二阶锥内点算法进行求解。
- 对于一般非线性规划问题 (目标或约束中包含一般非线性关系，模型为凸或非凸), 数学规划求解器会识别并调用一般非线性规划内点算法 (interior point method) 进行求解，至局部最优解。

注解

社区版本仅供学术研究，如需商用请联系天筹求解器团队。

1.2 约束规划求解器概述

天筹约束规划求解器是为求解复杂的约束规划问题而设计的一款高效、通用的优化引擎。约束规划求解器基于惰性子句生成（Lazy Clause Generation, LCG）方法，采用先进的 CP-SAT 求解技术，并深度融合多种工业级启发式策略，显著提升大规模问题的求解效率与鲁棒性。

求解器支持丰富的约束类型，涵盖以下五大类：

- 逻辑约束：包括布尔变量之间的逻辑关系（如 AND、OR 等），适用于建模决策逻辑。
- 线性约束：涉及整数或布尔变量的线性等式或不等式，常用于资源平衡与数量限制。
- 全局约束（Global Constraints）：提供高表达力的复合约束，如 Table（表约束）、AllDifferent（互异约束）、Element（元素索引约束）等，可大幅简化模型并加速求解。
- 特定问题约束：面向典型应用场景（如作业调度、车辆路径规划、三维装箱、资源分配等）定制的专用约束，提升建模效率与求解性能。
- 计算型约束：支持复杂函数表达式的约束，例如 LinMax（线性最大值）、LinProb（线性乘积）等，适用于需要内嵌计算逻辑的高级建模需求。

1.3 联系信息

天筹求解器可提供从问题建模、模型优化到算法调优、服务部署的全流程决策优化解决方案。无论您是希望进行现有求解方案的改进或自研替代，还是希望在新项目中应用数学优化工具，都欢迎随时联系我们。

天筹求解器还在快速迭代演进中，会持续提高求解性能与稳定性，推出更多用户友好特性。有关天筹求解器使用过程中的问题、建议和需求，均可发送邮件到：optverse@huawei.com

CHAPTER 2

安装说明

2.1 支持平台

数学规划求解器与约束规划求解器目前支持 Linux 系统平台，在以下系统测试可正常运行：

- **Ubuntu**: Ubuntu 18.04, Ubuntu 20.04, Ubuntu 22.04
- **CentOS**: CentOS 8, CentOS 9
- 低版本 Linux 系统需要满足 *glibc >= 2.33* 和 *libstdc++ >= 6.0.26*
- Python 版本需要满足在 3.10 和 3.13 之间

2.2 获取渠道

2.2.1 线下版本

- **软件安装包**: 用户可联系天筹求解器团队 获取，我们会有专人与您联系。

2.3 安装指南

2.3.1 线下版本安装

1. 将求解器安装包 `optverse.x.x.x.tar.gz` 拷贝到指定路径 (如 `/opt/`). 以下假设路径为 `<installdir>` .
2. 解压缩安装包：

```
tar -xzvf optverse.x.x.x.tar.gz
```

这将创建一个新的路径 `<installdir>/optverse` ，路径结构为：

```
./
├── bin/
│   └── optverse
├── dist/
│   ├── optvpy-X.X.X-cp310-cp310-linux_x86_64.whl
│   ├── optvpy-X.X.X-cp311-cp311-linux_x86_64.whl
│   ├── optvpy-X.X.X-cp312-cp312-linux_x86_64.whl
│   └── optvpy-X.X.X-cp313-cp313-linux_x86_64.whl
├── examples/
│   ├── c++/
│   │   ├── feasibility-relax.cpp
│   │   ├── lp-sensitivity-analysis.cpp
│   │   ├── lp1.cpp
│   │   ├── lp2.cpp
│   │   ├── lp3.cpp
│   │   ├── milp1.cpp
│   │   └── milp2.cpp
│   ├── CMakeLists.txt
│   ├── data/
│   │   ├── lp-example.lp
│   │   ├── milp-example.mps
│   │   └── testNetwork.mps
│   └── python/
│       ├── feasibility-relax.py
│       ├── lp-sensitivity-analysis.py
│       ├── lp1.py
│       ├── lp2.py
│       ├── lp3.py
│       ├── milp1.py
│       └── milp2.py
├── include/
│   └── optv_c++.h
└── lib/
    ├── libiomp5.so
    └── liboptverse.so
```

3. 设置 LD_LIBRARY_PATH:

```
echo 'export LD_LIBRARY_PATH="<installdir>/optverse/lib:${LD_LIBRARY_PATH}"' >> ~/.
↪bashrc
source ~/.bashrc
```

4. 进入 <installdir>/optverse/bin 路径，执行 ./optverse --version，如果显示以下信息则说明求解器已成功安装:

```
OptVerse Optimizer version 3.6.0 (Internal Release)
Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
```

注解

本产品将数学规划求解器与约束规划求解器统一集成于单一安装包，实现一次部署、双求解器就绪。

2.3.2 Python SDK 安装指南

安装包支持同时安装数学规划求解器和约束规划求解器的 Python SDK。安装包中包含统一的 wheel 安装文件，安装命令如下：

```
pip3 install optvpy-x.x.x-cp3x-cp3xm-linux_x86_64.whl
```

2.3.3 使用许可获取

SDK 需要使用许可 (license) 才能正常使用，请联系天筹求解器团队 获取，我们会有专人与您联系。

2.3.4 License 使用

- 把 license 文件放在某个目录下，如 `/opt/Optverse.lic`
- 将 `OPTV_LICENSE_FILE` 环境变量指向 license 文件: `export OPTV_LICENSE_FILE=/opt/Optverse.lic`

注解

将此变量导出到.bashrc 文件中，可方便多次连续调用。

```
echo "export OPTV_LICENSE_FILE=/opt/Optverse.lic" >> ~/.bashrc
source ~/.bashrc
cd <installdir>/optverse/bin
./optverse ../examples/data/lp-example.lp
```

CHAPTER 3

命令行工具

3.1 使用方法

3.1.1 显示帮助及版本信息

用法：

```
optverse [flag]    # Display help or version information
```

可选 flag 详见参数说明。

3.1.2 求解问题

用法：

```
optverse [options]* filename # Set solving parameters, read model file, and solve
```

可选求解参数列表见参数说明。

当前支持的文件格式有：

- `.mps`, `.lp` 文件格式
- `.gz`, `.z`, `.Z` 压缩文件格式
- `.fzn` 文件格式

注解

- `.fzn` 文件不支持压缩格式。
- 有关 `.mps` 格式的详细说明可参考*MPS 格式文件说明*，有关 `.lp` 格式的详细说明可参考*LP 格式文件说明*。

- 有关 `.fzn` 格式的详细说明可参考*FZN 格式文件说明*。

3.2 参数说明

-h, --help 显示帮助信息。

-v, --version 显示版本信息。

--method=<int> 求解算法。

允许的取值

取值	含义
`-1`	自动
`0`	原始单纯形方法
`1`	对偶单纯形方法
`2`	内点方法 (IPM)
`3`	确定性内点方法 (deterministic IPM)
`4`	并发求解方法 (concurrent solver)

默认
`-1`

--timeLimit=<double> 求解时间上限 (秒)。

取值范围
$[0.0, 1.79769 \times 10^{308}]$

默认
1.79769×10^{308}

--memLimit=<int> 求解内存上限 (MB)。

取值范围
$[100, 2147483647]$

默认
2147483647

--threads=<int> 求解线程数。

取值范围
[1, 平台线程]

默认
平台物理核数

--gap=<double> MIP 目标 gap 值。

取值范围
$[0.0, 1.79769 \times 10^{308}]$

默认
0.0001

--logFile=<string> 求解日志路径。

默认
./optv.log

--resultFile=<string> 解文件路径。

默认
./<filename>.sol

- 如果用户没有指定任何输入参数，将默认返回帮助信息。
- 输入问题模型文件只能作为最后一个输入参数。
- 如输入问题为网络流规划问题, 求解器将自动调用网络流单纯形法（network simplex method）进行求解。

3.3 使用示例

- 不输入任何参数，默认显示帮助信息

```
./optverse
```

- 显示帮助信息

```
./optverse -h
./optverse --help
```

- 显示版本信息

```
./optverse -v
./optverse --version
```

- 按默认设置求解问题文件 test-problem.mps 中的问题。

```
./optverse test-problem.mps
```

- 求解问题文件 test-problem.mps, 指定日志文件为 ./test-problem.log, 结果文件为 ./test-problem.sol。

```
./optverse --logFile=./test-problem.log --resultFile=./test-problem.sol␣
↪test-problem.mps
```

- 求解压缩格式问题文件 test-problem.mps.gz 并设置时间上限 7200 秒，内存使用上限 32GB，目标 gap 值 0.01%，使用线程数 4 并行求解。

```
./optverse --timeLimit=7200 --memLimit=32768 --gap=0.0001 --threads=4␣
↪test-problem.mps.gz
```

3.4 日志解析

3.4.1 LP 求解日志解析

求解 LP 问题文件 `lp-example.lp` 使用默认参数进行求解，指定结果文件为 `lp-example.sol`：

```
./optverse --resultFile=lp-example.sol examples/data/lp-example.lp
```

所得日志如下：

```
Set parameter "logFile" to "optv.log"
Set parameter "resultFile" to "lp-example.sol"
OptVerse Optimizer version "3.6.0"
Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.

Read problem /home/solver/optverse/examples/data/lp-example.lp
Read time: 0.00s

Optimize an LP model
  8 rows, 20 columns and 36 nonzeros

Presolve problem
Presolve time: 0.00s
After presolve:
  4 rows, 12 columns and 12 nonzeros

     Time    Iteration     Objective   Primal Num.Inf.  Dual Num.Inf.
     0.0s            0  0.000000e+00                4              0
     0.0s            4  1.380000e+02                0              0

Postsolving
Postsolve finished

     Time    Iteration     Objective   Primal Num.Inf.  Dual Num.Inf.
     0.0s            0  1.380000e+02                0              0

Solve results
  Status               Optimal solution found
  Objective            1.380000000000e+02
  Simplex iteration    4
  Time                 0.01

Write best solution /home/solver/optverse/lp-example.sol
```

上述日志包含了如下信息：

- 参数设置，包含: logFile、resultFile，可用参数请参考数学规划求解器参数
- 版本和版权信息。
- 问题读取时间。
- 问题信息，包含行数、列数、非零元个数。
- 预处理时间及预处理后的问题规模，包含新行数、列数和非零元个数。
- 单纯形算法迭代日志，如时间、迭代步数，当前目标值，原始及对偶可行性的违背量。

- 后处理信息。
- 求解结果概览，包含：
  - 求解状态，如 `Optimal solution found`, `Problem is infeasible`, `Problem is unbounded`, `Time limit reached`, `Numerical error`, `Symbolic error`, `Unknown`
  - 最优目标值
  - 迭代总数
  - 总耗时
- 运行期间的报警错误信息。

3.4.2 MILP 求解日志解析

使用默认参数求解 MILP 问题文件 `milp-example.mps`，指定结果文件为 *milp-example.sol*：

```
./optverse --resultFile=milp-example.sol examples/data/milp-example.mps
```

所得日志如下：

```
Set parameter "logFile" to "optv.log"
Set parameter "resultFile" to "milp-example.sol"
OptVerse Optimizer version "3.6.0" (Internal Release)
Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.

Read problem /home/solver/optverse/examples/data/milp-example.mps
Read time: 0.00s

Optimize an MILP model
  8 rows, 20 columns (20 binary, 0 integer, 0 continuous) and 36 nonzeros

Presolve problem
Presolve time: 0.00s
After presolve:
  8 rows, 20 columns (20 binary, 0 integer, 0 continuous) and 36 nonzeros

Start parallel solving, using up to 6 threads

    Time     Solved       Open       LPIter     BestBound       BestSol        Gap
  H   0.0s          0          0          0   0.000000e+00   2.980000e+02     ␣
↪100%
  H   0.0s          0          0          0   1.380000e+02   2.880000e+02   52.
↪08%
      0.0s          0          0          0   1.380000e+02   2.880000e+02   52.
↪08%
      0.0s          0          0          9   2.420000e+02   2.880000e+02   15.
↪97%
  H   0.0s          1          0          9   2.420000e+02   2.420000e+02    0.
↪00%
      0.0s          1          0          9   2.420000e+02   2.420000e+02    0.
↪00%

Solve results
  Status               Optimal solution found
```

```
  Best solution         2.420000000000e+02
  Best bound            2.420000000000e+02
  Gap                   0.0000%
  Node                  1
  LP iteration          9
  Time                  0.01

Write best solution /home/solver/optverse/milp-example.sol
```

上述日志包含了如下信息：

- 参数设置, 包含: logFile、resultFile，可用参数请参考数学规划求解器参数。
- 版本和版权信息。
- 问题读取时间。
- 原问题规模
- 预处理时间和预处理后问题规模
- 开始串行、并行求解（线程数），如：
  - Start sequential solving
  - Start parallel solving, using up to 4 threads
- 第一列（在 Time 列之前）仅可能出现 H 和 *
- Time, Solved, Open, LPIter, BestBound, BestSol, 和 Gap 信息
- 求解结果概览，包含：
  - 求解状态，仅可能出现 `Optimal solution found`, `Time limit reached`, `Memory limit reached`, 或 `Problem is infeasible`。
  - 最优解以及最优下界
  - Gap
  - 节点个数
  - LP 迭代总数
  - 总耗时
- 求解过程中的其他信息，例如警告和错误

3.4.3 CP 求解日志解析

使用默认参数求解 CP 问题 `fzn-example.fzn`：

```
./optverse --cpThreads=1 examples/data/fzn-example.fzn
```

所得日志如下：

```
Set parameter "cpThreads" to "1"
CP model is created with number of variables: 12.
CP model is created with number of constraints: 10.
CP solver is executed with the following parameters:
time limit: 9.223372036854776e+18 sec
deterministic time limit: 9.223372036854776e+18
number of threads: 1
```

(下页继续)

(续上页)

```
deterministic mode: 1
random seed: 1
abs gap: 1e-06
relative gap: 0.0

    Time    Solved      Open    It/Node    BestBound       BestSol       Gap
    0.0s         0         0       --   3.450000e+02         --          --
 H   0.0s         0         0       --   3.450000e+02   3.450000e+02    0.00%

 Solve results
    Status               Optimal solution found
    Best solution        3.450000000000e+02
    Best bound           3.450000000000e+02
    Gap                  0.0000%
    Node                 0
    Time                 0.00
```

上述日志包含了如下信息：

- 参数设置，本例中参数 `cpThreads` 被用于设置线程数
- 原问题规模
- 约束规划求解器参数
- 求解过程的相关信息：
  - 在 `Time` 列之前出现 `H`，表示找到一个可行解
  - 接下来每一列的含义为：求解时间（Time），已关闭的节点数（Solved），待探索的开放节点数（Open），每个节点的迭代次数（It/Node），当前最佳下界（BestBound），当前找到的最佳可行解（BestSol），相对误差（Gap，计算公式为 (BestSol - BestBound) / BestSol）
- 求解结果概览，包含：
  - 求解状态，如 `Optimal solution found`、`Problem is infeasible`、`Input model is feasible`、`Input model is invalid`、`Interrupted by user` 以及 `Unknown`。
  - 最优解以及最优下界
  - Gap
  - 节点个数
  - 求解时间
- 求解过程中的其他信息，例如警告和错误

CHAPTER 4

天筹求解器快速入门

4.1 天筹数学规划求解器快速入门

本章提供了一些基本的示例，用于演示如何调用数学规划求解器各语言接口进行数学规划问题的建模和求解。

4.1.1 C++ 接口

本节将通过一个简单的 C++ 示例来演示数学规划求解器 C++ 接口的使用。

这个示例将介绍如何：

- 设置求解器参数
- 构建模型
- 将构建模型写入文件
- 优化模型
- 打印最优解信息

待求解问题的数学公式如下所示：

$$
\begin{aligned}
\text{minimize} \quad & -x - 14y - 6z, \\
\text{subject to} \quad & 2x + y + z \leq 3 \\
& 3y + z \leq 6, \\
& 0 \leq x \leq 1 \\
& 0 \leq y \\
& 0 \leq z \leq 3 \\
\text{where} \quad & x,\ y \in \mathbb{Z},\ z \in \mathbb{R}.
\end{aligned}
$$

```cpp
/*
 * Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
 * Description: OptVerse c++ interface example - build and solve an MILP problem
 */
#include "optv_c++.h"
using namespace std;

int main()
{
    try {
        OPTVEnv env("milp1.log");
        // Before model construction: set parameters through environment
        env.Set(OPTVDblParam::TIME_LIMIT, 7200);

        OPTVModel model(env);
        // After model construction: set parameters through model
        model.Set(OPTVStrParam::RESULT_FILE, "milp1.sol");

        // Add variables to the model
        OPTVVar x = model.AddVar(0, 1, -1, OPTV_BINARY, "x");
        OPTVVar y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y");
        OPTVVar z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z");

        // Add constraint to the model
        OPTVConstr c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0");
        OPTVConstr c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1");

        model.Write("milp1.lp");
        model.Optimize();

        if (model.Get(OPTVIntAttr::SOL_COUNT) > 0) {
            cout << "Model Status: " << model.Get(OPTVIntAttr::STATUS) << endl;
            cout << "Best solution: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;

            cout << "x = " << x.Get(OPTVDblAttr::X) << endl;
            cout << "y = " << model.GetVar(1).Get(OPTVDblAttr::X) << endl;
            cout << "z = " << model.GetVarByName("z").Get(OPTVDblAttr::X) << endl;
            cout << "c0 = " << model.GetConstr(0).Get(OPTVDblAttr::X) << endl;
            cout << "c1 = " << model.GetConstrByName("c1").Get(OPTVDblAttr::X) <<␣
↪endl;
        } else {
            cout << "No feasible solution available!" << endl;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

包含头文件

要使用 C++ 接口，首先要包含头文件 `optv_c++.h`。

```
#include "optv_c++.h"
```

创建环境

第一步需要创建一个空的环境对象，然后将日志文件名提供给该构造函数。

```
OPTVEnv env("milp1.log");
```

模型创建前的参数配置

若需要在调用*OPTVModel* 构造函数 **前**修改求解器参数，用户必须通过环境对象设置这些参数。在本例中，我们展示了在创建模型对象前如何通过环境对象设置求解器的时限参数。

```
env.Set(OPTVDblParam::TIME_LIMIT, 7200);
```

模型创建后的参数配置

若需要在调用*OPTVModel* 构造函数 **后**修改求解器参数，用户则必须通过模型对象设置这些参数。在本例中，我们展示了如何通过模型对象配置求解日志文件参数。

```
model.Set(OPTVStrParam::RESULT_FILE, "milp1.sol");
```

创建模型

创建了环境后，下一步就是创建模型。每个模型是一个优化问题，它由一组变量，一组约束和关联属性 (变量边界、目标系数、约束右边值等) 组成。

创建空模型

如果用户需要通过 API 一步一步构造模型，首先需要做的是创建一个空模型。

```
OPTVModel model(env);
```

从模型文件加载

如果把 `.mps`[1] 或 `.lp`[2] 文件路径当做第二个参数传入，该文件则被读取，相应的模型可以通过 API 求解或修改。

```
OPTVEnv env("fileRead.log");
OPTVModel model(env, fileName);
```

更多细节请参考 `milp2.cpp`

[1] 有关 MPS 格式的说明，请参见文件格式。
[2] 有关 LP 格式的说明，请参见文件格式。

向模型中添加变量

如果上一步创建了一个空模型，那么用户 **必须**向该模型中添加变量和约束后，才可以进行求解。同样，也可以向通过文件读取的模型增加变量或约束。方法*OPTVModel::AddVar()* 提供了向模型对象添加变量的功能，前两个参数是变量的下界和上界，第三个参数是目标系数，第四个参数是变量类型，可选值为: *OPTV_CONTINUOUS*、*OPTV_INTEGER* 或*OPTV_BINARY*，最后一个参数是变量的名称。

```
OPTVVar x = model.AddVar(0, 1, -1, OPTV_BINARY, "x");
OPTVVar y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y");
OPTVVar z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z");
```

如果所有创建的变量全部都是*OPTV_CONTINUOUS*，那么该模型就是一个线性规划问题，程序会自动调用线性规划求解算法进行求解。可以参考样例代码 `lp1.cpp`.

向模型中添加约束

添加完变量后，用户可以继续添加约束。数学规划求解器的 API 支持双边约束表达式，即用户创建约束对象时需要同时提供该约束的上下界。第一个参数是约束表达式的类型，如*OPTVLinExpr*，接下来两个参数分别是约束的上、下界（左、右边值），最后一个参数为约束名称。

```
OPTVConstr c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0");
OPTVConstr c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1");
```

求解模型

模型构造及相应的参数配置完成后，用户可以通过调用*OPTVModel::Optimize()* 来求解模型。求解完成后，求解状态、最优解等属性值都被保存在*OPTVModel* 对象中。可以调用 OPTVModel::Optimize() 方法执行求解。

```
model.Optimize();
```

查询求解结果

模型求解结束后，用户可以通过*OPTVModel::Get()* 方法来查询模型的不同属性和输出所得解。首先，模型的*OPTVIntAttr::SOL_COUNT* 属性提示了可行解是否成功获取；然后，模型求解状态可以通过*OPTVIntAttr::STATUS* 查询；而且目前最佳的目标值可以通过*OPTVDblAttr::OBJ_VAL* 获取；最后，以下示例展示了查询变量或约束的三种不同方法。

```
if (model.Get(OPTVIntAttr::SOL_COUNT) > 0) {
    cout << "Model Status: " << model.Get(OPTVIntAttr::STATUS) << endl;
    cout << "Best solution: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;

    cout << "x = " << x.Get(OPTVDblAttr::X) << endl;
    cout << "y = " << model.GetVar(1).Get(OPTVDblAttr::X) << endl;
    cout << "z = " << model.GetVarByName("z").Get(OPTVDblAttr::X) << endl;
    cout << "c0 = " << model.GetConstr(0).Get(OPTVDblAttr::X) << endl;
    cout << "c1 = " << model.GetConstrByName("c1").Get(OPTVDblAttr::X) << endl;
} else {
    cout << "No feasible solution available!" << endl;
}
```

编译和运行

数学规划求解器的软件包中已经包含了本节介绍的样例程序，路径在 `${OPTV_HOME}/examples/c++`，同时提供了 CMake 文件用于在 Linux 平台下编译所有样例。

编译时只需要先进入到 `${OPTV_HOME}/examples` 文件夹，然后执行如下操作即可：

```
mkdir build && cd build
cmake ..
make -j $(nproc)
```

编译完成后，上述示例可以通过调用以下命令运行：

```
./milp1
```

如果需要求解特定的 `.mps` 或 `.lp` 格式文件所含模型，用户可以通过以下命令求解和查询模型目标值及状态：

```
./milp2 xxx.mps
```

4.1.2 Python 接口

本节将通过前一章节的*C++ 接口示例* 来演示数学规划求解器 Python 接口的使用。该示例在 Python 环境下构建了一个模型，并对其进行求解，最后输出最优解。

```
"""
   Copyright (c) Huawei Technologies Co., Ltd. 2024. All rights reserved.
   Description: OptVerse python interface example - build and solve an MILP problem

   The mathematical formulation for the problem to be solved is as follows:
     min     - x - 14 y - 6 z
     s.t.    2 x +    y +   z <= 3
                    3 y +   z <= 6
     where x is binary, y is integer, z is continuous
              0 <= x <= 1
              0 <= y
              0 <= z <= 3
"""

from optvpy import *

if __name__ == "__main__":
    # Create an environment and set log file
    env = OPTVEnv("milp1.log")
    env.Set(OPTVDblParam.TIME_LIMIT, 7200)

    # Create an empty model
    model = OPTVModel(env)
    model.Set(OPTVStrParam.RESULT_FILE, "milp1.sol")

    # Add variables to the model
    x = model.AddVar(0, 1, -1, OPTV_BINARY, "x")
    y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y")
    z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z")

    # Add constraint to the model
```

(下页继续)

(续上页)

```
    c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0")
    c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1")

    model.Write('milp1.lp')
    # Optimize the model
    model.Optimize()

    # Output solution
    print(f"Best solution: {model.Get(OPTVDblAttr.OBJ_VAL)}")
    print(f"x = {x.Get(OPTVDblAttr.X)}")
    print(f"y = {model.GetVar(1).Get(OPTVDblAttr.X)}")
    print(f"z = {model.GetVarByName('z').Get(OPTVDblAttr.X)}")
```

引入模块

要使用 Python 接口，首先要安装并引入模块 `optvpy`.

```
from optvpy import *
```

创建环境

第一步需要创建一个空的环境对象，然后将日志文件名提供给该构造函数。

```
# Create an environment and set log file
env = OPTVEnv("milp1.log")
```

模型创建前的参数配置

若需要在调用*OPTVModel* 构造函数 **前**修改求解器参数，用户必须通过环境对象设置这些参数。在本例中，我们展示了在创建模型对象前如何通过环境对象设置求解器的时限参数。

```
env.Set(OPTVDblParam.TIME_LIMIT, 7200)
```

模型创建后的参数配置

若需要在调用*OPTVModel* 构造函数 **后**修改求解器参数，用户则必须通过模型对象设置这些参数。在本例中，我们展示了如何通过模型对象配置求解日志文件参数。

```
model.Set(OPTVStrParam.RESULT_FILE, "milp1.sol")
```

创建模型

创建了环境后，下一步就是创建模型。每个模型是一个优化问题，它由一组变量，一组约束和关联属性 (变量边界、目标系数、约束右边值等) 组成。

创建空模型

如果用户需要通过 API 一步一步构造模型，首先需要做的是创建一个空模型。

```
# Create an empty model
model = OPTVModel(env)
```

从模型文件加载

如果把 `.mps`[1] 或 `.lp`[2] 文件路径当做第二个参数传入，该文件则被读取，相应的模型可以通过 API 求解或修改。

```
env = OPTVEnv("fileRead.log")
model = OPTVModel(env, fileName)
```

更多细节请参考 `milp2.py`.

向模型中添加变量

如果上一步创建了一个空模型，那么用户 **必须**向该模型中添加变量和约束后，才可以进行求解。同样，也可以向通过文件读取的模型增加变量或约束。*OPTVModel.AddVar()* 方法提供了向模型对象添加变量的功能，前两个参数是变量的下界和上界，第三个参数是目标系数，第四个参数是变量类型，可选值为: *OPTV_CONTINUOUS*, *OPTV_INTEGER*, 或*OPTV_BINARY*，最后一个参数是变量的名称。

```
# Add variables to the model
x = model.AddVar(0, 1, -1, OPTV_BINARY, "x")
y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y")
z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z")
```

如果所有创建的变量全部都是*OPTV_CONTINUOUS*，那么该模型就是一个线性规划问题，程序会自动调用线性规划求解算法进行求解。更多细节请参考样例代码 `lp1.py`.

向模型中添加约束

添加完变量后，用户可以继续添加约束。数学规划求解器的 API 支持双边约束表达式，即用户创建约束对象时需要同时提供该约束的上下界。第一个参数是约束表达式的类型，如*OPTVLinExpr*，接下来两个参数分别是约束的上、下界（左、右边值），最后一个参数为约束名称。

```
# Add constraint to the model
c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0")
c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1")
```

[1] 有关 MPS 格式的说明，请参见文件格式。
[2] 有关 LP 格式的说明，请参见文件格式。

求解模型

模型构造及相应的参数配置完成后，用户可以通过调用*OPTVModel.Optimize()* 来求解模型。求解完成后，求解状态、最优解等属性值都被保存在*OPTVModel* 对象中。

```
# Optimize the model
model.Optimize()
```

查询求解结果

模型求解结束后，用户可以通过*OPTVModel.Get()* 方法来查询模型的不同属性和输出所得解。例如，目前最佳的目标值可以通过*OPTVDblAttr.OBJ_VAL* 查询，以下示例展示了查询变量或约束的三种不同方法。

```
# Output solution
print(f"Best solution: {model.Get(OPTVDblAttr.OBJ_VAL)}")
print(f"x = {x.Get(OPTVDblAttr.X)}")
print(f"y = {model.GetVar(1).Get(OPTVDblAttr.X)}")
print(f"z = {model.GetVarByName('z').Get(OPTVDblAttr.X)}")
```

运行示例

数学规划求解器的软件包中已经包含了本节介绍的样例程序，路径在 `${OPTV_HOME}/examples/python`。

只需进入 `${OPTV_HOME}/examples` 路径并执行。

```
python3 milp1.py
```

如果需要求解特定的 `.mps` 或 `.lp` 格式文件所含模型，用户可以通过以下命令求解和查询模型目标值及状态:

```
python3 milp2.py xxx.mps
```

4.1.3 Java 接口

本节将通过前一章节的*C++ 接口示例* 来演示数学规划求解器 Java 接口的使用。该示例在 Java 环境下构建了一个模型，并对其进行求解，最后输出最优解。

```
import com.huaweicloud.optv.*;

public class Milp1 {
    public static void main(String[] args) {
        try {
            OPTVEnv env = new OPTVEnv("Milp1.log");
            env.set(OPTVDblParam.TIME_LIMIT, 7200);


            OPTVModel model = new OPTVModel(env);
            model.set(OPTVStrParam.RESULT_FILE, "Milp1.sol");

            // add variables to the model
            OPTVVar x = model.addVar(0, 1, -1, OPTV.OPTV_BINARY, "x");
            OPTVVar y = model.addVar(0, OPTV.OPTV_INF, -14, OPTV.OPTV_INTEGER, "y");
            OPTVVar z = model.addVar(0, 3, -6, OPTV.OPTV_CONTINUOUS, "z");
```

(下页继续)

(续上页)

```
            // add constraint to the model
            OPTVConstr c0 = model.addConstr(OPTV.add(OPTV.add(OPTV.mul(2, x), y), z),␣
↪-OPTV.OPTV_INF, 3, "c0");
            OPTVConstr c1 = model.addConstr(OPTV.add(OPTV.mul(3, y), z), -OPTV.OPTV_
↪INF, 6, "c1");

            model.write("Milp1.lp");
            model.optimize();

            if (model.get(OPTVIntAttr.SOL_COUNT) > 0) {
                System.out.println("Best solution: " + model.get(OPTVDblAttr.OBJ_
↪VAL));

                System.out.println("x = " + x.get(OPTVDblAttr.X));
                System.out.println("y = " + model.getVar(1).get(OPTVDblAttr.X));
                System.out.println("z = " + model.getVarByName("z").get(OPTVDblAttr.
↪X));
            } else {
                System.out.println("No feasible solution available!");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

引入模块

使用 Java 接口前需要导入安装包 `com.huaweicloud.optv`.

```
import com.huaweicloud.optv.*;
```

创建环境

第一步需要创建一个空的环境对象，然后将日志文件名提供给该构造函数。

```
OPTVEnv env = new OPTVEnv("Milp1.log");
```

模型创建前的参数配置

若需要在调用*OPTVModel* 构造函数 **前**修改求解器参数，用户必须通过环境对象设置这些参数。在本例中，我们展示了在创建模型对象前如何通过环境对象设置求解器的时限参数。

```
env.set(OPTVDblParam.TIME_LIMIT, 7200);
```

模型创建后的参数配置

若需要在调用*OPTVModel* 构造函数 **后**修改求解器参数，用户则必须通过模型对象设置这些参数。在本例中，我们展示了如何通过模型对象配置求解日志文件参数。

```
model.set(OPTVStrParam.RESULT_FILE, "Milp1.sol");
```

创建模型

创建了环境后，下一步就是创建模型。每个模型是一个优化问题，它由一组变量，一组约束和关联属性 (变量边界、目标系数、约束右边值等) 组成。

创建空模型

如果用户需要通过 API 一步一步构造模型，首先需要做的是创建一个空模型。

```
OPTVModel model = new OPTVModel(env);
```

从模型文件加载

如果把 `.mps`[1] 或 `.lp`[2] 文件路径当做第二个参数传入，该文件则被读取，相应的模型可以通过 API 求解或修改。

```
OPTVEnv env = new OPTVEnv("fileRead.log")
OPTVModel model = new OPTVModel(env, fileName)
```

更多细节请参考 `Milp2.java`

向模型中添加变量

如果上一步创建了一个空模型，那么用户 **必须**向该模型中添加变量和约束后，才可以进行求解。同样，也可以向通过文件读取的模型增加变量或约束。*OPTVModel.addVar()* 方法提供了向模型对象添加变量的功能，前两个参数是变量的下界和上界，第三个参数是目标系数，第四个参数是变量类型，可选值为: *OPTV_CONTINUOUS*, *OPTV_INTEGER*, 或*OPTV_BINARY*，最后一个参数是变量的名称。

```
// add variables to the model
OPTVVar x = model.addVar(0, 1, -1, OPTV.OPTV_BINARY, "x");
OPTVVar y = model.addVar(0, OPTV.OPTV_INF, -14, OPTV.OPTV_INTEGER, "y");
OPTVVar z = model.addVar(0, 3, -6, OPTV.OPTV_CONTINUOUS, "z");
```

如果所有创建的变量全部都是*OPTV_CONTINUOUS*，那么该模型就是一个线性规划问题，程序会自动调用线性规划求解算法进行求解。更多细节请参考样例代码 `Lp1.java`.

[1] 有关 MPS 格式的说明，请参见文件格式。
[2] 有关 LP 格式的说明，请参见文件格式。

向模型中添加约束

添加完变量后，用户可以继续添加约束。OptVerse 的 API 支持双边约束表达式，即用户创建约束对象时需要同时提供该约束的上下界。第一个参数是约束表达式的类型，如*OPTVLinExpr*，接下来两个参数分别是约束的上、下界（左、右边值），最后一个参数为约束名称。

```
// add constraint to the model
OPTVConstr c0 = model.addConstr(OPTV.add(OPTV.add(OPTV.mul(2, x), y), z), -OPTV.OPTV_
↪INF, 3, "c0");
OPTVConstr c1 = model.addConstr(OPTV.add(OPTV.mul(3, y), z), -OPTV.OPTV_INF, 6, "c1");
```

求解模型

模型构造及相应的参数配置完成后，用户可以通过调用*OPTVModel.optimize()* 来求解模型。求解完成后，求解状态、最优解等属性值都被保存在*OPTVModel* 对象中。

```
model.optimize();
```

查询求解结果

模型求解结束后，用户可以通过*OPTVModel.get()* 方法来查询模型的不同属性和输出所得解。例如，目前最佳的目标值可以通过*OPTVDblAttr.OBJ_VAL* 查询，以下示例展示了查询变量或约束的三种不同方法。

```
System.out.println("Best solution: " + model.get(OPTVDblAttr.OBJ_VAL));

System.out.println("x = " + x.get(OPTVDblAttr.X));
System.out.println("y = " + model.getVar(1).get(OPTVDblAttr.X));
System.out.println("z = " + model.getVarByName("z").get(OPTVDblAttr.X));
```

编译和运行

数学规划求解器的软件包中已经包含了本节介绍的样例程序，路径在 `${OPTV_HOME}/examples/java`。

编译时只需要先进入到 `${OPTV_HOME}/examples` 文件夹，然后执行如下操作即可：

```
javac -cp .:${OPTV_HOME}/lib/optv-java-sdk.jar Milp1.java
```

编译完成后，可执行以下命令：

```
java -cp .:${OPTV_HOME}/lib/optv-java-sdk.jar Milp1
```

如果需要求解特定的 `.mps` 或 `.lp` 格式文件所含模型，用户可以通过以下命令求解和查询模型目标值及状态：

```
java -cp .:${OPTV_HOME}/lib/optv-java-sdk.jar Milp2
```

4.1.4 可行性松弛

本节为用户提供一个简单示例，演示如何松弛不可行问题，使其变为可行。

待求解问题的数学公式如下所示：

$$\begin{array}{lll} \text{minimize} & x + y & \\ \text{subject to} & 2x + y & \leq 2 \\ & -3x + 2y & \leq -3 \\ & x - 2y & \leq -2 \\ & 0 & \leq x, y \\ & x, y \in \mathbb{R}. & \end{array} \tag{4.1}$$

C++

```cpp
#include "optv_c++.h"
using namespace std;

int main()
{
    try {
        OPTVEnv env("feasibility-relax.log");
        OPTVModel model(env);
        model.Set(OPTVStrParam::RESULT_FILE, "feasibility-relax.sol");

        // Add variables to the model, all variables are continuous
        OPTVVar x = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x");
        OPTVVar y = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "y");

        // Add constraint to the model
        OPTVConstr c0 = model.AddConstr(2 * x + y, -OPTV_INF, 2, "c0");
        OPTVConstr c1 = model.AddConstr(-3 * x + 2 * y, -OPTV_INF, -3, "c1");
        OPTVConstr c2 = model.AddConstr(x - 2 * y, -OPTV_INF, -2, "c2");

        model.FeasRelax();

        cout << "L1 violation: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;
        cout << "c2 upper bound = " << c2.Get(OPTVDblAttr::UB) << endl;
    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Python 接口

```python
from optvpy import *

if __name__ == "__main__":
    try:
        env = OPTVEnv("feasibility-relax.log")
        model = OPTVModel(env)
        model.Set(OPTVStrParam.RESULT_FILE, "feasibility-relax.sol")
        obj = 0.0

        # Add variables to the model, all variables are continuous
        x = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x")
        y = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "y")

        # Add constraint to the model
        c0 = model.AddConstr(2 * x + y, -OPTV_INF, 2, "c0")
        c1 = model.AddConstr(-3 * x + 2 * y, -OPTV_INF, -3, "c1")
        c2 = model.AddConstr(x - 2 * y, -OPTV_INF, -2, "c2")

        obj = model.FeasRelax()

        print(f"L1 violation: {model.OBJ_VAL}")
        print(f"c2 upper bound = {c2.UB}")
    except OPTVException as e:
        print(f"{e.GetErrorCode()}: {e.GetMessage()}")
    except Exception:
        print("Unknown exception")
```

Java 接口

```java
import com.huaweicloud.optv.*;

public class FeasibilityRelax {
    public static void main(String[] args) {
        try {
            OPTVEnv env = new OPTVEnv("FeasibilityRelax.log");
            OPTVModel model = new OPTVModel(env);
            model.set(OPTVStrParam.RESULT_FILE, "FeasibilityRelax.sol");

            // add variables to the model, all variables are continuous
            OPTVVar x = model.addVar(0, OPTV.OPTV_INF, 1, OPTV.OPTV_CONTINUOUS, "x");
            OPTVVar y = model.addVar(0, OPTV.OPTV_INF, 1, OPTV.OPTV_CONTINUOUS, "y");

            // add constraint to the model
            OPTVConstr c0 = model.addConstr(OPTV.add(OPTV.mul(2, x), y), -OPTV.OPTV_
↪INF, 2, "c0");
            OPTVConstr c1 = model.addConstr(OPTV.add(OPTV.mul(-3, x), OPTV.mul(2, y)),
↪ -OPTV.OPTV_INF, -3, "c1");
            OPTVConstr c2 = model.addConstr(OPTV.sub(x, OPTV.mul(2, y)), -OPTV.OPTV_
↪INF, -2, "c2");

            model.feasRelax();

            System.out.println("L1 violation: " + model.get(OPTVDblAttr.OBJ_VAL));
```

(下页继续)

(续上页)

```
            System.out.println("c2 upper bound = " + c2.get(OPTVDblAttr.UB));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

引入模块

要访问 OptVerse 接口，我们需要导入所需语言的库。

C++

```
#include "optv_c++.h"
```

Python 接口

```
from optvpy import *
```

Java 接口

```
import com.huaweicloud.optv.*;
```

创建模型

首先初始化一个空的环境对象，提供给*OPTVModel* 构造函数。

C++

```
OPTVEnv env("feasibility-relax.log");
OPTVModel model(env);
```

Python 接口

```
env = OPTVEnv("feasibility-relax.log")
model = OPTVModel(env)
```

Java 接口

```
OPTVEnv env = new OPTVEnv("FeasibilityRelax.log");
OPTVModel model = new OPTVModel(env);
```

添加变量和约束

C++

```
// Add variables to the model, all variables are continuous
OPTVVar x = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x");
OPTVVar y = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "y");

// Add constraint to the model
OPTVConstr c0 = model.AddConstr(2 * x + y, -OPTV_INF, 2, "c0");
OPTVConstr c1 = model.AddConstr(-3 * x + 2 * y, -OPTV_INF, -3, "c1");
OPTVConstr c2 = model.AddConstr(x - 2 * y, -OPTV_INF, -2, "c2");
```

Python 接口

```
# Add variables to the model, all variables are continuous
x = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x")
y = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "y")

# Add constraint to the model
c0 = model.AddConstr(2 * x + y, -OPTV_INF, 2, "c0")
c1 = model.AddConstr(-3 * x + 2 * y, -OPTV_INF, -3, "c1")
c2 = model.AddConstr(x - 2 * y, -OPTV_INF, -2, "c2")
```

Java 接口

```
// add variables to the model, all variables are continuous
OPTVVar x = model.addVar(0, OPTV.OPTV_INF, 1, OPTV.OPTV_CONTINUOUS, "x");
OPTVVar y = model.addVar(0, OPTV.OPTV_INF, 1, OPTV.OPTV_CONTINUOUS, "y");

// add constraint to the model
OPTVConstr c0 = model.addConstr(OPTV.add(OPTV.mul(2, x), y), -OPTV.OPTV_INF, 2, "c0");
OPTVConstr c1 = model.addConstr(OPTV.add(OPTV.mul(-3, x), OPTV.mul(2, y)), -OPTV.OPTV_
↪INF, -3, "c1");
OPTVConstr c2 = model.addConstr(OPTV.sub(x, OPTV.mul(2, y)), -OPTV.OPTV_INF, -2, "c2
↪");
```

使用不可行修复功能

调用 LP 不可行修复 C++ 接口函数*OPTVModel::FeasRelax()* 即可使用不可行修复功能。

C++

```
model.FeasRelax();
```

Python 接口

```
obj = model.FeasRelax()
```

Java 接口

```
model.feasRelax();
```

查询不可行修复结果

用户可以通过调用*model.Get(OPTVDblAttr::OBJ_VAL)* 查询原模型的不可行违背量的 L^1-范数，以获取不可行修复的结果。同时，约束界限所需修改可以通过查询相应约束的属性获取，如*c2.Get(OPTVDblAttr::UB)* .

C++

```
cout << "L1 violation: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;
cout << "c2 upper bound = " << c2.Get(OPTVDblAttr::UB) << endl;
```

Python 接口

```
print(f"L1 violation: {model.OBJ_VAL}")
print(f"c2 upper bound = {c2.UB}")
```

Java 接口

```
System.out.println("L1 violation: " + model.get(OPTVDblAttr.OBJ_VAL));
System.out.println("c2 upper bound = " + c2.get(OPTVDblAttr.UB));
```

4.1.5 不可约不可行子系统 (IIS)

本节演示如何使用 OptVerse SDK 通过 OptVerse 求解器执行 IIS 评估。此示例将构建一个模型，计算 IIS 并将其写入文件。

待求解问题的数学公式如下所示：

$$
\begin{array}{lrrrrl}
\text{minimize} & & x_1 & + x_2 + & x_3 & \\
\text{subject to} & 1 \leq & x_1 & - 4x_2 + & x_3 & \\
& & 3x_1 & - x_2 - & x_3 & \leq 2 \\
& 3 \leq & x_1 & + 2x_2 - & 3x_3 & \\
& & x_1 & - x_2 - & & \leq 5 \\
& & 5x_1 & + 3x_2 + & x_3 & \leq 20 \\
& & & 0 \leq x_1,\ x_2,\ x_3 & & \\
& & & x_1,\ x_2,\ x_3 \in \mathbb{R}. & &
\end{array}
\tag{4.2}
$$

C++

```cpp
#include "optv_c++.h"
using namespace std;

int main()
{
    try {
        OPTVEnv env("iis.log");
        OPTVModel model(env);

        // Add variables to the model, all variables are continuous
        OPTVVar x1 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1");
        OPTVVar x2 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2");
        OPTVVar x3 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3");

        // Add constraints to the model
        OPTVConstr c1 = model.AddConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1");
        OPTVConstr c2 = model.AddConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2");
        OPTVConstr c3 = model.AddConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3");
        OPTVConstr c4 = model.AddConstr(x1 - x2, -OPTV_INF, 5, "c4");
        OPTVConstr c5 = model.AddConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5");

        if (model.ComputeIIS()) {
            model.WriteIIS("iis.ilp");
        } else {
            cout << "IIS computation failed" << endl;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Python 接口

```python
from optvpy import *

if __name__ == "__main__":
    try:
        env = OPTVEnv("iis.log")
        model = OPTVModel(env)

        # Add variables to the model, all variables are continuous
        x1 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1")
        x2 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2")
        x3 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3")

        # Add constraints to the model
        c1 = model.AddConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1")
        c2 = model.AddConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2")
        c3 = model.AddConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3")
        c4 = model.AddConstr(x1 - x2, -OPTV_INF, 5, "c4")
        c5 = model.AddConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5")

        if model.ComputeIIS():
            model.WriteIIS("iis.ilp")
        else:
            print("IIS computation failed")
    except OPTVException as e:
        print(f"{e.GetErrorCode()}: {e.GetMessage()}")
    except Exception:
        print("Unknown exception")
```

Java 接口

```java
import com.huaweicloud.optv.*;

public class IISExample {
    public static void main(String[] args) {
        try {
            OPTVEnv env = new OPTVEnv("iis.log");
            OPTVModel model = new OPTVModel(env);

            // Add variables to the model, all variables are continuous
            OPTVVar x1 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1");
            OPTVVar x2 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2");
            OPTVVar x3 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3");

            // Add constraints to the model
            OPTVConstr c1 = model.addConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1");
            OPTVConstr c2 = model.addConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2");
            OPTVConstr c3 = model.addConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3");
            OPTVConstr c4 = model.addConstr(x1 - x2, -OPTV_INF, 5, "c4");
            OPTVConstr c5 = model.addConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5");

            if (model.computeIIS()) {
                model.writeIIS("iis.ilp");
            } else {
```

(下页继续)

(续上页)

```
                System.out.println("IIS computation failed");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

引入模块

要访问 OptVerse 接口，我们需要导入所需语言的库。

C++

```
#include "optv_c++.h"
```

Python 接口

```
from optvpy import *
```

Java 接口

```
import com.huaweicloud.optv.*;
```

创建模型

首先初始化一个空的环境对象，提供给*OPTVModel* 构造函数。

C++

```
OPTVEnv env("iis.log");
OPTVModel model(env);
```

Python 接口

```
env = OPTVEnv("iis.log")
model = OPTVModel(env)
```

Java 接口

```
OPTVEnv env = new OPTVEnv("iis.log");
OPTVModel model = new OPTVModel(env);
```

添加变量和约束

C++

```
// Add variables to the model, all variables are continuous
OPTVVar x1 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1");
OPTVVar x2 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2");
OPTVVar x3 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3");

// Add constraints to the model
OPTVConstr c1 = model.AddConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1");
OPTVConstr c2 = model.AddConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2");
OPTVConstr c3 = model.AddConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3");
OPTVConstr c4 = model.AddConstr(x1 - x2, -OPTV_INF, 5, "c4");
OPTVConstr c5 = model.AddConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5");
```

Python 接口

```
# Add variables to the model, all variables are continuous
x1 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3")

# Add constraints to the model
c1 = model.AddConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1")
c2 = model.AddConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2")
c3 = model.AddConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3")
c4 = model.AddConstr(x1 - x2, -OPTV_INF, 5, "c4")
c5 = model.AddConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5")
```

Java 接口

```
// Add variables to the model, all variables are continuous
OPTVVar x1 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x1");
OPTVVar x2 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x2");
OPTVVar x3 = model.addVar(0, OPTV_INF, 1, OPTV_CONTINUOUS, "x3");

// Add constraints to the model
OPTVConstr c1 = model.addConstr(x1 - 4*x2 + x3, 1, OPTV_INF, "c1");
OPTVConstr c2 = model.addConstr(3*x1 - x2 - x3, -OPTV_INF, 2, "c2");
OPTVConstr c3 = model.addConstr(x1 + 2*x2 - 3*x3, 3, OPTV_INF, "c3");
OPTVConstr c4 = model.addConstr(x1 - x2, -OPTV_INF, 5, "c4");
OPTVConstr c5 = model.addConstr(5*x1 + 3*x2 + x3, -OPTV_INF, 20, "c5");
```

使用 IIS 计算功能

IIS 可以通过调用*OPTVModel::ComputeIIS()* 函数获取。该函数返回一个布尔变量，表征 IIS 是否被成功计算。

C++

```
if (model.ComputeIIS()) {
    model.WriteIIS("iis.ilp");
} else {
    cout << "IIS computation failed" << endl;
}
```

Python 接口

```
if model.ComputeIIS():
    model.WriteIIS("iis.ilp")
else:
    print("IIS computation failed")
```

Java 接口

```
if (model.computeIIS()) {
    model.writeIIS("iis.ilp");
} else {
    System.out.println("IIS computation failed");
}
```

在该示例中，IIS 是 c_1, c_2, c_3 约束以及其涉及的变量 x_1, x_2, x_3 边界约束。具体地，$c_1 + (-c2) + c3 \implies 2 \leq -x_1 - x_2 - x_3$，但是由于涉及到的变量 x_1, x_2, x_3 都大于等于 0，导致矛盾，从而它们不能同时成立。

4.1.6 LP 灵敏度分析

本节将通过一个简单的 C++ 示例来演示数学规划求解器 LP 敏感度分析的使用。该示例构建了一个模型，并对其进行求解，最后输出敏感度分析信息。

待求解问题的数学公式如下所示：

$$\begin{aligned} \text{minimize} \quad & -x_1 - 2x_2 - 3x_3 & \\ \text{subject to} \quad & -x_1 + x_2 + x_3 & \leq 20 \\ & x_1 - 3x_2 + x_3 & \leq 30 \\ & x_1 \leq 40, x_2 \geq 0, x_3 \geq 0 & \\ & x_1, x_2, x_3 \in \mathbb{R}. & \end{aligned} \tag{4.3}$$

C++

```
/*
* Copyright (c) Huawei Technologies Co., Ltd. 2022-2023. All rights reserved.
* Description: OptVerse c++ interface example - LP sensitivity analysis␣
↪functionalities
*/

#include <iomanip>

#include "optv_c++.h"
using namespace std;

int main()
{
    try {
        OPTVEnv env("lp-sensitivity-analysis.log");
        OPTVModel model(env);
        model.Set(OPTVStrParam::RESULT_FILE, "lp-sensitivity-analysis.sol");

        // Construct the model
        OPTVVar x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1");
        OPTVVar x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2");
        OPTVVar x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3");
        std::vector<OPTVVar> x {x1, x2, x3};

        model.AddConstr(-x1 + x2 +x3, -OPTV_INF, 20, "c1");
        model.AddConstr(x1 - 3*x2 +x3, -OPTV_INF, 30, "c2");

        // Optimize the model
        model.Optimize();
        if (model.Get(OPTVIntAttr::SOL_COUNT) > 0) {
            std::vector<int> rows{0, 1};
            std::vector<int> cols{0, 1, 2};
            cout << "-----------------Display---------------" << endl;

            cout << "Optimal Solution:" << endl;
            cout << std::setprecision(2) << std::scientific;
            for (auto j: cols) {
                cout << "x"<< (j+1) << " = " << x[j].Get(OPTVDblAttr::X) << endl;
            }

            // Sensitivity analysis on variables

            // Reduced cost: how much the objective will change if we increase the␣
↪simple bound constraints by 1
            cout << "Reduced Cost:" << endl;
            for (auto j : cols) {
                cout << "x"<< (j+1) << " : " << model.GetVar(j).
↪Get(OPTVDblAttr::DUAL) << endl;
            }


            // Objective coefficient sensitivity info, within which the current␣
↪optimal basis would remain optimal
            cout << "objective SA:" << endl;
            std::vector<double> objLower{};
```

(下页继续)

(续上页)

```
            std::vector<double> objUpper{};
            bool ok = model.GetObjSA(objLower, objUpper, cols);
            for (auto j : cols) {
                cout << "x"<< (j+1) << " : [" << objLower[j] <<", " << objUpper[j] <<
↪"]" << endl;
            }

            // Lower bound sensitivity info, within which the current optimal basis␣
↪would remain optimal
            cout << "Lower bound SA:" << endl;
            std::vector<double> lbLower{};
            std::vector<double> lbUpper{};
            ok = model.GetVarLbSA(lbLower, lbUpper, cols);
            for (auto j : cols) {
                cout << "x"<< (j+1) << " : [" << lbLower[j] <<", " << lbUpper[j] << "]
↪" << endl;
            }

            // Upper bound sensitivity info, within which the current optimal basis␣
↪would remain optimal
            cout << "Upper bound SA:" << endl;
            std::vector<double> ubLower{};
            std::vector<double> ubUpper{};
            ok = model.GetVarUbSA(ubLower, ubUpper, cols);
            for (auto j : cols) {
                cout << "x"<< (j+1) << " : [" << ubLower[j] <<", " << ubUpper[j] << "]
↪" << endl;
            }

            // Sensitivity analysis on constraints

            // Shadow price: how much the objective will change if we increase the␣
↪right-hand side of a structural constraint by 1
            cout << "Shadow Price:" << endl;
            for (auto i : rows) {
                cout << "c"<< (i+1) << " : " << model.GetConstr(i).
↪Get(OPTVDblAttr::DUAL) << endl;
            }

            // LHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
            cout << "Constraint left-hand-side SA:" << endl;
            std::vector<double> lhsLower{};
            std::vector<double> lhsUpper{};
            ok = model.GetConstrLbSA(lhsLower, lhsUpper, rows);
            for (auto i : rows) {
                cout << "c"<< (i+1) << " : [" << lhsLower[i] <<", " << lhsUpper[i] <<
↪"]" << endl;
            }

            // RHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
            cout << "Constraint right-hand-side SA:" << endl;
            std::vector<double> rhsLower{};
            std::vector<double> rhsUpper{};
            ok = model.GetConstrUbSA(rhsLower, rhsUpper, rows);
            for (auto i : rows) {
```

(下页继续)

(续上页)

```
                cout << "c"<< (i+1) << " : [" << rhsLower[i] <<", " << rhsUpper[i] <<
↪"]" << endl;
            }
        } else {
            cout << std::defaultfloat << "No feasible solution available!" << endl;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

Python 接口

```
from __future__ import print_function
from optvpy import *

def print(*args):
    """ Print globally in scientific notation with a given precision """
    __builtins__.print(*("%.15e" % arg if isinstance(arg, float) else arg for arg in␣
↪args))

if __name__ == "__main__":
    env = OPTVEnv("lp-sensitivity-analysis.log")
    model = OPTVModel(env)
    model.Set(OPTVStrParam.RESULT_FILE, "lp-sensitivity-analysis.sol")

    # Construct the model
    x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
    x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
    x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
    x = [x1, x2, x3]
    model.AddConstr(-x1 + x2 + x3, -OPTV_INF, 20, "c1")
    model.AddConstr(x1 - 3*x2 + x3, -OPTV_INF, 30, "c2")

    # Optimize the model
    model.Optimize()
    if model.Get(OPTVIntAttr.SOL_COUNT) > 0:
        rows = [0, 1]
        cols = [0, 1, 2]

        print("-----------------Display---------------")

        print("Optimal Solution:")
        for j in range(3):
            print("x%s = %s" % (j+1, x[j].Get(OPTVDblAttr.X)))

        # Sensitivity analysis on variables

        # Reduced cost: how much the objective will change if we increase the simple␣
↪bound constraints by 1
        print("Reduced Cost:")
```

(下页继续)

(续上页)

```
        for i in cols:
            print("x%s : %s" % (i+1, model.GetVar(i).Get(OPTVDblAttr.DUAL)))

        # Objective coefficient sensitivity info, within which the current optimal␣
↪basis would remain optimal
        print("objective SA:")
        objLower = []
        objUpper = []
        ok = model.GetObjSA(objLower, objUpper, cols)
        for i in cols:
            print("x%s : [%s, %s]" % (i+1, objLower[i], objUpper[i]))

        # Lower bound sensitivity info, within which the current optimal basis would␣
↪remain optimal
        print("Lower bound SA:")
        lbLower = []
        lbUpper = []
        ok = model.GetVarLbSA(lbLower, lbUpper, cols)
        for i in cols:
            print("x%s : [%s, %s]" % (i+1, lbLower[i], lbUpper[i]))

        # Upper bound sensitivity info, within which the current optimal basis would␣
↪remain optimal
        print("Upper bound SA:")
        ubLower = []
        ubUpper = []
        ok = model.GetVarUbSA(ubLower, ubUpper, cols)
        for i in cols:
            print("x%s : [%s, %s]" % (i+1, ubLower[i], ubUpper[i]))

        # Sensitivity analysis on constraints

        # Shadow price: how much the objective will change if we increase the right-
↪hand side of a structural constraint by 1
        print("Shadow Price:")
        for i in rows:
            print("c%s : %s" % (i+1, model.GetConstr(i).Get(OPTVDblAttr.DUAL)))

        # LHS sensitivity info, within which the current optimal basis would remain␣
↪optimal
        print("Constraint left-hand-side SA:")
        lhsLower = []
        lhsUpper = []
        ok = model.GetConstrLbSA(lhsLower, lhsUpper, rows)
        for i in rows:
            print("c%s : [%s, %s]" % (i+1, lhsLower[i], lhsUpper[i]))

        # RHS sensitivity info, within which the current optimal basis would remain␣
↪optimal
        print("Constraint right-hand-side SA:")
        rhsLower = []
        rhsUpper = []
        ok = model.GetConstrUbSA(rhsLower, rhsUpper, rows)
        for i in rows:
            print("c%s : [%s, %s]" % (i+1, rhsLower[i], rhsUpper[i]))
    else:
        print("No feasible solution available!")
```

Java 接口

```
import com.huaweicloud.optv.*;
import java.util.*;

public class LpSensitivityAnalysis {
    public static void main(String[] args) {
        try {
            OPTVEnv env = new OPTVEnv("LpSensitivityAnalysis.log");
            OPTVModel model = new OPTVModel(env);
            model.set(OPTVStrParam.RESULT_FILE, "LpSensitivityAnalysis.sol");

            // construct the model
            OPTVVar x1 = model.addVar(-OPTV.OPTV_INF, 40, -1, OPTV.OPTV_CONTINUOUS,
↪"x1");
            OPTVVar x2 = model.addVar(0, OPTV.OPTV_INF, -2, OPTV.OPTV_CONTINUOUS, "x2
↪");
            OPTVVar x3 = model.addVar(0, OPTV.OPTV_INF, -3, OPTV.OPTV_CONTINUOUS, "x3
↪");
            OPTVVar[] x = {x1, x2, x3};
            // -x1 + x2 +x3
            model.addConstr(OPTV.add(OPTV.add(OPTV.sub(x1), x2), x3), -OPTV.OPTV_INF,␣
↪20, "c1");
            // x1 - 3*x2 +x3
            model.addConstr(OPTV.add(OPTV.sub(x1, OPTV.mul(3, x2)), x3), -OPTV.OPTV_
↪INF, 30, "c2");

            // optimize the model
            model.optimize();
            model.write("LpSensitivityAnalysis.lp");

            if (model.get(OPTVIntAttr.SOL_COUNT) > 0) {
                int[] rows = {0, 1};
                int[] cols = {0, 1, 2};

                System.out.println("-----------------Display---------------");

                System.out.println("Optimal Solution:");
                for (int j: cols) {
                    System.out.printf("x%d = %.2e\n", j+1, x[j].get(OPTVDblAttr.X));
                }

                // // sensitivity analysis on variables

                // reduced cost: how much the objective will change if we increase␣
↪the simple bound constraints by 1
                System.out.println("Reduced Cost:");
                for (int j : cols) {
                    System.out.printf("x%d : %.2e\n", j+1, model.getVar(j).
↪get(OPTVDblAttr.DUAL));
                }

                // objective coefficient sensitivity info, within which the current␣
↪optimal basis would remain optimal
                System.out.println("objective SA:");
                OPTVModel.SensitivityBounds objBounds = model.getObjSA(cols);
                for (int j : cols) {
```

(下页继续)

(续上页)

```
                    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, objBounds.
↪down[j], objBounds.up[j]);
                }

                // lower bound sensitivity info, within which the current optimal␣
↪basis would remain optimal
                System.out.println("Lower bound SA:");
                OPTVModel.SensitvityBounds lbBounds = model.getVarLbSA(cols);
                for (int j : cols) {
                    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, lbBounds.down[j],
↪ lbBounds.up[j]);
                }

                // upper bound sensitivity info, within which the current optimal␣
↪basis would remain optimal
                System.out.println("Upper bound SA:");
                OPTVModel.SensitvityBounds ubBound = model.getVarUbSA(cols);
                for (int j : cols) {
                    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, ubBound.down[j],␣
↪ubBound.up[j]);
                }

                // // sensitivity analysis on constraints

                // shadow price: how much the objective will change if we increase␣
↪the right-hand side of a structural constraint by 1
                System.out.println("Shadow Price:");
                for (int i : rows) {
                    System.out.printf("c%d : %.2e\n", i+1, model.getConstr(i).
↪get(OPTVDblAttr.DUAL));
                }

                // LHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
                System.out.println("Constraint left-hand-side SA:");
                OPTVModel.SensitvityBounds lhsBounds = model.getConstrLbSA(rows);
                for (int i : rows) {
                    System.out.printf("c%d : [ %.2e,  %.2e]\n", i+1, lhsBounds.
↪down[i], lhsBounds.up[i]);
                }

                // RHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
                System.out.println("Constraint right-hand-side SA:");
                OPTVModel.SensitvityBounds rhsBounds = model.getConstrUbSA(rows);
                for (int i : rows) {
                    System.out.printf("c%d : [ %.2e,  %.2e]\n", i+1, rhsBounds.
↪down[i], rhsBounds.up[i]);
                }
            } else {
                System.out.println("No feasible solution available!");
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

引入模块

要访问 OptVerse 接口，我们需要导入所需语言的库。

C++

```
#include "optv_c++.h"
```

Python 接口

```
from optvpy import *
```

Java 接口

```
import com.huaweicloud.optv.*;
```

创建模型

首先初始化一个空的环境对象，提供给*OPTVModel* 构造函数。

C++

```
OPTVEnv env("lp-sensitivity-analysis.log");
OPTVModel model(env);
```

Python 接口

```
env = OPTVEnv("lp-sensitivity-analysis.log")
model = OPTVModel(env)
model.Set(OPTVStrParam.RESULT_FILE, "lp-sensitivity-analysis.sol")
```

Java 接口

```
OPTVEnv env = new OPTVEnv("LpSensitivityAnalysis.log");
OPTVModel model = new OPTVModel(env);
```

添加变量和约束

C++

```
// Construct the model
OPTVVar x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1");
OPTVVar x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2");
OPTVVar x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3");
std::vector<OPTVVar> x {x1, x2, x3};

model.AddConstr(-x1 + x2 +x3, -OPTV_INF, 20, "c1");
model.AddConstr(x1 - 3*x2 +x3, -OPTV_INF, 30, "c2");
```

Python 接口

```
# Construct the model
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
x = [x1, x2, x3]
model.AddConstr(-x1 + x2 + x3, -OPTV_INF, 20, "c1")
model.AddConstr(x1 - 3*x2 + x3, -OPTV_INF, 30, "c2")
```

Java 接口

```
// construct the model
OPTVVar x1 = model.addVar(-OPTV.OPTV_INF, 40, -1, OPTV.OPTV_CONTINUOUS, "x1");
OPTVVar x2 = model.addVar(0, OPTV.OPTV_INF, -2, OPTV.OPTV_CONTINUOUS, "x2");
OPTVVar x3 = model.addVar(0, OPTV.OPTV_INF, -3, OPTV.OPTV_CONTINUOUS, "x3");
OPTVVar[] x = {x1, x2, x3};
// -x1 + x2 +x3
model.addConstr(OPTV.add(OPTV.add(OPTV.sub(x1), x2), x3), -OPTV.OPTV_INF, 20, "c1");
// x1 - 3*x2 +x3
model.addConstr(OPTV.add(OPTV.sub(x1, OPTV.mul(3, x2)), x3), -OPTV.OPTV_INF, 30, "c2
↪");
```

求解模型

完成模型创建及配置所需求解参数后，可以通过调用*OPTVModel::Optimize()* 求解该模型。

C++

```
model.Optimize();
```

Python 接口

```
model.Optimize()
```

Java 接口

```
model.optimize();
```

输出模型解的信息

C++

```
cout << "Optimal Solution:" << endl;
cout << std::setprecision(2) << std::scientific;
for (auto j: cols) {
    cout << "x"<< (j+1) << " = " << x[j].Get(OPTVDblAttr::X) << endl;
}
```

Python 接口

```
print("Optimal Solution:")
for j in range(3):
    print("x%s = %s" % (j+1, x[j].Get(OPTVDblAttr.X)))
```

Java 接口

```
System.out.println("Optimal Solution:");
for (int j: cols) {
    System.out.printf("x%d = %.2e\n", j+1, x[j].get(OPTVDblAttr.X));
}
```

递减成本

递减成本反映了目标值对变量界限的敏感度，具体地，它们对应变量界限每增加一个单位后目标值的改变量。

C++

```
cout << "Optimal Solution:" << endl;
cout << std::setprecision(2) << std::scientific;
for (auto j: cols) {
    cout << "x"<< (j+1) << " = " << x[j].Get(OPTVDblAttr::X) << endl;
}
```

Python 接口

```
print("Optimal Solution:")
for j in range(3):
    print("x%s = %s" % (j+1, x[j].Get(OPTVDblAttr.X)))
```

Java 接口

```
System.out.println("Optimal Solution:");
for (int j: cols) {
    System.out.printf("x%d = %.2e\n", j+1, x[j].get(OPTVDblAttr.X));
}
```

目标函数敏感度分析

对于指定的变量（通过 `cols` 参数设定），目标函数敏感性分析功能提供一个区间 [`objLower` , `objUpper`]，只要对应的目标系数在此区间内，无论如何变化，所得模型的最优基不变。

C++

```
// Objective coefficient sensitivity info, within which the current optimal basis␣
↪would remain optimal
cout << "objective SA:" << endl;
std::vector<double> objLower{};
std::vector<double> objUpper{};
bool ok = model.GetObjSA(objLower, objUpper, cols);
for (auto j : cols) {
    cout << "x"<< (j+1) << " : [" << objLower[j] <<", " << objUpper[j] << "]" << endl;
}
```

Python 接口

```
# Objective coefficient sensitivity info, within which the current optimal basis␣
↪would remain optimal
print("objective SA:")
objLower = []
objUpper = []
ok = model.GetObjSA(objLower, objUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, objLower[i], objUpper[i]))
```

Java 接口

```
// objective coefficient sensitivity info, within which the current optimal basis␣
↪would remain optimal
System.out.println("objective SA:");
OPTVModel.SensitivityBounds objBounds = model.getObjSA(cols);
for (int j : cols) {
    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, objBounds.down[j], objBounds.
↪up[j]);
}
```

变量上/下界敏感度分析

对于指定的变量（通过 cols 参数设定），上/下界敏感性分析功能提供一个区间（如下界的敏感度区间 [lbLower , lbUpper], 上界的敏感度区间 [ubLower , ubUpper]），只要对应的上/下界在此区间内，无论如何变化，所得模型的最优基不变。

C++

```
// Lower bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
cout << "Lower bound SA:" << endl;
std::vector<double> lbLower{};
std::vector<double> lbUpper{};
ok = model.GetVarLbSA(lbLower, lbUpper, cols);
for (auto j : cols) {
    cout << "x"<< (j+1) << " : [" << lbLower[j] <<", " << lbUpper[j] << "]" << endl;
}
```

Python 接口

```
# Lower bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
print("Lower bound SA:")
lbLower = []
lbUpper = []
ok = model.GetVarLbSA(lbLower, lbUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, lbLower[i], lbUpper[i]))
```

Java 接口

```
// lower bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
System.out.println("Lower bound SA:");
OPTVModel.SensitvityBounds lbBounds = model.getVarLbSA(cols);
for (int j : cols) {
    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, lbBounds.down[j], lbBounds.
↪up[j]);
}
```

C++

```
// Upper bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
cout << "Upper bound SA:" << endl;
std::vector<double> ubLower{};
std::vector<double> ubUpper{};
ok = model.GetVarUbSA(ubLower, ubUpper, cols);
for (auto j : cols) {
    cout << "x"<< (j+1) << " : [" << ubLower[j] <<", " << ubUpper[j] << "]" << endl;
}
```

Python 接口

```
# Upper bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
print("Upper bound SA:")
ubLower = []
ubUpper = []
ok = model.GetVarUbSA(ubLower, ubUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, ubLower[i], ubUpper[i]))
```

Java 接口

```
// upper bound sensitivity info, within which the current optimal basis would remain␣
↪optimal
System.out.println("Upper bound SA:");
OPTVModel.SensitvityBounds ubBound = model.getVarUbSA(cols);
for (int j : cols) {
    System.out.printf("x%d : [ %.2e,  %.2e]\n", j+1, ubBound.down[j], ubBound.up[j]);
}
```

影子价格

影子价格反映了目标值对约束右侧值的敏感度，具体地，它们对应约束右侧值每增加一个单位后目标值的改变量。

C++

```
// Shadow price: how much the objective will change if we increase the right-hand␣
↪side of a structural constraint by 1
cout << "Shadow Price:" << endl;
for (auto i : rows) {
    cout << "c"<< (i+1) << " : " << model.GetConstr(i).Get(OPTVDblAttr::DUAL) << endl;
}
```

Python 接口

```
# Shadow price: how much the objective will change if we increase the right-hand side␣
↪of a structural constraint by 1
print("Shadow Price:")
for i in rows:
    print("c%s : %s" % (i+1, model.GetConstr(i).Get(OPTVDblAttr.DUAL)))
```

Java 接口

```
// shadow price: how much the objective will change if we increase the right-hand␣
↪side of a structural constraint by 1
System.out.println("Shadow Price:");
for (int i : rows) {
    System.out.printf("c%d : %.2e\n", i+1, model.getConstr(i).get(OPTVDblAttr.DUAL));
}
```

约束左/右侧值敏感度分析

对于指定的约束（通过 rows 参数设定），左/右侧值敏感性分析功能提供一个区间，具体地，左侧值的敏感度区间 [lhsLower, lhsUpper]，右侧值的敏感度区间 [rhsLower, rhsUpper]。只要对应的左/右侧值在此区间内，无论如何变化，所得模型的最优基不变。

C++

```
// LHS sensitivity info, within which the current optimal basis would remain optimal
cout << "Constraint left-hand-side SA:" << endl;
std::vector<double> lhsLower{};
std::vector<double> lhsUpper{};
ok = model.GetConstrLbSA(lhsLower, lhsUpper, rows);
for (auto i : rows) {
    cout << "c"<< (i+1) << " : [" << lhsLower[i] <<", " << lhsUpper[i] << "]" << endl;
}
```

Python 接口

```
# LHS sensitivity info, within which the current optimal basis would remain optimal
print("Constraint left-hand-side SA:")
lhsLower = []
lhsUpper = []
ok = model.GetConstrLbSA(lhsLower, lhsUpper, rows)
for i in rows:
    print("c%s : [%s, %s]" % (i+1, lhsLower[i], lhsUpper[i]))
```

Java 接口

```
System.out.println("Constraint left-hand-side SA:");
OPTVModel.SensitvityBounds lhsBounds = model.getConstrLbSA(rows);
for (int i : rows) {
    System.out.printf("c%d : [ %.2e,  %.2e]\n", i+1, lhsBounds.down[i], lhsBounds.
↪up[i]);
}
```

C++

```
// RHS sensitivity info, within which the current optimal basis would remain optimal
cout << "Constraint right-hand-side SA:" << endl;
std::vector<double> rhsLower{};
std::vector<double> rhsUpper{};
ok = model.GetConstrUbSA(rhsLower, rhsUpper, rows);
for (auto i : rows) {
    cout << "c"<< (i+1) << " : [" << rhsLower[i] <<", " << rhsUpper[i] << "]" << endl;
}
```

Python 接口

```
# RHS sensitivity info, within which the current optimal basis would remain optimal
print("Constraint right-hand-side SA:")
rhsLower = []
rhsUpper = []
ok = model.GetConstrUbSA(rhsLower, rhsUpper, rows)
for i in rows:
    print("c%s : [%s, %s]" % (i+1, rhsLower[i], rhsUpper[i]))
```

Java 接口

```
System.out.println("Constraint right-hand-side SA:");
OPTVModel.SensitvityBounds rhsBounds = model.getConstrUbSA(rows);
for (int i : rows) {
    System.out.printf("c%d : [ %.2e,  %.2e]\n", i+1, rhsBounds.down[i], rhsBounds.
↪up[i]);
```

4.2 天筹约束规划求解器快速入门

本章提供了一些基本的示例，用于演示如何调用约束规划求解器各语言接口进行约束规划问题建模和求解。

4.2.1 C++ 接口

本节将通过一个简单的示例来演示约束规划求解器 C++ 接口的使用。

这个示例将介绍如何：

- 设置求解器参数
- 构建模型
- 将构建模型写入文件
- 优化模型
- 打印最优解信息

待求解问题的数学公式如下所示：

$$
\begin{aligned}
\text{minimize } & b_1 + b_2 + b_3 + b_4 + 2x_1 + 2x_2 \\
\text{subject to } & -x_1 + 4x_2 + 8 \leq 15 \\
& 3b_1 - b_2 + x_1 = [3,4] \cup [6,10] \cup [12,14] \\
& b_1 \vee b_2 \vee b_4 \\
& \neg b_1 \wedge b_3 \\
& b_2 + b_3 \geq 1 \\
& b_2 + b_4 \leq 1 \\
& b_2 + b_4 = 1 \\
& b_1 \rightarrow b_4 \\
\text{where } & b_1, b_2, b_3, b_4 \in [0,1] \\
& x_1 \in [2,5] \\
& x_2 \in [1,2] \cup [5,7] \cup [15,15]
\end{aligned}
$$

```cpp
/*
 * Copyright (c) Huawei Technologies Co., Ltd. 2025. All rights reserved.
 * Description: OptVerse c++ interface example - build and solve a CP-SAT problem
 */
#include "optv_c++.h"
using namespace std;

int main()
{
    try {
        OPTVEnv env;
        // Before model construction: set parameters through environment
        env.Set(OPTVDblParam::CP_TIME_LIMIT, 7200);

        OPTVCPModel model(env);
        // After model construction: set parameters through model
        model.Set(OPTVIntParam::CP_THREADS, 12);

        // Add variables to the model
        OPTVCPVar b1 = model.NewBoolVar("b1");
        OPTVCPVar b2 = model.NewBoolVar("b2");
        OPTVCPVar b3 = model.NewBoolVar("b3");
        OPTVCPVar b4 = model.NewBoolVar("b4");
        OPTVCPVar x1 = model.NewIntVar(2, 5, "x1");
```

(下页继续)

(续上页)

```
        // domain: [1, 2], [5, 7], [15, 15]
        OPTVCPVar x2 = model.NewIntVarFromDomain({1, 2, 5, 7, 15, 15}, "x2");
        OPTVCPVar c1 = model.NewConstant(11);

        // Add constraint to the model
        OPTVCPConstr lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_
↪INF, 15);
        // domain: [3, 4], [6, 10], [12, 14]
        std::vector<int64_t> domain = {3, 4, 6, 10, 12, 14};
        OPTVCPConstr lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1,␣
↪domain);

        OPTVCPConstr boolOr = model.AddBoolOr({b1, b2, b4});
        OPTVCPConstr boolAnd = model.AddBoolAnd({~b1, b3});
        OPTVCPConstr boolALO = model.AddAtLeastOne({b2, b3});
        OPTVCPConstr boolAMO = model.AddAtMostOne({b2, b4});
        OPTVCPConstr boolEO = model.AddExactlyOne({b2, b4});
        OPTVCPConstr boolImpl = model.AddImplication({b1, b4});

        boolOr.OnlyEnforceIf({b3, b4});

        model.Write("cp.fzn");
        OPTVCPSolver solver;
        auto status = solver.Solve(model);

        switch (status) {
            case OPTV_CP_UNKNOWN:
                cout << "Model is unknown." << endl;
                break;
            case OPTV_CP_MODEL_INVALID:
                cout << "Model is invalid." << endl;
                break;
            case OPTV_CP_FEASIBLE:
                cout << "Model is feasible." << endl;
                break;
            case OPTV_CP_INFEASIBLE:
                cout << "Model is infeasible." << endl;
                break;
            case OPTV_CP_OPTIMAL: {
                std::vector<OPTVCPVar> vars{b1, b2, b3, b4, x1, x2};
                const auto& result = solver.GetValues(vars);
                cout << "Optimal solution found. Best solution: " << endl;
                for (int i = 0; i < result.size(); ++i) {
                    cout << "  " << vars[i].GetName() << " = " << result[i] << endl;
                }

                double objVal = solver.GetObjectiveValue();
                break;
            }
            case OPTV_CP_USER_INTERRUPT:
                cout << "Solver was interrupted by the user." << endl;
                break;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
```

(下页继续)

(续上页)

```
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

包含头文件

要使用 C++ 接口，首先要包含头文件 `optv_c++.h`。

```
#include "optv_c++.h"
```

创建环境

第一步需要创建一个空的环境对象。

```
OPTVEnv env;
```

模型创建前的参数配置

若需要在调用*OPTVCPModel* 构造函数 **前**修改求解器参数，用户必须通过环境对象设置这些参数。在本例中，我们展示了在创建模型对象前如何通过环境对象设置求解器的时限参数。

```
env.Set(OPTVDblParam::CP_TIME_LIMIT, 7200);
```

模型创建后的参数配置

若需要在调用*OPTVCPModel* 构造函数 **后**修改求解器参数，用户则必须通过模型对象设置这些参数。在本例中，我们展示了如何通过模型对象设置线程数量。

```
model.Set(OPTVIntParam::CP_THREADS, 12);
```

创建模型

创建了环境后，下一步就是创建模型。每个模型是一个优化问题，它由一组变量，一组约束和关联属性 (变量边界、目标系数、约束右端项等) 组成。

创建空模型

如果用户需要通过 API 一步一步构造模型，首先需要做的是创建一个空模型。

```
OPTVCPModel model(env);
```

从模型文件加载

如果把 `.mps`[1] 、`.lp`[2] 或 `.fzn`[3] 文件路径当做第二个参数传入，该文件则被读取，相应的模型可以通过 API 求解或修改。

```
OPTVEnv env;
OPTVCPModel(env, fileName);
```

向模型中添加变量

如果上一步创建了一个空模型，那么用户 **必须**向该模型中添加变量和约束后，才可以进行求解。同样，也可以通过添加变量或约束来扩展从文件加载的模型。

可使用以下方法向模型中添加变量：

- *OPTVCPModel::NewIntVar()*
- *OPTVCPModel::NewIntVarFromDomain()*
- *OPTVCPModel::NewBoolVar()*
- *OPTVCPModel::NewConstant()*

```
OPTVCPVar b1 = model.NewBoolVar("b1");
OPTVCPVar b2 = model.NewBoolVar("b2");
OPTVCPVar b3 = model.NewBoolVar("b3");
OPTVCPVar b4 = model.NewBoolVar("b4");
OPTVCPVar x1 = model.NewIntVar(2, 5, "x1");
// domain: [1, 2], [5, 7], [15, 15]
OPTVCPVar x2 = model.NewIntVarFromDomain({1, 2, 5, 7, 15, 15}, "x2");
OPTVCPVar c1 = model.NewConstant(11);
```

向模型中添加约束

变量添加完成后，用户可以继续添加约束。约束规划求解器支持以下类型的约束：

- *Linear*
- *Boolean Or*
- *Boolean And*
- *Boolean Xor*
- *Implication*
- *At Least One*
- *At Most One*
- *Exactly One*
- *Allowed Assignments*;
- *Forbidden Assignments*;
- *Interval*

[1] 有关 MPS 格式的说明，请参见文件格式。
[2] 有关 LP 格式的说明，请参见文件格式。
[3] 有关 FZN 格式的说明，请参见文件格式。

- *No Overlap*
- *No Overlap 2D*;
- *Cumulative*
- *All Different*;
- *Element*;
- *Circuit*;
- *Routes*;
- *Max Equality*;
- *Min Equality*;
- *Abs Equality*;
- *Multiplication Equality*;
- *Division Equality*;
- *Modulo Equality*;
- *Inverse*.

```
OPTVCPConstr lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_INF,␣
↪15);
// domain: [3, 4], [6, 10], [12, 14]
std::vector<int64_t> domain = {3, 4, 6, 10, 12, 14};
OPTVCPConstr lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, domain);

OPTVCPConstr boolOr = model.AddBoolOr({b1, b2, b4});
OPTVCPConstr boolAnd = model.AddBoolAnd({~b1, b3});
OPTVCPConstr boolALO = model.AddAtLeastOne({b2, b3});
OPTVCPConstr boolAMO = model.AddAtMostOne({b2, b4});
OPTVCPConstr boolEO = model.AddExactlyOne({b2, b4});
OPTVCPConstr boolImpl = model.AddImplication({b1, b4});

boolOr.OnlyEnforceIf({b3, b4});
```

求解模型

模型构建完成并设置好参数后，可以使用*OPTVCPSolver::Solve()* 求解模型。求解完成后，求解状态与求解结果将保存在*OPTVCPSolver* 对象中。

```
OPTVCPSolver solver;
auto status = solver.Solve(model);
```

查询求解结果

求解完成后，可以通过*OPTVCPSolver*访问以求解模型中的相关数据。函数*OPTVCPSolver::Solve()*在求解完模型后会返回求解状态，用于判断是否找到可行解。使用*OPTVCPSolver::GetObjectiveValue()*可以获取当前找到的最优目标值。而*OPTVCPSolver::GetSolution()*、*OPTVCPSolver::GetValue()*、*OPTVCPSolver::GetValues()*等方法可以获取变量的取值。

```
switch (status) {
    case OPTV_CP_UNKNOWN:
        cout << "Model is unknown." << endl;
        break;
    case OPTV_CP_MODEL_INVALID:
        cout << "Model is invalid." << endl;
        break;
    case OPTV_CP_FEASIBLE:
        cout << "Model is feasible." << endl;
        break;
    case OPTV_CP_INFEASIBLE:
        cout << "Model is infeasible." << endl;
        break;
    case OPTV_CP_OPTIMAL: {
        std::vector<OPTVCPVar> vars{b1, b2, b3, b4, x1, x2};
        const auto& result = solver.GetValues(vars);
        cout << "Optimal solution found. Best solution: " << endl;
        for (int i = 0; i < result.size(); ++i) {
            cout << "  " << vars[i].GetName() << " = " << result[i] << endl;
        }

        double objVal = solver.GetObjectiveValue();
        break;
    }
    case OPTV_CP_USER_INTERRUPT:
        cout << "Solver was interrupted by the user." << endl;
        break;
}
```

编译和运行

约束规划求解器的软件包中已经包含了本节介绍的样例程序，路径在`${OPTV_HOME}/examples/c++`，同时提供了 CMake 文件用于在 Linux 平台下编译所有样例。

编译时只需要先进入到`${OPTV_HOME}/examples`文件夹，然后执行如下操作即可：

```
mkdir build && cd build
cmake ..
make -j $(nproc)
```

编译完成后，上述示例可以通过调用以下命令运行：

```
./cp
```

4.2.2 Python 接口

本节将通过一个简单的示例来演示约束规划求解器 Python 接口的使用。

这个示例将介绍如何：

- 设置求解器参数
- 构建模型
- 将构建模型写入文件
- 优化模型
- 打印最优解信息

待求解问题的数学公式如下所示：

$$
\begin{aligned}
\text{minimize } & b_1 + b_2 + b_3 + b_4 + 2x_1 + 2x_2 \\
\text{subject to } & -x_1 + 4x_2 + 8 \leq 15 \\
& 3b_1 - b_2 + x_1 = [3,4] \cup [6,10] \cup [12,14] \\
& b_1 \vee b_2 \vee b_4 \\
& \neg b_1 \wedge b_3 \\
& b_2 + b_3 \geq 1 \\
& b_2 + b_4 \leq 1 \\
& b_2 + b_4 = 1 \\
& b_1 \rightarrow b_4 \\
\text{where } & b_1, b_2, b_3, b_4 \in [0,1] \\
& x_1 \in [2,5] \\
& x_2 \in [1,2] \cup [5,7] \cup [15,15]
\end{aligned}
$$

```
"""
   Copyright (c) Huawei Technologies Co., Ltd. 2025. All rights reserved.
   Description: OptVerse python interface example - build and solve a CP-SAT problem
"""

from optvpy import *

if __name__ == "__main__":
    # Create an environment
    env = OPTVEnv()
    env.Set(OPTVDblParam.CP_TIME_LIMIT, 7200)

    # Create an empty model
    model = OPTVCPModel(env)
    # After model construction: set parameters through model
    model.Set(OPTVIntParam.CP_THREADS, 12)

    # Add variables to the model
    b1 = model.NewBoolVar("b1")
    b2 = model.NewBoolVar("b2")
    b3 = model.NewBoolVar("b3")
    b4 = model.NewBoolVar("b4")
    x1 = model.NewIntVar(2, 5, "x1")
    # domain: [1, 2], [5, 7], [15, 15]
```

(下页继续)

(续上页)

```
    x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
    c1 = model.NewConstant(11)

    # Add constraint to the model
    lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3 <= 15)
    # domain: [3, 4], [6, 10], [12, 14]
    domain = [3, 4, 6, 10, 12, 14]
    lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, domain)

    boolOr = model.AddBoolOr([b1, b2, b4])
    boolAnd = model.AddBoolAnd([~b1, b3])
    boolALO = model.AddAtLeastOne([b2, b3])
    boolAMO = model.AddAtMostOne([b2, b4])
    boolEO = model.AddExactlyOne([b2, b4])
    boolImpl = model.AddImplication([b1, b4])

    boolOr.OnlyEnforceIf(b3, b4)

    model.Write("cp.fzn")
    solver = OPTVCPSolver()
    status = solver.Solve(model)

    if status == OPTV_CP_UNKNOWN:
        print("Model is unknown.")
    elif status == OPTV_CP_MODEL_INVALID:
        print("Model is invalid.")
    elif status == OPTV_CP_FEASIBLE:
        print("Model is feasible.")
    elif status == OPTV_CP_INFEASIBLE:
        print("Model is infeasible.")
    elif status == OPTV_CP_OPTIMAL:
        objVal = solver.GetObjectiveValue()
        print("Optimal solution found. Best objective value: ", objVal)
        vars = [b1, b2, b3, b4, x1, x2]
        result = solver.GetValues(vars)
        print("Optimal solution found. Best solution: ")
        for i in range(len(result)):
            print(vars[i].GetName(), " ", result[i])
    elif status == OPTV_CP_USER_INTERRUPT:
        print("Solver was interrupted by the user.")
```

引入模块

要使用 Python 接口，首先要安装并引入模块 `optvpy`.

```
from optvpy import *
```

创建环境

第一步需要创建一个空的环境对象。

```
# Create an environment
env = OPTVEnv()
```

模型创建前的参数配置

若需要在调用*OPTVCPModel* 构造函数 **前**修改求解器参数，用户必须通过环境对象设置这些参数。在本例中，我们展示了在创建模型对象前如何通过环境对象设置求解器的时限参数。

```
env.Set(OPTVDblParam.CP_TIME_LIMIT, 7200)
```

模型创建后的参数配置

若需要在调用*OPTVCPModel* 构造函数 **后**修改求解器参数，用户则必须通过模型对象设置这些参数。在本例中，我们展示了如何通过模型对象设置线程数量。

```
model.Set(OPTVIntParam.CP_THREADS, 12)
```

创建模型

创建了环境后，下一步就是创建模型。每个模型是一个优化问题，它由一组变量，一组约束和关联属性 (变量边界、目标系数、约束右端项等) 组成。

创建空模型

如果用户需要通过 API 一步一步构造模型，首先需要做的是创建一个空模型。

```
# Create an empty model
model = OPTVCPModel(env)
```

从模型文件加载

如果把 `.mps`[1] 、`.lp`[2] 或 `.fzn`[3] 文件路径当做第二个参数传入，该文件则被读取，相应的模型可以通过 API 求解或修改。

```
env = OPTVEnv()
model = OPTVCPModel(env, fileName)
```

[1] 有关 MPS 格式的说明，请参见文件格式。
[2] 有关 LP 格式的说明，请参见文件格式。
[3] 有关 FZN 格式的说明，请参见文件格式。

向模型中添加变量

如果上一步创建了一个空模型，那么用户 **必须**向该模型中添加变量和约束后，才可以进行求解。同样，也可以通过添加变量或约束来扩展从文件加载的模型。

可使用以下方法向模型中添加变量：

- *OPTVCPModel.NewIntVar()*
- *OPTVCPModel.NewIntVarFromDomain()*
- *OPTVCPModel.NewBoolVar()*
- *OPTVCPModel.NewConstant()*

```
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")
x1 = model.NewIntVar(2, 5, "x1")
# domain: [1, 2], [5, 7], [15, 15]
x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
c1 = model.NewConstant(11)
```

向模型中添加约束

变量添加完成后，用户可以继续添加约束。约束规划求解器支持以下类型的约束：

- *Linear*
- *Boolean Or*
- *Boolean And*
- *Boolean Xor*
- *Implication*
- *At Least One*
- *At Most One*
- *Exactly One*
- *Allowed Assignments*;
- *Forbidden Assignments*;
- *Interval*
- *No Overlap*
- *Cumulative*
- *All Different*。

```
lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3 <= 15)
# domain: [3, 4], [6, 10], [12, 14]
domain = [3, 4, 6, 10, 12, 14]
lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, domain)

boolOr = model.AddBoolOr([b1, b2, b4])
```

(下页继续)

(续上页)

```
boolAnd = model.AddBoolAnd([~b1, b3])
boolALO = model.AddAtLeastOne([b2, b3])
boolAMO = model.AddAtMostOne([b2, b4])
boolEO = model.AddExactlyOne([b2, b4])
boolImpl = model.AddImplication([b1, b4])

boolOr.OnlyEnforceIf(b3, b4)
```

求解模型

模型构建完成并设置好参数后，可以使用*OPTVCPSolver.Solve()* 求解模型。求解完成后，求解状态与求解结果将保存在*OPTVCPSolver* 对象中。

```
solver = OPTVCPSolver()
status = solver.Solve(model)
```

查询求解结果

求解完成后，可以通过*OPTVCPSolver* 访问以求解模型中的相关数据。函数*OPTVCPSolver.Solve()* 在求解完模型后会返回求解状态，用于判断是否找到可行解。使用*OPTVCPSolver.GetObjectiveValue()* 可以获取当前找到的最优目标值。而*OPTVCPSolver.GetSolution()* 、*OPTVCPSolver.GetValue()*、*OPTVCPSolver.GetValues()* 等方法可以获取变量的取值。

```
if status == OPTV_CP_UNKNOWN:
    print("Model is unknown.")
elif status == OPTV_CP_MODEL_INVALID:
    print("Model is invalid.")
elif status == OPTV_CP_FEASIBLE:
    print("Model is feasible.")
elif status == OPTV_CP_INFEASIBLE:
    print("Model is infeasible.")
elif status == OPTV_CP_OPTIMAL:
    objVal = solver.GetObjectiveValue()
    print("Optimal solution found. Best objective value: ", objVal)
    vars = [b1, b2, b3, b4, x1, x2]
    result = solver.GetValues(vars)
    print("Optimal solution found. Best solution: ")
    for i in range(len(result)):
        print(vars[i].GetName(), " ", result[i])
elif status == OPTV_CP_USER_INTERRUPT:
    print("Solver was interrupted by the user.")
```

运行示例

数学规划求解器的软件包中已经包含了本节介绍的样例程序，路径在 `${OPTV_HOME}/examples/python`。只需进入 `${OPTV_HOME}/examples` 路径并执行。

```
python3 cp.py
```

CHAPTER 5

文件格式

本章描述了天筹求解器支持的各种文件格式。

天筹支持以下类型的模型文件：

- `.mps`
- `.qps`[1]
- `.lp`
- `.nl`[2]
- `.fzn`[3]

同时，天筹求解器也支持以下压缩格式（其中暂不支持压缩的 `.nl` 文件读取）：

- `.gz`
- `.z`
- `.Z`

5.1 MPS 格式文件说明

MPS 文件是运筹优化求解器常用的文件格式之一，它采用列格式书写且区分大小写; 每行数据表示一条记录，记录分为两种类型：指示符记录和数据记录，记录中包含空格，且常采用空格作为分割符。根据书写的方式不同常分为 **Fixed** 格式 (fixed format) 和 **Free** 格式 (free format), 其中 Fixed 格式字段必须书写在固定的范围列内，而 Free 格式限制更为宽松常采用空格隔离字段。固定格式的行列名常为 8 个字符，空格也算作字符，变量名中可以存在空格；自由格式则无字符数量限制，但变量名最好不要超过 255 个字符且不能包含空格；指示符记录用来标识数据段的, 每条指示符记录从第一列开始书写，常见指示符类型包括五个必选项和四个可选项。

[1] `.mps` 的别名，有时候用来表征 QP，QCP，QCQP 问题。

[2] 一种可以用来表征线性及非线性模型的格式，最初由常用的建模语言 AMPL 提出。这种格式包含了通用数学模型的完整代数表达式，从而比 `.mps` 或 `.lp` 读写更快、更准确。请从 AMPL MP library 查看更多细节。

[3] 目前只支持约束规划问题的 `.fzn` 文件。

指示符记录用于标识数据段。每个指示符记录都从第一列开始编写。共有五种必需指示符类型和四种可选指示符类型。

是否必须	用途	作用
NAME	是	用于指定问题名称；不同于其他指示符记录，名称记录包含数据。
OBJSENSE	否	选用, 制定模型优化方向为最小化（默认）或最大化。
ROWS	是	用于指定每个约束的名称和含义。
COLUMNS	是	用于指定分配给每个变量（列）的名称和对应于该变量的非零约束系数。
RHS	是	用于指定右侧向量的名称和每个约束（行）的值。
RANGES	否	用于指定限制为处于两个值之间的区间内的约束；还会指定区间端点。
BOUNDS	否	用于指定每个变量（列）必须保持的限制。如果省略，则默认边界为 $[0, \infty]$。
QUADOBJ/QMATRI	否	选用, 指定目标函数中的二次项。
QCMATRIX	否	选用, 指定二次约束函数中的二次项。
ENDATA	是	用于表示数据结束；始终为 MPS 文件中的最后一个条目。

5.1.1 MPS 文件样例

下面混合整数问题的例子来解释每个指示符和数据记录的含义与使用方式。

所描述模型的数学公式如下所示

$$\min \ x_1 + 2x_5 - x_8$$

满足条件:

$$\begin{aligned} 2.5 \leq 3x_1 + x_2 - 2x_4 - x_5 - x_8, \\ 2x_2 + 1.1x_3 \leq 2.1, \\ x_3 + x_6 = 4.0, \\ 1.8 \leq 2.8x_4 - 1.2x_7 \leq 5.0, \\ 3.0 \leq 5.6x_1 + x_5 + 1.9x_8 \leq 15.0, \end{aligned}$$

这里

$$\begin{aligned} 2.5 \leq x_1 \\ 0 \leq x_2 \quad \leq 4.1 \\ 0 \leq x_3 \\ 0 \leq x_4 \\ 0.5 \leq x_5 \quad \leq 4.0 \\ 0 \leq x_6 \\ 0 \leq x_7 \\ 0 \leq x_8 \quad \leq 4.3 \end{aligned}$$

这里,

$$\begin{aligned} x_1, x_2 \in \mathbb{R} \\ x_3, x_4 \in \{0, 1\}. \end{aligned}$$

对应 MPS 文件

```
NAME          EXAMPLE
ROWS
 N  OBJ
 G  ROW01
 L  ROW02
 E  ROW03
 G  ROW04
 L  ROW05
COLUMNS
    COL01     OBJ                1.0
    COL01     ROW01              3.0   ROW05              5.6
    COL02     ROW01              1.0   ROW02              2.0
*
*  Mark COL03 and COL04 as integer variables.
*
    INT1      'MARKER'                 'INTORG'
    COL03     ROW02              1.1   ROW03              1.0
    COL04     ROW01             -2.0   ROW04              2.8
    INT1END   'MARKER'                 'INTEND'
*
    COL05     OBJ                2.0
    COL05     ROW01             -1.0   ROW05              1.0
    COL06     ROW03              1.0
    COL07     ROW04             -1.2
    COL08     OBJ               -1.0
    COL08     ROW01             -1.0   ROW05              1.9
RHS
    RHS1      ROW01              2.5
    RHS1      ROW02              2.1
    RHS1      ROW03              4.0
    RHS1      ROW04              1.8
    RHS1      ROW05             15.0
RANGES
    RNG1      ROW04              3.2
    RNG1      ROW05             12.0
BOUNDS
 LO BND1      COL01              2.5
 UP BND1      COL02              4.1
 LO BND1      COL05              0.5
 UP BND1      COL05              4.0
 UP BND1      COL08              4.3
ENDATA
```

注解

以星号 (*) 开头的行为注释行。

5.1.2 数据记录格式

分为 Fixed 格式和 Free 格式, Fixed 格式对参数出现的位置要求极其严格。

Fixed 格式

对于 Fixed 格式如果定义字符偏移值 Inumchar 等于 8，则数据记录对应的位置为：

记录	字段 1	字段 2	字段 3	字段 4	字段 5	字段 6
内容	2-3	5-12	15-22	25-36	40-47	50-61

如果字符偏移量不等于 8，则数据记录对应的位置也会改变：

参数记录	开始位置	结束位置
字段 1	2	3
字段 2	5	5 + Inumchar - 1
字段 3	字段 2 后的第三列	开始位置 +Inumchar-1
字段 4	字段 3 后的第三列	开始位置 +11
字段 5	字段 4 后的第四列	开始位置 +Inumchar-1
字段 6	字段 5 后的第五列	开始位置 +11

注解

1. 字段 2、3、5 字符段，其中的字符串中不应该带有空格，如果有空格也会被移除。
2. 字段 4、6 是数据段，其数值必须能在 12 个字符内完成书写，这六个字段外的列必须是空格否则无法识别。

Free 格式

对于 Free 格式其基本形式也采用 6 字段的方式书写，其形式略有不同：

1. Free 格式则无需固定位置，可以在除第二列外的任何位置，不同字段通过空格分割。
2. 但是字段对应的顺序必须和 Fixed 格式相同，超过 6 个字段的记录仅读取前 6 个字段后续不再处理。
3. 如不是从第一列开始且采用 `$$` 的字段，会被当成注释，该行记录后续部分不会被处理。
4. 第 72-80 列自动忽略。

5.1.3 NAME 节

MPS 文件第一个指示符记录是问题名字段，表示模型的名字，其对应的对应字符段为：

字段 1	字段 2	字段 3	字段 4
NAME	空格	模型名字	BINARY、FREE 或者空格

对于 Fixed 格式 NAME 必须书写在 1-4 列, modelName 必须书写在列范围 $[5+\text{Inumchar}+2, 5+2\cdot\text{Inumchar}+1]$ 内，字段 4 必须书写在列范围 $[5+2\cdot\text{Inumchar}+4, 5+2\cdot\text{Inumchar}+15]$ 内。

关于字段 3 的 modelName，对于 Fixed 格式是可选项，而对于 Free 格式则是必选的。如果文件是二进制格式，Fixed 格式则要求指定字段 4 为 `BINARY`，否则留空即可。

5.1.4 OBJSENSE 节

该指示符非标准 MPS 格式具有，常作为扩展格式使用。是可选指示符记录，Fixed 格式不能包含该节。若未显式指定，则默认此模型采用最小化优化方向；OptVerse 代码中可以使用以下两个节来扩展 MPS 标准：OBJSENSE 和 OBJNAME，两者可在 NAME 节之后指定。OBJSENSE 用于设置目标函数含义，而 OBJNAME 用于从文件内的可用行中选择目标函数。如果这些节均未出现在 MPS 文件中，那么默认问题为最小化问题，并且目标函数由 ROWS 节中遇到的第一个可用行决定。如果使用这些选项，那么它们必须按顺序出现 NAME 节之后作为第一个和第二个节。OBJSENSE 的可选值是 `MAX` 或 `MIN`.

例如：

```
NAME          example.mps
OBJSENSE
  MAX
OBJNAME
  rowname
```

5.1.5 ROWS 节

标题字段 ROWS 表征该参数段的开始位置，自此需要从第一列开始书写；参数记录按行书写，每行包含名称和约束含义，每条记录占一行，格式为：

字段 1	字段 2
约束指示符	约束名字

字段 1 必须在 2-3 列，字段 1 与字段 2 之间至少需要一个空格。约束指示符表示约束的类型，包含了一个用于指定该行的含义的单个字母。可接受的值为：

1. E 表示该行等于
2. L 表示该行小于等于
3. G 表示该行大于等于
4. N 表示自由行

字段 2 包含字符标识（最大长度为 255 个字符），字段 3-6 不可在 ROWS 节中留白。MPS 文件将目标函数也当成一条约束，以标志符类型 `N` 包含在 ROWS 节中。如果定义了多个自由行，则仅将第一个作为目标函数，其他自由行会被丢弃。

多目标优化

通过标志符 N 来定义多个目标函数。为确保这些目标函数被正确解析，需在同一行中使用空格分隔的方式提供以下参数：

1. 优先级
2. 权重（可以取负数）
3. 绝对容差
4. 相对容差

在天筹求解器中进行多目标优化时，所有的目标函数都要是线性的。

下面的示例定义了两个目标函数，其中第一个目标函数的优先级为 2，两个目标函数的权重均为 1。

```
N  OBJ0 2 1 0 0
N  OBJ1 1 1 0 0
```

5.1.6 COLUMNS 节

COLUMNS 用来定义各列（变量）名字。在 COLUMNS 节中，约束矩阵的所有列均使用其名称和所有非零元素进行指定。数据记录格式为：

字段 1	字段 2	字段 3	字段 4	字段 5	字段 6
空白	列的名字	约束 1 的名字	约束 1 中该列的系数（非零）	约束 2 的名字	约束 2 中该列的系数（非零）

如果字段 2 为空白，则说明该记录和上一行的记录具有相同的列名。字段 3 是约束或者目标函数中的非零系数; 字段 5、6 和字段 3、4 的含义相同，是可选项。记录中的 ROWS 约束名称无顺序要求。

若出现‘MARKER’‘INTORG’及‘MARKER’‘INTEND’，则分别表示整数变量的起止，介于这些关键字之间的列都是整数变量。

5.1.7 RHS 节

在 RHS 节中，指定了约束的非零右侧值。数据记录格式为：

字段 1	字段 2	字段 3	字段 4	字段 5	字段 6
空白	RHS 的名字	约束 1 的名字	约束 1 的 RHS 非零值	约束 2 的名字	约束 2 的 RHS 非零值

这里：

1. 第一列要留空。
2. 第二列表示 RHS 名字，可以任取，若为空则和上一条记录名字相同。
3. 每个 RHS 必须具有唯一的名称。
4. 第四列表示所对应的约束的 RHS 值。
5. 第五列、第六列分别与第三列、第四列的含义相同，为可选字段。

此外，还可以在 MPS 格式的文件的 RHS 节中声明模型的目标偏移量。使用此功能时，将偏移量的负值声明为目标函数的 RHS. 例如，以下行声明目标偏移量为 3.1415：

```
rhs       obj      -3.1415
```

5.1.8 RANGES 节

RANGES 节包含了约束右侧值（RHS）的范围。数据记录格式为：

字段 1	字段 2	字段 3	字段 4	字段 5	字段 6
空白	右侧范围向量标识	约束 1 的名字	约束 1 标识的范围值	约束 2 的名字（可选）	约束 2 标识的范围值（可选）

指定右侧范围的效果取决于指定行的含义以及该范围具有正系数还是负系数。对于给定的行，*rhs* 为右侧值，而 *range* 为对应的范围值。

范围值的解释如下：

行类型	范围值符号	生成的 RHS 上限	生成的 RHS 下限
G	+ 或 -	rhs + |range|	rhs
L	+ 或 -	rhs	rhs - |range|
E	+	rhs + range	rhs
E	-	rhs	rhs + range

5.1.9 BOUNDS 节

变量的界限值可在 BOUNDS 节中指定。数据记录格式为：

字段 1	字段 2	字段 3	字段 4	字段 5	字段 6
边界类型	边界名称	列的名字	列的边界值	空白	空白

注解

1. 字段 1: 界限类型，从左侧第二列开始书写。
2. OPTV 支持界限类型值为：
   - LO: 下限
   - LI: 下限，整数
   - UP: 上限
   - UI: 上限，整数
   - FX: 固定值（上下限相同）
   - FR: 自由变量（下限为 -∞ 且上限为 +∞）
   - MI: 负无穷大（下限为 -∞）

- PL: 正无穷大（上限为 +∞）
- BV: 二进制变量，整数（0 或 1）

3. 天筹求解器暂不支持界类型值为 - SC: 半连续变量的上限
4. 字段 5 和 6 不可用在 BOUNDS 节中。
5. 如果界类型不是 LO, UP, FX, 或者 UI 则忽略字段 4 的取值。
6. 界中出现的变量的顺序无需和 COLUMNS 中相同。

5.1.10 QUADOBJ/QMATRIX 节

QUADOBJ 节定义了目标函数中的二次型，是可选的。如果未提供该节，则问题被当作（混合整数）线性规划问题处理。

字段 1	字段 2	字段 3	字段 4	字段 5	字段 6
空白	变量 1 名字	变量 2 名字	系数值	空白	空白

该节中的每行表示了 Q 矩阵的上三角部分的一个非零值，每个值隐含了 1/2 系数。

例如：

```
QUADOBJ
    X1        X1        2.0
    X1        X2        4.0
    X2        X2        6.0
```

描述的 Q 如下：

$$Q = \frac{1}{2}\begin{bmatrix} 2 & 4 \\ 4 & 6 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}$$

QMATRIX 节是 QUADOBJ 节的替代，不像 QUADOBJ 节只描述上三角部分，它描述了完整的矩阵，例如，上述矩阵可以在 QMATRIX 节中描述为：

```
QMATRIX
    X1        X1        2.0
    X1        X2        4.0
    X2        X1        4.0
    X2        X2        6.0
```

5.1.11 QCMATRIX 节

QCMATRIX 节扩展了 MPS 的功能，使其可以兼容二次约束问题。具体地，每个含有二次项的约束，需要单独的 QCMATRIX 节，约束名需要在 QCMATRIX 行提供，矩阵格式类似于 QMATRIX，需要完整的矩阵。不过，这里不再有除以 2 的默认操作。

```
QCMATRIX c1
    X1        X1        2.0
    X1        X2        4.0
    X2        X1        4.0
```

(下页继续)

(续上页)

```
    X2        X2        6.0

QCMATRIX c3
    X1        X1        1.0
```

5.2 LP 格式文件说明

LP 格式作为一种在学界与业界都十分常用的文件格式，具有比 MPS 格式更易阅读的优点。

与 MPS 文件是列格式不同，LP 文件采用行表达式描述问题，且没有固定的书写范围和长度。但从处理性能和阅读难易的角度出发，各家求解器都规范了每行的最字节长度。天筹求解器以 16 KB 的 buffer 进行读取。虽然超过此规格的行仍然可以正确读取，但是建议将每行的规格控制在 16 KB 以内。下面我们给出一个标准的 LP 文件：

```
\ENCODING-ISO-8859-1

\Problem name:Maximize
Maximize
 obj: x1 + 2 x2 + 3 x3 + x4
Subject To
 r1: - x1 + x2 + x3 + 10 x4 <= 20
 r2: x1 - 3 x2 + x3 <= 30
 r3: x2 - 3.5 x4 = 0
Bounds
 0 <= x1 <= 40
 2 <= x4 <= 3
Generals
 x4
Binaries
 x3
End
```

`Problem`, `Maximize`, `Subject To`, `Bounds`, `Generals`, `Binaries`, `General Constraints`, 和 `End` 分别是各节的指示词。它们出现的顺序需要以本章节接下来的介绍为准，否则会影响读取效果。各节指示词必须是单独一行书写，且对大小写不敏感。

下面我们对文件的各部分给出详细的描述。

5.2.1 LP 格式 BNF 规范

本章节介绍天篝求解器 Backus-Naur 形式规范。

```
⟨LP File⟩ ::=[⟨Model Name⟩]
              ⟨Objective Section⟩
              ⟨Constraint Section⟩
              [⟨Bounds Section⟩]
              [⟨Var Type Section⟩]
              [⟨SOS Section⟩]
              [⟨GenCon Section⟩]
              "END"
⟨Model Name⟩ ::=("PROBLEM" |"PROB" )⟨IDENT⟩
⟨Objective Section⟩ ::=⟨Objective Sense⟩([⟨Objective⟩]|⟨Multi Objective⟩+)
⟨Objective Sense⟩ ::=⟨Min Sense⟩|⟨Max Sense⟩
⟨Min Sense⟩ ::=("MINIMIZE" |"MINIMUM" |"MIN" )
⟨Max Sense⟩ ::=("MAXIMIZE" |"MAXIMUM" |"MAX" )
⟨Objective⟩ ::=[⟨IDENT⟩':' ][⟨Linear Term⟩|⟨Quadratic Region⟩]+
⟨Multi Objective⟩ ::=[⟨IDENT⟩':' ]⟨Multi Params⟩[⟨Linear Objective⟩]
⟨Constraint Section⟩ ::=("SUBJECT TO" |"SUBJECT" |"ST" )[⟨Constraint⟩|⟨Indicator Constraint⟩]+
⟨Bounds Section⟩ ::=("BOUNDS" |"BOUND" )[⟨Bound⟩]+
⟨Var Type Section⟩ ::=(⟨Binary Section⟩|⟨General Section⟩)+
⟨Binary Section⟩ ::=("BINARIES" |"BINARY" |"BIN" )[⟨IDENT⟩]+
⟨General Section⟩ ::=("GENERALS" |"GENERAL" |"GEN" )[⟨IDENT⟩]+
⟨Constraint⟩ ::=[⟨IDENT⟩':' ][⟨Linear Term⟩|⟨Quadratic Region⟩] + ⟨SENSE⟩⟨Signed Number⟩
⟨Linear Term⟩ ::=['+' |'-' ] + [⟨Signed Number⟩]⟨IDENT⟩
⟨Quadratic Region⟩ ::='[' ⟨Quadratic Term⟩ + ']'
⟨Quadratic Term⟩ ::=['+' |'-' ] + [⟨Signed Number⟩](⟨IDENT⟩"^ 2" |⟨IDENT⟩'*' ⟨IDENT⟩)
⟨SOS Section⟩ ::=("SOS" )⟨SOS Constraint⟩+
⟨SOS Constraint⟩ ::=[⟨IDENT⟩':' ]'S' ['1' |'2' ]"::" ⟨SOS Weight⟩ + \n
⟨SOS Weight⟩ ::=⟨IDENT⟩':' ⟨Signed Number⟩
⟨GenCon Section⟩ ::=⟨GenCon Indicator⟩[⟨General Constraint⟩]+
⟨GenCon Indicator⟩ ::=("G.C." |"GENCON" |"GENERAL CONSTRAINT" |"GENERAL CONSTRAINTS" )
⟨General Constraint⟩ ::=[⟨IDENT⟩':' ]⟨IDENT⟩'=' ⟨GENCON TYPE⟩⟨GenCon Var List⟩
⟨GenCon Var List⟩ ::='(' ⟨IDENT⟩[',' ⟨IDENT⟩] + ')'
⟨Multi Params⟩ ::="Priority=" ⟨DIGIT⟩ + "Weight=" ['+' |'-' ]⟨NUMBER⟩"AbsTol=" ⟨NUMBER⟩"RelTol=" ⟨NUMBER⟩
⟨Indicator Constraint⟩ ::=[⟨IDENT⟩':' ]⟨IDENT⟩'=' ('0' |'1' )"->" ⟨Constraint⟩
```

⟨Bound⟩ ::=⟨Num or Inf⟩⟨SENSE⟩⟨IDENT⟩⟨SENSE⟩⟨Num or Inf⟩
|⟨Num or Inf⟩⟨SENSE⟩⟨IDENT⟩
|⟨IDENT⟩⟨SENSE⟩⟨Num or Inf⟩
|⟨IDENT⟩"FREE"
⟨SENSE⟩ ::='<' ['=']
|'>' ['=']
|'=' ['=']
⟨Num or Inf⟩ ::=⟨NUMBER⟩
|['+' |'-']("INFINITY" |"INF")
⟨Signed Number⟩ ::=['+' |'-'] + ⟨NUMBER⟩

表 1: BNF 符号定义

符号	描述
⟨A⟩	非终结符
⟨A⟩ := ...	替代 ⟨A⟩ 为 '...'
[⟨A⟩]	⟨A⟩, 可选
'A'	终结符
"A"	终结字符串
(⟨A⟩)	⟨A⟩
(⟨A⟩|⟨B⟩)	⟨A⟩ 或 ⟨B⟩
[⟨A⟩|⟨B⟩]	⟨A⟩ 或 ⟨B⟩, 可选
⟨A⟩+	一个或多个 ⟨A⟩
[⟨A⟩]+	一个或多个 ⟨A⟩, 可选 (基本上 *)

5.2.2 LP 格式的注释

反斜杠 (\) 后面的内容被认定为注释，求解器不对其进行解析，而是将其忽略；对于文件中的空白行同样进行忽略。例如：

```
\ENCODING=ISO-8859-1

\Problem name:Maximize
```

5.2.3 LP 格式的关键字

LP 格式中的关键字可选用以下三种格式之一：

1. 全部大写 (如 `PROBLEM`)
2. 全部小写 (如 `problem`)
3. 首字母大写 (如 `Problem`)

关键词 (指示词)	可选项	含义
problem	prob	问题名称
minimize	min, minimum	最小化问题
maximize	max, maximum	最大化问题
subject to	st, s.t., such that	约束满足
bounds	bound	表达式的界
binaries	bin, binary	0-1 变量
generals	gen, general, int, integer, integers	整数变量
sos		SOS 约束
general constraints	gencon, gencons, general constraint, g.c.	一般约束
and		一般函数 AND
or		一般函数 OR
inf	infinity	无穷大
free		自由变量
end		结束

问题最小 (大) 化关键词可以可选地作为格式的第一行有效字段，如果省略则默认为最小化问题；end 关键词标注问题描述的结束，end 之后即便还存在字段也不会被解析。

5.2.4 LP 格式的节

在介绍节之前，我们首先介绍两个概念：**令牌**和 **表达式**。令牌是单次能够处理的最小单位，不可通过插入任意字符分割，如空格或换行符。我们将数值、字符串、关键词、操作符、关系符视为令牌。表达式是加入空格进行组合的令牌组，我们仅支持采用空格作为令牌组的令牌分隔符。如表达式 10 x1 + x2 + 2000 中，10 和 2000 是数值令牌，变量名 x1 和 x2 是字符串令牌，令牌之间采用空格分隔；10,x1,+,x2,+,2000 则不是令牌组或所谓的表达式，而仅仅是个字符串。

字符串是由零个或多个字符组成的有限序列，可以包含特殊字符，但不可以以这些符号 *, +, < >, =, (,), ^, :, /, ,, 或 .，以及数字开头，也不可以作为关键词)。

表达式是加入空格进行组合的令牌组，我们仅支持采用空格作为令牌组的令牌分隔符。如表达式 10 x1 + x2 + 2000 中，10 和 2000 是数值令牌，变量名 x1 和 x2 是字符串令牌，令牌之间采用空格分隔；10,x1,+,x2,+,2000 则不是令牌组或所谓的表达式，而仅仅是个字符串（字符串是由零个或多个字符组成的有限序列，可以包含特殊字符，但不可以以这些符号 *, +, < >, =, (,), ^, :, /, ,, 或 .，以及数字开头，也不可以作为关键词)。同时，它也不是变量名，因为变量名、约束名、目标函数名需要满足如下条件：彼此不同，名字长度不超过 255 个字符，不能以数字或者操作符 +, −, *, ^ 及关系符 <, >, = 开头，此外，也不可以是所谓的关键词。变量名最好也不要采用 E + 数字形式，如 E9 或 e9，因为这样会被解析为数字。建议使用字符和数字组合，如 var001.

Problem Name 节 [可选]

用户可以在此节中为模型指定名称，若缺省，求解器会以 optv 作为临时的模型名称。

目标函数

目标函数可采用 maximize 或 minimize 关键词开头，形式为：

```
minimize
-x1 + x2 + -x3 + 0.5 x1 + 100 + x4 + 20
```

如上形式可以看出：

1. *minimize* 单独一行。
2. 变量可以直接与负号 (-) 连接，如 x1 和 x3.
3. 变量可以多次出现，我们聚合多次系数，本例中为 x1 的系数最终为 -1 + 0.5.
4. 偏移量可以出现在表达式中间或结尾，如 100 和 20. 如果出现多个偏移量，它们会被自动加和。

在实际使用中目标函数可以提供一个名字，并且可以多行书写，但需注意一些细节，如：

```
minimize
obj: -x1 + x2 + -x3 + 0.5 x1
+ 100 + x4 + x5 + 1.5 + 2.0
x6
```

目标函数名后需要加冒号 (:) 作为分割；换行时以令牌为最小的组成单位。

多目标优化

天筹求解器也支持求解多目标优化问题。多目标优化问题的模型必须使用线性目标函数（目标函数中不能包含二次项），但可以设置一个空的目标函数。一个多目标优化问题模型中的每个目标都必需有一个表头，该表头需包含以下信息：

1. [可选] 目标名称，名称后使用冒号表示名称结束
2. 优先级
3. 权重（可以取负数）
4. 绝对容差
5. 相对容差

下面这个示例展示了各个参数的格式以及如何指定多个目标：

```
obj1: Priority=2 Weight=1.0 AbsTol=1.0e-6 RelTol=0.0
 x0 + x1 + 2
obj2: Priority=1 Weight=1.0 AbsTol=1.0e-6 RelTol=0.0
 3 x0 + 2 x1 - 3
```

Quadratic Objectives 节

目标函数可以含有二次项，该部分需放入方括弧 `[ <quad terms> ]` 并由 `/ 2` 随后。这里，空格符是可选的。注意，若未读取到 `/ 2`，相应的数据将被转换并以警告形式通知，否则用户需要在相应的 `.lp` 文件中将数值乘以 2 以避免该转换和警告；另一方面，除以 2 以外的数值会导致解析错误。

一般来说，二次项有以下两种写法：

1. `x^2` 或 `x ^ 2`
2. `x*x` 或 `x * x`

这两种方式都可以被正确解析和读取，`*` 和 `^` 操作符左右的空格符都是可选的。其次，混合项由 `*` 标识，对角元可以使用 `*` 或 `^`. 如果二次项未被提供系数，那么默认值 `1.0` 会被用来当做其系数。

另外，建议将表达式中二次项按照下标由低到高排序以提高效率，例如：

```
minimize
obj: -2.0 x1 +3.0 x2 -1.0 x3 + [ x1^2 + x2^2 + x1 * x2 + x3^2 + x1 * x3 + x2 * x3 ] /␣
↪2
```

重新排列上述示例中目标函数表达式的二次项部分为：

```
minimize
obj: -2.0 x1 +3.0 x2 -1.0 x3 + [ x1^2 + x1 * x2 + x1 * x3 + x2^2 + x2 * x3 + x3^2 ] /␣
↪2
```

约束章节

约束满足节的关键词 `subject to` 同样需要单独一行书写。每个新的约束必须另起一行。约束可以提供约束名，在约束名之后跟冒号，约束名必须遵循与变量名称相同的规则。若约束名缺省，求解器会自动为约束增加临时名称，`R_#`，这里 `#` 是当前的约束总计数。

约束被定义为通过变量、常数、操作符组成的线性表达式，后跟关系符和数字右侧系数。使用以下关系符之一可以直观地指示约束的意义：`>=`, `<=`, 或 `=`. 例如，这是一个命名约束：

```
subject to:
depts01: -x1 + 0.5 x2 <= 40
```

但不允许关系符两边都存在表达式，如：

```
subject to:
depts02: -x1 + 0.5 x2 <= 40 + x1
```

和双侧关系符，如：

```
subject to:
depts03: 0 <= -x1 + 0.5 x2 <= 40 + x1 <= 40
```

Quadratic Constraints 节

与目标函数表达式相同，约束表达式也可以多行书写，但需要保持一致性。

约束表达式也可以含有二次项，该部分需放入方括弧 `[ <quad terms> ]`，但与目标函数中的二次项不同，这里 **没有** `/ 2` 随后。以下两种二次项的写法都被支持：

1. `x^2` 或 `x ^ 2`
2. `x*x` 或 `x * x`

这两种方式都可以被正确解析和读取，`*` 和 `^` 操作符左右的空格符都是可选的。其次，混合项由 `*` 标识，对角元可以使用 `*` 或 `^`. 如果二次项未被提供系数，那么默认值 `1.0` 会被用来当做其系数。

另外，建议将表达式中二次项按照下标由低到高排序以提高效率，例如：

```
subject to:
c1: -2.0 x1 +3.0 x2 -1.0 x3 + [ x1^2 + x2^2 + x1 * x2 + x3^2 + x1 * x3 + x2 * x3 ]
```

重新排列上述示例中目标函数表达式的二次项部分为：

```
subject to:
c1: -2.0 x1 +3.0 x2 -1.0 x3 + [ x1^2 + x1 * x2 + x1 * x3 + x2^2 + x2 * x3 + x3^2 ]
```

Indicator Constraints 节

指示约束（Indicator Constraint）是天筹求解器中用于表达逻辑逻辑关系的条件约束，用于刻画如果某个 0-1 变量取特定值，则某条线性约束必须被满足的逻辑关系。指示约束允许在满足某些条件的情况下，禁用或者激活某一线性约束。指示约束的格式为：约束名称（可选），随后依次是 0-1 变量、等号、变量的值（0 或 1）、箭头”->”，最后是未标记的线性约束。

例如：

```
ind0: x = 0 -> y + z <= 20
```

Bounds 节 [可选]

`bounds` 节是可选的，如果存在则表明需要对变量使用提供的变量界。根据界被提供的个数，每一行表达式可以分为单边界或双边界。单边界形如 `x >= 10` 或 `x < 10`，双边界形如 `10 <= x <= 15`. 这两种界都不允许多行书写。如果没有提供变量界，则默认采用求解器的内部值。如果为变量定义了单个边界，则将使用适当的默认边界作为第二个边界。

这里，有几条准则：

1. 固定边界表达式不被允许，即不能写为 `x1 = 20` 而要统一写为 `20 <= x1 <= 20`.
2. 支持 `+(-)inf`，如 `x1 > -inf`.
3. 支持 `free`, 如 `x1 free`.

Generals 和 Binaries 节 [可选]

LP 文件的 `generals` 和 `binaries` 节用于指示在可行解中必须具有整数值的变量。鉴于两者之间的相似性，我们将其放在同一个部分介绍，实际上他们是否同时存在互不影响。注册在这两节的变量将具有的默认边界的定义。对于在 `generals` 部分注册的变量，默认范围是 $[0, 10^{20}]$，而 `binaries` 部分注册的变量，默认边界是 0 和 1. 变量注册的方式是通过将变量罗列到对应节，如：

```
Generals
x4 x5 x6 x7
x8 x9

Binaries
x1 x2 x3 x4
```

可见，不同变量之间通过空格分割，允许多行书写, 同时我们允许变量注册到不同节，如变量 `x4`，最终的变量类型为其可行区间最小的类型。

变量赋值可以采用任意顺序，也可以多次赋值（只有最后一次赋值生效）。

SOS Constraints 节 [可选]

`sos` 节支持用户为模型增加 SOS 约束。这里，每个 SOS 约束必须单独新起一行，且以新行结束。可以向 SOS 约束提供约束名（若约束名缺省，求解器会自动为约束增加临时名称，`SOS_#`，这里 `#` 是当前的 SOS 约束总计数），在约束名之后跟冒号。然后，SOS 约束名后需要跟随 SOS 类型指示符（`S1` 或 `S2` ）和两个冒号。最后，SOS 约束由变量及权重组合的列表表征和定义。

细节请参考如下 SOS 节：

```
SOS
   sos1: S1::   x : 1.0    y : 2.0
   sos2: S2::   x : 1.0    z : 2.0
```

General Constraints 节 [可选]

表 2: 天筹求解器支持的一般约束类型

约束	关键字	描述
And	`AND`	只有所有变量非零时，结果才非零。
Or	`OR`	只要有一个变量非零，结果就非零。
Max	`MAX`	返回提供的所有变量取值中的最大者。
Min	`MIN`	返回提供的所有变量取值中的最小者。

类似于 `subject to` 节，`general constraints` 节支持用户为模型增加一般约束。同样地，每个一般函数需要单独新起一行，可以提供约束名，在约束名之后跟冒号，约束名必须遵循与变量名称相同的规则。若约束名缺省，求解器会自动为约束增加临时名称，`GC_#`，这里 `#` 是当前的一般约束总计数。

.lp 格式中的一般约束需要遵循以下规范：首先，若指定约束名称，则名称后需要加 `:`；然后，需要一个变量名和等号联合表征该约束的结果；其次，在等号后，用户需指定约束函数类型，如 `AND` 或 `OR`；最后，约束函数中的参数列表需要以 `,` 分隔。

细节请参考以下示例：

```
General Constraints
    and1: r1 = AND ( x1, x2, x3 )
    and2: r2 = AND ( x3, x1 )
    or1:  r3 = OR ( x1, x2, x3 )
    or2:  r4 = OR ( x3, x1 )
    max1: r5 = MAX ( x1, x2, 10.0 )
    min1: r6 = MIN ( x1, x2, 10.0 )
```

注解

现阶段我们求解器仅支持线性问题，对于非线性字段和半连续、半整型暂不支持。

5.3 FZN 文件格式说明

FlatZinc 模型由变量声明、约束定义以及目标函数的定义（若问题为优化问题）组成。

下面，我们提供一个标准的 FZN 文件：

```
var bool: BOOL0;
var bool: BOOL1;
var bool: BOOL2;
var 0..10: INT0;
var 0..20: INT1;
var {0, 2, 5, 8}: INT2;
array [1..3] of var 1..20: x;
var 0..1000: solution;
constraint array_bool_and([BOOL1, BOOL2], true);
constraint array_bool_or([BOOL0, BOOL1], true);
constraint int_lin_ne_imp([-2, 1, 1],[INT0, INT1, INT2], 5, BOOL0);
constraint int_lin_ge([-2, 2], [x[1], x[3]], 15);
constraint int_lin_eq([1, 1, 1, -1], [x[1], x[2], x[3], solution], 0);
solve maximize solution;    % objective: maximize(x[1] + x[2] + x[3])
```

FZN 问题包含三个部分：变量、约束和目标函数。这些部分的顺序很重要，必须遵循本文档中规定的顺序。

FlatZinc 模型由多条语句组成，每条语句末尾都以分号 ; 结束。FlatZinc 中注释以 % 开头，表示该行其余部分为注释内容。

FlatZinc 使用 UTF-8 字符集。非 ASCII 字符只能出现在字符串字面量中。

FlatZinc 语法区分大小写（`foo` 和 `Foo` 是不同的名称）。标识符必须以字母（`[A-Za-z]`）开头，后接任意数量的字母、数字或下划线（`[A-Za-z0-9_]`）。此外，变量名或参数名的标识符可以以下划线开头。但谓词名称、谓词参数及注解的标识符不得以下划线开头。

以下关键字为保留字，不能用作标识符：

`annotation`、`any`、`array`、`bool`、`case`、`constraint`、`diff`、`div`、`else`、`elseif`、`endif`、`enum`、`false`、`float`、`function`、`if`、`in`、`include`、`int`、`intersect`、`let`、`list`、`maximize`、`minimize`、`mod`、`not`、`of`、`satisfy`、`subset`、`superset`、`output`、`par`、`predicate`、`record`、`set`、`solve`、`string`、`symdiff`、`test`、`then`、`true`、`tuple`、`union`、`type`、`var`、`where`、`xor`。

请注意，其中部分关键字在 FlatZinc 中并未实际使用，之所以保留，是因为它们在 Zinc 和 MiniZinc 中属于关键字。

FlatZinc 语法对空白字符不敏感。且没有预设的行长度上限。

5.3.1 FZN 格式章节

变量节

该部分允许用户为模型指定变量及变量数组。声明变量时需使用关键字 `var`，并同时指定变量的类型、取值范围（非布尔变量）以及变量名。例如，一个布尔变量可表示为：`var bool: b1;`。对于整数变量，还需明确其定义域。这可以通过区间形式指定，例如：`var 0..10: x1;`，也可以通过枚举值集合指定，例如：`var {0, 2, 3, 7, 8}: x2;`。此外，用户还可以声明数组。此时需使用关键字 array，并在方括号中给出数组元素的索引范围。例如：`array [1..3] of var bool: b;` 或 `array [1..3] of var 1..20: x;`。

约束章节

该部分允许用户为模型指定约束条件。当前天筹约束规划求解器支持以下类型的约束：

Linear:

- bool_lin_eq(array [int] of int: as, array [int] of var bool: bs, var int: c)
- bool_lin_le(array [int] of int: as, array [int] of var bool: bs, int: c)
- bool2int(var bool: a, var int: b)
- int_plus(var int: a, var int: b, var int: c)
- set_in(var int: x, var set of int: S)
- set_in_reif(var int: x, var set of int: S, var bool: r)
- bool_not(var bool: a, var bool: b)

Boolean Or:

- array_bool_or(array [int] of var bool: as, var bool: r)
- bool_clause(array [int] of var bool: as, array [int] of var bool: bs)
- bool_or(var bool: a, var bool: b, var bool: r)

Boolean And:

- array_bool_and(array [int] of var bool: as, var bool: r)
- bool_and(var bool: a, var bool: b, var bool: r)

Boolean Xor:

- array_bool_xor(array [int] of var bool: as)
- bool_xor(var bool: a, var bool: b, var bool: r)

Table:

- table_bool(array [int] of var bool: x, array [int, int] of bool: t)
- table_int(array [int] of var int: x, array [int, int] of int: t)

No Overlap:

- disjunctive_strict(array [int] of var int: s, array [int] of var int: d)

No Overlap 2D:

- fzn_diffn(array[int] of var int: x, array[int] of var int: y, array[int] of var int: dx, array[int] of var int: dy)

- fzn_diffn_nonstrict(array[int] of var int: x, array[int] of var int: y, array[int] of var int: dx, array[int] of var int: dy)

Cumulative:

- disjunctive(array [int] of var int: s, array [int] of var int: d)
- cumulative(array [int] of var int: s, array [int] of int: d, array [int] of int: r, int: b)
- var_cumulative(array [int] of var int: s, array [int] of int: d, array [int] of int: r, var int: b)

All Different:

- all_different_int(array [int] of var int: x)

Element:

- array_bool_element(var int: b, array [int] of bool: as, var bool: c)
- array_var_bool_element(var int: b, array [int] of var bool: as, var bool: c)
- array_var_int_element(var int: b, array [int] of var int: as, var int: c)
- array_int_element(var int: b, array [int] of int: as, var int: c)

Circuit:

- circuit(array[int] of var int: x, int: base)
- subcircuit(array[int] of var int: x, int: base)

Max Equality:

- int_max(var int: a, var int: b, var int: c)
- maximum_int(var int: m, array [int] of var int: x)
- array_int_maximum(var int: m, array[int] of var int: x)

Min Equality:

- int_min(var int: a, var int: b, var int: c)
- minimum_int(var int: m, array [int] of var int: x)
- array_int_minimum(var int: m, array[int] of var int: x)

Abs Equality:

- int_abs(var int: a, var int: b)

Multiplication Equality:

- int_times(var int: a, var int: b, var int: c)

Division Equality:

- int_div(var int: a, var int: b, var int: c)

Modulo Equality:

- int_mod(var int: a, var int: b, var int: c)

Inverse:

- fzn_inverse(array [int] of var int: f, array [int] of var int: invf)

Linear Constraint 支持 `_imp` 或 `_reif` 为后缀：

- bool_eq(var bool: a, var bool: b)

- bool_ge(var bool: a, var bool: b)
- bool_gt(var bool: a, var bool: b)
- bool_le(var bool: a, var bool: b)
- bool_lt(var bool: a, var bool: b)
- bool_ne(var bool: a, var bool: b)
- int_eq(var int: a, var int: b)
- int_ge(var int: a, var int: b)
- int_gt(var int: a, var int: b)
- int_le(var int: a, var int: b)
- int_lt(var int: a, var int: b)
- int_ne(var int: a, var int: b)
- int_lin_eq(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_ge(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_gt(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_le(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_lt(array [int] of int: as, array [int] of var int: bs, int: c)
- int_lin_ne(array [int] of int: as, array [int] of var int: bs, int: c)

使用后缀时，需要引入一个额外的布尔变量。例如 `bool_eq_reif(var bool: a, var bool: b, var bool: r);`。后缀 `_reif` 表示变量 `r` 等价于该约束的真值（即当约束成立时 `r` 为真，否则为假）。缀 `_imp` 则为约束添加一个控制变量，用于条件性地启用或禁用该约束。

目标函数

目标函数以 `solve` 关键字开头。根据问题类型，其后应使用相应的关键字：对于满足性问题使用 `satisfy`，对于优化问题则使用 `minimize` 或者 `maximize` 。在最小化或最大化问题中，需指定目标变量。其格式如下：

```
solve maximize INT0;
```

CHAPTER 6

属性介绍

6.1 数学规划求解器属性介绍

6.1.1 模型属性

这些是模型级别的属性，意味着它们针对整个模型，而非模型中的某个特定变量或约束。若是想要查询这些属性的值，直接在模型对象上调用 `Get()` 方法。对于允许用户修改的属性，可以使用 `Set()` 方法更改某一属性的值。

关于如何查询或修改属性，请参考使用示例 。

表 1: 数学规划求解器模型属性

名称	描述
NUM_INT_VARS	整数变量总数
NUM_VARS	变量总数
NUM_CONSTRS	约束总数
NUM_LIN_CONSTRS	线性约束总数
NUM_QUAD_CONSTRS	二次约束总数
NUM_CONES	锥约束个数
SOL_COUNT	存储的解的个数
OBJ_SENSE	优化目标方向
STATUS	求解状态
OBJ_BOUND	目标函数的最优界
OBJ_VAL	原始解的目标函数值
DUAL_OBJ_VAL	对偶目标函数值
OBJ_OFFSET	目标函数的常数项
POOL_OBJ_VAL	解池中第 N 个最优解对应的目标函数值
MIP_GAP	混合整数规划模型的相对最优性间隙
RUN_TIME	求解时间
FINGERPRINT	模型指纹
NAME	模型名称

NUM_INT_VARS

类型： int

是否可修改： 否

整数变量总数，包括 0/1 变量和其他整数变量。

NUM_VARS

类型： int

是否可修改： 否

变量总数。

NUM_CONSTRS

类型： int

是否可修改： 否

约束总数。

NUM_LIN_CONSTRS

类型： int

是否可修改： 否

线性约束总数。

NUM_QUAD_CONSTRS

类型： int

是否可修改： 否

二次约束总数。

NUM_CONES

类型： int

是否可修改： 否

模型中的锥体数。

SOL_COUNT

类型： int

是否可修改： 否

存储解的个数：对于 LP 问题，若获得最优解则取值为 1，否则取 0；对于 MIP 问题，该属性对应找到的解（包含最优和次最优）的个数。

OBJ_SENSE

类型： int

是否可修改： 是

优化目标方向，取值为-1 代表最大化问题，取值为 1 代表最小化问题。

参考*OPTVSense*。

STATUS

类型： int

是否可修改： 否

模型求解状态。

名称	取值	描述
OPTV_OPTIMAL	1	模型求解成功（最优解）
OPTV_INFEASIBLE	2	模型被证明不可行
OPTV_INF_OR_UNBD	3	模型被证明不可行或无界
OPTV_UNBOUNDED	4	模型被证明无界
OPTV_TIME_LIMIT	5	求解时间超出给定时限 TIME_LIMIT
OPTV_MEM_LIMIT	6	求解内存占用超过给定阈值 MEM_LIMIT
OPTV_INTERRUPTED	7	用户自行终止（如通过:kbd:*Ctrl*+*C*）或在 Callback 中终止
OPTV_UNKNOWN	0	未知状态

OBJ_BOUND

类型： double

是否可修改： 否

已证明的目标函数值最优界，仅适用于 MIP 求解范畴。

OBJ_VAL

类型： double

是否可修改： 否

模型的原始解对应的目标值。

DUAL_OBJ_VAL

类型： double

是否可修改： 否

模型的对偶目标值。

OBJ_OFFSET

类型： double

是否可修改： 是

目标函数中的常数项。

POOL_OBJ_VAL

类型： double

是否可修改： 否

该属性仅适用于 MIP 求解范畴，当通过*SOLUTION_NUMBER* 指定第 N 个 MIP 解时，*OBJ_VAL* 返回该解对应的模型目标值。

MIP_GAP

类型： double

是否可修改： 否

MIP 相对最优性间隙，仅适用于 MIP 求解范畴，具体定义可参考*GAP* 。

RUN_TIME

类型： double

是否可修改： 否

最近一次求解任务的执行时间。

FINGERPRINT

模型指纹

类型： long

是否可修改： 否

该参数对应由模型内部数据和属性决定的 hash 值，不同的模型（包括变量或约束添加的顺序）应具有不同的模型指纹。

NAME

类型： str

是否可修改： 是

模型名称。

6.1.2 变量属性

这些是与变量相关的属性。若是想要获取这些属性的值，直接在变量对象上调用 `Get()` 方法。对于允许用户修改的属性，可以使用 `Set()` 方法更改某一属性的值。

关于如何查询或修改属性，请参考使用示例 。

表 2: 数学规划求解器变量属性

名称	描述
BASIS	变量的基状态
BRANCH_PRIORITY	MIP 变量分支优先级
PROHIBITED	变量是否被预处理屏蔽
LB	变量下界
UB	变量上界
OBJ	变量在目标函数中的系数
START	MIP 热启动值
X	变量的原始解值
XN	变量在第 N 个解中的原始解值
DUAL	变量的对偶解值
NAME	变量名称
VAR_TYPE	变量类型
IIS_INTEGRAL	IIS 中的整数可行性指示器
IIS_LB	IIS 中的下界指示器
IIS_UB	IIS 中的上界指示器

BASIS

类型： int

是否可修改： 是

变量的基状态。

名称	取值	描述
OPTV_BASIC	0	基变量
OPTV_NONBASIC_LOWER	-1	非基，在下界
OPTV_NONBASIC_UPPER	-2	非基，在上界
OPTV_SUPERBASIC	-3	超基变量

BRANCH_PRIORITY

类型： int

是否可修改： 是

默认值： 0

该数值仅适用于 MIP 问题求解，用于引导 MIP 搜索过程的分支环节，以决定在哪些取值非整数的整数变量处进行分支，较大的取值表示较高的分支优先度。如果存在优先度相同的情况，则采用原始的分支决策。

PROHIBITED

类型： int

是否可修改： 是

表示变量是否被 LP 预处理屏蔽。取值为 0 表示未被屏蔽，取值为 1 表示被屏蔽。该参数不适用于 MIP 问题。

LB

类型： double

是否可修改： 是

变量的下界值。

注解

对于 0/1 变量，该属性只允许取值为 `0` 或 `1`．若赋值超出上述允许值 10^{-6}, 该操作会被禁止且相应的警告信息会被抛出。

UB

类型： double

是否可修改： 是

变量的上界值。

注解

对于 0/1 变量，该属性只允许取值为 `0` 或 `1`。若赋值超出上述允许值 10^{-6}, 该操作会被禁止且相应的警告信息会被抛出。

OBJ

类型： double

是否可修改： 是

变量在目标函数中的系数。

START

类型： double

是否可修改： 是

MIP 热启动值，该属性仅适用于 MIP 求解范畴，默认为*OPTV_UNDF*。

X

类型： double

是否可修改： 否

变量的原始解值。

XN

类型： double

是否可修改： 否

变量的原始解值。

该属性仅适用于 MIP 求解范畴。类似于属性*X*, 当通过*SOLUTION_NUMBER* 指定第 N 个 MIP 解时，`XN` 返回变量的原始解值。

DUAL

类型： double

是否可修改： 否

变量的对偶解值。

NAME

类型： str

是否可修改： 是

变量名称。

VAR_TYPE

类型： char

变量类型。

是否可修改： 是

表 3: 天筹 MILP 变量类型

名称	取值	描述
OPTV_CONTINUOUS	`'C'`	连续型
OPTV_INTEGER	`'I'`	整数型
OPTV_BINARY	`'B'`	0/1 型

连续变量可取给定范围内的任意浮点数值；整数变量取值限定在给定范围内的所有整数；0/1 变量只可取 0 或者 1。

IIS_INTEGRAL

类型： int

是否可修改： 是

当前 IIS 中变量的整数可行性指示符。值为 `1` 表示该变量在 IIS 中因整数性约束而不可行；值为 `0` 表示该变量的整数性并非导致不可行的原因。

注解

此属性仅在变量类型为整数（VAR_TYPE 为 `'I'` 或 `'B'`）时有效。

IIS_LB

类型： int

是否可修改： 是

当前 IIS 中变量下界的可行性指示符。值为 `1` 表示该变量的下界属于 IIS。值为 `0` 表示该下界不是不可行性所必需的。

注解

可以将此属性设置为 `1` 以强制将变量的下界包含在 IIS 中，或者设置为 `0` 以将其排除。修改此属性可能会导致 IIS 变为无效。

IIS_UB

类型： int

是否可修改： 是

变量在当前 IIS 中的上界可行性指示符。值为 `1` 表示该变量的上界是 IIS 的一部分。值为 `0` 表示该上界不是造成不可行所必需的。

注解

可以将此属性设置为 `1` 以强制将变量的上限包含在 IIS 中，或设置为 `0` 以将其排除。修改此属性可能会导致 IIS 变得不可行。

6.1.3 约束属性

这些是与约束相关的属性。若是想要获取这些属性的值，直接在约束对象上调用 `Get()` 方法。对于允许用户修改的属性，可以使用 `Set()` 方法更改某一属性的值。

关于如何查询或修改属性，请参考使用示例 。

表 4: 数学规划求解器约束属性

名称	描述
IIS_CONSTR	约束是否属于计算得到的不可约不一致子系统（IIS）
BASIS	约束的基状态
PROHIBITED	约束是否被预处理屏蔽
LB	约束的左端项
UB	约束的右端项
X	约束的原始解值
XN	约束在第 N 个解中的原始解值
DUAL	约束的对偶解值
NAME	约束名称

IIS_CONSTR

类型： bool

是否可修改： 否

表示指定约束是否属于计算所得的不可约不一致子系统 (IIS)；如果没有计算出 IIS，那么该属性不可获取。

BASIS

类型： int

是否可修改： 是

约束的基状态。

名称	取值	描述
OPTV_BASIC	0	基变量
OPTV_NONBASIC_LOWER	-1	非基，在下界
OPTV_NONBASIC_UPPER	-2	非基，在上界
OPTV_SUPERBASIC	-3	超基变量

PROHIBITED

类型： int

是否可修改： 是

表示约束是否被 LP 预处理屏蔽。取值为 0 表示未被屏蔽，取值为 1 表示被屏蔽。该参数不适用于 MIP 问题。

LB

类型： double

是否可修改： 是

约束的下界。

UB

类型： double

是否可修改： 是

约束的上界。

X

类型： double

是否可修改： 否

约束的原始解值。

XN

类型： double

是否可修改： 否

约束的原始解值。

该属性仅适用于 MIP 求解范畴。类似于属性*X*, 当通过*SOLUTION_NUMBER* 指定第 N 个 MIP 解时，`XN` 返回约束的原始解值。

DUAL

类型： double

是否可修改： 否

约束的对偶解值。

NAME

类型： str

是否可修改： 是

约束名称。

6.1.4 二次约束属性

这些是与二次约束相关的属性。若是想要获取这些属性的值，直接在约束对象上调用 `Get()` 方法。对于允许用户修改的属性，可以使用 `Set()` 方法更改某一属性的值。

关于如何查询或修改属性，请参考使用示例 。

表 5: 数学规划求解器二次约束属性

名称	描述
PROHIBITED	约束是否被预处理屏蔽
X	约束的原始解值
DUAL	约束的对偶解值
NAME	约束名称
QCSENSE	约束含义
QCRHS	约束右端项

DUAL

类型： double

是否可修改： 否

二次约束的对偶解值，注意其存在性受最优解和模型的几何结构的影响。

NAME

类型： str

是否可修改： 是

二次约束名称。

PROHIBITED

类型： int

是否可修改： 是

表示约束是否被 QCP 预处理屏蔽, 取值为 0 表示未被屏蔽，取值为 1 表示被屏蔽该参数不适用于 MIP 问题。

QCRHS

类型： double

是否可修改： 是

二次约束的右端项。注意超过 1e20 的二次约束右端值通常会导致数值问题。

QCSENSE

类型： char

是否可修改： 是

约束含义，可取值包括‘<’，‘>’，‘=’, 分别对应小于等于、大于等于和等式约束。

X

类型： double

是否可修改： 否

二次约束原始解值。

6.1.5 SOS 约束属性

这些是 SOS 属性，它们与模型中的特殊有序集（SOS）约束相关联。若是想要查询这些属性的值，直接在 OPTVSOS 对象上调用 Get() 方法。对于允许用户修改的属性，可以使用 Set() 方法更改某一属性的值。

关于如何查询或修改属性，请参考使用示例 。

表 6: SOS 约束属性

名称	描述
SOS_TYPE	SOS 约束类型 ∈ {1,2}
NAME	SOS 约束名称

SOS_TYPE

类型： int

是否可修改： 否

SOS 类型，可取值为 1 或 2.

NAME

类型： str

是否可修改： 是

SOS 约束名称。

6.1.6 一般约束属性

这些是与一般约束相关的属性。若是想要查询这些属性的值，直接在一般约束对象上调用 Get() 方法。对于允许用户修改的属性，可以使用 Set() 方法更改某一属性的值。

关于如何查询或修改属性，请参考使用示例 。

表 7: 数学规划求解器一般约束属性

名称	描述
GENCONSTR_TYPE	一般约束类型

GENCONSTR_TYPE

类型： int

是否可修改： 否

一般约束类型。

表 8: 一般约束的类型

名称	取值	描述
OPTV_GENCONSTR_AND	0	只有所有变量非零时，结果才非零。
OPTV_GENCONSTR_OR	1	只要有一个变量非零，结果就非零。
OPTV_GENCONSTR_INDICA	2	返回值为布尔型，当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0。

6.1.7 多目标优化属性

这些是用于设置和查询多目标优化的属性。

关于如何查询或修改属性，请参考使用示例 。

表 9: 天筹数学规划多目标优化属性

名称	描述
NUM_OBJ	多目标优化中目标函数的个数
OBJ_N	变量在第 N 个目标函数中的系数
OBJ_N_VAL	第 N 个目标函数的值
OBJ_N_WEIGHT	第 N 个目标函数的权重（用于对同一层级上的多个目标函数进行加权组合）
OBJ_N_OFFSET	多目标优化中目标函数的常数项
OBJ_N_REL_TOL	第 N 个目标函数的相对最优性间隙
OBJ_N_ABS_TOL	第 N 个目标函数的绝对最优性间隙
OBJ_N_NAME	第 N 个目标函数的名称

NUM_OBJ

类型： int

是否可修改： 是

多目标优化中目标函数的个数。改变该属性会增减模型中的目标函数个数。将该属性的值设置为 `0`，会将原问题转化为可行性问题。

OBJ_N

类型： double

是否可修改： 是

变量在第 `n` 个目标函数中的系数。可以通过参数*OBJ_NUMBER* 来设置 `n` 。

OBJ_N_VAL

类型： double

是否可修改： 否

多目标优化中，解池中的第 k 个解在目标函数 n 中的值。可以通过参数*SOLUTION_NUMBER* 来查询 k 的值，通过参数*OBJ_NUMBER* 设置 n 的值。

OBJ_N_WEIGHT

类型： double

是否可修改： 是

多目标优化中，目标函数 n 的权重。可以通过参数*OBJ_NUMBER* 来设置 n 。

该属性值的符号决定了多目标优化的方向：

正号表示该多目标的优化方向等于*OPTVSense::MINIMIZE*；

负号表示该多目标的优化方向等于*OPTVSense::MAXIMIZE*。

OBJ_N_OFFSET

类型： double

是否可修改： 是

多目标优化中，目标函数 n 的常数项。可以通过参数*OBJ_NUMBER* 来设置 n 。

OBJ_N_REL_TOL

类型： double

是否可修改： 是

多目标优化中，第 n 个目标函数在低优先级目标优化时允许的相对劣化程度。可以通过参数*OBJ_NUMBER* 来设置 n 。

OBJ_N_ABS_TOL

类型： double

是否可修改： 是

多目标优化中，第 n 个目标函数在低优先级目标优化时允许的绝对劣化程度。可以通过参数*OBJ_NUMBER* 来设置 n 。

OBJ_N_NAME

类型： str

是否可修改： 是

在多目标优化中，查询或设置目标函数 n 的名称。可以通过参数*OBJ_NUMBER* 设置 n。

6.1.8 解的质量属性

关于如何查询或修改属性，请参考使用示例 。

表 10: 数学规划求解器解质量属性

名称	描述
MAX_VIO	最大违反度
BOUND_VIO	最大变量界违反度
CONSTR_VIO	最大约束违反度
INT_VIO	最大整性违反度
BOUND_VIO_INDEX	最大变量界违反下标
CONSTR_VIO_INDEX	最大约束违反下标
INT_VIO_INDEX	最大整性违反下标
BOUND_VIO_SUM	变量界违反度之和
CONSTR_VIO_SUM	约束违反度之和
INT_VIO_SUM	整性违反度之和

MAX_VIO

类型： double

是否可修改： 否

所有约束（变量界、约束、整性）的最大违反度。

BOUND_VIO

类型： double

是否可修改： 否

变量界的最大违反度。

CONSTR_VIO

类型： double

是否可修改： 否

约束最大违反度。

INT_VIO

类型： double

是否可修改： 否

整性约束最大违反度。

BOUND_VIO_INDEX

类型： int

是否可修改： 否

变量界违反度最大的变量的下标。

CONSTR_VIO_INDEX

类型： int

是否可修改： 否

约束违反度最大的约束的下标。

INT_VIO_INDEX

类型： int

是否可修改： 否

整性违反度最大的变量的下标。

BOUND_VIO_SUM

类型： double

是否可修改： 否

变量界违反度之和。

CONSTR_VIO_SUM

类型： double

是否可修改： 否

约束违反度之和。

INT_VIO_SUM

类型： double

是否可修改： 否

整性违反度之和。

6.2 属性使用示例

天筹数学规划求解器的属性接口采用正交设计：通过一组精简而统一的方法，即可访问和操作大量不同的属性。具体而言：

- 属性名称及其语义在所有支持的编程语言 API 中完全一致，仅存在细微的语言特定语法（或 修 饰）差异。
- 要查询或修改某个属性，只需在您所选的编程语言中，确定其数据类型（例如 double、integer 等）所对应的属性类型即可。

数据类型及其对应的属性类型如下：

数据类型	属性类型
`int`	OPTVIntAttr
`double`	OPTVDblAttr
`long`	OPTVLongAttr
`string`	OPTVStrAttr
`char`	OPTVCharAttr
`bool`	OPTVBoolAttr

以下展示了如何在不同语言 API 中查询变量的下界：

语言	示例
C++	*var.Get(OPTVDblAttr::LB)*
java	*var.get(OPTVDblAttr.LB)*
python	*var.Get(OPTVDblAttr.LB)* 或 var.LB

需要注意的是，属性的更改不会立即生效。只有当用户更新或者优化模型时才会生效（例如在 C++ 中，使用*OPTVModel::Update()* 或*OPTVModel::Optimize()* ）。

CHAPTER 7

参数参考

7.1 数学规划求解器参数说明

7.1.1 终止条件

这些参数会影响算法的终止行为。如果算法超出其中任意一项限制，将终止运行并返回一个非最优的终止状态。请注意，算法在达到指定限制时不会立即停止，而可能在超出设置限制一段时间后才进行终止检查。

表 1: 终止条件参数

名称	描述
TIME_LIMIT	模型求解时间上限（秒）。
MEM_LIMIT	求解器使用的内存限制, 单位 MB。
SUBMIP_NODES	子 MIP 启发式策略的搜索节点数上限。
START_NODE_LIMIT	MIP 分支定界节点个数上限。

TIME_LIMIT 求解时间上限

类型： double

模型求解时间上限（秒）。

默认值： 10^{20}

最小值： 0

最大值： 10^{20}

如果求解时间超出该数值，求解器则会终止并返回状态*OPTV_TIME_LIMIT*。

注意求解计时受制于单步迭代计时。

MEM_LIMIT 内存限制

类型： int

求解器使用的内存限制, 单位 MB。

默认值： 2147483647

最小值： 100

最大值： 2147483647

该参数用来限制求解器运行占用的 RAM 空间不超过指定阈值，若超出该阈值，求解器将终止运行并返回状态*OPTV_MEM_LIMIT*。

SUBMIP_NODES 启发式节点数上限

类型： int

MIP 启发式策略模块访问的节点个数上限。

默认值： 500

最小值： 0

最大值： 2147483647

该参数用以限制 MIP 启发式策略访问的节点个数，访问节点越多，越有可能产生更高质量的解，但通常也耗时更久。

注解

本参数仅适用于 MIP 问题。

START_NODE_LIMIT 分支定界节点数上限

类型： int

MIP 分支定界节点个数上限。

默认值： -1

最小值： -3

最大值： 2147483647

该参数用于控制 MIP 热启动策略，及策略执行时进行子 MIP 搜索的分支定界节点数。默认选项允许 MIP 热启动执行，且由求解器自动确定此节点数；取值为-2 时，仅当 MIP 热启动的变量赋值完整时才进行热启动处理；取值为-3 将完全禁用 MIP 热启动。

注解

本参数仅适用于 MIP 问题。

7.1.2 容差

这些参数用于控制允许的可行性或最优性违反程度。

表 2: 容差参数

名称	描述
FEAS_TOL	原始问题约束可行性精度。
INT_TOL	整数变量的整数性精度。
MARKOWITZ_TOL	用于限制选择矩阵分解主元时的数值误差。
GAP	MIP 相对最优性间隙。

FEAS_TOL 约束可行性精度

类型： double

原始问题约束可行性精度。

默认值： 10^{-6}

最小值： 10^{-9}

最大值： 0.01

若该参数取较小的值，所得解的约束违背量则越小，但是迭代步数可能更多。

INT_TOL 整数性精度

类型： double

整数变量的整数性精度。

默认值： 10^{-6}

最小值： 10^{-9}

最大值： 0.01

若该参数取较小的值，所得解的整数性违背量则越小，但是求解性能（时耗）有可能被影响；取值较大一般不影响性能（时耗）。

MARKOWITZ_TOL 矩阵分解精度

类型： double

适用于单纯形方法，用于限制选择矩阵分解主元时的数值误差。

默认值： 0.1

最小值： 10^{-4}

最大值： 0.999

取值较大将得到更好的数值稳定性，但可能影响求解性能（时耗）。

GAP 相对偏差

类型： double

MIP 相对最优性间隙。

默认值： 0.0001

最小值： 0

最大值： 10^{100}

设 f_P 为原始目标值（primal objective），f_D 为对偶目标值（dual objective），则二者的相对差值 g 可由公式 $g = \frac{|f_P - f_D|}{|f_P|}$ 求解。

在 MIP 求解过程中，一旦目标值的上下界相对偏差比 `gap` 小，求解器则会终止并返回状态*OPTV_OPTIMAL*。

7.1.3 日志

这些参数用于控制日志信息的输出。

表 3: 日志参数

名称	描述
OUTPUT_FLAG	禁止日志打印。

OUTPUT_FLAG 日志打印控制

类型： int

启用或禁用求解器打印日志。

默认值： 0

最小值： 0

最大值： 1

如果该参数设置为 0，求解器不会打印日志。若是设置为 1，求解器正常打印日志。

7.1.4 输入/输出

这些参数可用于读取或写出信息。

表 4: 输入/输出参数

名称	描述
LOG_FILE	日志文件路径。
PARAM_FILE	参数文件路径。
REPLAY_FILE	SDK 命令回放文件路径。
RESULT_FILE	结果文件路径。
SOL_FORMAT	输出解的形式（稀疏或者稠密）。

LOG_FILE 日志文件路径

类型： str

日志文件路径。

默认值： `"./optv.log"`

如果*OUTPUT_FLAG* 被设置为 0，则本参数将被忽略。

注解

通过 SDK 构建模型时，默认不会输出日志文件。

使用命令行工具时，该参数的默认值为 `"./optv.log"`。

PARAM_FILE 参数文件路径

类型： str

参数文件路径。

默认值： `""`

用于为求解器指定参数文件，默认值为空。

REPLAY_FILE SDK 命令回放文件

类型： str

输出回放文件的路径。

默认值： `""`

用于指定 SDK 命令回放文件存放的路径，若为空，则不创建命令回放文件。

注解

该参数 **必须**在 `OPTVModel` 对象创建前，通过 `OPTVEnv` 对象设置。

RESULT_FILE 结果文件路径

类型： str

结果文件路径。

默认值： `"./<input_file>.sol"`

注解

通过 SDK 构建模型时，默认不进行输出结果文件，**必须**通过该参数指定生成解文件。

使用命令行工具时，结果文件的输出路径默认是输入模型文件的路径。

SOL_FORMAT 解文件风格控制

类型： int

输出解的形式（稀疏或者稠密）。

默认值： 0

最小值： 0

最大值： 1

通过调整该参数，用户可以控制解文件中是否包含取值为 0 的变量。若设置该参数为 0，解文件将省去取值为 0 的变量，为稀疏格式；否则，解文件将包含取值为 0 的变量，为稠密格式。

7.1.5 LP

这些参数用于控制 LP 算法。

表 5: LP 相关参数

名称	描述
CROSSOVER	内点法 crossover 策略。
METHOD	LP 求解算法（如单纯形、内点等）。
OPTIMAL_BASIS	清理所得解获取最优基解选项。

CROSSOVER

类型： int

内点法 crossover 策略。

默认值： 2

最小值： 0

最大值： 4

crossover 将内点解（通常为可行解，但一般不是基解）转化为基本可行解，它由以下三步组成：

1) 原始推进步 - 将原始变量推至边界。

2) 对偶推进步 - 将对偶变量推至边界。

3) 清理 - 应用单纯形方法清理任何原始或对偶不可行性。

该参数用于配置前两步的步骤及清理方法的选项。

取值	第一种推进方法	第二种推进方法	不可行性清理方法
0	禁用	禁用	禁用
1	对偶	原始	原始
2 (默认)	对偶	原始	对偶
3	原始	对偶	原始
4	原始	对偶	对偶

METHOD 优化算法

类型： int

LP 问题优化算法选择。

默认值： -1

最小值： -1

最大值： 4

该参数用于指定 LP 问题的优化方法。

取值	方法
-1	自动
0	原始单纯形方法
1	对偶单纯形方法
2	内点方法 (IPM)
3	确定性内点方法 (deterministic IPM)
4	并发求解方法 (concurrent solver)

注解

该参数即影响 LP 问题的求解，也影响 MIP 问题的首个 LP 松弛问题的求解。

OPTIMAL_BASIS 最优基

类型： int

求解结束时是否同时需要最优基。

默认值： 1 (开启)

最小值： 0 (关闭)

最大值： 1 (开启)

该高级参数允许用户获取可行解后跳过基状态清理过程，若不关心所得的可行解是基解与否，可关闭该选项。关闭该选项后，求解器不会执行基状态的清理操作。

警告

若关闭该选项，所得解有基状态不正确的风险，而且这个参数会影响热启动的效果。仅适用于查询问题是否可行的情景，如果用户需要获取变量或约束的取值 (*var.Get(OPTVDblAttr::X)* 或 *constr.Get(OPTVDblAttr::X)*)，则需要 **开启**该选项 (默认环境下为开启状态)。

7.1.6 MIP

这些参数用于控制 MIP 算法。

表 6: MIP 算法参数

名称	描述
BRANCH_DIR	MIP 分支方向。
VAR_BRANCH	MIP 变量分支方法。
MIP_FOCUS	MIP 高层级的解策略。
HEURISTICS	MIP 启发式策略模块占时比例。
POOL_SOLUTIONS	MIP 解池的最大存储（解）数量。
SOLUTION_NUMBER	用来指定 MIP 解。
PRESOLVE_LEVEL	预处理程度控制。
SEED	初始随机数种子。
THREADS	线程数。

BRANCH_DIR 分支方向

类型： int

MIP 分支方向。

默认值： 0

最小值： -1

最大值： 1

该参数控制分支定界搜索中子节点的选取，取值-1 表示优先尝试向下圆整分支节点，取值 +1 表示优先尝试向上圆整分支节点。默认值为 0，表示自动选取子节点。

VAR_BRANCH 变量分支方法

类型： int

MIP 变量分支方法。

默认值： -1

最小值： -1

最大值： 2

该参数控制变量分支策略，具体信息如下：

取值	描述
-1	自动
0	Reliability 分支方法
1	Strong branching 方法
2	Pseudo cost branching 方法

MIP_FOCUS 解策略

类型： int

MIP 高层级的解策略。

默认值： 0

最小值： 0

最大值： 3

该参数控制 MIP 求解器的优先度策略，默认为均衡式，具体信息如下：

取值	描述
0	均衡式。
1	优先快速找到可行解。
2	优先证明最优性。
3	优先改善下界。

HEURISTICS 启发式模块

类型： double

MIP 启发式策略模块占总求解时间的比例。

默认值： 0.05

最小值： 0

最大值： 1

该参数控制 MIP 求解器启发式策略模块的执行，方式是控制模块的时间消耗占总求解时间的比例，默认值为 0.05, 即占总时耗的 5%。较高的该参数值可以使 MIP 求解器找到更多或更好的可行解，但是目标值的最优界改进会更慢。

POOL_SOLUTIONS 解池容量

类型： int

MIP 解池的最大存储（解）数量。

默认值： 10

最小值： 1

最大值： 1000000

该参数用于设置 MIP 解池存储的解的数量上限。

SOLUTION_NUMBER 解索引

类型： int

用来指定 MIP 解。

默认值： 0

最小值： 0

最大值： 1000000

用户可以通过该参数指定要查询的解，从而可获悉更多相关信息，如*XN* 或*POOL_OBJ_VAL*；取值为 0 时，上述查询信息等同于*X* 或*OBJ_VAL*.

注解

本参数仅适用于 MIP 问题。

PRESOLVE_LEVEL 预处理控制

类型： int

预处理程度控制。

默认值： -1

最小值： -1

最大值： 2

用户通过该参数控制预处理的程度。

取值	程度
-1	自动
0	关闭预处理
1	保守倾向
2	激进倾向（耗时多，处理后的模型更紧凑）

SEED 随机数种子

类型： int

初始随机数种子。

默认值： 0

最小值： 0

最大值： 2147483647

此参数所提供的值将用作伪随机数生成的初始种子值。

THREADS 线程数

类型： int

求解器运行期间使用最大线程数。

默认值： 机器可用最大线程数除以 2。

最小值： 1

最大值： 可用的虚拟处理器数量。

7.1.7 割平面

这些参数会影响 MIP 割平面的生成。

表 7: 割平面参数

名称	描述
CUT_PASSES	根节点 (MIP) 割平面方法最大轮数。
CUTS	MIP 割平面方法强度。

CUT_PASSES 割平面轮数

类型： int

根节点 (MIP) 割平面方法最大轮数。

默认值： 2147483647

最小值： 0

最大值： 2147483647

该参数用以控制根节点割平面方法，使得轮数不超过设定值。

CUTS 割平面方法强度

类型： int

MIP 割平面方法强度。

默认值： -1

最小值： -1

最大值： 3

该参数全局地控制割平面方法强度。默认值为-1，允许求解器进行自适应的调整；取值 0 则会关闭割平面方法；取值 1 会应用轻量级的割平面方法，取值 2 对应中等强度的割平面方法，而取值 3 则应用高强度的割平面方法。

7.1.8 多目标优化

这些参数在处理多目标优化问题时可能会用到。

表 8: 多目标优化参数

名称	描述
OBJ_NUMBER	目标函数索引。

OBJ_NUMBER 目标函数索引

类型： int

在多目标优化中，需要操作的目标函数的索引。

默认值： 0

最小值： 0

最大值： 2147483647

若是用户需要查询或者修改某个目标函数的相关属性，该参数用以定位该子目标函数。

7.2 参数示例

天筹求解器的参数设计遵循正交性原则：通过简洁且一直的接口，即可高效访问和操作所有参数。具体而言：

- 跨语言一致性：所有受支持的编程语言 API 中，参数名称及其语义完全一致，仅在语法细节（如语言特有的装饰符或调用方式）上略有差异。
- 直观的参数设置：只需根据参数的数据类型（如双精度浮点数、整数等），选择对应编程语言中相应的参数类型即可完成设置。

各数据类型及其对应的参数类型如下所示：

数据类型	数值类型
`int`	参数类型
`double`	OPTVIntParam
`string`	OPTVDblParam

以下展示了如何使用 MILP 求解器的不同语言 API 设置此参数。

语言	参数集示例
C++	*`model.Set(OPTVIntParam::THREADS, 10)`*
java	*`model.set(OPTVIntParam.THREADS, 10)`*
python	*`model.Set(OPTVIntParam.THREADS, 10)`* 或 model.Threads = 10

需要注意的是，在天筹数学规划求解器中，参数的修改不会立即生效。只有更新或优化模型时（在 C++ 中使用*`OPTVModel::Update()`* 或*`OPTVModel::Optimize()`*），参数修改才会生效。

同样地，下面展示了在天筹约束规划求解器中如何使用不同语言的 API 来设置参数。

语言	参数集示例
C++	*model.Set(OPTVIntParam::CP_THREADS, 10)*
python	*model.Set(OPTVIntParam.CP_THREADS, 10)*

7.3 约束规划求解器参数说明

7.3.1 终止条件

这些参数用于控制算法的终止条件。如果算法超出其中任意一项限制，将终止运行并返回一个非最优的终止状态。请注意，算法在达到指定限制时不会立即停止，而是在后续的某个检查点才会检测到该限制已被超出。因此实际终止时间可能会明显晚于限制被突破的时刻。

表 9: 终止条件参数

名称	描述
CP_TIME_LIMIT	模型求解时间上限（秒）。
CP_DETER_TIME_LIMIT	约束规划求解器的确定性时间限制。

CP_TIME_LIMIT

类型： double

模型求解时间上限（秒）。

默认值： $2^{63} - 1$

最小值： 0

最大值： $2^{63} - 1$

CP_DETER_TIME_LIMIT

类型： double

约束规划求解器的确定性时间限制。

默认值： $2^{63} - 1$

最小值： 0

最大值： $2^{63} - 1$

7.3.2 容差

这些参数用于控制允许的可行性或最优性违反程度。

表 10: 容差参数

名称	描述
CP_GAP	当前最优可行解（上界）与最佳理论界（下界）之间的相对差值。
CP_ABS_GAP	当前最优可行解（上界）与最佳理论界（下界）之间的绝对差值。

CP_GAP

类型： double

当前最优可行解（上界）与最佳理论界（下界）之间的相对差值。

默认值： 0

最小值： 0

最大值： 10^{100}

设 f_P 为原始目标值（primal objective），f_D 为对偶目标值（dual objective），则二者的相对差值 g 可由公式 $g = \frac{|f_P - f_D|}{|f_P|}$ 求解。

当目标函数的上下界之间的相对间隙小于参数 `gap` 所指定的值时，求解器将终止并返回状态码*OPTV_CP_OPTIMAL*。

CP_ABS_GAP

类型： double

当前最优可行解（上界）与最佳理论界（下界）之间的绝对差值。

默认值： 10^{-6}

最小值： 0

最大值： 10^{100}

7.3.3 算法控制

约束规划求解器的算法控制参数。

表 11: 算法控制参数

名称	描述
CP_THREADS	线程数。
CP_SOLVER_SEED	约束规划求解器的随机种子。
CP_DETERMINISTIC_MODE	是否以确定性模式进行求解。
CP_PRESOLVE	约束规划求解器是否开启预处理。
CP_LP_LEVERAGE_LEVEL	约束规划求解器中 LP 求解器的使用级别。
CP_HEURISTIC_MODE	是否以启发式模式进行求解。
CP_CORE_BASE_OPT	是否基于核心方法进行优化。

CP_THREADS

类型： int

线程数。

默认值： 1

最小值： 1

最大值： 可用的虚拟处理器数量。

CP_SOLVER_SEED

类型： int

约束规划求解器的随机种子。

默认值： 1

最小值： 0

最大值： $2^{63} - 1$

此参数所提供的值将用作伪随机数生成的初始种子值。

CP_DETERMINISTIC_MODE

类型： int

是否以确定性模式进行求解。

默认值： 1

最小值： 0

最大值： 1

CP_PRESOLVE

类型： int

约束规划求解器是否开启预处理。

默认值： 1

最小值： 0

最大值： 1

CP_LP_LEVERAGE_LEVEL

类型： int

约束规划求解器中 LP 求解器的使用级别。

默认值： 1

最小值： 0

最大值： $2^{31} - 1$

CP_HEURISTIC_MODE

类型： int

是否以启发式模式进行求解。

默认值： 0

最小值： 0

最大值： 1

CP_CORE_BASE_OPT

类型： int

是否基于核心方法进行优化。

默认值： 0

最小值： 0

最大值： 1

CHAPTER 8

API 参考手册

8.1 环境

环境类，用于管理天筹数学规划求解器以及约束规划求解器的参数配置。此外，许可文件需要通过环境变量 `OPTV_LICENSE_FILE` 来配置。

8.1.1 C++ API 参考手册

class **OPTVEnv**

OPTVEnv()

OPTVEnv::**OPTVEnv**()

返回
环境变量，参数为默认。

注解

默认配置下日志文件被设定为 `optv.log` 。

OPTVEnv::**OPTVEnv**(const char *logFileName)

参数
logFileName –日志文件路径，格式为 C-风格字符串。

返回
环境变量，参数为默认，日志文件由参数指定。

OPTVEnv::**OPTVEnv**(const std::string &logFileName)

参数
logFileName –日志文件路径，格式为字符串。

返回
环境变量，参数为默认，日志文件由参数指定。

OPTVEnv::**OPTVEnv**(const *OPTVEnv* &xenv)

参数
xenv –需要被复制的环境对象。

返回
环境对象 `xenv` 的副本。

OPTVEnv::**OPTVEnv**(const *OPTVEnv* *xenv)

参数
xenv –需要被复制的环境对象的指针。

返回
环境对象指针 `xenv` 所指对象的副本。

Get()

获取求解器的参数值。

可用参数请参考*数学规划求解器参数*。

OPTVEnv::**Get**(*OPTVIntParam* param)

参数
param –整数型参数。

返回
参数取值。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVEnv::**Get**(*OPTVDblParam* param)

参数
param –双精度型参数。

返回
参数取值。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVEnv::**Get**(*OPTVStrParam* param)

参数
param –字符串型参数。

返回
参数取值。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

Set()

通过环境对象设置求解器参数，相应修改会作为默认，被应用到后续创建的所有模型。模型建立后，只有调用*OPTVModel::Update()* 才可以激活新的参数修改。

可用参数请参考数学规划求解器参数。

OPTVEnv::**Set**(*OPTVIntParam* param, int value)

参数

- **param** –整数型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVEnv::**Set**(*OPTVDblParam* param, double value)

参数

- **param** –双精度型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVEnv::**Set**(*OPTVStrParam* param, const std::string &value)

参数

- **param** –字符串型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

ResetParams()

将所有的参数重置为默认值。

OPTVEnv::**ResetParams**()

返回
无。

8.1.2 JAVA API 参考手册

class OPTVEnv

OPTVEnv()

OPTVEnv.**OPTVEnv**()

返回
环境变量，参数为默认。

返回类型
OPTVEnv

注解

默认配置下日志文件被设定为 `optv.log` 。

OPTVEnv.**OPTVEnv**(String *logFileName*)

参数

- **logFileName** –日志文件路径，格式为 C-风格字符串。

返回
环境变量，参数为默认，日志文件由参数指定。

返回类型
OPTVEnv

OPTVEnv.**OPTVEnv**(OPTVEnv *xenv*)

参数

- **xenv** –需要被复制的环境对象。

返回
环境对象 xenv 的副本。

返回类型
OPTVEnv

get()

获取求解器的参数值。

可用参数请参考数学规划求解器参数。

OPTVEnv.**get**(OPTVIntParam *param*)

参数

- **param** –整数型参数。

返回
参数取值。

返回类型
int

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

OPTVEnv.**get**(OPTVDblParam *param*)

参数

- **param** –双精度型参数。

返回
参数取值。

返回类型
double

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

OPTVEnv.**get**(OPTVStrParam *param*)

参数

- **param** –字符串型参数。

返回
参数取值。

返回类型
String

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

set()

通过环境对象设置求解器参数，相应修改会作为默认，被应用到后续创建的所有模型。模型建立后，只有调用*OPTVModel.update()* 才可以激活新的参数修改。

可用参数请参考数学规划求解器参数。

OPTVEnv.**set**(OPTVIntParam *param*, int *value*)

参数

- **param** –整数型参数。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

OPTVEnv.**set**(OPTVDblParam *param*, double *value*)

参数

- **param** –双精度型参数。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

OPTVEnv.**set**(OPTVStrParam *param*, String *value*)

参数

- **param** –字符串型参数。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

8.1.3 Python API 参考手册

class OPTVEnv

OPTVEnv()

OPTVEnv.**OPTVEnv**()

返回
环境变量，参数为默认。

返回类型
OPTVEnv

注解

默认配置下日志文件被设定为 `optv.log` 。

OPTVEnv.**OPTVEnv**(*logFileName*)

参数
logFileName (*str*) –日志文件路径。

返回
环境变量，参数为默认，日志文件由参数指定。

返回类型
OPTVEnv

OPTVEnv.**OPTVEnv**(*xenv*)

参数
xenv (OPTVEnv) –需要被复制的环境对象。

返回
环境对象 `xenv` 的副本。

返回类型
OPTVEnv

__getattr__()

OPTVEnv.**__getattr__**(*param*)

通过属性访问方式获取求解器参数值。

这允许使用点表示法访问参数，例如，`env.TIME_LIMIT` 而不是 `env.Get(OPTVDblParam.TIME_LIMIT)`。

参数
param –作为属性的参数名（例如 `TIME_LIMIT`、`THREADS`）。

返回
参数取值。

返回类型
int|float|str

引发
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

示例

```
env = OPTVEnv()
env.Set(OPTVDblParam.TIME_LIMIT, 1234.0)
print(env.TIME_LIMIT)   # 1234.0
```

__setattr__()

OPTVEnv.**__setattr__**(*param*, *value*)

使用属性式访问设置求解器参数。

这允许使用点表示法设置参数，例如，`env.TIME_LIMIT = 7200.0` 而不是 `env.Set(OPTVDblParam.TIME_LIMIT, 7200.0)`。

参数

- **param** –作为属性的参数名（例如 `TIME_LIMIT`、`THREADS`）。
- **value** –要设置的值。

引发
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

示例

```
env = OPTVEnv()

env.TIME_LIMIT = 1234.0
env.THREADS = 2
print(env.TIME_LIMIT)   # 1234.0
print(env.THREADS)      # 2
```

Get()

获取求解器的参数值。

可用参数请参考数学规划求解器参数。

OPTVEnv.**Get**(*param*)

参数
param(OPTVIntParam/OPTVDblParam/OPTVStrParam)–需要查询的参数，包括整数型、双精度型、字符串型。

返回
参数取值。

返回类型
int|float|str

引发
OPTVErrCode.UNKNOWN_PARAMETER–传入了未知参数。

示例

```
env = OPTVEnv()
env.Set(OPTVDblParam.TIME_LIMIT, 7200.0)
print(env.Get(OPTVDblParam.TIME_LIMIT)) # 7200.0
```

Set()

通过环境对象设置求解器参数，相应修改会作为默认被应用到后续创建的所有模型。模型建立后，只有调用*OPTVModel.Update()*才可以激活新的参数修改。

可用参数请参考数学规划求解器参数。

OPTVEnv.**Set**(*param*, *value*)

参数

- **param**(OPTVIntParam/OPTVDblParam/OPTVStrParam)–需要设置的参数，包括整数型、双精度型、字符串型。
- **value**(*int*/*float*/*str*)–要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_PARAMETER–传入了未知参数。

示例

```
env = OPTVEnv()
env.Set(OPTVDblParam.TIME_LIMIT, 7200.0)
env.Set(OPTVIntParam.THREADS, 2)
env.Set(OPTVStrParam.LOG_FILE, 'optv_new.log')

print(env.Get(OPTVDblParam.TIME_LIMIT)) # 7200.0
```

(下页继续)

(续上页)

```
print(env.Get(OPTVIntParam.THREADS))     # 2
print(env.Get(OPTVStrParam.LOG_FILE))    # optv_new.log
```

ResetParams()

将所有的参数重置为默认值。

OPTVEnv.**ResetParams**()

返回
无。

返回类型
None

示例

```
env = OPTVEnv()
env.Set(OPTVDblParam.TIME_LIMIT, 7200.0)
env.Set(OPTVIntParam.THREADS, 1)
env.Set(OPTVStrParam.LOG_FILE, 'optv_new.log')

# Reset all configured parameters
env.ResetParams()

print(env.Get(OPTVDblParam.TIME_LIMIT)) # 1e+20
print(env.Get(OPTVIntParam.THREADS))    # 1
print(env.Get(OPTVStrParam.LOG_FILE))   # ""
```

8.2 数学规划求解器 API 参考手册

8.2.1 C++ API 参考手册

变量

class **OPTVVar**

天筹求解器的变量对象，用于从模型访问变量。注意变量不可由其构造函数创建，而需要通过模型的*OPTVModel::AddVar()* 或*OPTVModel::AddVars()* 接口添加变量到模型。

Get()

查询变量属性。

可用属性请参考*天筹属性*。

OPTVVar::**Get**(*OPTVIntAttr* attr)

参数
attr –整数型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知变量属性。

OPTVVar::**Get**(*OPTVCharAttr* attr)

参数
attr –字符型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知变量属性。

OPTVVar::**Get**(*OPTVDblAttr* attr)

参数
attr –双精度型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知变量属性。

OPTVVar::**Get**(*OPTVStrAttr* attr)

参数
attr –字符串型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知变量属性。

Set()

设置变量属性，只有调用*OPTVModel::Update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVVar::**Set**(*OPTVIntAttr* attr, int value)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知变量属性。

OPTVVar::**Set**(*OPTVCharAttr* attr, char value)

参数

- **attr** –字符型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知变量属性。

OPTVVar::**Set**(*OPTVDblAttr* attr, double value)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知变量属性。

OPTVVar::**Set**(*OPTVStrAttr* attr, const std::string &value)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知变量属性。

Index()

获得变量在模型中的索引值。

OPTVVar::**Index**() const

返回
变量索引/下标。

返回值

- **-1** –该变量在模型中不存在。

- **-2** –该变量已从模型中删除。
- **>=0** –该变量在模型中的索引/下标。

SameAs()

判断两个变量是否为同一个对象。

bool *OPTVVar*::**SameAs**（const *OPTVVar* &v2）const

参数
v2 –需要与当前变量对比的变量。

返回
变量是否相同。

返回值

- **true** –变量相同。
- **false** –变量不同。

列

class **OPTVColumn**

天筹求解器的列对象，用于以列的形式向模型添加变量，用户可以使用该功能向模型同时新增变量并添加其至已有约束。

在下面的例子中，我们构造一个初始模型并展示如何通过列对象增加变量，本示例展示的模型为*Quick Start* 章节的*milp1* .

```
#include "optv_c++.h"

OPTVEnv env;
OPTVModel model(env);

OPTVVar x = model.AddVar(0, 1, -1, OPTV_BINARY, "x");
OPTVVar y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y");

OPTVConstr c0 = model.AddConstr(2 * x + y, -OPTV_INF, 3, "c0");
OPTVConstr c1 = model.AddConstr(3 * y, -OPTV_INF, 6, "c1");
```

具体地，构建的模型形式如下：

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

```
OPTVColumn col;

col.AddTerm(1, c0);
col.AddTerm(1, c1);

OPTVVar z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, col, "z");
```

通过将 col 添加到模型，相应的约束矩阵变为：

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 3 & 1 \end{bmatrix}$$

AddTerm()

向列对象（尾部）添加单项。

OPTVColumn::**AddTerm**(double coef, const *OPTVConstr* &constr)

该函数向列对象添加单项。

参数

- **coef** –系数值。
- **constr** –约束对象。

返回

无。

AddTerms()

向列对象（尾部）添加多项。

OPTVColumn::**AddTerms**(const double *coefs, const *OPTVConstr* *constrs, int cnt)

该功能向列对象添加多项。

参数

- **coefs** –C-风格的系数序列。
- **constrs** –C-风格的约束序列。
- **cnt** –添加项的个数。

返回

无。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –constrs 或 coefs 为空指针或 cnt 取负值。

Clear()

清空与该列相关内部容器，使*OPTVColumn::Size()* 返回 0.

OPTVColumn::**Clear**()

返回

无。

GetCoeff()

据指定索引值获得列对象中对应的约束系数。

OPTVColumn::**GetCoef**(int i) const

参数
i –指定的索引值，对应列中的约束。

返回
列对象中对应该索引, i, 的约束系数。

Raises OPTVErrCode::INDEX_OUT_OF_RANGE
若 i ∉ [0, this->Size()]。

GetConstr()

根据指定索引值获得列对象中对应的约束。

OPTVColumn::**GetConstr**(int i) const

参数
i –指定的索引值，对应列中的约束。

返回
列对象中对应该索引, i, 的约束对象。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –若 i ∉ [0, this->Size()]。

Remove()

从列对象中删除单/多项。

OPTVColumn::**Remove**(int i)
根据指定索引值从该列对象中删除项。

参数
i –指定的索引值。

返回
无。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –若 i ∉ [0, this->Size()]。

OPTVColumn::**Remove**(const *OPTVConstr* &constr)
从列对象中删除指定约束的所有相关项。

参数
constr –指定要删除的约束。

返回
无。

抛出
OPTVErrCode::*NOT_IN_MODEL* –指定的约束，constr, 在模型中不存在。

Size()

获得该列的规模。

OPTVColumn::**Size**() const

该数值统计对应列中的元素个数，无论是否有重复项，例如，若某约束被多次添加，本数值依然相应增加。

返回

该列的规模。

线性表达式

class **OPTVLinExpr**

线性表达式*OPTVLinExpr*类用于构建线性的目标函数或者约束函数。线性表达式对象由变量和常量组成，边获取边构造，如下例所示的表达式 `1 * x + 2 * y + 3 * x`，它包含以下元素：

索引	系数	变量
0	1	x
1	2	y
2	3	x

AddTerms()

该功能向当前表达式（后）添加一组新的项。

OPTVLinExpr::**AddTerms**(const double *coefs, const *OPTVVar* *vars, int cnt)

此函数将 $\text{coefs}_i * \text{vars}_i, \forall i \in [0, \text{cnt}]$ 添加到当前线性表达式后。

参数

- **coefs** –C-风格的系数序列。
- **vars** –C-风格的变量序列。
- **cnt** –添加项的个数。

返回

无。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –`vars` 或 `coefs` 为空指针或 `cnt` 取负值。

Clear()

清空该表达式，*OPTVLinExpr::Size()* 将返回 0。同时，该表达式从数值意义上将等同于 0。

OPTVLinExpr::**Clear**()

返回
无。

GetCoef()

据指定索引值获得表达式中对应变量的系数。

OPTVLinExpr::**GetCoef**(int i) const

参数
i –变量在线性表达式中的索引。

返回
对应的变量系数。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –若 i $\notin [0, \texttt{this->Size()}]$。

GetVar()

根据指定索引值获得表达式中的对应变量。

OPTVLinExpr::**GetVar**(int i) const

参数
i –变量在线性表达式中的索引。

返回
指定位置上的变量。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –若 i $\notin [0, \texttt{this->Size()}]$。

GetConstant()

获得当前表达式中的常数项。

OPTVLinExpr::**GetConstant**() const

返回
表达式中的常量数值。

GetValue()

获得该表达式的取值。

OPTVLinExpr::**GetValue**() const

返回
对应原始解的表达式的取值，即该表达式的变量值与系数的乘积之和。

抛出
OPTVErrCode::*DATA_NOT_AVAILABLE* –当前变量取值不存在。

Remove()

从表达式中删除项。

OPTVLinExpr::**Remove**(int i)
根据指定索引值从该表达式中删除项。

参数
i –指定的索引值。

返回
无。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –若 i ∉ [0, this->Size()]。

OPTVLinExpr::**Remove**(const *OPTVVar* &v)
指定的变量（在模型中）不存在。

参数
v –指定要删除的变量。

返回
是否成功删除任何项。

返回值
- **true** –成功删除任何项。
- **false** –未删除任何项。

抛出
OPTVErrCode::*NOT_IN_MODEL* –成功删除与指定变量有关的项。

SameAs()

判断两个线性表达式是否为同一个对象。

OPTVLinExpr::**SameAs**(const *OPTVLinExpr* &e2) const

参数
e2 –需要与当前线性表达式对比的线性表达式。

返回
线性表达式是否相同。

返回值

- **true** –线性表达式相同。
- **false** –线性表达式不同。

Size()

获得该表达式的规模。

OPTVLinExpr::**Size**() const

该函数返回的数值对应线性表达式中的变量个数，不论变量是否重复出现。例如表达式 x+y 对应 Size = 2，而 x+y+x 对应 Size = 3。

返回
线性表达式大小。

线性约束

class **OPTVConstr**

天筹求解器的线性约束对象，用于从模型访问约束。注意约束不可由其构造函数创建，而需要通过模型的*OPTVModel::AddConstr()* 或*OPTVModel::AddConstrs()* 接口添加约束至模型。此外，二次约束的相应接口为*OPTVModel::AddQConstr()*.

Get()

查询约束属性。

可用属性请参考*天筹属性*。

OPTVConstr::**Get**(*OPTVIntAttr* attr)

参数
attr –整数型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVConstr::**Get**(*OPTVDblAttr* attr)

参数
attr –双精度型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVConstr::**Get**(*OPTVStrAttr* attr)

参数
attr –字符串型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

Set()

设置约束属性，只有调用*OPTVModel::Update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVConstr::**Set**(*OPTVIntAttr* attr, int value)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVConstr::**Set**(*OPTVDblAttr* attr, double value)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVConstr::**Set**(*OPTVStrAttr* attr, const std::string &value)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

Index()

获得约束在模型中的索引值。

OPTVConstr::**Index**() const

返回
约束索引。

返回值

- **-1** –该约束在模型中不存在。
- **-2** –该约束已从模型中删除。
- **>=0** –该约束在模型中的索引/下标。

SameAs()

查询两约束是否相同。

OPTVConstr::**SameAs**(const *OPTVConstr* &c2) const

参数
c2 –需要与当前约束对比的约束。

返回
约束是否相同。

返回值

- **true** –约束相同。
- **false** –约束不同。

二次表达式

class **OPTVQuadExpr**

OPTVQuadExpr 类用于构建二次约束和目标函数。此类型的对象由变量和常数项构造而成。二次表达式由三种类型的项组成：二次项、线性项和常数项。二次项存储在包含第一个变量、第二个变量和系数的容器中。线性分量通过内部线性表达式对象进行访问。二次表达式中的索引严格指向二次分量，线性表达式则被独立处理。

如下例所示的二次表达式 `1*x*x + 2*x*y + 3*x*y + 4*y*y + 5*x + 6*y + 7*x` 的描述如下：

索引	系数	变量 1	变量 2
0	1	x	x
1	2	x	y
2	3	x	y
3	4	y	y

要访问同一表达式的线性分量，请使用*OPTVQuadExpr::GetLinExpr()* 接口。该接口将返回一个*OPTVLinExpr* 对象，代表该表达式的线性部分。该线性表达式可以正常访问，即：

索引	系数	变量
0	5	x
1	6	y
2	7	x

AddConstant()

向二次表达式增加常数项。

OPTVQuadExpr::**AddConstant**(double c)

此函数向当前二次表达式添加常数项，但不是覆盖。

参数

c –要添加的常数。

返回

无。

AddTerm()

在二次表达式的末尾添加一个线性项。

OPTVQuadExpr::**AddTerm**(double coeff, const *OPTVVar* &var)

此函数会将 coeff * var 项添加到表达式末尾。

参数

- **coeff** –该函数向当前二次表达式后添加 coeff * var 项。
- **var** –变量对象。

返回

无。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –需添加的变量 var 为空。

向当前二次表达式后添加一个二次性项。

OPTVQuadExpr::**AddTerm**(double coeff, const *OPTVVar* &var1, const *OPTVVar* &var2)

该函数向当前二次表达式后添加 coeff * var1 * var2 项。

参数

- **coeff** –该函数向当前二次表达式后添加 coeff * var 项。
- **var1** –该项的第一个变量。
- **var2** –该项的第二个变量。

返回

无。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –需添加的项中变量 var1 或 var2 为空。

AddTerms()

向当前二次表达式后添加多个线性项。

OPTVQuadExpr::**AddTerms**(const double *coeff, const *OPTVVar* *var, int cnt)

该函数向当前二次表达式后添加 $\text{coeff}_i * \text{var}_i, \forall i \in [0, \text{cnt}]$.

参数

- **coeff** –C-风格的系数序列。
- **var** –C-风格的变量序列。
- **cnt** –添加项的个数。

返回

无。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –参数 `var` 或 `coeff` 为空指针，或 `cnt` 取负值。

向当前二次表达式后添加多个二次性项。

OPTVQuadExpr::**AddTerms**(const double *coeff, const *OPTVVar* *var1, const *OPTVVar* *var2, int cnt)

该函数向当前二次表达式后添加 $\text{coeff}_i * \text{var1}_i * \text{var2}_i, \forall i \in [0, \text{cnt}]$.

参数

- **coeff** –C-风格的系数序列。
- **var1** –C-风格的变量 1 序列。
- **var2** –C-风格的变量 2 序列。
- **cnt** –添加项的个数。

返回

无。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –参数 `var1`, `var2` 或 `coeff` 为空指针，或 `cnt` 取负值。

Clear()

清空该表达式。

OPTVQuadExpr::**Clear**()

该函数清空该表达式涉及的所有存储容器，*OPTVQuadExpr::Size()* 将返回 `0`. 同时，该表达式也清空相应的线性表达式部分，重置常数项为 `0`.

返回

无。

GetCoeff()

据指定索引值获得表达式中对应二次项的系数。

OPTVQuadExpr::**GetCoeff**(int i) const

参数
i –表达式中该术语的索引。

返回
对应的二次项系数。

抛出
OPTVErrCode::*DATA_NOT_AVAILABLE* –若 i $\notin [0, \texttt{this->Size()}]$。

GetConstant()

获得当前表达式中的常数项。

OPTVQuadExpr::**GetConstant**() const

返回
表达式中的常量数值。

GetLinExpr()

获取二次表达式的线性部分。

OPTVQuadExpr::**GetLinExpr**() const

返回
二次表达式的线性部分。

GetValue()

获得该二次表达式的取值。

OPTVQuadExpr::**GetValue**() const

返回
对应原始解的表达式的取值，即该表达式的变量值与系数的乘积之和。

抛出
OPTVErrCode::*DATA_NOT_AVAILABLE* –当前变量取值不存在。

GetVar1()

根据指定索引值获得表达式中的对应二次项的第一个变量。

OPTVQuadExpr::**GetVar1**(int i) const

该函数返回表达式中的对应二次项的第一个变量，如表达式 `x*x + x*y + y*y` 的描述如下：

索引	变量 1	变量 2
0	x	x
1	x	y
2	y	y

参数

i –指定的二次项对应的索引值。

返回

对应二次项的第一个变量。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –若 i $\notin$ [0, this->Size()]。

GetVar2()

根据指定索引值获得表达式中的对应二次项的第二个变量。请参考*OPTVQuadExpr::GetVar1()* 和示例。

OPTVQuadExpr::**GetVar2**(int i) const

该函数返回表达式中的对应二次项的第二个变量。

参数

i –指定的二次项对应的索引值。

返回

对应二次项的第二个变量。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –若 i $\notin$ [0, this->Size()]。

Remove()

从二次表达式中删除项。

OPTVQuadExpr::**Remove**(int i)

根据指定索引值从二次表达式中删除二次项，但不可通过此函数删除线性项的任何部分。

参数

i –指定的索引值。

返回

无。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –若 i $\notin$ [0, this->Size()]。

从表达式中删除变量。

OPTVQuadExpr::**Remove**(const *OPTVVar* &v)

从二次表达式中（包括线性部分）删除指定变量涉及的所有项。

参数

v –指定要删除的变量。

返回

是否成功删除任何项。

返回值

- **true** –成功删除任何项。
- **false** –未删除任何项。

抛出

OPTVErrCode::*NOT_IN_MODEL* –成功删除与指定变量有关的项。

SameAs()

查询两二次表达式是否相同。

OPTVQuadExpr::**SameAs**(const *OPTVQuadExpr* &e2) const

参数

e2 –需要与当前二次表达式对比的线性表达式。

返回

二次表达式是否相同。

返回值

- **true** –二次表达式相同。
- **false** –二次表达式不同。

Size()

获得该表达式的规模。

OPTVQuadExpr::**Size**() const

该数值对应当前二次表达式中的 二**次**项的个数，不论重复与否。例如，表达式 `x*x + y*y + y` 对应 `Size = 2`，而表达式 `x*x + x*x + y*y + y` 对应 `Size = 3`。

返回

该二次表达式的规模。

二次约束

class **OPTVQConstr**

二次约束类用于访问模型中的二次约束。注意二次约束也不可以由其构造函数直接创建，而是通过接口*OPTVModel::AddQConstr()* 将其添加到模型。

警告

当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的，否则会抛出*OPTVErrCode::MATRIX_NOT_PSD* 异常。

Get()

查询二次约束属性。

可用属性请参考*天筹属性*。

OPTVQConstr::**Get**(*OPTVIntAttr* attr) const

参数
attr –整数型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVQConstr::**Get**(*OPTVCharAttr* attr) const

参数
attr –字符型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVQConstr::**Get**(*OPTVDblAttr* attr) const

参数
attr –双精度型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVQConstr::**Get**(*OPTVStrAttr* attr) const

参数
attr –字符串型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

Set()

设置二次约束属性，只有调用*OPTVModel::Update()*后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVQConstr::**Set**(*OPTVIntAttr* attr, int value)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回

无。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVQConstr::**Set**(*OPTVCharAttr* attr, char value)

参数

- **attr** –字符型属性。
- **value** –要设置的值。

返回

无。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVQConstr::**Set**(*OPTVDblAttr* attr, double value)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回

无。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVQConstr::**Set**(*OPTVStrAttr* attr, const std::string &value)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回

无。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

Index()

获取二次约束在模型中的索引值。

OPTVQConstr::**Index**() const

返回
二次约束索引。

返回值

- **-1** –该二次约束在模型中不存在。
- **-2** –该二次约束已从模型中删除。
- **>=0** –该二次约束在模型中的索引/下标。

SameAs()

查询两二次约束是否相同。

OPTVQConstr::**SameAs**(const *OPTVQConstr* &c2) const

参数
c2 –需要与当前二次约束对比的二次约束。

返回
二次约束是否相同。

返回值

- **true** –二次约束相同。
- **false** –二次约束不同。

仿射表达式

class **OPTVAffineExpr**

OPTVAffineExpr 类用于封装仿射变换 $Fx + g$，是构建锥约束的基础。一个*OPTVAffineExpr* 对象包含以下内容：

- **线性系数**：稀疏矩阵 F ，采用坐标（COO）格式存储非零系数及其对应的变量索引。通过逐行添加线性表达式来构建仿射表达式（COO 存储为内部实现，用户通过 SDK 对象提供数据）。
- **模型变量**：稀疏向量 x 。用户可以通过添加线性表达式 对象，提供所需的变量。
- **常数项**：用于平移线性组合的偏移向量 g。

在创建*OPTVCone* 对象时，*OPTVAffineExpr* 提供了仿射变换的数学结构，锥类型参数则指定了该表达式必须满足的几何约束 C 。这种设计将几何约束与数学表达式分离，使得用户能够基于简单的仿射单元，灵活地组合构建复杂的锥约束。

有关锥的详细介绍可见锥约束。

AddRow()

向仿射表达式中添加单行数据。

OPTVAffineExpr::**AddRow**(const *OPTVLinExpr* &expr)

参数
expr –添加到仿射表达式中的线性表达式对象。

返回
无。

AddRows()

向仿射表达式中添加多行数据。

OPTVAffineExpr::**AddRows**(const *OPTVLinExpr* *expr, int count)

参数

- **expr** –添加到仿射表达式中的线性表达式数组。每个线性表达式对应仿射表达式中的一行。
- **count** –添加的线性表达式个数（线性表达式数组的长度）。

返回
无。

抛出
OPTVErrCode::*INVALID_ARGUMENT* –count 为负数。

GetColumnIndices()

该函数会返回一个向量。向量中的每个值对应仿射表达式中的一个非零元素的列索引。仿射表达式在内部以COO（坐标）格式存储，其中每个非零元素由其行索引、列索引及其值共同标识。

OPTVAffineExpr::**GetColumnIndices**() const

返回
一个整数向量，其每个元素对应一个非零元素的列索引。该向量中元素的顺序与*OPTVAffineExpr::GetRowIndices()* 以及*OPTVAffineExpr::GetValues()* 返回结果的顺序保持一致。

GetOffset()

OPTVAffineExpr::**GetOffset**(int index) const

参数
index –查询偏移量的行索引。

返回
一个双精度浮点数，表示索引为 index 的行的偏移量。

抛出
std::out_of_range –index 的值大于仿射表达式中的行数。

GetOffsets()

OPTVAffineExpr::**GetOffsets**() const

返回
一个双精度浮点数的向量，每个元素对应仿射表达式中某一行的偏移量。

GetRowIndices()

该函数返回一个向量。向量中的每个元素对应一个非零元素的行索引。仿射表达式在内部以 COO（坐标）格式存储，其中每个非零元素由其行索引、列索引及其值共同标识。

OPTVAffineExpr::**GetRowIndices**() const

返回
一个整数向量，其每个元素代表一个非零元素的行索引。向量中的元素顺序与*OPTVAffineExpr::GetColumnIndices()* 及其*OPTVAffineExpr::GetValues()* 的返回结果的顺序一致。

GetValues()

该函数返回一个向量。向量中的每个元素对应仿射表达式中一个非零元素的值。非零元素的位置由*OPTVAffineExpr::GetRowIndices()* 与*OPTVAffineExpr::GetColumnIndices()* 返回的行索引以及列索引确定。

OPTVAffineExpr::**GetValues**() const

返回
一个双精度浮点数向量，其每个元素对应一个非零元素的值。向量中元素的位置与*OPTVAffineExpr::GetRowIndices()* 以及*OPTVAffineExpr::GetColumnIndices()* 返回结果的顺序一致。

SetOffset()

设置指定行的偏移量。

OPTVAffineExpr::**SetOffset**(int index, double value)

参数

- **index** –需要设置偏移量的行的索引。
- **value** –偏移量的值。

抛出
`std::out_of_range` –`index` 的值大于仿射表达式中的行数。

Size()

查询仿射表达式中元素的数量。

OPTVAffineExpr::**Size**() const

函数的返回值为仿射表达式中元素的个数，重复的元素会被多次统计。例如，若同一线性表达式被多次添加，仿射表达式中的元素数量将随之增加。

返回
仿射表达式中的元素数量。

锥约束

class **OPTVCone**

天筹求解器（OptVerse）的锥约束对象。用于访问模型中锥约束的相关信息。推荐使用*OPTVModel::AddCone()* 接口创建锥对象，并向模型中添加锥约束，不建议使用构造函数直接创建锥对象。

锥约束在数学上定义为：

$$Fx + g \in C$$

其中：

- F 是一个 $m \times n$ 的系数矩阵，用于对优化变量进行线性转换，
- x 是一个 n 维的优化变量向量，
- g 是一个 m 维的常数偏移向量（即右端项），
- C 是约束仿射表达式取值范围的锥。
- 表达式 $Fx + g$ 生成一个属于锥 C 的 m 维向量。

矩阵 F 采用坐标（COO）稀疏格式存储，以提高内存效率，是一个仅包含非零元素的行索引、列索引、值的三元组。用户可以按行向仿射表达式中添加线性表达式，将表达式 $Fx + g$ 构建为仿射表达式。

天筹求解器（OptVerse）支持以下锥约束类型：

- 二阶锥
- 旋转二阶锥

仿射表达式以及锥约束类型的详细介绍请见仿射表达式与锥类型。

Get()

查询锥约束某一属性的值。

OPTVCone::**Get**(*OPTVStrAttr* attr)

参数
attr –字符串型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

GetType()

获取锥约束的类型。

有关锥约束类型的详细信息请见*OPTVConeType*。

OPTVCone::**GetType**() const

返回
锥约束的类型。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –锥约束已从模型中移除。

Index()

返回锥线性约束在模型中的索引。

OPTVCone::**Index**() const

返回
锥约束的索引。

返回值

- **-1** –锥约束不在任何模型中。
- **-2** –锥约束已从模型中移除。
- **>=0** –锥线性约束在模型中的索引。

SameAs()

判断两个锥约束是否为同一个对象。

OPTVCone::**SameAs**(const *OPTVCone* &c2) const

参数
c2 –用于比对是否相同的锥约束。

返回
锥约束是否相同。

返回值

- **true** –锥约束相同
- **false** –锥约束不同

Set()

设置锥约束属性的值。在调用*OPTVModel::Update()* 之前，该设置不会在模型内部生效。

可用属性请参考*天筹属性*。

OPTVCone::**Set**(*OPTVStrAttr* attr, const std::string &value)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回

无。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

SOS 约束

class **OPTVSOS**

天筹求解器的 SOS 约束对象，用以从模型中获取 SOS 约束数据。这里，用户需要通过*OPTVModel::AddSOS()* 接口向模型中添加 SOS 约束，而非借助于构造函数。

特殊顺序集，special ordered set (SOS)，是指一组有序集合里，至多有一个非零值（SOS1 型）或至多有两个非零值（SOS2 型）。SOS 约束是对一组变量的取值进行限制的特殊约束，可以额外地指定模型中的整数条件。这类约束单独列出来，可以加快线性规划的求解速度。

具体地，天筹求解器的支持下列两种 SOS 约束：

- SOS 类型 1 要求指定的一组变量至多有一个变量可取非零值。
- SOS 类型 2 指定的一组变量至多有两个变量可取非零值，且取非零值的变量顺序要求相邻。

Get()

查询 SOS 约束的属性。

可用属性请参考*天筹属性*。

OPTVSOS::**Get**(*OPTVIntAttr* attr)

参数

attr –整数型属性。

返回

属性值。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVSOS::**Get**(*OPTVStrAttr* attr)

参数
attr –字符串型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

Set()

设置 SOS 约束属性，只有调用*OPTVModel::Update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVSOS::**Set**(*OPTVIntAttr* attr, int value)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVSOS::**Set**(*OPTVStrAttr* attr, const std::string &value)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

Index()

获取 SOS 约束在模型中的索引值。

OPTVSOS::**Index**() const

返回
SOS 约束索引。

返回值

- **-1** –该 SOS 约束在模型中不存在。
- **-2** –该 SOS 约束已从模型中删除。
- **>=0** –该 SOS 约束在模型中的索引/下标。

SameAs()

查询两 SOS 约束是否相同。

OPTVSOS::**SameAs**(const *OPTVSOS* &c2) const

参数
c2 –需要与当前 SOS 约束对比的 SOS 约束。

返回
SOS 约束是否相同。

返回值

- **true** –SOS 约束相同。
- **false** –SOS 约束不同。

一般约束

class **OPTVGenConstr**

天筹求解器一般约束，用于从模型中获取一般约束数据。特别地。一般约束需要通过 OPTVModel::AddGenConstrXxx 向模型中添加约束而构造，而不是借助于任何构造函数。

表 1: 天筹求解器支持的一般约束类型

约束	类型	接口	描述
And	*OPTV_GENCONSTR_*	*OPTVModel::AddGenCons*	只有所有变量非零时，结果才非零。
Or	*OPTV_GENCONSTR_*	*OPTVModel::AddGenCons*	只要有一个变量非零，结果就非零。
Indicator	*OPTV_GENCONSTR_*	*OPTVModel::AddGenCons*	返回值为布尔型，当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0。

Get()

查询一般约束属性。

可用属性请参考*天筹属性*。

OPTVGenConstr::**Get**(*OPTVIntAttr* attr)

参数
attr –整数型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVGenConstr::**Get**(*OPTVDblAttr* attr)

参数
attr –双精度型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVGenConstr::**Get**(*OPTVStrAttr* attr)

参数
attr –字符串型属性。

返回
属性值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

Set()

设置一般约束属性，只有调用*OPTVModel::Update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVGenConstr::**Set**(*OPTVDblAttr* attr, double value)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

OPTVGenConstr::**Set**(*OPTVStrAttr* attr, const std::string &value)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知约束属性。

Index()

获得一般约束在模型中的索引值。

OPTVGenConstr::**Index**() const

返回
约束索引。

返回值

- **-1** –该约束在模型中不存在。
- **-2** –该约束已从模型中删除。
- **>=0** –该约束在模型中的索引/下标。

SameAs()

查询两个一般约束是否为同一对象。

OPTVGenConstr::**SameAs**(const *OPTVGenConstr* &c2) const

参数
c2 –需要与当前约束对比的约束。

返回
约束是否相同。

返回值

- **true** –约束相同。
- **false** –约束不同。

模型

class **OPTVModel**

OptVerse 优化模型对象。

`OPTVModel` 类是 OptVerse 中用于构建和求解优化模型的核心对象。它存储了所有变量、约束条件、目标函数及求解器设置。

模型具有 ** 惰性更新 ** 行为：对模型所做的更改（如添加变量、约束、修改属性等）不会立即反映到底层求解器中。在优化之前，调用*OPTVModel::Update()* 将模型状态与求解器同步。

典型模型生命周期:

1. 使用其中一个构造函数创建一个*OPTVModel*
2. 使用*OPTVModel::AddVar()* 或*OPTVModel::AddVars()* 来添加变量
3. 使用*OPTVModel::AddConstr()*、*OPTVModel::AddQConstr()* 或专门的约束方法来添加约束
4. 使用*OPTVModel::SetObjective()*（或通过*OPTVModel::AddVar()* / *OPTVModel::AddVars()* 参数）设置目标函数
5. （可选）直接在模型上配置求解器参数
6. 调用*OPTVModel::Update()* 进行同步
7. 调用*OPTVModel::Optimize()* 进行求解

有关典型模型构建示例，请参阅快速开始。

OPTVModel()

explicit *OPTVModel*::**OPTVModel**(const *OPTVEnv* *env)

创建一个与给定环境关联的空模型。

参数

env –指向天筹环境的指针。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –env 为空。

---

explicit *OPTVModel*::**OPTVModel**(const *OPTVEnv* &env)

创建一个与给定环境关联的空模型。

参数

env –对天筹环境的引用。

OPTVModel::**OPTVModel**(const *OPTVEnv* *env, const std::string &fileName)

通过读取文件来创建模型。

参数

- **env** –指向天筹环境的指针。
- **fileName** –模型文件的路径（MPS、LP 等）。

抛出

- *OPTVErrCode*::*INVALID_ARGUMENT* –env 为空。
- *OPTVException* –无法从文件中读取模型。

OPTVModel::**OPTVModel**(const *OPTVEnv* &env, const std::string &fileName)

通过读取文件来创建模型。

参数

- **env** –对天筹环境的引用。
- **fileName** –模型文件的路径（MPS、LP 等）。

抛出

OPTVException –无法从文件中读取模型。

OPTVModel::**OPTVModel**(const *OPTVEnv* *env, const std::string &content, const std::string &format)

通过字符串读取来创建一个模型。

参数

- **env** –指向天筹环境的指针。
- **content** –将模型内容表示为字符串。
- **format** –模型格式（例如：”*mps*”、”*lp*”）。

抛出

- *OPTVErrCode*::*INVALID_ARGUMENT* –env 为空。
- *OPTVException* –从字符串读取模型失败。

OPTVModel::**OPTVModel**(const *OPTVEnv* &env, const std::string &content, const std::string &format)

通过字符串读取来创建一个模型。

参数

- **env** –对天筹环境的引用。
- **content** –将模型内容表示为字符串。
- **format** –模型格式（例如：”*mps*”、”*lp*”）。

抛出

OPTVException –从字符串读取模型失败。

AddCone()

向模型中添加单个锥约束。

OPTVModel::**AddCone**(const *OPTVAffineExpr* &affine, *OPTVConeType* type, const std::string &name = "")

该函数将向模型中添加一条锥约束。

参数

- **affine** –锥约束的仿射表达式 $Fx + g$。
- **type** –锥约束类型（如，二阶锥、旋转二阶锥）。
- **name** –锥约束名称。

返回

添加到模型中的锥约束对象。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –type 不是*OPTVConeType::SOC* 或者*OPTVConeType::RSOC* 。仿射表达式为空。type 为*OPTVConeType::RSOC* ，且仿射表达式中的元素个数小于 *3* 。

AddConstr()

向模型中添加单个线性约束。

OPTVModel::**AddConstr**(const *OPTVLinExpr* &expr, double lhs, double rhs, const std::string &name = "")

该函数向模型中添加一个线性约束。当需要一次性添加大量约束时，建议使用批量模式（*OPTVModel::AddConstrs()*），虽然大多数情况下，效率差别不大。

参数

- **expr** –约束的线性表达式部分。
- **lhs** –约束的下界。
- **rhs** –约束的上界。
- **name** –约束名称。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddConstrs()

向模型中添加多个线性约束。

OPTVModel::**AddConstrs**(const *OPTVLinExpr* *expr, const double *lhs, const double *rhs, const std::string *names, int count)

该函数一次性向模型中添加多个线性约束，尤其适用于添加大量约束的情形。

参数

- **expr** –约束的线性表达式序列。
- **lhs** –约束的左端值序列。
- **rhs** –约束的右端值序列。
- **names** –约束的名字序列。
- **count** –添加约束的个数。

返回

添加到模型中的约束对象向量。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddQConstr()

向模型中添加单个二次约束。

OPTVModel::**AddQConstr**(const *OPTVQuadExpr* &expr, double lhs, double rhs, const std::string &name = "")

该函数向模型中添加一个二次约束。

参数

- **expr** –约束的二次表达式部分。
- **lhs** –约束的下界。
- **rhs** –约束的上界。
- **name** –约束名称。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddGenConstrAnd()

向模型中添加单个 AND 一般约束。

OPTVModel::**AddGenConstrAnd**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, const std::string &name = "")

该函数向模型中添加单个 AND 一般约束：只有所有变量非零时，结果才非零。

参数

- **resultant** –约束的布尔型返回值。
- **vars** –所有参与检验的变量为真时，运算结果为真。
- **len** –vars 序列长度。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddGenConstrOr()

向模型中添加单个 OR 一般约束。

OPTVModel::**AddGenConstrOr**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, const std::string &name = "")

该函数向模型中添加单个 OR 一般约束：只要有一个变量非零，结果就非零。

参数

- **resultant** –约束的布尔型返回值。

- **vars** –参与检验的变量至少有一个为真时，运算结果为真。
- **len** –vars 序列长度。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddGenConstrIndicator()

向模型中添加单个 INDICATOR 一般约束。

OPTVModel::**AddGenConstrIndicator** (const *OPTVVar* &binVar, int binVal, const *OPTVLinExpr* &expr, char sense, double rhs, const std::string &name = "")

该函数向模型中添加单个 INDICATOR 一般约束：当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0.

参数

- **binVar** –二进制指示变量。
- **binVal** –要求线性约束必须满足的二进制指示变量的值 (0 or 1).
- **expr** –参与检验的线性约束表达式。
- **sense** – 线性约束的含义，如*OPTV_SENSE_LESS*，*OPTV_SENSE_EQUAL*，或*OPTV_SENSE_GREATER*.
- **rhs** –参与检验的线性约束右端值。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddGenConstrAbs()

向模型中添加单个 ABS 一般约束：$r = \text{abs}(x)$。此约束条件保证因变量 r 的值为自变量 x 的绝对值，即 $r = |x|$, 其中 $r \geq 0$。

形如 $r = \text{abs}(x)$ 的绝对值约束要求变量 r 等于变量 x 的绝对值。该约束施加了关系 $r = |x|$，其中 $r \geq 0$。

OPTVModel::**AddGenConstrAbs** (const *OPTVVar* &resultant, const *OPTVVar* &inputVar, const std::string &name = "")

结果变量，该变量的值为另一变量的绝对值。

参数

- **resultant** –该变量的绝对值将被结果变量采用。
- **inputVar** –添加到模型的一般约束对象。

- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddGenConstrNorm()

向模型中添加一个 NORM 约束。

向 “NORM“ 约束定义为：$r = \text{norm}\{x_1, \ldots, x_n\}$，其中因变量 r 表示向量 $x_1, \ldots, x_n$ 的范数。

OPTVModel::**AddGenConstrNorm**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, int normType, const std::string &name = "")

该函数向模型中添加单个 NORM 一般约束。

参数

- **resultant** –结果变量，其值为参数向量的范数。
- **vars** –用于计算向量范数的变量。
- **len** –vars 序列长度。
- **normType** –范数类型：-1 为 L^∞ 范数，0 为 L^0 范数，1 为 L^1 范数。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddGenConstrMax()

向模型中添加单个 MAX 约束：$r = \max\{x_1, \ldots, x_n\}$。

形式为 $r = \max\{x_1, \ldots, x_n\}$ 的 MAX 约束确保变量 r 等于参数向量 $x_1, \ldots, x_n$ 中的最大值。

OPTVModel::**AddGenConstrMax**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, const std::string &name = "", double constant = -OPTV_INF)

该函数向模型中添加单个 MAX 约束。

参数

- **resultant** –从中取最大值的一组变量。
- **vars** –变量向量，该向量的最大值会被结果变量采用。
- **len** –vars 序列长度。
- **name** –约束名称。
- **constant** –[**可选**] 参与 MAX 操作的常量。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddGenConstrMin()

向模型中添加单个 MIN 约束：$r = \min\{x_1, \ldots, x_n\}$ 。

形式为 $r = \min\{x_1, \ldots, x_n\}$ 的 MIN 约束确保变量 r 等于参数向量 $x_1, \ldots, x_n$ 中的最小值。

OPTVModel::**AddGenConstrMin**(const *OPTVVar* &resultant, const *OPTVVar* *vars, int len, const std::string &name = "", double constant = OPTV_INF)

该函数向模型中添加单个 MIN 约束。

参数

- **resultant** –从中取最小值的一组变量。
- **vars** –变量向量，该向量的最小值会被结果变量采用。
- **len** –vars 序列长度。
- **name** –约束名称。
- **constant** –[**可选**] 参与 MIN 操作的常量。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddSOS()

向模型中添加单个 SOS 约束。

OPTVModel::**AddSOS**(const *OPTVVar* *vars, const double *weights, int len, int type, const std::string &name = "")

参数

- **vars** –SOS 约束所涉及变量的 C-风格序列。
- **weights** –SOS 的变量权重的 C-风格序列。
- **len** –vars 和 weights 的序列长度。
- **type** –SOS 类型，取值为 1 或 2。
- **name** –SOS 约束的名字。

返回

添加到模型的 SOS 约束对象。

抛出

- *OPTVErrCode*::*INVALID_ARGUMENT* –type 取值不是 1 或 2。
- *OPTVErrCode*::*NOT_IN_MODEL* –vars 序列中的某个变量不属于当前模型。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –当前模型存在无效下标，比如模型关联的环境发生溢出时，执行本操作有可能抛出此错误。

AddVar()

向模型中添加单个变量。

OPTVModel::**AddVar**(double lb, double ub, double obj, char type, const std::string &name = "")

该函数向模型中添加一个变量。当需要一次性添加大量变量时，建议使用批量模式（*OPTVModel::AddVars()*），虽然大多数情况下，效率差别不大。

参数

- **lb** –变量的下界值。
- **ub** –变量的上界值。
- **obj** –变量在目标函数中的（线性）系数。
- **type** –变量类型，取值包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY* 。
- **name** –变量名称。

返回

添加到模型中的变量对象。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –未知变量类型，name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

OPTVModel::**AddVar**(double lb, double ub, double obj, char type, const *OPTVColumn* &col, const std::string &name = "")

该函数向模型中添加一个变量，并可以设置现有约束中的相应系数。当需要一次性添加大量变量时，建议使用批量模式（*OPTVModel::AddVars()*），虽然大多数情况下，效率差别不大。

参数

- **lb** –变量的下界值。
- **ub** –变量的上界值。
- **obj** –变量在目标函数中的（线性）系数。
- **type** –变量类型，取值包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY* 。
- **col** –设置约束系数的列对象。
- **name** –变量名称。

返回

添加到模型中的变量对象。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –未知变量类型，name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddVars()

向模型中添加多个变量。

OPTVModel::**AddVars**(const double *lb, const double *ub, const double *obj, const char *type, const std::string *names, int count)

该函数一次性向模型中添加多个变量，尤其适用于添加大量变量的情形。

参数

- **lb** –变量的下界序列。
- **ub** –变量的上界序列。
- **obj** –变量序列对应的（线性）目标系数。
- **type** –变量类型序列，包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY* 。
- **names** –变量名称序列。
- **count** –添加变量的个数。

返回

添加到模型的变量向量。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –type 包含未知变量类型，name 为空，数值参数中存在 NaN 或者 name 包含的变量名与模型中的其他组件重名。

OPTVModel::**AddVars**(const double *lb, const double *ub, const double *obj, const char *type, const *OPTVColumn* *col, const std::string *names, int count)

该函数向模型中添加多个变量，并可以设置现有约束中的相应系数。当需要一次性添加大量变量时，建议使用此功能，虽然大多数情况下，效率差别不大。

参数

- **lb** –变量的下界序列。
- **ub** –变量的上界序列。
- **obj** –变量序列对应的（线性）目标系数。
- **type** –变量类型序列，包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY* 。
- **col** –设置约束系数的列对象序列。
- **names** –变量名称序列。
- **count** –添加变量的个数。

返回

添加到模型的变量向量。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –type 包含未知变量类型，name 为空，数值参数中存在 NaN 或者 name 包含的变量名与模型中的其他组件重名。

CheckSolution()

检验指定的解是否可行。

OPTVModel::**CheckSolution**(const std::vector<double> &solution, double tol = 1e-8) const

该函数供用户检验指定的解是否满足当前模型的可行性要求，但 **不可以**检验内部存储的解。

参数

- **solution** –(列）解向量。
- **tol** –约束可行性整数性检查精度（**可选**，默认值为 10^{-8}）。

返回

解对模型是否可行。

返回值

- **false** –解对模型不可行。
- **true** –解对模型是可行的。

ComputeIIS()

计算不可约不一致子系统（IIS）。

有关 IIS 的细节请参考*IIS 快速入门*。

OPTVModel::**ComputeIIS**()

返回

IIS 是否计算成功。

返回值

- **false** –IIS 计算不成功。
- **true** –成功计算 IIS。

FeasRelax()

进行不可行问题的诊断及分析。

更多细节请参考*LP 不可行诊断及修复快速入门*。

OPTVModel::**FeasRelax**()

该函数进行不可行问题的诊断及分析，目前仅支持 L^1 范数。

返回

不可行问题修复的状态。

返回值

- **0** –当前问题是可行的。
- **-1** –不可行问题修复失败。
- **>0** –Phase 1 最小违背量的目标函数值。

GetCoef()

查询给定约束中指定变量的系数。

OPTVModel::**GetCoef**(const *OPTVConstr* &con, const *OPTVVar* &var) const

参数

- **con** –需要查询的约束。
- **var** –需要查询的变量。

返回

查询变量在该约束中的系数。

抛出

- *OPTVErrCode*::*INVALID_ARGUMENT* –约束或变量不存在。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –数据不存在，或约束的线性表达式部分不可用。

GetColumn()

获取指定变量对应的列对象。

OPTVModel::**GetColumn**(const *OPTVVar* &v) const

参数

v –需要查询的变量。

返回

指定变量对应的列对象。

抛出

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –该变量不存在于当前模型。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –变量索引值*Index* 超过了模型的变量总数。若当前模型需要执行更新，或者当前变量属于其它模型时，该错会被抛出。

GetCone()

获取指定位置的锥约束。

OPTVModel::**GetCone**(int i) const

参数

i –锥约束的索引。

返回

索引为 i 的锥约束。

抛出

OPTVErrCode::*INDEX_OUT_OF_RANGE* –索引值大于模型中锥约束的数量。

GetCones()

获取模型所有的锥约束。

OPTVModel::**GetCones**() const

返回
包含模型所有锥约束的向量。

GetConstrByName()

通过名称获取约束。

OPTVModel::**GetConstrByName**(const std::string &name) const

参数
name –需要访问的约束名称。

返回
名称匹配 name 的约束。

抛出
OPTVErrCode::*DATA_NOT_AVAILABLE* –给定的名称为空，或模型不存在命名为 name 的约束。

GetConstr()

根据指定索引值获得模型中对应的约束。

OPTVModel::**GetConstr**(int i) const

参数
i –线性约束在模型中的索引。

返回
索引为 i 的约束。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –指定的索引值超出了模型的约束总数。

此函数中的参数 *i* 是指全局下标，与约束的类型无关。例如，用户依次添加了一个线性约束和一个二次约束，那么该线性约束的索引值为 *0* 而二次约束的索引值为 *1*.

GetConstrs()

获得模型中的所有线性约束。

OPTVModel::**GetConstrs**() const

返回
包含模型中所有线性约束的向量。

GetGenConstr()

OPTVGenConstr *OPTVModel*::**GetGenConstr**(int type, int index) const

获取指定类型的单个通用约束。

参数

- **type** –通用约束类型（例如*OPTV_GENCONSTR_AND*、*OPTV_GENCONSTR_OR*）。
- **index** –要检索的通用约束的索引。

返回

所查询的通用约束。

Raises OPTVErrCode.INVALID_ARGUMENT

无效的约束类型。

Raises OPTVErrCode.INDEX_OUT_OF_RANGE

索引超出范围。

GetGenConstrs()

std::vector<*OPTVGenConstr*> *OPTVModel*::**GetGenConstrs**(int type) const

获取指定类型的所有常规约束。

参数

type –通用约束类型（例如*OPTV_GENCONSTR_AND*、*OPTV_GENCONSTR_OR*）。

返回

指定类型的所有通用约束的向量。

Raises OPTVErrCode.INVALID_ARGUMENT

无效的约束类型。

GetGenConstrAnd()

获得模型中特定的 AND 一般约束数据。

OPTVModel::**GetGenConstrAnd**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant,
std::vector<*OPTVVar*> &vars) const

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –参与检验该一般约束的所有变量的列表。

抛出

OPTVErrCode::*NOT_IN_MODEL* –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

GetGenConstrOr()

获得模型中特定的 OR 一般约束数据。

OPTVModel::**GetGenConstrOr**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars) const

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –参与检验该一般约束的所有变量的列表。

抛出

OPTVErrCode::*NOT_IN_MODEL* –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

GetGenConstrIndicator()

获得模型中特定的 INDICATOR 一般约束数据。

OPTVModel::**GetGenConstrIndicator**(const *OPTVGenConstr* &genConstr, *OPTVVar* &binVar, int &binVal, *OPTVLinExpr* &expr, char &sense, double &rhs) const

参数

- **genConstr** –需要查询的一般约束。
- **binVar** –二进制指示变量。
- **binVal** –要求线性约束必须满足的二进制指示变量的值 (0 or 1).
- **expr** –参与检验的线性约束表达式。
- **sense** – 线性约束的含义，如*OPTV_SENSE_LESS*，*OPTV_SENSE_EQUAL*，或*OPTV_SENSE_GREATER*.
- **rhs** –参与检验的线性约束右端值。

抛出

OPTVErrCode::*NOT_IN_MODEL* –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

GetGenConstrAbs()

获取模型中指定的 ABS 一般约束数据。

OPTVModel::**GetGenConstrAbs**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, *OPTVVar* &argument) const

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **argument** –该一般约束的参数变量。

抛出

OPTVErrCode::*NOT_IN_MODEL* –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

GetGenConstrNorm()

获得模型中指定的 NORM 一般约束数据。

OPTVModel::**GetGenConstrNorm**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars, int &normType) const

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –用于计算范数的变量列表。
- **normType** –范数类型（-1 为 L^{∞} 范数，0 为 L^{0} 范数，1 为 L^{1} 范数）。

抛出

OPTVErrCode::*NOT_IN_MODEL* –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

GetGenConstrMin()

获得模型中指定的 MIN 一般约束数据。

OPTVModel::**GetGenConstrMin**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars, double &constant)

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –参与检验该一般约束的所有变量的列表。
- **constant** –参与该一般约束的常量操作数。

抛出

OPTVErrCode::*NOT_IN_MODEL* –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

GetGenConstrMax()

获取模型中指定的 MAX 一般约束数据。

OPTVModel::**GetGenConstrMax**(const *OPTVGenConstr* &genConstr, *OPTVVar* &resultant, std::vector<*OPTVVar*> &vars, double &constant)

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。

- **vars** –参与检验该一般约束的所有变量的列表。
- **constant** –参与该一般约束的常量操作数。

抛出

OPTVErrCode::*NOT_IN_MODEL* –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

GetQConstr()

根据指定索引值获得模型中对应的二次约束。

OPTVModel::**GetQConstr**(int i) const

参数

i –线性约束在模型中的索引。

返回

索引为 i 的约束。

抛出

OPTVErrCode::*INDEX_OUT_OF_RANGE* –指定的索引值超出了模型的约束总数。

对应的二次约束。

GetQConstrs()

获得模型中的所有二次约束。

OPTVModel::**GetQConstrs**() const

返回

包含模型中所有二次约束的向量。

GetSOS()

根据索引值获取指定类型的 SOS 约束。

OPTVModel::**GetSOS**(int i, int type) const

参数

- **i** –指定 SOS 约束在模型中的索引值。
- **type** –SOS 类型，取值为 1 或 2。

返回

对应索引值 i 的指定类型的 SOS 约束。

抛出

- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –索引值 i 超出了模型的该类型的 SOS 约束总数。
- *OPTVErrCode*::*INVALID_ARGUMENT* –type 取值不是 1 或 2。

这里，参数 i 特指某一类型 SOS 约束集中的索引值，例如，如果用户先后添加了类型为 1 和 2 的两个 SOS 约束，那么两者的索引值都是 0.

GetSOSData()

OPTVModel::**GetSOSData**(const *OPTVSOS* &sos, std::vector<*OPTVVar*> &vars, std::vector<double> &weights) const

获取 SOS 约束的内部数据。

参数

- **sos** –当前访问的 SOS 约束对象。
- **vars** –变量序列的空向量。如果非空，该容器在本次函数调用中会被清空。
- **weights** –变量权重序列的空向量。如果非空，该容器在本次函数调用中会被清空。

返回

无。

抛出

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –SOS 约束是由默认构造函数生成的。
- *OPTVErrCode*::*NOT_IN_MODEL* –SOS 约束属于另外一个模型，或已经从本模型中被删除。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –约束下表越界、超出容器范围，可能需要对当前模型执行更新。

GetSOSs()

获取指定类型的所有 SOS 约束。

OPTVModel::**GetSOSs**(int type) const

参数

type –SOS 类型，取值为 `1` 或 `2`。

返回

包含模型中所有类型为 `type` 的 SOS 约束的向量。

抛出

- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –SOS 约束的索引值超出了模型的该类型的 SOS 约束总数，可能需要对当前模型执行更新。
- *OPTVErrCode*::*INVALID_ARGUMENT* –`type` 取值不是 `1` 或 `2`。

GetObjective() 获取线性目标函数

获取线性目标函数。

OPTVModel::**GetObjective**() const

返回

目标函数的线性表达式。

GetObjectiveN()

在多目标优化中，获取指定目标函数的线性表达式。

OPTVModel::**GetObjectiveN**(int objId) const

参数
objId –目标函数的索引。

返回
表示指定的目标函数的线性表达式对象。

抛出
OPTVErrCode::*VALUE_OUT_OF_RANGE* –多目标优化的目标函数的索引值大于模型中目标函数的数量。

GetQObjective()

获取二次目标函数。

OPTVModel::**GetQObjective**() const

返回
目标函数的二次表达式。

GetRow()

获取指定约束对应的线性函数表达式。

OPTVModel::**GetRow**(const *OPTVConstr* &c) const

参数
c –指定的约束对象。

返回
该约束对应的线性表达式。

抛出

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –该约束不存在于当前模型。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –当前约束的索引*Index* 超过了模型的约束总数。若当前模型需要执行更新，或者当前约束属于其它模型时，该错会被抛出。

Get()

参数

获取求解器的参数值。

可用参数请参考数学规划求解器参数。

OPTVModel::**Get**(*OPTVIntParam* param) const

参数
param –整数型参数。

返回
参数取值。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVModel::**Get**(*OPTVDblParam* param) const

参数
param –双精度型参数。

返回
参数取值。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVModel::**Get**(*OPTVStrParam* param) const

参数
param –字符串型参数。

返回
参数取值。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

属性

获取属性的取值。

可用属性请参考*天筹属性*。

OPTVModel::**Get**(*OPTVIntAttr* attr) const

参数
attr –整数型属性。

返回
属性取值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

OPTVModel::**Get**(*OPTVDblAttr* attr) const

参数
attr –双精度型属性。

返回
属性取值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

OPTVModel::**Get**(*OPTVLongAttr* attr) const

参数
attr –长整型属性。

返回
属性取值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

OPTVModel::**Get**(*OPTVStrAttr* attr) const

参数
attr –字符串型属性。

返回
属性取值。

抛出
OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

GetVarByName()

通过名称获取变量。

OPTVModel::**GetVarByName**(const std::string &name) const

参数
name –需要访问的变量名称。

返回
名称匹配 `name` 的变量。

抛出
OPTVErrCode::*DATA_NOT_AVAILABLE* –给定的名称为空，或模型不存在命名为 `name` 的变量。

GetVar()

根据指定索引值获得模型中对应的变量。

OPTVModel::**GetVar**(int i) const

参数
i –变量在模型中的索引。

返回
在模型中索引 `i` 的变量。

抛出

OPTVErrCode::*INDEX_OUT_OF_RANGE* –指定的索引值超出了模型的变量总数。

GetVars()

获得模型中的所有变量。

OPTVModel::**GetVars**() const

返回

包含模型中所有变量的向量。

Optimize()

对模型进行求解。

OPTVModel::**Optimize**()

返回

无。

抛出

OPTVErrCode::*MATRIX_NOT_PSD* –模型中包含二次目标函数或者二次约束，而该表达式的矩阵不满足正半定性质。当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的。

求解器采用哪种优化算法取决于求解器参数配置和模型属性，例如变量类型被用以自动识别 LP, QP, MIP 及网络流问题。

Presolve()

对模型进行预处理。

OPTVModel::**Presolve**()

该函数会应用预处理流程，返回一个新的*OPTVModel* 对象。原模型的状态被修改为 被预处理.

返回

预处理后的模型。

Read()

从文件中读取读取模型和基。

OPTVModel::**Read**(const std::string &fileName)

该函数供读取基信息，也可以读取模型文件，然而建议由构造函数读取文件中的模型，参考文件格式。

表 2: 支持读入的文件格式

.mps	.lp	.bas

表 3: 支持读入的压缩格式

.gz	.z	.Z

参数
fileName –C++ 字符串文件名。

返回
无。

抛出
OPTVErrCode::*ERROR_FILE_READ* –文件名或后缀为空，文件存在格式错误。

Remove()

从模型中删除变量。

OPTVModel::**Remove**(const *OPTVVar* &var)

该函数将变量从模型中删除，对应的索引会被相应地调整。

例如，若下列模型 `model` 构建时包含以下变量：

变量	索引
x0	0
x1	1
x2	2
x3	3

调用 `model.Remove(x0)` 后，相应变化为：

变量	索引
x1	0
x2	1
x3	2

参数
var –指定要删除的变量。

返回
无。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –模型不包含此变量。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –该变量属于其他模型，而不属于当前模型。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –变量的下标超出了模型的变量总数，可能需要执行模型更新。

从模型中删除约束。

OPTVModel::**Remove**(const *OPTVConstr* &constr)

该函数将约束从模型中删除，对应的索引会被相应地调整。

例如，若模型 `model` 创建时包含如下约束：

约束	索引
c0	0
c1	1
c2	2
c3	3

调用 `model.Remove(c0)` 后，相应变化为：

约束	索引
c1	0
c2	1
c3	2

参数

constr –指定要删除的约束。

返回

无。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –模型不包含此约束。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –该约束属于其他模型，而不属于当前模型。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –约束的下标超出了模型的变量总数，可能需要执行模型更新。

OPTVModel::**Remove**(const *OPTVGenConstr* &genConstr)

从模型中删除单个一般约束。执行后对应的相同类型的一般约束索引会被相应地调整。

例如，若下列模型 `model` 构建时包含以下一般约束：

约束	类型	索引
and0	AND	0
and1	AND	1
or0	OR	0
or1	OR	1

调用 `model.Remove(and0)` 后，相应变化为：

约束	类型	索引
and1	AND	0
or0	OR	0
or1	OR	1

参数
genConstr –指定要删除的一般约束。

返回
无。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –模型不包含此一般约束。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –该一般约束属于其他模型，而不属于当前模型。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –该一般约束的下标超出了模型的变量总数，可能需要执行模型更新。

OPTVModel::**Remove**(const *OPTVSOS* &sos)

从模型中删除单个 SOS 约束。与其他约束类型不同的是，删除该 SOS 约束后，相同类型的约束的索引**不会**改变。

例如，若模型 `model` 创建时包含如下约束：

约束	类型	索引
sos1_1	1	0
sos1_2	1	1
sos1_3	1	2
sos2_1	2	0

调用 `model.Remove(sos1_2)` 后，相应变化为：

约束	类型	索引
sos1_1	1	0
sos1_3	1	2
sos2_1	2	0

参数
sos –指定要删除的 SOS 约束。

返回
无。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –SOS 约束不属于任何模型。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –SOS 不属于当前模型。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –SOS 约束的索引值超出了模型的该类型的 SOS 约束总数，可能需要对当前模型执行更新。

OPTVModel::**Remove**(const *OPTVCone* &cone)

该函数从模型中移除一个锥约束。移除操作将导致后续锥约束的索引发生前移。

约束	类型	索引
c0	SOC	0
c1	RSOC	1
c2	SOC	2
c3	RSOC	3

调用 `model.Remove(c0)` 后，相应变化为：

约束	类型	索引
c1	RSOC	0
c2	SOC	1
c3	RSOC	2

参数

cone –需要移除的锥约束对象。

返回

无。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –锥约束不在任何模型中。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –锥约束属于另一个模型，不在当前模型中。
- *OPTVErrCode*::*INDEX_OUT_OF_RANGE* –锥约束的索引大于模型中锥约束的数量。添加锥约束后，没有将其更新到模型中，则可能发生此情况。

Relax()

进行线性规划 (LP) 松弛。

OPTVModel::**Relax**() const

该函数将原模型中的所有整数类型变量替换为*OPTV_CONTINUOUS* 变量，生成并返回一个新的*OPTVModel* 对象，对原模型不做任何修改。

返回

模型的 LP 松弛副本。

Reset()

将模型重置为 未求解成功状态。

OPTVModel::**Reset**(bool clearAll = false)

该函数会清除内部解的信息，默认情况下，包括解的值、LP 基和模型状态。`clearAll` 标志是否清除额外的模型属性值（包括 MIP 热启动值，MIP 分支优先度，LP 的禁用属性状态）。

参数

clearAll –清除额外信息（默认为 `false`）。

返回

无。

GetObjSA()

目标函数敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel::**GetObjSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

参数

- **down** –敏感度区间左端点。
- **up** –敏感度区间右端点。
- **list** –需要进行目标函数敏感度分析的变量索引集合。

返回

敏感度分析是否成功。

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

GetVarLbSA()

变量下界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel::**GetVarLbSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

参数

- **down** –下界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** –下界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** –需要进行变量下界敏感度分析的约束索引集合。

返回

敏感度分析是否成功。

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

GetVarUbSA()

变量上界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel::**GetVarUbSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

参数

- **down** –上界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** –上界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** –需要进行变量上界敏感度分析的约束索引集合。

返回

敏感度分析是否成功。

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

GetConstrLbSA()

约束左端值/下界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel::**GetConstrLbSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

参数

- **down** –下界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** –下界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** –需要进行左端值敏感度分析的约束索引集合。

返回

敏感度分析是否成功。

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

GetConstrUbSA()

约束右端值/上界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel::**GetConstrUbSA**(std::vector<double> &down, std::vector<double> &up, const std::vector<int> &list) const

参数

- **down** –上界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** –上界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** –需要进行右端值敏感度分析的约束索引集合。

返回

敏感度分析是否成功。

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

SetCallback()

OPTVModel::**SetCallback**(*OPTVCallback* *callback)

该函数将用户定义的回调函数与模型关联。根据回调类型（参见*OPTVCBType*），回调将在求解过程的不同阶段被触发。

参数

callback –指向用户自定义回调类的指针。

返回

无。

SetCoef()

设置给定约束中指定变量的系数。

OPTVModel::**SetCoef**(const *OPTVConstr* &con, const *OPTVVar* &var, double newValue)

参数

- **con** –需要修改的约束。
- **var** –需要修改系数的变量。
- **newValue** –新的系数值。

返回

无。

抛出

- *OPTVErrCode*::*INVALID_ARGUMENT* –约束或变量不存在，或数值参数中存在 NaN。

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –数据不存在，或约束的线性表达式部分不可用。

SetObjective()

设置线性目标函数。

OPTVModel::**SetObjective**(const *OPTVLinExpr* &expr, *OPTVSense* sense = *OPTVSense*::*MINIMIZE*)

该函数清除当前目标函数（相对于在其基础上进行叠加）。若未指定 sense 参数，求解器会采用默认的（*OPTVSense::MINIMIZE*）。

参数

- **expr** –需要设定的目标函数线性表达式。
- **sense** – **可选**，最小化或最大化，默认值为*OPTVSense::MINIMIZE*，参考*OPTVSense*.

返回

无。

设置二次目标函数。

OPTVModel::**SetObjective**(const *OPTVQuadExpr* &expr, *OPTVSense* sense = *OPTVSense*::*MINIMIZE*)

该函数清除当前目标函数（相对于在其基础上进行叠加）。若未指定 sense 参数，求解器会采用默认的（*OPTVSense::MINIMIZE*）。

参数

- **expr** –需要设定的目标函数二次表达式。
- **sense** – **可选**，最小化或最大化，默认值为*OPTVSense::MINIMIZE*，参考*OPTVSense*.

返回

无。

SetObjectiveN()

在多目标优化中，设置线性目标函数。

OPTVModel::**SetObjectiveN**(int objId, const *OPTVLinExpr* &expr, const std::string &name, int priority, double weight = 1.0, double absTolerance = 0.0, double relTolerance = 0.0)

当模型中有多个目标函数时，该函数会替换当前具有相同 Id 的目标函数（而不是进行累加），否则会向模型中添加一个新的目标函数。目标函数的优化方向由参数 weight 的符号决定。

参数

- **objId** –待修改目标函数的索引。
- **expr** –需要设定的目标函数表达式。
- **name** –目标函数的名称。
- **priority** –目标函数的优先级（高优先级的目标函数会被优先求解）。
- **weight** –[**可选**, 默认值 1.0] 目标函数的权重（用于确定目标函数的优化方向，以及处理多个目标函数拥有相同优先级的情况）。

- **absTolerance** –[**可选**, 默认值 0.0] 目标函数的绝对容差（决定在后续的优化过程中该目标函数允许的最大绝对偏差）。
- **relTolerance** –[**可选**, 默认值 0.0] 目标函数的相对容差（决定在后续优化过程中该目标函数允许的最大相对偏差）。

返回
无。

Set()

参数

设置求解器的参数取值，一旦模型被创建，该接口仅提供控制此模型专用参数的接口，直到调用*OPTVModel::Update()*方法后，新修改才会在内部生效。

可用参数请参考数学规划求解器参数。

OPTVModel::**Set**(*OPTVIntParam* param, int value)

参数

- **param** –整数型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVModel::**Set**(*OPTVDblParam* param, double value)

参数

- **param** –双精度型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVModel::**Set**(*OPTVStrParam* param, const std::string &value)

参数

- **param** –字符串型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

属性

设置求解器的属性值，同样，直到调用*OPTVModel::Update()* 方法后，新修改才会在内部生效。

可用属性请参考*天筹属性*。

OPTVModel::**Set**(*OPTVIntAttr* attr, int value)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回

无。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

OPTVModel::**Set**(*OPTVDblAttr* attr, double value)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回

无。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

OPTVModel::**Set**(*OPTVStrAttr* attr, const std::string &value)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回

无。

抛出

OPTVErrCode::*UNKNOWN_ATTRIBUTE* –未知属性。

Update()

执行模型更新。

OPTVModel::**Update**()

该函数会进行模型更新，或 lazy 构建（如必要）。只有调用该函数 **后**，模型的更新才会生效。SDK 构建的模型需要调用此函数来保证内部模型的即时性。例如，用户调用*OPTVModel::Optimize()* 前 **不需要**调用此函数，因为该函数在内部已被默认执行。

返回

无。

WriteIIS()

将不可约不一致子系统（IIS）写入文件（.ilp 格式）。

有关 IIS 的细节请参考*IIS 快速入门*。

OPTVModel::**WriteIIS**(const std::string &fileName)

参数
fileName –写出的文件名，必须包含 .ilp 后缀（例如 iis.ilp）.

返回
无。

抛出
OPTVErrCode::*ERROR_FILE_WRITE* –不支持的文件格式，或者未计算出 IIS.

注解

此函数 **不会**计算 IIS，若 IIS 未被计算，本函数将抛出异常。

Write()

将模型或基写出到文件。

OPTVModel::**Write**(const std::string &fileName)

支持的文件格式请参考文件格式。

表 4: 支持写出的文件格式

.mps	.lp	.bas	.ilp

表 5: 支持写出的压缩格式

.gz	.z	.Z

参数
fileName –C++ 字符串文件名。

返回
无。

抛出
OPTVErrCode::*ERROR_FILE_WRITE* –文件名或后缀为空或错误。

临时约束

class **OPTVTempConstr**

天筹临时约束对象。OPTVTempConstr 类作为重载运算符形成的约束表达式中间结果的占位符。该类不包含任何成员函数。相反，表达式和变量类型（*OPTVVar* 、*OPTVLinExpr* 、*OPTVQuadExpr* ）的比较操作（==、<=、>=）会返回 OPTVTempConstr 对象。这些对象通常被约束添加方法立即使用，不适用于存储在用户自定义变量中。

OPTVTempConstr 允许用户创建多种不同类型的约束:

- **线性约束**：线性约束的形式为 Expr1 sense Expr2，其中 Expr1 与 Expr2 可以是*OPTVVar* 对象、*OPTVLinExpr* 对象或数值常量，而 sense 是以下比较操作符之一：==、<=、>=。例如，x + y <= 1 + z 与 x + y == 5 都是有效的线性约束。
- **二次约束**：二次约束的形式为 Expr1 sense Expr2，其中 Expr1 与 Expr2 可以是*OPTVVar* 、*OPTVLinExpr* 、*OPTVQuadExpr*，或者常量，而 sense 则是以下比较操作符的其中之一：==, <=, >=。例如，x*y <= 1 + z 与 x*x + y*y <= z*z 都是有效的二次约束。需要注意的是 Expr1 与 Expr2 至少有一个是*OPTVQuadExpr* ，否则该约束为线性约束。

可以参考以下示例:

```
model.AddConstr(x + y >= 1, "c0");

model.AddQConstr(x*x + y*y <= z, "c1");
```

回调

class **OPTVCallback**

天筹求解器的回调对象。

该类为抽象基类，用户可以从中派生出自己的回调子类。在求解器的求解过程中，*OPTVCallback::RunCallback()* 函数将会被定期调用。根据回调类型的不同，可以在求解过程中查询不同的数据。

用户可以通过*OPTVModel::SetCallback()* 函数将回调提供给指定的模型。可以设置多个回调。

下面的示例中，向模型提供了两个自定义的回调。一个回调用于查询解的信息，另一个则用于查询 MIP 的节点信息。

```
#include "optv_c++.h"
#include <iostream>

using namespace std;

class SDKMipSolutionCallback final : public OPTVCallback {
public:
    SDKMipSolutionCallback() : OPTVCallback(OPTVCBType::MIP_SOLUTION) {}
    ~SDKMipSolutionCallback() override = default;

    [[nodiscard]] int GetNTriggered() const { return nTriggered_; }

    OPTVVar varToWatch;
protected:
    void RunCallback() final
    {
        ++nTriggered_;
```

(下页继续)

(续上页)

```
        std::cout << "Callback 1: " << std::endl;
        cout << "Time: " << this->GetInfo(OPTVCBDblInfo::RUN_TIME);
        cout << ", dual bound: " << this->GetInfo(OPTVCBDblInfo::MIP_DUAL_BOUND);
        cout << ", new incumbent: " << this->GetInfo(OPTVCBDblInfo::MIP_INCUMBENT_
↪OBJ);
        cout << ", new incumbent sol @ i: " << this->GetSolution(varToWatch) << " ";
        cout << "type: " << static_cast<int>(this->GetType()) << endl;
    }


private:
    int nTriggered_{0};
};

class SDKMipNodeCallback final : public OPTVCallback {
public:
    SDKMipNodeCallback() : OPTVCallback(OPTVCBType::MIP_NODE) {}
    ~SDKMipNodeCallback() override = default;

    [[nodiscard]] int GetNTriggered() const { return nTriggered_; }

    OPTVVar varToWatch;
protected:
    void RunCallback() final
    {
        ++nTriggered_;
        std::cout << "Callback 2: " << std::endl;
        cout << "n solution: " << this->GetInfo(OPTVCBIntInfo::MIP_NUM_SOLUTION) << ",
↪ ";
        cout << "Time: " << this->GetInfo(OPTVCBDblInfo::RUN_TIME);
        this->SetSolution(varToWatch, 0);
        std::cout << "type: " << static_cast<int>(this->GetType()) << std::endl;

        this->Abort();
    }


private:
    int nTriggered_{0};
};

int main()
{
    OPTVEnv env();
    OPTVModel model(env, "cb_demo.mps");

    model.Update();
    OPTVVar x1 = model.GetVarByName("x1");

    SDKMipSolutionCallback sdkMipSolutionCallback;
    SDKMipNodeCallback sdkMipNodeCallback;
    sdkMipSolutionCallback.varToWatch = x1;
    sdkMipNodeCallback.varToWatch = x1;

    model.SetCallback(&sdkMipSolutionCallback);
    model.SetCallback(&sdkMipNodeCallback);
```

(下页继续)

(续上页)

```
    model.Optimize();

    return 0;
}
```

Abort()

中断求解进程。

OPTVCallback::**Abort**()

返回
无。

GetInfo()

OPTVCallback::**GetInfo**(*OPTVCBIntInfo* info) const

参数
info –需要查询的整型回调信息。

返回
查询到的数据。

抛出

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –当前的回调类型不支持查询该类型数据的信息。
- *OPTVErrCode*::*INVALID_ARGUMENT* –查询未知的回调数据类型。

OPTVCallback::**GetInfo**(*OPTVCBDblInfo* info) const

参数
info –需要查询的双精度型回调信息。

返回
查询到的数据。

抛出

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –当前的回调类型不支持查询该类型数据的信息。
- *OPTVErrCode*::*INVALID_ARGUMENT* –查询未知的回调数据类型。

OPTVCallback::**GetInfo**(*OPTVCBLongInfo* info) const

参数
info –需要查询的长整型回调信息。

返回
查询到的数据。

抛出

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –当前的回调类型不支持查询该类型数据的信息。
- *OPTVErrCode*::*INVALID_ARGUMENT* –查询未知的回调数据类型。

OPTVCallback::**GetInfo**(*OPTVCBStrInfo* info) const

参数
info –需要查询的字符串型回调信息。

返回
查询到的数据。

抛出

- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –当前的回调类型不支持查询该类型数据的信息。
- *OPTVErrCode*::*INVALID_ARGUMENT* –查询未知的回调数据类型。

GetNodeRel()

OPTVCallback::**GetNodeRel**(const *OPTVVar* &var)

仅在回调类型为*OPTVCBType::MIP_NODE* 时可以调用。

参数
var –期望获取松弛解的变量。

返回
变量 var 在当前节点松弛解的值。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –变量 var 不在当前模型中。如果变量 var 被删除，或者数据未刷新，可能会抛出该错误。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –无法获取节点松弛解的信息，或者回调类型不是*OPTVCBType::MIP_NODE*。

OPTVCallback::**GetNodeRel**()

仅在回调类型为*OPTVCBType::MIP_NODE* 时可以调用。

返回
包含当前节点所有变量松弛解的向量。

抛出
OPTVErrCode::*DATA_NOT_AVAILABLE* –无法获取节点松弛解的信息，或者回调类型不是*OPTVCBType::MIP_NODE*。

GetSolution()

OPTVCallback::**GetSolution**(const *OPTVVar* &var)

仅当回调类型为*OPTVCBType::MIP_SOLUTION*时可用。

参数

var –用于获取在当前候选解中的值的变量。

返回

变量 var 在当前候选解中的值。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –变量 var 不在当前模型中。如果变量 var 被删除，或者数据未刷新，可能会抛出该错误。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –无法获取候选解信息，或者回调类型不是*OPTVCBType::MIP_SOLUTION*.

OPTVCallback::**GetSolution**()

仅当回调类型为*OPTVCBType::MIP_SOLUTION*时可用。

返回

包含模型当前候选解信息的向量。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –无法获取候选解信息，或者回调类型不是*OPTVCBType::MIP_SOLUTION*.

GetType()

获取回调类型。详细信息请见*OPTVCBType*.

OPTVCallback::**GetType**() const

返回

回调类型。

RunCallback()

这是一个虚函数，在创建用户自定义的回调类时需要重写。该函数将在回调触发时执行，以便用户获取回调信息并执行其他自定义逻辑。

OPTVCallback::**RunCallback**()

返回

无。

SetSolution()

OPTVCallback::**SetSolution**(const *OPTVVar* &var, double value)

仅在回调类型为*OPTVCBType::MIP_NODE*时可以调用。

参数

- **var** –希望设置其值的变量。
- **value** –需要设置的值。

返回

无。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –变量 var 不在当前模型中。如果变量 var 被删除，或者数据未刷新，可能会抛出该错误。
- *OPTVErrCode*::*DATA_NOT_AVAILABLE* –无法获取节点松弛解的信息，或者回调类型不是*OPTVCBType::MIP_NODE*。

OPTVCallback::**SetSolution**(const std::vector<double> &values)

仅在回调类型为*OPTVCBType::MIP_NODE*时可以调用。

参数

values –包含为整个模型设置的所有解值的向量。

返回

无。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –无法获取节点松弛解的信息，或者回调类型不是*OPTVCBType::MIP_NODE*。

UseSolution()

OPTVCallback::**UseSolution**()

通过*OPTVCallback::SetSolution()*设置变量的值后，用户可以在类型为*OPTVCBType::MIP_NODE*的回调类中使用 UseSolution，这样求解器可以在此基础上，调用启发式算法生成一个新的候选解。

返回

新候选解目标值，如果给定的变量取值不可行或者未找到新的候选解为*OPTV_INF*。

Raises OPTVErrCode::DATA_NOT_AVAILABLE

回调类型不是*OPTVCBType::MIP_NODE*.

回调信息

OPTVCBDblInfo

enum **OPTVCBDblInfo**

表 6: 双精度型回调信息

名称	描述
MIP_DUAL_BOUND	当前求解阶段的下界。
MIP_INCUMBENT_OBJ	新的可行解。
MIP_PRIMAL_BOUND	当前求解阶段的上界。
RUN_TIME	求解时间。
SIMPLEX_OBJ	单纯形目标值。

MIP_DUAL_BOUND

enumerator **MIP_DUAL_BOUND**

当前求解阶段的下界。可以在除了*OPTVCBType::MESSAGE* 之外的其他类型的回调类中使用。

MIP_INCUMBENT_OBJ

enumerator **MIP_INCUMBENT_OBJ**

新可行解的目标函数值。只能在类型为*OPTVCBType::MIP_SOLUTION* 的回调类中使用。

MIP_PRIMAL_BOUND

enumerator **MIP_PRIMAL_BOUND**

当前求解阶段的上界。可以在除了*OPTVCBType::MESSAGE* 之外的其他类型的回调类中使用。

RUN_TIME

enumerator **RUN_TIME**

求解时间。可以在任何类型的回调类中使用。

SIMPLEX_OBJ

enumerator **SIMPLEX_OBJ**

单纯形目标值。只能在类型为*OPTVCBType::SIMPLEX* 的回调类中使用。

OPTVCBIntInfo

enum **OPTVCBIntInfo**

表 7: 整型回调信息

名称	描述
MIP_NODE_STATUS	节点 Lp 模型的状态。
MIP_NUM_SOLUTION	发现的可行解的数量。
SIMPLEX_NUM_ITERS	单纯形迭代次数。

MIP_NODE_STATUS

enumerator **MIP_NODE_STATUS**

当前节点的 LP 模型的状态。只能在类型为*OPTVCBType::MIP_NODE* 的回调类中使用该参数

MIP_NUM_SOLUTION

enumerator **MIP_NUM_SOLUTION**

发现的可行解数量。可以在除了*OPTVCBType::MESSAGE* 以外的其他类型的回调类中使用。

SIMPLEX_NUM_ITERS

enumerator **SIMPLEX_NUM_ITERS**

单纯形迭代次数。只能在类型为*OPTVCBType::SIMPLEX* 的回调中使用。

OPTVCBLongInfo

enum **OPTVCBLongInfo**

表 8: 长整型回调信息

名称	描述
MIP_NUM_NODE	搜索的节点数量。
MIP_NUM_OPENNODE	待处理的开放节点数量。

MIP_NUM_NODE

enumerator **MIP_NUM_NODE**

已搜索的节点数量。可以在除了*OPTVCBType::MESSAGE* 之外的其他类型的回调类中使用。

MIP_NUM_OPENNODE

enumerator **MIP_NUM_OPENNODE**

待处理的开放节点数量。只能在类型为*OPTVCBType::MIP* 的回调类中使用。

OPTVCBStrInfo

enum **OPTVCBStrInfo**

表 9: 字符串回调信息

名称	描述
MESSAGE	下一条日志信息。

MESSAGE

enumerator **MESSAGE**

下一条要打印的日志信息。只有在类型为*OPTVCBType::MESSAGE* 的回调类中才能使用。

回调类型

天筹求解器通过 `OPTVCBType` 来区分不同类型的回调功能。不同的回调类型在对应的上下文中被触发。回调类型在实例化回调类时指定，且之后不可更改。

enum **OPTVCBType**

enumerator **MESSAGE**

在打印日志信息之前触发。

enumerator **MIP**

在 MIP 求解过程中的各个阶段触发。

enumerator **MIP_NODE**

在对节点进行 LP 求解后触发。

enumerator **MIP_SOLUTION**

在找到新候选解后触发。

enumerator **SIMPLEX**

在单纯形优化过程中触发。

异常

class **OPTVException**

异常类。

该类表示由天筹求解器抛出的异常。它继承自 `std::exception`，并提供对错误码 (*OPTVErrCode*) 和描述性消息的访问。

使用*OPTVException::GetErrorCode()* 确定具体错误类型，并通过*OPTVException::GetMessage()* 获取人类可读的描述信息。

OPTVException::**GetMessage**() const

获取异常的错误信息。

返回
异常的错误信息。

OPTVException::**GetErrorCode**() const

获取异常的错误码。

返回
异常的错误码。

关于*OPTVErrCode* 更多细节请参考*错误码*。

错误码

OPTVErrCode 用来产生特定的*OPTVException* 异常对象，并给出相应的详细信息。

enum class **OPTVErrCode**

天筹错误码。

enumerator **ERROR**

错误：错误信息，一般从求解器内部调用抛出。

enumerator **WARNING**

警告：警告信息，一般从求解器内部调用抛出。

enumerator **UNKNOWN**

未知：遇到未知错误。

enumerator **INVALID_ARGUMENT**

无效参数：传入函数的参数不满足条件。

例如在设置变量类型*OPTVCharAttr::VAR_TYPE* 时，传入未定义的属性值，或者用 0 除线性表达式。

enumerator **UNKNOWN_ATTRIBUTE**

未知属性：访问或者设置未定义的属性。

例如*OPTVConstr::Get(OPTVIntAttr::NUM_VARS)* 将抛出此错误码，因为*OPTVIntAttr::NUM_VARS* 不是模型的属性。

enumerator **DATA_NOT_AVAILABLE**

数据不可用：访问的属性值不可用。

例如在调用*OPTVModel::Optimize()* 之前，试图获取解的信息*OPTVVar::Get(OPTVDblAttr::X)* 将会抛出此错误。

enumerator **INDEX_OUT_OF_RANGE**

下标越界：用于获取对象的索引值超出了允许范围。

例如，当前模型仅有 2 个变量，调用*model.GetVar(10)* 会抛出此异常。

enumerator **UNKNOWN_PARAMETER**

未知参数：访问未定义的参数。

enumerator **VALUE_OUT_OF_RANGE**

数值越界：参数值超出了允许范围。例如尝试获取不存在的*OPTVSense*.

enumerator **ERROR_FILE_READ**

文件读取失败：读取文件时发生错误。

enumerator **ERROR_FILE_WRITE**

文件写出失败：写出文件时发生错误。

enumerator **LICENSE_CHECK_FAILED**

许可检查失败：未发现有效许可。

注：许可文件 `OPTV_LICENSE_FILE` 需要通过环境变量设置，细节请参考安装说明。

enumerator **INVALID_MODEL_INPUT**

模型输入无效：试图使用无效的模型输入信息。

enumerator **NOT_IMPLEMENTED**

功能不可用：试图使用的功能暂未开放。

enumerator **NOT_IN_MODEL**

模型组件不存在：访问的变量或约束在模型中不存在。

enumerator **MATRIX_NOT_PSD**

非正半定矩阵：二次表达式的矩阵不满足正半定性质。

当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的。

优化方向

enum class **OPTVSense**

目标含义，默认值为 1，表征最小化问题。

enumerator **MINIMIZE**

最小化问题 (+1)

enumerator **MAXIMIZE**

最大化问题 (-1)

锥类型

enum class **OPTVConeType**

锥类型。

锥类型用于指定锥约束中的仿射表达式 必须满足几何约束 C。

enumerator **SOC**

二阶锥

二阶锥 C_s 的定义如下：

$$C_s^n = \left\{ x \in \mathbb{R}^n \mid x_1 \geq \sqrt{x_2^2 + x_3^2 + \cdots + x_n^2} \right\}.$$

enumerator **RSOC**

旋转二阶锥。

旋转二阶锥 C_r 的定义如下：

$$C_r^n = \left\{ x \in \mathbb{R}^n \mid 2x_1 x_2 \geq x_3^2 + \cdots + x_n^2,\ x_1, x_2 \geq 0 \right\}.$$

有关锥的详细介绍可见锥约束。

常量

OPTV_INF

取值

```
1.0e+100
```

描述

表示一个无穷大的值。

OPTV_UNDF

取值

```
1.0e+101
```

描述

未定义的数值。

变量类型

OPTV_BINARY

取值

```
B
```

描述

0-1 整数变量类型。

OPTV_INTEGER

取值

```
I
```

描述

整数变量类型。

OPTV_CONTINUOUS

取值

```
C
```

描述

连续变量类型。

模型求解状态

OPTV_UNKNOWN

取值

0

描述

未知模型求解状态（默认）。

OPTV_OPTIMAL

取值

1

描述

模型找到最优解，可以查询到最优解。

OPTV_INFEASIBLE

取值

2

描述

模型被证明无解。

OPTV_INF_OR_UNBD

取值

3

描述

模型被证明无解或者无界。

OPTV_UNBOUNDED

取值

4

描述

模型被证明无界。

OPTV_TIME_LIMIT

取值

5

描述

模型在指定时间*OPTVDblParam::TIME_LIMIT* 内没有完成求解。

OPTV_MEM_LIMIT

取值

6

描述

模型求解使用内存超过预设值*OPTVIntParam::MEM_LIMIT*.

OPTV_INTERRUPTED

取值

7

描述

用户终止模型求解（如通过 Ctrl+C）.

变量和约束的基状态

OPTV_BASIC

取值

0

描述

变量或约束为基。

OPTV_NONBASIC_LOWER

取值

-1

描述

变量或约束非基，取下界值。

OPTV_NONBASIC_UPPER

取值

-2

描述

变量或约束非基，取上界值。

OPTV_SUPERBASIC

取值

-3

描述

变量或约束为超基。

天筹一般约束类型

OPTV_GENCONSTR_AND

取值

0

描述

只有所有变量非零时，结果才非零。

OPTV_GENCONSTR_OR

取值

1

描述

只要有一个变量非零，结果就非零。

OPTV_GENCONSTR_INDICATOR

取值

2

描述

返回值为布尔型，当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0。

OPTV_GENCONSTR_ABS

取值

3

描述

绝对值约束：`x == Abs(y)`。

OPTV_GENCONSTR_NORM

取值

4

描述

欧几里得范数约束：`x == Norm(y1, y2, ...)`。

OPTV_GENCONSTR_MIN

取值

5

描述

最小约束：x == Min(y1, y2, ...)。

OPTV_GENCONSTR_MAX

取值

6

描述

最大约束：x == Max(y1, y2, ...)。

约束含义

OPTV_SENSE_LESS

取值

<

描述

小于或等于。

OPTV_SENSE_EQUAL

取值

=

描述

等于。

OPTV_SENSE_GREATER

取值

>

描述

大于或等于。

OPTV_SENSE_NOT_EQUAL

取值

!

描述

不等于。

CP Infinity

OPTV_CP_INF

取值

```
int64_t max
```

描述

Infinity value for CP solver.

8.2.2 Java API 参考手册

变量

class OPTVVar

天筹求解器的变量对象，用于从模型访问变量。注意变量不可由其构造函数创建，而需要通过模型的*OPTVModel.addVar()* 或*OPTVModel.addVars()* 接口添加变量到模型。

---

get()

查询变量属性。

可用属性请参考*天筹属性*。

OPTVVar.**get**(OPTVIntAttr *attr*)

参数

- **attr** –整数型属性。

返回
属性值。

返回类型
int

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

OPTVVar.**get**(OPTVCharAttr *attr*)

参数

- **attr** –字符型属性。

返回
属性值。

返回类型
char

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

OPTVVar.**get**(OPTVDblAttr *attr*)

参数

- **attr** –双精度型属性。

返回
属性值。

返回类型
double

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

OPTVVar.**get**(OPTVStrAttr *attr*)

参数

- **attr** –字符串型属性。

返回
属性值。

返回类型
String

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

set()

设置变量属性，只有调用*OPTVModel.update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVVar.**set**(OPTVIntAttr *attr*, int *value*)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

OPTVVar.**set** (OPTVCharAttr *attr*, char *value*)

参数

- **attr** –字符型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

OPTVVar.**set** (OPTVDblAttr *attr*, double *value*)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

OPTVVar.**set** (OPTVStrAttr *attr*, String *value*)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

index()

获得变量在模型中的索引值。

OPTVVar.**index**()

返回
变量索引/下标。

返回类型
int

返回值

- **-1** –该变量在模型中不存在。
- **-2** –该变量已从模型中删除。
- **>=0** –该变量在模型中的索引/下标。

sameAs()

判断两个变量是否为同一个对象。

OPTVVar.**sameAs**(OPTVVar *v2*)

参数

- **v2** –需要与当前变量对比的变量。

返回
变量是否相同。

返回类型
boolean

返回值

- **true** –变量相同。
- **false** –变量不同。

列

class OPTVColumn

天筹求解器的列对象，用于以列的形式向模型添加变量，用户可以使用该功能向模型同时新增变量并添加其至已有约束。

在下面的例子中，我们构造一个初始模型并展示如何通过列对象增加变量，本示例展示的模型为*Quick Start* 章节的*milp1* .

```
import com.huaweicloud.optv.*;

OPTVEnv env;
OPTVModel model(env);

OPTVVar x = model.addVar(0, 1, -1, OPTV_BINARY, "x");
OPTVVar y = model.addVar(0, OPTV_INF, -14, OPTV_INTEGER, "y");
```

(下页继续)

(续上页)

```
OPTVConstr c0 = model.addConstr(2 * x + y, -OPTV_INF, 3, "c0");
OPTVConstr c1 = model.addConstr(3 * y, -OPTV_INF, 6, "c1");
```

具体地，构建的模型形式如下：

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

```
OPTVColumn col;

col.addTerm(1, c0);
col.addTerm(1, c1);

OPTVVar z = model.addVar(0, 3, -6, OPTV_CONTINUOUS, col, "z");
```

通过将 `col` 添加到模型，相应的约束矩阵变为：

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 3 & 1 \end{bmatrix}$$

AddTerm()

向列对象（尾部）添加单项。

OPTVColumn.**addTerm**(double *coef*, OPTVConstr *constr*)

该函数向列对象添加单项。

参数

- **coef** –系数值。
- **constr** –约束对象。

返回

无。

返回类型

void

AddTerms()

向列对象（尾部）添加多项。

OPTVColumn.**addTerms**(double[] *coefs*, OPTVConstr[] *constrs*, int *cnt*)

该功能向列对象添加多项。

参数

- **coefs** –系数序列。
- **constrs** –约束序列。
- **cnt** –添加项的个数。

返回
无。

返回类型
void

抛出
OPTVErrCode.INVALID_ARGUMENT –constrs 或 coefs 为空，或 cnt 取值为负。

Clear()

清空与该列相关内部容器，使*OPTVColumn.size()* 返回 0.

OPTVColumn.**clear**()

返回
无。

返回类型
void

GetCoeff()

据指定索引值获得列对象中对应的约束系数。

OPTVColumn.**getCoef**(int *i*)

参数

- **i** –指定的索引值，对应列中的约束。

返回
列对象中对应该索引, i, 的约束系数。

返回类型
double

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0,$ this->Size() $]$。

GetConstr()

根据指定索引值获得列对象中对应的约束。

OPTVColumn.**getConstr**(int *i*)

参数

- **i** –指定的索引值，对应列中的约束。

返回
列对象中对应该索引, i, 的约束对象。

返回类型
OPTVConstr

Throw OPTVErrCode.INDEX_OUT_OF_RANGE
若 i $\notin [0,$ this->Size() $]$。

Remove()

从列对象中删除单/多项。

OPTVColumn.**remove**(int *i*)

根据指定索引值从该列对象中删除项。

参数

- **i** –指定的索引值。

返回
无。

返回类型
void

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0, \texttt{this->Size()}]$。

OPTVColumn.**remove**(OPTVConstr *constr*)

从列对象中删除指定约束的所有相关项。

参数

- **constr** –指定要删除的约束。

返回
是否成功删除任何项。

返回
无。

返回类型
void

抛出
OPTVErrCode.NOT_IN_MODEL –指定的约束，constr, 在模型中不存在。

Size()

获得该列的规模。

OPTVColumn.**size**()

该数值统计对应列中的元素个数，无论是否有重复项，例如，若某约束被多次添加，本数值依然相应增加。

返回
该列的规模。

返回类型
int

线性表达式

class OPTVLinExpr

线性表达式*OPTVLinExpr*类用于构建线性的目标函数或者约束函数。线性表达式对象由变量和常量组成，边获取边构造，如下例所示的表达式 `1 * x + 2 * y + 3 * x`，它包含以下元素：

索引	系数	变量
0	1	x
1	2	y
2	3	x

---

addTerms()

该功能向当前表达式（后）添加一组新的项。

OPTVLinExpr.**addTerms**(double[] *coefs*, OPTVVar[] *vars*, int *cnt*)

此函数将 $\text{coefs}_i * \text{vars}_i, \forall i \in [0, \text{cnt}]$ 添加到当前线性表达式后。

参数

- **coefs** –系数序列。
- **vars** –变量序列。
- **cnt** –添加项的个数。

返回
无。

返回类型
void

抛出
OPTVErrCode.INVALID_ARGUMENT –`vars` 或 `coefs` 为空指针或 `cnt` 取负值。

clear()

清空该表达式，*OPTVLinExpr.size()* 将返回 `0`. 同时，该表达式从数值意义上将等同于 `0`.

OPTVLinExpr.**clear**()

返回
无。

返回类型
void

getCoef()

据指定索引值获得表达式中对应变量的系数。

OPTVLinExpr.**getCoef**(int *i*)

参数

- **i** –变量在线性表达式中的索引。

返回
对应的变量系数。

返回类型
double

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0, \texttt{this->Size()}]$。

getVar()

根据指定索引值获得表达式中的对应变量。

OPTVLinExpr.**getVar**(int *i*)

参数

- **i** –变量在线性表达式中的索引。

返回
指定位置上的变量。

返回类型
OPTVVar

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0, \texttt{this->Size()}]$。

getConstant()

获得当前表达式中的常数项。

OPTVLinExpr.**getConstant**()

返回
表达式中的常量数值。

返回类型
double

getValue()

获得该表达式的取值。

OPTVLinExpr.**getValue**()

返回
对应原始解的表达式的取值，即该表达式的变量值与系数的乘积之和。

返回类型
double

抛出
OPTVErrCode.DATA_NOT_AVAILABLE –当前变量取值不存在。

remove()

从表达式中删除项。

OPTVLinExpr.**remove**(int *i*)

根据指定索引值从该表达式中删除项。

参数

- **i** –指定的索引值。

返回
无。

返回类型
void

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –若 $i \notin [0, \texttt{this->Size()}]$。

OPTVLinExpr.**remove**(OPTVVar *v*)

指定的变量（在模型中）不存在。

参数

- **v** –指定要删除的变量。

返回
是否成功删除任何项。

返回类型
boolean

返回值

- **true** –成功删除任何项。
- **false** –未删除任何项。

抛出
OPTVErrCode.NOT_IN_MODEL –成功删除与指定变量有关的项。

sameAs()

判断两个线性表达式是否为同一个对象。

OPTVLinExpr.**sameAs**(OPTVLinExpr *e2*)

参数

- **e2** –需要与当前线性表达式对比的线性表达式。

返回
线性表达式是否相同。

返回类型
boolean

返回值

- **true** –线性表达式相同。
- **false** –线性表达式不同。

size()

获得该表达式的规模。

OPTVLinExpr.**size**()

该函数返回的数值对应线性表达式中的变量个数，不论变量是否重复出现。例如表达式 `x+y` 对应 `Size = 2`，而 `x+y+x` 对应 `Size = 3`。

返回
线性表达式大小。

返回类型
int

线性约束

class OPTVConstr

天筹求解器的线性约束对象，用于从模型访问约束。注意约束不可由其构造函数创建，而需要通过模型的*OPTVModel.addConstr()* 或*OPTVModel.addConstrs()* 接口添加约束至模型。此外，二次约束的相应接口为*OPTVModel.addQConstr()*.

---

get()

查询约束属性。

可用属性请参考*天筹属性*。

OPTVConstr.**get**(OPTVIntAttr *attr*)

参数

- **attr** –整数型属性。

返回
属性值。

返回类型
int

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVConstr.**get**(OPTVDblAttr *attr*)

参数

- **attr** –双精度型属性。

返回
属性值。

返回类型
double

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVConstr.**get**(OPTVStrAttr *attr*)

参数

- **attr** –字符串型属性。

返回
属性值。

返回类型
String

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

set()

设置约束属性，只有调用*OPTVModel.update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVConstr.**set**(OPTVIntAttr *attr*, int *value*)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVConstr.**set**(OPTVDblAttr *attr*, double *value*)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVConstr.**set**(OPTVStrAttr *attr*, String *value*)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

index()

获得约束在模型中的索引值。

OPTVConstr.**index**()

返回
约束索引。

返回值

- **-1** –该约束在模型中不存在。
- **-2** –该约束已从模型中删除。
- **>=0** –该约束在模型中的索引/下标。

sameAs()

查询两约束是否相同。

OPTVConstr.**sameAs**(OPTVConstr *c2*)

参数

- **c2** –需要与当前约束对比的约束。

返回
约束是否相同。

返回类型
boolean

返回值

- **true** –约束相同。
- **false** –约束不同。

二次表达式

class OPTVQuadExpr

二次表达式类*OPTVQuadExpr* 用于构建二次的目标函数或约束。

如下例所示的二次表达式 `1*x*x + 2*x*y + 3*x*y + 4*y*y + 5*x + 6*y + 7*x` 的描述如下：

索引	Coeff	变量 1	变量 2
0	1	x	x
1	2	x	y
2	3	x	y
3	4	y	y

要访问同一表达式的线性部分，可以使用*OPTVQuadExpr.getLinExpr()* 接口。这样就可以正常访问线性部分，即：

索引	系数	变量
0	5	x
1	6	y
2	7	x

addConstant()

向二次表达式增加常数项。

OPTVQuadExpr.**addConstant**(double *c*)

此函数向当前二次表达式添加常数项，但不是覆盖。

参数

- **c** –要添加的常数。

返回

无。

返回类型

void

addTerm()

在二次表达式的末尾添加一个线性项。

OPTVQuadExpr.**addTerm**(double *coeff*, OPTVVar *var*)

此函数会将 coeff * var 项添加到表达式末尾。

参数

- **coeff** –该函数向当前二次表达式后添加 coeff * var 项。
- **var** –变量对象。

返回

无。

返回类型

void

抛出

OPTVErrCode.INVALID_ARGUMENT –需添加的变量 `var` 为空。

向当前二次表达式后添加一个二次性项。

OPTVQuadExpr.**addTerm**(double *coeff*, OPTVVar *var1*, OPTVVar *var2*)

该函数向当前二次表达式后添加 coeff * var1 * var2 项。

参数

- **coeff** –该函数向当前二次表达式后添加 coeff * var 项。
- **var1** –该项的第一个变量。
- **var2** –该项的第二个变量。

返回

无。

返回类型

void

抛出

OPTVErrCode.INVALID_ARGUMENT –需添加的项中变量 `var1` 或 `var2` 为空。

addTerms()

向当前二次表达式后添加多个线性项。

OPTVQuadExpr.**addTerms**(double[] *coeff*, OPTVVar[] *var*, int *cnt*)

该函数向当前二次表达式后添加 $\text{coeff}_i * \text{var}_i, \forall i \in [0, \text{cnt}]$.

参数

- **coeff** –C-风格的系数序列。
- **var** –C-风格的变量序列。
- **cnt** –添加项的个数。

返回

无。

返回类型

void

抛出

OPTVErrCode.INVALID_ARGUMENT –变量 `var` 或 `coeff` 为空，或 `cnt` 取负值。

向当前二次表达式后添加多个二次性项。

OPTVQuadExpr.**addTerms**(double[] *coeff*, OPTVVar[] *var1*, OPTVVar[] *var2*, int *cnt*)

该函数向当前二次表达式后添加 $\text{coeff}_i * \text{var1}_i * \text{var2}_i, \forall i \in [0, \text{cnt}]$.

参数

- **coeff** –C-风格的系数序列。
- **var1** –C-风格的变量 1 序列。
- **var2** –C-风格的变量 2 序列。
- **cnt** –添加项的个数。

返回

无。

返回类型

void

抛出

OPTVErrCode.INVALID_ARGUMENT –变量 `var1`，`var2` 或 `coeff` 为空，或 `cnt` 取负值。

clear()

清空该表达式。

OPTVQuadExpr.**clear**()

该函数清空该表达式涉及的所有存储容器，*OPTVQuadExpr.size()* 将返回 `0`. 同时，该表达式也清空相应的线性表达式部分，重置常数项为 `0`.

返回

无。

返回类型
void

getCoeff()

据指定索引值获得表达式中对应二次项的系数。

OPTVQuadExpr.**getCoeff** (int *i*)

参数

- **i** –表达式中该术语的索引。

返回
对应的二次项系数。

返回类型
double

抛出
OPTVErrCode.DATA_NOT_AVAILABLE –若 i $\notin [0, \texttt{this->Size()}]$。

getConstant()

获得当前表达式中的常数项。

OPTVQuadExpr.**getConstant**()

返回
表达式中的常量数值。

返回类型
double

getLinExpr()

获取二次表达式的线性部分。

OPTVQuadExpr.**getLinExpr**()

返回
二次表达式的线性部分。

返回类型
OPTVLinExpr

getValue()

获得该二次表达式的取值。

OPTVQuadExpr.**getValue**()

返回
对应原始解的表达式的取值，即该表达式的变量值与系数的乘积之和。

返回类型
double

抛出
OPTVErrCode.DATA_NOT_AVAILABLE –当前变量取值不存在。

getVar1()

根据指定索引值获得表达式中的对应二次项的第一个变量。

OPTVQuadExpr.**getVar1** (int *i*)
该函数返回表达式中的对应二次项的第一个变量，如表达式 `x*x + x*y + y*y` 的描述如下：

索引	变量 1	变量 2
0	x	x
1	x	y
2	y	y

参数

- **i** –指定的二次项对应的索引值。

返回
对应二次项的第一个变量。

返回类型
OPTVVar

抛出
OPTVErrCode.DATA_NOT_AVAILABLE –若 $i \notin [0, \texttt{this->Size()}]$。

getVar2()

根据指定索引值获得表达式中的对应二次项的第二个变量。请参考*OPTVQuadExpr.getVar1()* 和示例。

OPTVQuadExpr.**getVar2** (int *i*)
该函数返回表达式中的对应二次项的第二个变量。

参数

- **i** –指定的二次项对应的索引值。

返回
对应二次项的第二个变量。

返回类型
OPTVVar

抛出
OPTVErrCode.DATA_NOT_AVAILABLE –若 i ∉ [0, this->Size()]。

remove()

从二次表达式中删除项。

OPTVQuadExpr.**remove** (int *i*)

根据指定索引值从二次表达式中删除二次项，但不可通过此函数删除线性项的任何部分。

参数

- **i** –指定的索引值。

返回
无。

返回类型
void

抛出
OPTVErrCode.DATA_NOT_AVAILABLE –若 i ∉ [0, this->Size()]。

从表达式中删除变量。

OPTVQuadExpr.**remove** (OPTVVar *v*)

从二次表达式中（包括线性部分）删除指定变量涉及的所有项。

参数

- **v** –指定要删除的变量。

返回
无。

返回类型
void

抛出
OPTVErrCode.NOT_IN_MODEL –成功删除与指定变量有关的项。

sameAs()

查询两二次表达式是否相同。

OPTVQuadExpr.**sameAs** (OPTVQuadExpr *e2*)

参数

- **e2** –需要与当前二次表达式对比的线性表达式。

返回
二次表达式是否相同。

返回类型
boolean

返回值

- **true** –二次表达式相同。
- **false** –二次表达式不同。

size()

获得该表达式的规模。

OPTVQuadExpr.**size**()

该数值对应当前二次表达式中的 二**次**项的个数，不论重复与否。例如，表达式 `x*x + y*y + y` 对应 `Size = 2`，而表达式 `x*x + x*x + y*y + y` 对应 `Size = 3`。

返回
该二次表达式的规模。

返回类型
int

二次约束

class OPTVQConstr

二次约束类用于访问模型中的二次约束。注意二次约束也不可以由其构造函数直接创建，而是通过接口*OPTVModel.addQConstr()* 将其添加到模型。

警告

当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的，否则会抛出*OPTVErrCode.MATRIX_NOT_PSD* 异常。

---

get()

查询二次约束属性。

可用属性请参考*天筹属性*。

OPTVQConstr.**get(OPTVIntAttr attr) const**()

参数

- **attr** –整数型属性。

返回
属性值。

返回类型
int

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVQConstr.**get(OPTVCharAttr attr) const**()

参数

- **attr** –字符型属性。

返回
属性值。

返回类型
char

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVQConstr.**get(OPTVDblAttr attr) const**()

参数

- **attr** –双精度型属性。

返回
属性值。

返回类型
double

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVQConstr.**get(OPTVStrAttr attr) const**()

参数

- **attr** –字符串型属性。

返回
属性值。

返回类型
String

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

set()

设置二次约束属性，只有调用*OPTVModel.update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVQConstr.**set**(OPTVIntAttr *attr*, int *value*)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVQConstr.**set**(OPTVCharAttr *attr*, char *value*)

参数

- **attr** –字符型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVQConstr.**set**(OPTVDblAttr *attr*, double *value*)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVQConstr.**set**(OPTVStrAttr *attr*, String *value*)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

index()

获取二次约束在模型中的索引值。

OPTVQConstr.**index**()

返回
二次约束索引。

返回类型
int

返回值

- **-1** –该二次约束在模型中不存在。
- **-2** –该二次约束已从模型中删除。
- **>=0** –该二次约束在模型中的索引/下标。

sameAs()

查询两二次约束是否相同。

OPTVQConstr.**sameAs**(OPTVConstr *c2*)

参数

- **c2** –需要与当前约束对比的约束。

返回
约束是否相同。

返回类型
boolean

返回值

- **true** –约束相同。
- **false** –约束不同。

仿射表达式

class OPTVAffineExpr

OPTVAffineExpr 类用于封装仿射变换 $Fx + g$，是构建锥约束的基础。一个*OPTVAffineExpr* 对象包含以下内容：

- **线性系数**：稀疏矩阵 F ，采用坐标（COO）格式存储非零系数及其对应的变量索引。通过逐行添加线性表达式来构建仿射表达式（COO 存储为内部实现，用户通过 SDK 对象提供数据）。
- **模型变量**：稀疏向量 x 。用户可以通过添加线性表达式 ，提供所需的变量。
- **常数项**：用于平移线性组合的偏移向量 g。

在创建*OPTVCone* 对象时，*OPTVAffineExpr* 提供了仿射变换的数学结构，锥类型参数则指定了该表达式必须满足的几何约束 C 。这种设计将几何约束与数学表达式分离，使得用户能够基于简单的仿射单元，灵活地组合构建复杂的锥约束。

有关锥的详细介绍可见锥约束。

addRow()

向仿射表达式中添加单行数据。

OPTVAffineExpr.**addRow**(OPTVLinExpr *expr*)

参数

- **expr** –添加到仿射表达式中的线性表达式对象。

返回
无。

返回类型
void

addRows()

向仿射表达式中添加多行数据。

OPTVAffineExpr.**addRows**(OPTVLinExpr[] *expr*, int *count*)

参数

- **expr** –添加到仿射表达式中的线性表达式数组。每个线性表达式对应仿射表达式中的一行。
- **count** –添加的线性表达式个数（线性表达式数组的长度）。

返回
无。

返回类型
void

抛出
OPTVErrCode.INVALID_ARGUMENT –count 为负数。

getColumnIndices()

该函数会返回一个向量。向量中的每个值对应仿射表达式中的一个非零元素的列索引。仿射表达式在内部以COO（坐标）格式存储，其中每个非零元素由其行索引、列索引及其值共同标识。

OPTVAffineExpr.**getColumnIndices**()

返回
一个整数向量，其每个元素对应一个非零元素的列索引。该向量中元素的顺序与*OPTVAffineExpr.getRowIndices()* 以及*OPTVAffineExpr.getValues()* 返回结果的顺序保持一致。

返回类型
int[]

getOffset()

OPTVAffineExpr.**getOffset**(int *index*)

参数

- **index** –查询偏移量的行索引。

返回
一个双精度浮点数，表示索引为 index 的行的偏移量。

返回类型
int

抛出
java.lang.IndexOutOfBoundsException –index 的值大于仿射表达式中的行数。

getOffsets()

OPTVAffineExpr.**getOffsets**()

返回
一个双精度浮点数的向量，每个元素对应仿射表达式中某一行的偏移量。

返回类型
double[]

getRowIndices()

该函数返回一个向量。向量中的每个元素对应一个非零元素的行索引。仿射表达式在内部以 COO（坐标）格式存储，其中每个非零元素由其行索引、列索引及其值共同标识。

OPTVAffineExpr.**getRowIndices**()

返回
一个整数向量，其每个元素代表一个非零元素的行索引。向量中的元素顺序与*OPTVAffineExpr.getColumnIndices()* 及其*OPTVAffineExpr.getValues()* 的返回结果的顺序一致。

返回类型
int[]

getValues()

该函数返回一个向量。向量中的每个元素对应仿射表达式中一个非零元素的值。非零元素的位置由*OPTVAffineExpr.getRowIndices()* 与*OPTVAffineExpr.getColumnIndices()* 返回的行索引以及列索引确定。

OPTVAffineExpr.**getValues**()

返回
一个双精度浮点数向量，其每个元素对应一个非零元素的值。向量中元素的位置与*OPTVAffineExpr.getRowIndices()* 以及*OPTVAffineExpr.getColumnIndices()* 返回结果的顺序一致。

返回类型
double[]

setOffset()

设置指定行的偏移量。

OPTVAffineExpr.**setOffset**(int *index*, double *value*)

参数

- **index** –需要设置偏移量的行的索引。
- **value** –偏移量的值。

返回类型
void

抛出
`java.lang.IndexOutOfBoundsException` –index 的值大于仿射表达式中的行数。

size()

查询仿射表达式中元素的数量。

OPTVAffineExpr.**size**()

函数的返回值为仿射表达式中元素的个数，重复的元素会被多次统计。例如，若同一线性表达式被多次添加，仿射表达式中的元素数量将随之增加。

返回
仿射表达式中的元素数量。

返回类型
int

锥约束

class OPTVCone

天筹求解器（OptVerse）的锥约束对象。用于访问模型中锥约束的相关信息。推荐使用*OPTVModel.addCone()* 接口创建锥对象，并向模型中添加锥约束，不建议使用构造函数直接创建锥对象。

锥约束在数学上定义为：

$$Fx + g \in C$$

其中：

- F 是一个 $m \times n$ 的系数矩阵，用于对优化变量进行线性转换，
- x 是一个 n 维的优化变量向量，
- g 是一个 m 维的常数偏移向量（即右端项），
- C 是约束仿射表达式取值范围的锥。
- 表达式 $Fx + g$ 生成一个属于锥 C 的 m 维向量。

矩阵 F 采用坐标（COO）稀疏格式存储，以提高内存效率，是一个仅包含非零元素的行索引、列索引、值的三元组。用户可以按行向仿射表达式中添加线性表达式，将表达式 $Fx + g$ 构建为仿射表示。

天筹求解器（OptVerse）支持以下锥约束类型：

- *Second-order cone*
- *Rotated second-order cone*

仿射表达式以及锥类型的详细信息请见仿射表达式 与锥类型。

---

get()

查询锥约束某一属性的值。

OPTVCone.**get**(OPTVStrAttr *attr*)

参数

- **attr** –字符串型属性。

返回
属性值。

返回类型
String

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

getType()

获取锥约束的类型。

有关锥约束类型的详细信息请见*OPTVConeType*。

OPTVCone.**getType**()

返回
锥约束的类型。

返回类型
OPTVConeType

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –锥约束已从模型中移除。

index()

返回锥线性约束在模型中的索引。

OPTVCone.**index**()

返回
锥约束的索引。

返回类型
int

返回值

- **-1** –锥约束不在任何模型中。
- **-2** –锥约束已从模型中移除。
- **>=0** –锥线性约束在模型中的索引。

sameAs()

判断两个锥约束是否为同一个对象。

OPTVCone.**sameAs**(OPTVCone *c2*)

参数

- **c2** –用于比对是否相同的锥约束。

返回
锥约束是否相同。

返回类型
boolean

返回值

- **true** –锥约束相同
- **false** –锥约束不同

Set()

设置锥约束属性的值。在调用*OPTVModel.update()*之前，该设置不会在模型内部生效。

可用属性请参考*天筹属性*。

OPTVCone.**set**(OPTVStrAttr *attr*, String *value*)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

SOS 约束

class OPTVSOS

天筹求解器的 SOS 约束对象，用以从模型中获取 SOS 约束数据。这里，用户需要通过*OPTVModel.addSOS()*接口向模型中添加 SOS 约束，而非借助于构造函数。

特殊顺序集，special ordered set (SOS)，是指一组有序集合里，至多有一个非零值（SOS1 型）或至多有两个非零值（SOS2 型）。SOS 约束是对一组变量的取值进行限制的特殊约束，可以额外地指定模型中的整数条件。这类约束单独列出来，可以加快线性规划的求解速度。

具体地，天筹求解器的支持下列两种 SOS 约束：

- SOS 类型 1 要求指定的一组变量至多有一个变量可取非零值。
- SOS 类型 2 指定的一组变量至多有两个变量可取非零值，且取非零值的变量顺序要求相邻。

---

get()

查询 SOS 约束的属性。

可用属性请参考*天筹属性*。

OPTVSOS.**get**(OPTVIntAttr *attr*)

参数

- **attr** –整数型属性。

返回
属性值。

返回类型
int

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVSOS.**get**(OPTVStrAttr *attr*)

参数

- **attr** –字符串型属性。

返回
属性值。

返回类型
String

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

set()

设置 SOS 约束属性，只有调用*OPTVModel.update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVSOS.**set** (OPTVIntAttr *attr*, int *value*)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVSOS.**set** (OPTVStrAttr *attr*, String *value*)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

index()

获取 SOS 约束在模型中的索引值。

OPTVSOS.**index**()

返回
SOS 约束索引。

返回类型
int

返回值

- **-1** –该 SOS 约束在模型中不存在。
- **-2** –该 SOS 约束已从模型中删除。
- **>=0** –该 SOS 约束在模型中的索引/下标。

sameAs()

查询两 SOS 约束是否相同。

OPTVSOS.**sameAs**(OPTVSOS *c2*)

参数

- **c2** –需要与当前 SOS 约束对比的 SOS 约束。

返回
SOS 约束是否相同。

返回类型
boolean

返回值

- **true** –SOS 约束相同。
- **false** –SOS 约束不同。

一般约束

class OPTVGenConstr

天筹求解器的一般约束，用于从模型中获取一般约束数据。特别地。一般约束需要通过 `OPTVModel.addGenConstrXxx` 向模型中添加约束而构造，而不是借助于任何构造函数。

表 10: 天筹求解器支持的一般约束类型

约束	类型	接口	描述
And	*OPTV_GENCONSTR_A*	*OPTVModel.addGenConstrAnd()*	只有所有变量非零时，结果才非零。
Or	*OPTV_GENCONSTR_O*	*OPTVModel.addGenConstrOr()*	只要有一个变量非零，结果就非零。
Indicator	*OPTV_GENCONSTR_I*	*OPTVModel.addGenConstrIndicator*	返回值为布尔型，当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0。

get()

查询一般约束属性。

可用属性请参考*天筹属性*。

OPTVGenConstr.**get**(OPTVIntAttr *attr*)

参数

- **attr** –整数型属性。

返回
属性值。

返回类型
int

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVGenConstr.**get**(OPTVDblAttr *attr*)

参数

- **attr** –双精度型属性。

返回
属性值。

返回类型
double

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVGenConstr.**get**(OPTVStrAttr *attr*)

参数

- **attr** –字符串型属性。

返回
属性值。

返回类型
String

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

set()

设置一般约束属性，只有调用*OPTVModel.update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVGenConstr.**set**(OPTVDblAttr *attr*, int *value*)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

OPTVGenConstr.**set**(OPTVStrAttr *attr*, String *value*)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

index()

获得一般约束在模型中的索引值。

OPTVGenConstr.**index**()

返回
约束索引。

返回类型
int

返回值

- **-1** –该约束在模型中不存在。
- **-2** –该约束已从模型中删除。
- **>=0** –该约束在模型中的索引/下标。

sameAs()

查询两个一般约束是否为同一对象。

OPTVGenConstr.**sameAs**(OPTVGenConstr *c2*)

参数

- **c2** –需要与当前约束对比的约束。

返回
约束是否相同。

返回类型
boolean

返回值

- **true** –约束相同。
- **false** –约束不同。

模型

class OPTVModel

OPTVModel()

OPTVModel(OPTVEnv *env*)

创建一个与给定环境关联的空模型。

参数

- **env** –天箭模型环境变量。

OPTVModel(OPTVEnv *env*, String *fileName*)

通过读取文件来创建模型。

参数

- **env** –天箭模型环境变量。
- **fileName** –模型文件的路径（MPS、LP 等）。

addCone()

向模型中添加单个锥约束。

OPTVModel.**addCone**(OPTVAffineExpr *affine*, OPTVConeType *type*, String *name* = *""*)

该函数将向模型中添加一条锥约束。

参数

- **affine** –锥约束的仿射表达式 $Fx + g$。
- **type** –锥约束类型（如，二阶锥、旋转二阶锥）。
- **name** –锥约束名称。

返回

添加到模型中的锥约束对象。

返回类型

OPTVCone

抛出

OPTVErrCode.INVALID_ARGUMENT –type 不是*OPTVConeType.SOC* 或者*OPTVConeType.RSOC* 。仿射表达式为空。type 为*OPTVConeType.RSOC* ，且仿射表达式中的元素个数小于 *3* 。

addConstr()

向模型中添加单个线性约束。

OPTVModel.**addConstr**(OPTVLinExpr *expr*, double *lhs*, double *rhs*, String *name*="")

该函数向模型中添加一个线性约束。当需要一次性添加大量约束时，建议使用批量模式(*OPTVModel.addConstrs()*)，虽然大多数情况下，效率差别不大。

参数

- **expr** –约束的线性表达式部分。
- **lhs** –约束的下界。
- **rhs** –约束的上界。
- **name** –约束名称。

返回

添加到模型中的约束对象。

返回类型

OPTVConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addConstrs()

向模型中添加多个线性约束。

OPTVModel.**addConstrs**(OPTVLinExpr[] *expr*, double[] *lhs*, double[] *rhs*, String[] *names*, int *count*)

该函数一次性向模型中添加多个线性约束，尤其适用于添加大量约束的情形。

参数

- **expr** –约束的线性表达式序列。
- **lhs** –约束的左端值序列。
- **rhs** –约束的右端值序列。
- **names** –约束的名字序列。
- **count** –添加约束的个数。

返回

添加到模型的约束列表。

返回类型

OPTVConstr[]

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addQConstr()

向模型中添加单个二次约束。

OPTVModel.**addQConstr**(OPTVQuadExpr *expr*, double *lhs*, double *rhs*, String *name*="")

该函数向模型中添加一个二次约束。

参数

- **expr** –约束的二次表达式部分。
- **lhs** –约束的下界。
- **rhs** –约束的上界。
- **name** –约束名称。

返回

添加到模型中的约束对象。

返回类型

OPTVQConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addGenConstrAnd()

向模型中添加单个 AND 一般约束。

OPTVModel.**addGenConstrAnd**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, String *name* = "")

该函数向模型中添加单个 AND 一般约束：只有所有变量非零时，结果才非零。

参数

- **resultant** –约束的布尔型返回值。
- **vars** –所有参与检验的变量序列。
- **len** –vars 序列长度。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addGenConstrOr()

向模型中添加单个 OR 一般约束。

OPTVModel.**addGenConstrOr**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, String *name* = "")

该函数向模型中添加单个 OR 一般约束：只要有一个变量非零，结果就非零。

参数

- **resultant** –约束的布尔型返回值。
- **vars** –所有参与检验的变量的序列。
- **len** –vars 序列长度。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addGenConstrIndicator()

向模型中添加单个 INDICATOR 一般约束。

OPTVModel.**addGenConstrIndicator**(OPTVVar *binVar*, int *binVal*, OPTVLinExpr *expr*, char *sense*, double *rhs*, String *name* = "")

该函数向模型中添加单个 INDICATOR 一般约束：当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0.

参数

- **binVar** –二进制指示变量。
- **binVal** –要求线性约束必须满足的二进制指示变量的值 (0 or 1).
- **expr** –参与检验的线性约束表达式。
- **sense** –线性约束的含义，如*OPTV_SENSE_LESS*，*OPTV_SENSE_EQUAL*，或*OPTV_SENSE_GREATER*。
- **rhs** –参与检验的线性约束右端值。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addGenConstrAbs()

向模型中添加单个 ABS 一般约束：$r = \text{abs}(x)$。此约束条件保证因变量 r 的值为自变量 x 的绝对值，即 $r = |x|$, 其中 $r \geq 0$。

形如 $r = \text{abs}(x)$ 的绝对值约束要求变量 r 等于变量 x 的绝对值。该约束施加了关系 $r = |x|$，其中 $r \geq 0$。

OPTVModel.**addGenConstrAbs** (OPTVVar *resultant*, OPTVVar *inputVar*, String *name*)

结果变量，该变量的值为另一变量的绝对值。

参数

- **resultant** –该变量的绝对值将被结果变量采用。
- **inputVar** –添加到模型的一般约束对象。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

AddGenConstrNorm()

向模型中添加一个 NORM 约束。

向 “NORM“ 约束定义为：$r = \text{norm}\{x_1, \ldots, x_n\}$，其中因变量 r 表示向量 $x_1, \ldots, x_n$ 的范数。

OPTVModel.**addGenConstrNorm** (OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, int *normType*, String *name*)

该函数向模型中添加单个 NORM 一般约束。

参数

- **resultant** –结果变量，其值为参数向量的范数。
- **vars** –用于计算向量范数的变量。
- **len** –vars 序列长度。
- **normType** –范数类型：-1 为 L^∞ 范数，0 为 L^0 范数，1 为 L^1 范数。
- **name** –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addGenConstrMax()

向模型中添加单个 MAX 约束：$r = \max\{x_1, \ldots, x_n\}$。

形式为 $r = \max\{x_1, \ldots, x_n\}$ 的 MAX 约束确保变量 r 等于参数向量 $x_1, \ldots, x_n$ 中的最大值。

OPTVModel.**addGenConstrMax**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, String *name* = *""*, double *constant* = *-OPTV_INF*)

该函数向模型中添加单个 MAX 约束。

参数

- **resultant** –从中取最大值的一组变量。
- **vars** –向模型中添加单个 MAX 约束：$r = \max\{x_1, \ldots, x_n\}$ 。
- **len** –vars 序列长度。
- **name** –约束名称。
- **constant** –[**可选**] 参与 MAX 操作的常量。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addGenConstrMin()

向模型中添加单个 MIN 约束：$r = \min\{x_1, \ldots, x_n\}$ 。

形式为 $r = \min\{x_1, \ldots, x_n\}$ 的 MIN 约束确保变量 r 等于参数向量 $x_1, \ldots, x_n$ 中的最小值。

OPTVModel.**addGenConstrMin**(OPTVVar *resultant*, OPTVVar[] *vars*, int *len*, String *name* = *""*, double *constant* = *OPTV_INF*)

该函数向模型中添加单个 MIN 约束。

参数

- **resultant** –从中取最小值的一组变量。
- **vars** –变量序列，该向量的最小值会被结果变量采用。
- **len** –vars 序列长度。
- **name** –约束名称。
- **constant** –[**可选**] 参与 MIN 操作的常量。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

抛出

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addSOS()

向模型中添加单个 SOS 约束。

OPTVModel.**addSOS**(OPTVVar[] *vars*, double[] *weights*, int *len*, int *type*, String *name*="")

参数

- **vars** –SOS 约束所涉及变量的序列。
- **weights** –SOS 的变量权重的序列。
- **len** –vars 和 weights 的序列长度。
- **type** –SOS 类型，取值为 1 或 2。
- **name** –SOS 约束的名字。

返回
添加到模型的 SOS 约束对象。

返回类型
OPTVSOS

抛出

- *OPTVErrCode.INVALID_ARGUMENT* –type 取值不是 1 或 2。
- *OPTVErrCode.NOT_IN_MODEL* –vars 序列中的某个变量不属于当前模型。
- *OPTVErrCode.INDEX_OUT_OF_RANGE* –当前模型存在无效下标，比如模型关联的环境发生溢出时，执行本操作有可能抛出此错误。

addVar()

向模型中添加单个变量。

OPTVModel.**addVar**(double *lb*, double *ub*, double *obj*, OPTVCharAttr *type*, String *name* = "")

该函数向模型中添加一个变量。当需要一次性添加大量变量时，建议使用批量模式（*OPTVModel.addVars()*），虽然大多数情况下，效率差别不大。

参数

- **lb** –变量的下界值。
- **ub** –变量的上界值。
- **obj** –变量在目标函数中的（线性）系数。
- **type** –变量类型，取值包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **name** –变量名称。

返回
添加到模型中的变量对象。

返回类型
OPTVVar

抛出
OPTVErrCode.INVALID_ARGUMENT –未知变量类型，name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

OPTVModel.**addVar**(double *lb*, double *ub*, double *obj*, char *type*, OPTVColumn *col*, String *name* = "")

该函数向模型中添加一个变量，并可以设置现有约束中的相应系数。当需要一次性添加大量变量时，建议使用批量模式(*OPTVModel.addVars()*)，虽然大多数情况下，效率差别不大。

参数

- **lb** –变量的下界值。
- **ub** –变量的上界值。
- **obj** –变量在目标函数中的（线性）系数。
- **type** –变量类型，取值包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** –设置约束系数的列对象。
- **name** –变量名称。

返回

添加到模型中的变量对象。

返回类型

OPTVVar

抛出

OPTVErrCode.INVALID_ARGUMENT –未知变量类型，name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

addVars()

向模型中添加多个变量。

OPTVModel.**addVars**(double[] *lb*, double[] *ub*, double[] *obj*, char[] *type*, String[] *names*, int *count*)

该函数一次性向模型中添加多个变量，尤其适用于添加大量变量的情形。

参数

- **lb** –变量的下界序列。
- **ub** –变量的上界序列。
- **obj** –变量序列对应的（线性）目标系数。
- **type** –变量类型序列，包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **names** –变量名称序列。
- **count** –添加变量的个数。

返回

添加到模型的变量序列。

返回类型

OPTVVar[]

抛出

OPTVErrCode.INVALID_ARGUMENT –type 包含未知变量类型，name 为空，数值参数中存在 NaN 或者 name 包含的变量名与模型中的其他组件重名。

OPTVModel.**addVars**(double[] *lb*, double[] *ub*, double[] *obj*, char[] *type*, OPTVColumn[] *col*, String[] *names*, int *count*)

该函数向模型中添加多个变量，并可以设置现有约束中的相应系数。当需要一次性添加大量变量时，建议使用此功能，虽然大多数情况下，效率差别不大。

参数

- **lb** –变量的下界序列。
- **ub** –变量的上界序列。
- **obj** –变量序列对应的（线性）目标系数。
- **type** –变量类型序列，包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** –设置约束系数的列对象序列。
- **names** –变量名称序列。
- **count** –添加变量的个数。

返回
添加到模型的变量序列。

返回类型
OPTVVar[]

抛出
OPTVErrCode.INVALID_ARGUMENT –`type` 包含未知变量类型，`name` 为空，数值参数中存在 `NaN` 或者 `name` 包含的变量名与模型中的其他组件重名。

checkSolution()

检验指定的解是否可行。

OPTVModel.**checkSolution**(double[] *solution*, double *tol=1e-8*)

该函数供用户检验指定的解是否满足当前模型的可行性要求，但 **不可以**检验内部存储的解。

参数

- **solution** –（列）解向量。
- **tol** –约束可行性整数性检查精度（**可选**，默认值为 10^{-8}）。

返回
解对模型是否可行。

返回类型
boolean

返回值

- **false** –解对模型不可行。
- **true** –解对模型是可行的。

computeIIS()

计算不可约不一致子系统（IIS）。

有关 IIS 的细节请参考*IIS 快速入门*。

OPTVModel.**computeIIS**()

返回
IIS 是否计算成功。

返回类型
boolean

返回值

- **false** –IIS 计算不成功。
- **true** –成功计算 IIS。

feasRelax()

进行不可行问题的诊断及分析。

更多细节请参考*LP 不可行诊断及修复快速入门*。

OPTVModel.**feasRelax**()

该函数进行不可行问题的诊断及分析，目前仅支持 L^1 范数。

返回
不可行问题修复的状态。

返回类型
int

返回值

- **0** –当前问题是可行的。
- **-1** –不可行问题修复失败。
- **>0** –Phase 1 最小违背量的目标函数值。

getCoef()

查询给定约束中指定变量的系数。

OPTVModel.**getCoef**(OPTVConstr *con*, OPTVVar *var*)

参数

- **con** –需要查询的约束。
- **var** –需要查询的变量。

返回
查询变量在该约束中的系数。

返回类型
double

抛出

- *OPTVErrCode.INVALID_ARGUMENT* –约束或变量不存在。
- *OPTVErrCode.DATA_NOT_AVAILABLE* –数据不存在，或约束的线性表达式部分不可用。

GetColumn()

获取指定变量对应的列对象。

OPTVModel.**getColumn**(OPTVVar *v*)

参数

- **v** –需要查询的变量。

返回
指定变量对应的列对象。

返回类型
OPTVColumn

Throw OPTVErrCode.DATA_NOT_AVAILABLE
该变量不存在于当前模型。

Throw OPTVErrCode.INDEX_OUT_OF_RANGE
变量索引值*Index*超过了模型的变量总数。若当前模型需要执行更新，或者当前变量属于其它模型时，该错会被抛出。

getConstrByName()

通过名称获取约束。

OPTVModel.**getConstrByName**(String *name*)

参数

- **name** –需要访问的约束名称。

返回
名称匹配 name 的约束。

返回类型
OPTVConstr

抛出
OPTVErrCode.DATA_NOT_AVAILABLE –给定的名称为空，或模型不存在命名为 name 的约束。

getCone()

获取指定位置的锥约束。

OPTVModel.**getCone**(int *i*)

参数

- **i** –锥约束的索引。

返回
索引为 i 的锥约束。

返回类型
OPTVCone

Throw OPTVErrCode.INDEX_OUT_OF_RANGE
索引值大于模型中锥约束的数量。

getCones()

获取模型所有的锥约束。

OPTVModel.**getCones**()

返回
包含模型所有圆锥约束的数组。

返回类型
OPTVCone[]

getConstr()

根据指定索引值获得模型中对应的约束。

OPTVModel.**getConstr**(int *i*)

参数

- **i** –线性约束在模型中的索引。

返回
索引为 i 的约束。

返回类型
OPTVConstr

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –指定的索引值超出了模型的约束总数。

getConstrs()

获得模型中的所有线性约束。

OPTVModel.**getConstrs**()

返回
包含模型中所有线性约束的向量。

返回类型
OPTVConstr[]

getGenConstrAnd()

获得模型中特定的 AND 一般约束数据。

OPTVModel.**getGenConstrAnd**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*)

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –参与检验该一般约束的所有变量的列表。

返回
vars 序列中变量的个数。

返回类型
int

抛出
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

getGenConstrOr()

获得模型中特定的 OR 一般约束数据。

OPTVModel.**getGenConstrOr**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*)

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –参与检验该一般约束的所有变量的列表。

返回
vars 序列中变量的个数。

返回类型
int

抛出
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

getGenConstrIndicator()

获得模型中特定的 INDICATOR 一般约束数据。

OPTVModel.**getGenConstrIndicator**(OPTVGenConstr *genConstr*, OPTVVar *binVar*, int *binVal*, OPTVLinExpr *expr*, char *sense*, double *rhs*)

参数

- **genConstr** –需要查询的一般约束。
- **binVar** –二进制指示变量。
- **binVal** –要求线性约束必须满足的二进制指示变量的值 (0 or 1).
- **expr** –参与检验的线性约束表达式。
- **sense** – 线性约束的含义，如*OPTV_SENSE_LESS*，*OPTV_SENSE_EQUAL*，或*OPTV_SENSE_GREATER*。
- **rhs** –参与检验的线性约束右端值。

返回
无。

返回类型
void

抛出
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

getGenConstrAbs()

获取模型中指定的 ABS 一般约束数据。

OPTVModel.**getGenConstrAnd**(OPTVGenConstr *genConstr*, OPTVVar *resvar*, OPTVVar *argvar*)

参数

- **genConstr** –需要查询的一般约束。
- **resvar** –该一般约束对应的结果变量。
- **argvar** –该一般约束的参数变量。

返回
无。

返回类型
void

抛出
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

getGenConstrNorm()

获得模型中指定的 NORM 一般约束数据。

OPTVModel.**getGenConstrNorm**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*, int[] *normType*)

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –用于计算范数的变量列表。
- **normType** –范数类型（-1 为 L^{∞} 范数，0 为 L^{0} 范数，1 为 L^{1} 范数）。

返回
无。

返回类型
void

抛出
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

getGenConstrMin()

获得模型中指定的 MIN 一般约束数据。

OPTVModel.**getGenConstrMin**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*, double[] *constant*)

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –参与检验该一般约束的所有变量的列表。
- **constant** –参与该一般约束的常量操作数。

返回
无。

返回类型
void

抛出
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

getGenConstrMax()

获取模型中指定的 MAX 一般约束数据。

OPTVModel.**getGenConstrMax**(OPTVGenConstr *genConstr*, OPTVVar *resultant*, OPTVVar[] *vars*, double[] *constant*)

参数

- **genConstr** –需要查询的一般约束。
- **resultant** –该一般约束对应的结果变量。
- **vars** –参与检验该一般约束的所有变量的列表。
- **constant** –参与该一般约束的常量操作数。

返回
无。

返回类型
void

抛出
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

getQConstr()

根据指定索引值获得模型中对应的二次约束。

OPTVModel.**getQConstr(int i) const**()

参数

- **i** –线性约束在模型中的索引。

返回
索引为 i 的约束。

返回类型
OPTVQConstr

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –指定的索引值超出了模型的约束总数。

对应的二次约束。

getQConstrs()

获得模型中的所有二次约束。

OPTVModel.**getQConstrs() const**()

返回
包含模型中所有二次约束的向量。

返回类型
OPTVQConstr[]

getSOS()

根据索引值获取指定类型的 SOS 约束。

OPTVModel.**getSOS(int i, int type) const**()

参数

- **i** –指定 SOS 约束在模型中的索引值。
- **type** –SOS 类型，取值为 1 或 2。

返回
对应索引值 i 的指定类型的 SOS 约束。

返回类型
OPTVSOS

抛出

- *OPTVErrCode.INDEX_OUT_OF_RANGE* –索引值 i 超出了模型的该类型的 SOS 约束总数。
- *OPTVErrCode.INVALID_ARGUMENT* –type 取值不是 1 或 2。

这里，参数 i 特指某一类型 SOS 约束集中的索引值，例如，如果用户先后添加了类型为 1 和 2 的两个 SOS 约束，那么两者的索引值都是 0.

getSOSData()

OPTVModel.**getSOSData**(OPTVSOS *sos*, OPTVVar[] *vars*, double[] *weights*)

获取 SOS 约束的内部数据。

参数

- **sos** –当前访问的 SOS 约束对象。
- **vars** –变量序列的空向量。如果非空，该容器在本次函数调用中会被清空。
- **weights** –变量权重序列的空向量。如果非空，该容器在本次函数调用中会被清空。

返回
无。

返回类型
void

抛出

- *OPTVErrCode.DATA_NOT_AVAILABLE* –SOS 约束是由默认构造函数生成的。
- *OPTVErrCode.NOT_IN_MODEL* –SOS 约束属于另外一个模型，或已经从本模型中被删除。
- *OPTVErrCode.INDEX_OUT_OF_RANGE* –约束下表越界、超出容器范围，可能需要对当前模型执行更新。

getSOSs()

获取指定类型的所有 SOS 约束。

OPTVModel.**getSOSs(int type) const**()

参数

- **type** –SOS 类型，取值为 `1` 或 `2`。

返回
包含模型中所有类型为 `type` 的 SOS 约束的序列。

返回类型
OPTVSOS[]

抛出

- *OPTVErrCode.INDEX_OUT_OF_RANGE* –SOS 约束的索引值超出了模型的该类型的 SOS 约束总数，可能需要对当前模型执行更新。
- *OPTVErrCode.INVALID_ARGUMENT* –`type` 取值不是 `1` 或 `2`。

getObjective()

获取线性目标函数。

OPTVModel.**getObjective**()

返回
目标函数的线性表达式。

返回类型
OPTVLinExpr

getObjectiveN()

在多目标优化中，获取指定目标函数的线性表达式。

OPTVModel.**getObjectiveN**(int *objId*)

参数

- **objId** –目标函数的索引。

返回
表示指定的目标函数的线性表达式对象。

返回类型
OPTVLinExpr

抛出
OPTVErrCode.VALUE_OUT_OF_RANGE –多目标优化的目标函数的索引值大于模型中目标函数的数量。

getQObjective()

获取二次目标函数。

OPTVModel.**getQObjective() const**()

返回
目标函数的二次表达式。

返回类型
OPTVQuadExpr

getRow()

获取指定约束对应的线性函数表达式。

OPTVModel.**getRow**(OPTVConstr *c*)

参数

- **c** –指定的约束对象。

返回
该约束对应的线性表达式。

返回类型
OPTVLinExpr

抛出

- *OPTVErrCode.DATA_NOT_AVAILABLE* –该约束不存在于当前模型。
- *OPTVErrCode.INDEX_OUT_OF_RANGE* –当前约束的索引*Index*超过了模型的约束总数。若当前模型需要执行更新，或者当前约束属于其它模型时，该错会被抛出。

get()

参数

获取求解器的参数值。

可用参数请参考参数。

OPTVModel.**get**(OPTVIntParam *param*)

参数

- **param** –整数型参数。

返回
参数取值。

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

OPTVModel.**get**(OPTVDblParam *param*)

参数

- **param** –双精度型参数。

返回
参数取值。

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

OPTVModel.**get**(OPTVStrParam *param*)

参数

- **param** –字符串型参数。

返回
参数取值。

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

属性

获取属性的取值。

可用属性请参考*天筹属性*。

OPTVModel.**get**(OPTVIntAttr *attr*)

参数

- **attr** –整数型属性。

返回
属性取值。

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

OPTVModel.**get**(OPTVDblAttr *attr*)

参数

- **attr** –双精度型属性。

返回
属性取值。

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

OPTVModel.**get**(OPTVLongAttr *attr*)

参数

- **attr** –长整型属性。

返回
属性取值。

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

OPTVModel.**get**(OPTVStrAttr *attr*)

参数

- **attr** –字符串型属性。

返回
属性取值。

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

getVarByName()

通过名称获取变量。

OPTVModel.**getVarByName**(String *name*)

参数

- **name** –需要访问的变量名称。

返回
名称匹配 `name` 的变量。

返回类型
OPTVVar

抛出
OPTVErrCode.DATA_NOT_AVAILABLE –给定的名称为空，或模型不存在命名为 `name` 的变量。

getVar()

根据指定索引值获得模型中对应的变量。

OPTVModel.**getVar**(int *i*)

参数

- **i** –变量在模型中的索引。

返回
在模型中索引 `i` 的变量。

返回类型
OPTVVar

抛出
OPTVErrCode.INDEX_OUT_OF_RANGE –指定的索引值超出了模型的变量总数。

getVars()

获得模型中的所有变量。

OPTVModel.**getVars**()

返回
包含模型中所有变量的数组。

返回类型
OPTVVar[]

optimize()

对模型进行求解。

OPTVModel.**optimize**()

返回
无。

返回类型
void

抛出
OPTVErrCode.MATRIX_NOT_PSD –模型中包含二次目标函数或者二次约束，而该表达式的矩阵不满足正半定性质。当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的。

求解器采用哪种优化算法取决于求解器参数配置和模型属性，例如变量类型被用以自动识别 LP, QP, MIP 及网络流问题。

presolve()

对模型进行预处理。

OPTVModel.**presolve**()
该函数会应用预处理流程，返回一个新的*OPTVModel* 对象。原模型的状态被修改为 被预处理.

返回
预处理后的模型。

返回类型
OPTVModel

read()

从文件中读取读取模型和基。

OPTVModel.**read**(String *fileName*)

该函数供读取基信息，也可以读取模型文件，然而建议由构造函数读取文件中的模型，参考文件格式。

表 11: 支持读入的文件格式

.mps	.lp	.bas

表 12: 支持读入的压缩格式

.gz	.z	.Z

参数

- **fileName** –C++ 字符串文件名。

返回

无。

返回类型

void

抛出

OPTVErrCode.ERROR_FILE_READ –文件名或后缀为空，文件存在格式错误。

relax()

进行线性规划 (LP) 松弛。

OPTVModel.**relax**()

该函数将原模型中的所有整数类型变量替换为*OPTV_CONTINUOUS* 变量，生成并返回一个新的*OPTVModel* 对象，对原模型不做任何修改。

返回

模型的 LP 松弛副本。

返回类型

OPTVModel

remove()

从模型中删除变量。

OPTVModel.**remove**(OPTVVar *var*)

该函数将变量从模型中删除，对应的索引会被相应地调整。

例如，若下列模型 `model` 构建时包含以下变量:

变量	索引
x0	0
x1	1
x2	2
x3	3

调用 `model.Remove(c0)` 后，相应变化为：

变量	索引
x1	0
x2	1
x3	2

参数

- **var** –指定要删除的变量。

返回

无。

返回类型

void

抛出

- *OPTVErrCode.NOT_IN_MODEL* –模型不包含此变量。
- *OPTVErrCode.DATA_NOT_AVAILABLE* –该变量属于其他模型，而不属于当前模型。
- *OPTVErrCode.INDEX_OUT_OF_RANGE* –变量的下标超出了模型的变量总数，可能需要执行模型更新。

OPTVModel.**remove**(OPTVConstr *constr*)

该函数将约束从模型中删除，对应的索引会被相应地调整。

例如，若模型 `model` 创建时包含如下约束：

约束	索引
c0	0
c1	1
c2	2
c3	3

调用 `model.remove(and0)` 后，相应变化为：

约束	索引
c1	0
c2	1
c3	2

参数

- **constr** –指定要删除的约束。

返回
无。

返回类型
void

抛出

- *OPTVErrCode.NOT_IN_MODEL* –模型不包含此约束。
- *OPTVErrCode.DATA_NOT_AVAILABLE* –该约束属于其他模型，而不属于当前模型。
- *OPTVErrCode.INDEX_OUT_OF_RANGE* –约束的下标超出了模型的变量总数，可能需要执行模型更新。

OPTVModel.**remove**(OPTVGenConstr *genConstr*)

从模型中删除单个一般约束。执行后对应的相同类型的一般约束索引会被相应地调整。

例如，若下列模型 `model` 构建时包含以下一般约束：

约束	类型	索引
and0	AND	0
and1	AND	1
or0	OR	0
or1	OR	1

调用 `model.remove(sos1_2)` 后，相应变化为：

约束	类型	索引
and1	AND	0
or0	OR	0
or1	OR	1

参数

- **genConstr** –指定要删除的一般约束。

返回
无。

返回类型
void

抛出

- *OPTVErrCode.NOT_IN_MODEL* –模型不包含此约束。
- *OPTVErrCode.DATA_NOT_AVAILABLE* –该约束属于其他模型，而不属于当前模型。
- *OPTVErrCode.INDEX_OUT_OF_RANGE* –该一般约束的下标超出了模型的变量总数，可能需要执行模型更新。

OPTVModel.**remove**(OPTVSOS *sos*)

从模型中删除单个 SOS 约束。与其他约束类型不同的是，删除该 SOS 约束后，相同类型的约束的索引**不会**改变。

例如，若模型 `model` 创建时包含如下约束：

约束	类型	索引
sos1_1	1	0
sos1_2	1	1
sos1_3	1	2
sos2_1	2	0

调用 `model.Remove(c0)` 后，锥约束的索引会变为：

约束	类型	索引
sos1_1	1	0
sos1_3	1	2
sos2_1	2	0

参数

- **sos** –指定要删除的 SOS 约束。

返回
无。

返回类型
void

抛出

- *OPTVErrCode.NOT_IN_MODEL* –SOS 约束不属于任何模型。
- *OPTVErrCode.DATA_NOT_AVAILABLE* –SOS 不属于当前模型。
- *OPTVErrCode.INDEX_OUT_OF_RANGE* –SOS 约束的索引值超出了模型的该类型的 SOS 约束总数，可能需要对当前模型执行更新。

OPTVModel.**remove**(OPTVCone *cone*)

该函数从模型中移除一个锥约束。移除操作将导致后续锥约束的索引发生前移。

约束	类型	索引
c0	SOC	0
c1	RSOC	1
c2	SOC	2
c3	RSOC	3

调用 `model.remove(and0)` 后，相应变化为：

约束	类型	索引
c1	RSOC	0
c2	SOC	1
c3	RSOC	2

参数

- **cone** –需要移除的锥约束对象。

返回
无。

抛出

- *OPTVErrCode.NOT_IN_MODEL* –锥约束不在任何模型中。
- *OPTVErrCode.DATA_NOT_AVAILABLE* –锥约束属于另一个模型，不在当前模型中。
- *OPTVErrCode.INDEX_OUT_OF_RANGE* –锥约束的索引大于模型中锥约束的数量。添加锥约束后，没有将其更新到模型中，则可能发生此情况。

reset()

将模型重置为 *未求解成功*状态。

OPTVModel.**reset**(boolean *clearAll=false*)

该函数会清除内部解的信息，默认情况下，包括解的值、LP 基和模型状态。`clearAll` 标志是否清除额外的模型属性值（包括 MIP 热启动值，MIP 分支优先度，LP 的禁用属性状态）。

参数

- **clearAll** –清除额外信息（默认为 `false`）。

返回
无。

返回类型
void

getObjSA()

目标函数敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**getObjSA**(double[] *down*, double[] *up*, int[] *list*)

参数

- **down** –敏感度区间左端点。
- **up** –敏感度区间右端点。
- **list** –需要进行目标函数敏感度分析的变量索引集合。

返回
敏感度分析是否成功。

返回类型
boolean

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

getVarLbSA()

变量下界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**getVarLbSA** (double[] *down*, double[] *up*, int[] *list*)

参数

- **down** –下界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** –下界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** –需要进行变量下界敏感度分析的约束索引集合。

返回
敏感度分析是否成功。

返回类型
boolean

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

getVarUbSA()

变量上界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**getVarUbSA** (double[] *down*, double[] *up*, int[] *list*)

参数

- **down** –上界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** –上界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** –需要进行变量上界敏感度分析的约束索引集合。

返回
敏感度分析是否成功。

返回类型
boolean

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

getConstrLbSA()

约束左端值/下界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**getConstrLbSA**(double[] *down*, double[] *up*, int[] *list*)

参数

- **down** –下界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** –下界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** –需要进行左端值敏感度分析的约束索引集合。

返回
敏感度分析是否成功。

返回类型
boolean

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

getConstrUbSA()

约束右端值/上界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**getConstrUbSA**(double[] *down*, double[] *up*, int[] *list*)

参数

- **down** –上界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** –上界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** –需要进行右端值敏感度分析的约束索引集合。

返回
敏感度分析是否成功。

返回类型
boolean

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

setCoef()

设置给定约束中指定变量的系数。

OPTVModel.**setCoef**(OPTVConstr *con*, OPTVVar *var*, double *newValue*)

参数

- **con** –需要修改的约束。
- **var** –需要修改系数的变量。
- **newValue** –新的系数值。

返回
无。

返回类型
void

抛出

- *OPTVErrCode.INVALID_ARGUMENT* –约束或变量不存在，或数值参数中存在 NaN。
- *OPTVErrCode.DATA_NOT_AVAILABLE* –数据不存在，或约束的线性表达式部分不可用。

setObjective()

设置线性目标函数。

OPTVModel.**setObjective**(OPTVLinExpr *expr*, OPTVSense *sense=OPTVSense.MINIMIZE*)

该函数清除当前目标函数（相对于在其基础上进行叠加）。若未指定 sense 参数，求解器会采用默认的（*OPTVSense.MINIMIZE*）。

参数

- **expr** –需要设定的目标函数表达式。
- **sense** – **可选**，最小化或最大化，默认值为:*OPTVSense.MINIMIZE*，参考*OPTVSense*.

返回
无。

返回类型
void

设置二次目标函数。

OPTVModel.**setObjective**(OPTVQuadExpr *expr*, OPTVSense *sense=OPTVSense.MINIMIZE*)

该函数清除当前目标函数（相对于在其基础上进行叠加）。若未指定 sense 参数，求解器会采用默认的（*OPTVSense.MINIMIZE*）。

参数

- **expr** –需要设定的目标函数表达式。
- **sense** – **可选**，最小化或最大化，默认值为:*OPTVSense.MINIMIZE*，参考*OPTVSense*.

返回
无。

返回类型
void

setObjectiveN()

在多目标优化中，设置线性目标函数。

OPTVModel.**setObjectiveN**(int *objId*, OPTVLinExpr *expr*, String *name*, int *priority*, double *weight* = *1.0*, double *absTolerance* = *0.0*, double *relTolerance* = *0.0*)

当模型中有多个目标函数时，该函数会替换当前具有相同 `Id` 的目标函数（而不是进行累加），否则会向模型中添加一个新的目标函数。目标函数的优化方向由参数 `weight` 的符号决定。

参数

- **objId** –待修改目标函数的索引。
- **expr** –需要设定的目标函数表达式。
- **name** –目标函数的名称。
- **priority** –目标函数的优先级（高优先级的目标函数会被优先求解）。
- **weight** –[**可选**, 默认值 `1.0`] 目标函数的权重（用于确定目标函数的优化方向，以及处理多个目标函数拥有相同优先级的情况）。
- **absTolerance** –[**可选**, 默认值 `0.0`] 目标函数的绝对容差（决定在后续的优化过程中该目标函数允许的最大绝对偏差）。
- **relTolerance** –[**可选**, 默认值 `0.0`] 目标函数的相对容差（决定在后续优化过程中该目标函数允许的最大相对偏差）。

返回
无。

返回类型
void

set()

参数

设置求解器的参数取值，一旦模型被创建，该接口仅提供控制此模型专用参数的接口，直到调用*OPTVModel.update()* 方法后，新修改才会在内部生效。

可用参数请参考*参数*。

OPTVModel.**set**(OPTVIntParam *param*, int *value*)

参数

- **param** –整数型参数。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

OPTVModel.**set**(OPTVDblParam *param*, double *value*)

参数

- **param** –双精度型参数。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

OPTVModel.**set**(OPTVStrParam *param*, String *value*)

参数

- **param** –字符串型参数。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

属性

设置求解器的属性值，同样，直到调用*OPTVModel.update()* 方法后，新修改才会在内部生效。

可用属性请参考*天筹属性*。

OPTVModel.**set**(OPTVIntAttr *attr*, int *value*)

参数

- **attr** –整数型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

OPTVModel.**set**(OPTVDblAttr *attr*, double *value*)

参数

- **attr** –双精度型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

OPTVModel.**set**(OPTVStrAttr *attr*, String *value*)

参数

- **attr** –字符串型属性。
- **value** –要设置的值。

返回
无。

返回类型
void

抛出
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

update()

执行模型更新。

OPTVModel.**update**()

该函数会进行模型更新，或 lazy 构建（如必要）。只有调用该函数 **后**，模型的更新才会生效。SDK 构建的模型需要调用此函数来保证内部模型的即时性。例如，用户调用*OPTVModel.optimize()* 前 **不需要**调用此函数，因为该函数在内部已被默认执行。

返回
无。

返回类型
void

writeIIS()

将不可约不一致子系统（IIS）写入文件（.ilp 格式）。

有关 IIS 的细节请参考*IIS 快速入门*。

OPTVModel.**writeIIS**(String *fileName*)

参数

- **fileName** –写出的文件名，必须包含 .ilp 后缀（例如 iis.ilp）.

返回
无。

返回类型
void

抛出
OPTVErrCode.ERROR_FILE_WRITE –不支持的文件格式，或者未计算出 IIS.

注解

此函数 **不会**计算 IIS，若 IIS 未被计算，本函数将抛出异常。

write()

将模型或基写出到文件。

OPTVModel.**write**(String *fileName*)

支持的文件格式请参考文件格式。

表 13: 支持写出的文件格式

.mps	.lp	.bas	.ilp

表 14: 支持写出的压缩格式

.gz	.z	.Z

参数

- **fileName** –C++ 字符串文件名。

返回
无。

返回类型
void

抛出
OPTVErrCode.ERROR_FILE_WRITE –文件名或后缀为空或错误。

错误码

OPTVErrCode 用来产生特定的*OPTVException* 异常对象，并给出相应的详细信息。

class OPTVErrCode

天筹错误码。

OPTVErrCode.**ERROR**

错误：错误信息，一般从求解器内部调用抛出。

OPTVErrCode.**WARNING**

警告：警告信息，一般从求解器内部调用抛出。

OPTVErrCode.**UNKNOWN**

未知：遇到未知错误。

OPTVErrCode.**INVALID_ARGUMENT**

无效参数：传入函数的参数不满足条件。

例如在设置变量类型*OPTVCharAttr.VAR_TYPE* 时，传入未定义的属性值，或者用 0 除线性表达式。

OPTVErrCode.**UNKNOWN_ATTRIBUTE**

未知属性：访问或者设置未定义的属性。

例如*OPTVConstr.get(OPTVIntAttr.NUM_VARS)* 将抛出此错误码，因为*OPTVIntAttr.NUM_VARS* 不是模型的属性。

OPTVErrCode.**DATA_NOT_AVAILABLE**

数据不可用：访问的属性值不可用。

例如在调用*OPTVModel.optimize()* 之前，试图获取解的信息*OPTVVar.get(OPTVDblAttr.X)* 将会抛出此错误。

OPTVErrCode.**INDEX_OUT_OF_RANGE**

下标越界：用于获取对象的索引值超出了允许范围。

例如，当前模型仅有 2 个变量，调用*model.getVar(10)* 会抛出此异常。

OPTVErrCode.**UNKNOWN_PARAMETER**

未知参数：访问未定义的参数。

OPTVErrCode.**VALUE_OUT_OF_RANGE**

数值越界：参数值超出了允许范围。例如尝试获取不存在的*OPTVSense*.

OPTVErrCode.**ERROR_FILE_READ**

文件读取失败：读取文件时发生错误。

OPTVErrCode.**ERROR_FILE_WRITE**

文件写出失败：写出文件时发生错误。

OPTVErrCode.**LICENSE_CHECK_FAILED**

许可检查失败：未发现有效许可。

注：许可文件 OPTV_LICENSE_FILE 需要通过环境变量设置，细节请参考安装说明。

OPTVErrCode.**INVALID_MODEL_INPUT**

模型输入无效：试图使用无效的模型输入信息。

OPTVErrCode.**NOT_IMPLEMENTED**

功能不可用：试图使用的功能暂未开放。

OPTVErrCode.**NOT_IN_MODEL**

模型组件不存在：访问的变量或约束在模型中不存在。

OPTVErrCode.**MATRIX_NOT_PSD**

非正半定矩阵：二次表达式的矩阵不满足正半定性质。

当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的。

异常

class OPTVException

异常类。

此类表示由天筹求解器抛出的异常。它提供了访问错误代码 (*OPTVErrCode*) 和描述性消息的功能。

使用*OPTVException.getErrorCode()* 来确定具体的错误类型，并使用*OPTVException.getMessage()* 获取便于理解的描述信息。

OPTVException.**getMessage**()

获取异常的错误信息。

返回
异常的错误信息。

返回类型
String

OPTVException.**getErrorCode**()

获取异常的错误码。

返回
异常的错误码。

返回类型
OPTVErrCode

关于*OPTVErrCode* 更多细节请参考错误码。

优化方向

class OPTVSense

目标含义，默认值为 1，表征最小化问题。

OPTVSense.**MINIMIZE**

最小化问题 (+1)

类型
int

取值
1

OPTVSense.**MAXIMIZE**

最大化问题 (-1)

类型
int

取值
-1

锥类型

class OPTVConeType

锥类型。

锥类型用于指定锥约束中的仿射表达式 必须满足几何约束 C。

OPTVConeType.**SOC**

二阶锥

二阶锥 C_s 的定义如下：

$$C_s^n = \left\{ x \in \mathbb{R}^n \mid x_1 \geq \sqrt{x_2^2 + x_3^2 + \cdots + x_n^2} \right\}.$$

OPTVConeType.**RSOC**

旋转二阶锥。

旋转二阶锥 C_r 的定义如下：

$$C_r^n = \left\{ x \in \mathbb{R}^n \mid 2x_1 x_2 \geq x_3^2 + \cdots + x_n^2,\ x_1, x_2 \geq 0 \right\}.$$

有关锥的详细介绍可见锥约束。

常量

OPTV_INF

表示一个无穷大的值。

类型
float

取值
1.0e+100

OPTV_UNDF

未定义的数值。

类型
float

取值
1.0e+101

变量类型

OPTV_BINARY

0-1 整数变量类型。

类型
char

取值
'B'

OPTV_INTEGER

整数变量类型。

类型
char

取值
'I'

OPTV_CONTINUOUS

连续变量类型。

类型
char

取值
'C'

模型求解状态

OPTV_UNKNOWN

未知模型求解状态（默认）。

类型
int

取值
0

OPTV_OPTIMAL

模型找到最优解，可以查询到最优解。

类型
int

取值
1

OPTV_INFEASIBLE

模型被证明无解。

类型
int

取值
2

OPTV_INF_OR_UNBD

模型被证明无解或者无界。

类型
int

取值
3

OPTV_UNBOUNDED

模型被证明无界。

类型
int

取值
4

OPTV_TIME_LIMIT

模型在指定时间*OPTVDblParam.TIME_LIMIT* 内没有完成求解。

类型
int

取值
5

OPTV_MEM_LIMIT

模型求解使用内存超过预设值*OPTVIntParam.MEM_LIMIT*.

类型
int

取值
6

OPTV_INTERRUPTED

用户终止模型求解（如通过 `Ctrl+C`）.

类型
int

取值
7

变量和约束的基状态

OPTV_BASIC

变量或约束为基。

类型
int

取值
0

OPTV_NONBASIC_LOWER

变量或约束非基，取下界值。

类型
int

取值
-1

OPTV_NONBASIC_UPPER

变量或约束非基，取上界值。

类型
int

取值
-2

OPTV_SUPERBASIC

变量或约束为超基。

类型
int

取值
-3

天筹一般约束类型

OPTV_GENCONSTR_AND

只有所有变量非零时，结果才非零。

类型
int

取值
0

OPTV_GENCONSTR_OR

只要有一个变量非零，结果就非零。

类型
int

取值
1

OPTV_GENCONSTR_INDICATOR

返回值为布尔型，当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0。

类型
int

取值
2

约束含义

OPTV_SENSE_LESS
小于或等于。

类型
char

取值
<

OPTV_SENSE_EQUAL
等于。

类型
char

取值
=

OPTV_SENSE_GREATER
大于或等于。

类型
char

取值
>

8.2.3 Python API 参考手册

变量

class OPTVVar

天筹求解器的变量对象，用于从模型访问变量。注意变量不可由其构造函数创建，而需要通过模型的*OPTVModel.AddVar()* 或*OPTVModel.AddVars()* 接口添加变量到模型。

Get()

查询变量属性。

可用属性请参考*天筹属性*。

OPTVVar.**Get**(*attr*)

参数
attr (OPTVIntAttr/OPTVDblAttr/OPTVCharAttr/OPTVStrAttr) –需要查询的变量属性。

返回
属性值。

返回类型
intlfloatlstr

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

示例

```
x = model.AddVar(0, 1, 0, OPTV_BINARY, "x")
model.Update()
# query the name of x
result = x.Get(OPTVStrAttr.NAME)
print(result) # x
```

Set()

设置变量属性，只有调用*OPTVModel.Update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVVar.**Set**(*attr*, *value*)

参数

- **attr** (OPTVIntAttr/OPTVCharAttr/OPTVDblAttr/OPTVStrAttr) –需要设置的变量属性。
- **value** (*int*/*float*/*str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

示例

```
x = model.AddVar(0, OPTV_INF, 0, OPTV_CONTINUOUS, "x")
x.Set(OPTVDblAttr.Ub, 100)
```

Index()

获得变量在模型中的索引值。

OPTVVar.**Index**()

返回
变量索引/下标。

返回类型
int

返回值

- **-1** –该变量在模型中不存在。
- **-2** –该变量已从模型中删除。
- **>=0** –该变量在模型中的索引/下标。

示例

```
x = model.AddVar(0, 1, 0, OPTV_BINARY, "x")
y = model.AddVar(0, 1, 0, OPTV_BINARY, "y")
x_ind = x.Index()
y_ind = y.Index()
print(x_ind, y_ind) # 0, 1
```

SameAs()

判断两个变量是否为同一个对象。

OPTVVar.**SameAs**(*v2*)

参数
v2 –需要与当前变量对比的变量。

返回
变量是否相同。

返回类型
bool

返回值

- **true** –变量相同。
- **false** –变量不同。

示例

```
x = model.AddVar(0, 1, 0, OPTV_BINARY, "x")
y = model.AddVar(0, 1, 0, OPTV_BINARY, "y")
z = x
print(x.SameAs(z)) # True
print(x.SameAs(y)) # False
```

矩阵变量

class OPTVMVar

天筹求解器的矩阵变量对象。OPTVMVar 类是一个构建多维变量，并且支持 NumPy 多维数组运算的类。我们建议通过模型的*OPTVModel.AddMVar()* 方法添加多维变量，而不是通过内置类方法转换生成。

Copy()

深度复制一个 OPTVMVar 对象。

OPTVMVar.**Copy**()

返回
原 OPTVMVar 对象的拷贝。

返回类型
OPTVMVar

示例

```
matrix_vars = model.AddMVar((2, 2), name="x")
copy_matrix_vars = matrix_vars.Copy()
```

Diagonal()

创建一个包含原 OPTVMVar 对象指定对角线上的变量的新 OPTVMVar 对象。

OPTVMVar.**Diagonal**(*offset=0*, *axis1=0*, *axis2=1*)

参数

- **offset** (*int*) –可选参数，表示对角线的偏移量，默认值为 0。若是大于 0，表示对角线向上的偏移量；若是小于 0，表示对角线向下的偏移量。
- **axis1** (*int*) –可选参数，作为二维子 OPTVMVar 的第一轴的轴，对角线从这里开始。默认值为 0。
- **axis2** (*int*) –可选参数，作为二维子 OPTVMVar 的第二轴的轴，对角线从这里开始。默认值为 1。

返回
一个包含原 OPTVMVar 对象指定对角线上的变量的新 OPTVMVar 对象。

返回类型
OPTVMVar

示例

```
matrix_vars = model.AddMVar((8, 8), name="x")
diagonal_main = matrix_vars.Diagonal()    # the main diagonal of␣
↪matrix_vars
diagonal_sub1 = matrix_vars.Diagonal(1)   # the OPTVVar above the main␣
↪diagonal
diagonal_sub1 = matrix_vars.Diagonal(-2) # the OPTVVar below the main␣
↪diagonal
```

FromList()

将元素为 OPTVVar 的列表转化为 OPTVMVar 对象。该方法为类生成方法，无需 OPTVMVar 对象。

OPTVMVar.**FromList**(*varlist*)

参数
varlist (*list*) –元素为 OPTVVar 的列表。

返回
新的 OPTVMVar 对象，其维度取决于参数 varlist 的维度。

返回类型
OPTVMVar

示例

```
x0 = model.AddVar(0, 1, 0, OPTV_BINARY, name="x0")
x1 = model.AddVar(0, 1, 0, OPTV_BINARY, name="x1")
x2 = model.AddVar(0, 1, 0, OPTV_BINARY, name="x2")
x3 = model.AddVar(0, 1, 0, OPTV_BINARY, name="x3")
matrix_1d = OPTVMMVar.FromList([x0, x1, x2, x3])      # 1-D OPTVMVar
matrix_2d = OPTVMMVar.FromList([[x0, x1], [x2, x3]]) # 2-D OPTVMVar
```

FromVar()

将 OPTVVar 对象转化为一个 0 维的 OPTVMVar 对象。该方法为类生成方法，无需 OPTVMVar 对象。

OPTVMVar.**FromVar**(*var*)

参数
var (OPTVVar) –OPTVVar 对象。

返回
新的 0 维 OPTVMVar 对象。

返回类型
OPTVMVar

示例

```
x = model.AddVar(0, 1, 0, OPTV_BINARY, name="x")
matrix_1d = OPTVMMVar.FromVar(x)
```

Get()

查询 OPTVMVar 对象中每个变量的属性。

可用属性请参考天筹属性。

OPTVMVar.**Get**(*attr*)

参数
attr (OPTVIntAttr/OPTVDblAttr/OPTVCharAttr/OPTVStrAttr) –需要查询的变量属性，包括整数型、双精度型、字符和字符串型。

返回
一个 NumPy 数组，维度与 OPTVMVar 的维度相同，其元素为对应变量相关属性的取值。

返回类型
numpy.ndarray

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

示例

```
matrix_vars = model.AddMVar((2, 2), name="x")
print(matrix_vars.Get(OPTVCharAttr.VAR_TYPE))
```

Set()

设置 OPTVMVar 对象中所有变量的属性，只有调用*OPTVModel.Update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVMVar.**Set**(*attr*, *newvalue*)

参数

- **attr** (OPTVIntAttr / OPTVCharAttr / OPTVDblAttr / OPTVStrAttr) –需要设置的变量属性。
- **newvalue** (*int* / *float* / *str*) –需要设置的属性的值，其形状必须与 OPTVMVar 对象的形状相同。也可以是一个标量，它将自动扩展为正确的形状。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知变量属性。

示例

```
matrix_vars = model.AddMVar((2, 2), name="x")
matrix_vars.Set(OPTVIntAttr.BASIS, 1)
```

Item()

若是 OPTVMVar 中只有一个 OPTVVar 对象，返回该 OPTVVar 的拷贝。若是在 OPTVMVar 有多个变量时调用该方法，会产生 ValueError。

OPTVMVar.**Item**()

返回
OPTVMVar 的唯一元素 OPTVVar 的拷贝。

返回类型
OPTVVar

引发
ValueError –OPTVMVar 中有多个元素。

示例

```
matrix_vars = model.AddMVar((2, 2), name="x")
matrix_var = matrix_vars[0, 1] # A 0-dimensional OPTVMVar containing␣
↪a single OPTVVar object
var = matrix_vars[0, 1].Item() # The OPTVVar object that resides␣
↪within
```

Reshape()

返回一个新的 OPTVMVar 对象，其元素与元 OPTVMVar 相同，形状与参数 shape 相同。

OPTVMVar.**Reshape**(*shape*)

参数
shape –一个整数或者元组，表示新 OPTVMVar 对象的形状。

返回
OPTVMVar 对象，其形状与给定形状相同。

返回类型
OPTVMVar

示例

```
matrix_vars = model.AddMVar((2, 2), name="x")
new_matrix_vars = matrix_vars.Reshape(4) # 1-D
```

ToList()

将 OPTVMVar 对象转换为包含其所有变量的列表。

OPTVMVar.**ToList**()

返回
包含 OPTVMVar 所有变量的列表。

返回类型
list[*OPTVVar*]

示例

```
matrix_vars = model.AddMVar(3, name="x")
vars_list = matrix_vars.ToList()
print(vars_list) # [<optv.Var x_0> <optv.Var x_1> <optv.Var x_2>]
```

Transpose()

生成一个新的 OPTVMVar 对象，它是原 OPTVMVar 对象的转置。

OPTVMVar.**Transpose**()

返回
新的 OPTVMVar 对象，为原 OPTVMVar 对象的转置。

返回类型
OPTVMVar

示例

```
matrix_vars = model.AddMVar((3, 1), name="x")
matrix_vars_T = matrix_vars.Transpose()
print(matrix_vars_T.Shape) # (1, 3)
matrix_vars = model.AddMVar((2, 3), name="x")
matrix_vars_T = matrix_vars.Transpose()
print(matrix_vars_T.Shape) # (3, 2)
```

Ndim 维度

获取 OPTVMVar 对象的维度。

`OPTVMVar.`**`Ndim`**

返回
OPTVMVar 对象的维度。

返回类型
int

示例

```
matrix_vars = model.AddMVar((2, 2), name="x")
print(matrix_vars.Ndim) # "2"
matrix_vars = model.AddMVar((2), name="x")
print(matrix_vars.Ndim) # "1"
```

Shape

获取 OPTVMVar 对象的形状。

`OPTVMVar.`**`Shape`**

返回
OPTVMVar 对象的形状。

返回类型
tuple[int, ···]

示例

```
matrix_vars = model.AddMVar((2, 2), name="x")
print(matrix_vars.Shape) # (2, 2)
matrix_vars = model.AddMVar(3, name="x")
print(matrix_vars.Shape) # (3,)
```

Size

获取 OPTVMVar 对象中的元素个数。

`OPTVMVar.Size`

返回
OPTVMVar 对象中的元素个数。

返回类型
int

示例

```
matrix_vars = model.AddMVar((2, 2), name="x")
print(matrix_vars.Size) # 4
matrix_vars = model.AddMVar(3, name="x")
print(matrix_vars.Size) # 3
```

T

类似于类方法 Transpose()。

`OPTVMVar.T`

返回
新的 OPTVMVar 对象，为原 OPTVMVar 对象的转置。

返回类型
OPTVMVar

示例

```
matrix_vars = model.AddMVar((3, 1), name="x")
matrix_vars_T = matrix_vars.T
print(matrix_vars_T.Shape) # (1, 3)
matrix_vars = model.AddMVar((2, 3), name="x")
matrix_vars_T = matrix_vars.T
print(matrix_vars_T.Shape) # (3, 2)
```

列

class OPTVColumn

天筹求解器的列对象，用于以列的形式向模型添加变量，用户可以使用该功能向模型同时新增变量并添加其至已有约束。

在下面的例子中，我们构造一个初始模型并展示如何通过列对象增加变量，本示例展示的模型为*Quick Start* 章节的*milp1* .

```
from optvpy import *

env = OPTVEnv()
model = OPTVModel(env)

x = model.AddVar(0, 1, -1, OPTV_BINARY, "x")
```

(下页继续)

(续上页)

```
y = model.AddVar(0, OPTV_INF, -14, OPTV_INTEGER, "y")

c0 = model.AddConstr(2 * x + y, -OPTV_INF, 3, "c0")
c1 = model.AddConstr(3 * y, -OPTV_INF, 6, "c1")
```

具体地，构建的模型形式如下：

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}$$

```
col = OPTVColumn()

col.addTerm(1, c0)
col.addTerm(1, c1)

z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, col, "z");
```

通过将 col 添加到模型，相应的约束矩阵变为：

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 3 & 1 \end{bmatrix}$$

AddTerm()

向列对象（尾部）添加单项。

OPTVColumn.**AddTerm**(*coef*, *constr*)

该函数向列对象添加单项。

参数

- **coef** (*float*) –系数值。
- **constr** (OPTVConstr) –约束对象。

返回

无。

返回类型

None

示例

```
col = OPTVColumn()

col.AddTerm(1.0, x)
col.AddTerm(3.0, y)
col.AddTerm(2.0, z)
```

AddTerms()

向列对象（尾部）添加多项。

OPTVColumn.**AddTerms**(*coefs*, *constrs*)

该功能向列对象添加多项。

参数

- **coefs** (*list[float]*) –系数值列表。
- **constrs** (*list[OPTVConstr]*) –约束列表。

返回

无。

返回类型

None

引发

OPTVErrCode.INVALID_ARGUMENT –constrs 或 coefs 为空。

示例

```
constrs = self.model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
```

Clear()

清空与该列相关内部容器，使*OPTVColumn.Size()* 返回 0.

OPTVColumn.**Clear**()

返回

无。

返回类型

None

示例

```
model.Update()

constrs = model.GetConstrs()
coefs = [1.0, 2.0, 3.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
print(col.Size()) # 3

col.Clear()
print(col.Size()) # 0
```

GetCoef()

据指定索引值获得列对象中对应的约束系数。

OPTVColumn.**GetCoef**(*i*)

参数
i (*int*) –指定的索引值，对应列中的约束。

返回
列对象中对应该索引, i, 的约束系数。

返回类型
float

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i ∉ [0, this->Size()]。

示例

```
col = OPTVColumn()

col.AddTerm(1.0, x)
col.AddTerm(3.0, y)
col.AddTerm(2.0, z)

print(col.GetCoef(0)) # 1.0
```

GetConstr()

根据指定索引值获得列对象中对应的约束。

OPTVColumn.**GetConstr**(*i*)

参数
i (*int*) –指定的索引值，对应列中的约束。

返回
列对象中对应该索引, i, 的约束对象。

返回类型
OPTVConstr

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i ∉ [0, this->Size()]。

示例

```
constrs = model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)

col.GetConstr(0)
```

Remove()

从列对象中删除单/多项。

OPTVColumn.**Remove**(*i*)

根据指定索引值从该列对象中删除项。

参数

i (*int*) –指定的索引值。

返回

无。

返回类型

None

引发

OPTVErrCode.INDEX_OUT_OF_RANGE –若 i ∉ [0, this->Size()]。

示例

```
model.Update()

constrs = model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
print(col.Size()) # 3

col.Remove(1)
print(col.Size()) # 2
```

OPTVColumn.**Remove**(*constr*)

从列对象中删除指定约束的所有相关项。

参数

constr (OPTVConstr) –指定要删除的约束。

返回

是否成功删除任何项。

返回

无。

返回类型

None

引发

OPTVErrCode.NOT_IN_MODEL –指定的约束，constr, 在模型中不存在。

示例

```
model.Update()

constrs = model.GetConstrs()
```

(下页继续)

(续上页)

```
coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
print(col.Size()) # 3

col.Remove(constrs[1])
print(col.Size()) # 2
```

Size()

获得该列的规模。

OPTVColumn.**Size**()

该数值统计对应列中的元素个数，无论是否有重复项，例如，若某约束被多次添加，本数值依然相应增加。

返回

该列的规模。

返回类型

int

示例

```
model.Update()

constrs = model.GetConstrs()

coefs = [1.0, 3.0, 2.0]

col = OPTVColumn()
col.AddTerms(coefs, constrs)
print(col.Size()) # 3
```

线性表达式

class OPTVLinExpr

线性表达式*OPTVLinExpr*类用于构建线性的目标函数或者约束函数。线性表达式对象由变量和常量组成，边获取边构造，如下例所示的表达式 `1 * x + 2 * y + 3 * x`，它包含以下元素：

索引	系数	变量
0	1	x
1	2	y
2	3	x

AddTerms()

该功能向当前表达式（后）添加一组新的项。

OPTVLinExpr.**AddTerms**(*coefs*, *vars*, *cnt*)

参数

- **coefs** (*list[float]*) –系数值列表。
- **vars** (*list[OPTVVar]*) –新添加项的变量系数。
- **cnt** (*int*) –新添加项的变量列表。

返回
无。

返回类型
None

引发
OPTVErrCode.INVALID_ARGUMENT –vars 或 coefs 为空，或 cnt 取负值。

示例

```
expr = x + 1
expr.AddTerms([2.0, 3.0], [y, z], 2)
model.Update()
print(expr) # x + 2*y + 3*z + 1
```

Clear()

清空该表达式，*OPTVLinExpr.Size()* 将返回 0. 同时，该表达式从数值意义上将等同于 0.

OPTVLinExpr.**Clear**()

返回
无。

返回类型
None

示例

```
expr = x + 1
expr.Clear()
print(expr)       # 0
print(expr.Size)  # 0
```

GetCoef()

据指定索引值获得表达式中对应变量的系数。

OPTVLinExpr.**GetCoef**(*i*)

参数
i (*int*) –变量在线性表达式中的索引。

返回
对应的变量系数。

返回类型
float

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0, \texttt{this->Size()}]$。

示例

```
expr = 2*x + 3*y + z + 1
result = expr.GetCoef(1)
print(result) # 3
```

GetVar()

根据指定索引值获得表达式中的对应变量。

OPTVLinExpr.**GetVar**(*i*)

参数
i (*int*) –变量在线性表达式中的索引。

返回
指定位置上的变量。

返回类型
OPTVVar

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0, \texttt{this->Size()}]$。

示例

```
expr = 2*x + 3*y + 1
result = expr.GetVar(0)
print(result) # x
```

GetConstant()

获得当前表达式中的常数项。

OPTVLinExpr.**GetConstant**()

返回
表达式中的常量数值。

返回类型
float

示例

```
expr = 2*x + 3*y + z + 6
result = expr.GetConstant()
print(result) # 6
```

GetValue()

获得该表达式的取值。

OPTVLinExpr.**GetValue**()

返回
对应原始解的表达式的取值，即该表达式的变量值与系数的乘积之和。

返回类型
float

引发
OPTVErrCode.DATA_NOT_AVAILABLE –当前变量取值不存在。

示例

```
model.Optimize()
obj = model.GetObjective()
result = obj.GetValue()
```

Remove()

从表达式中删除项。

OPTVLinExpr.**Remove**(*i*)
根据指定索引值从该表达式中删除项。

参数
i (*int*) –指定的索引值。

返回
无。

返回类型
None

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0, \texttt{this->Size()}]$。

示例

```
expr = 2*x + 3*y + 1
expr.Remove(0)
print(expr) # 3*y + 1
```

OPTVLinExpr.**Remove**(*v*)

指定的变量（在模型中）不存在。

参数

v (OPTVVar) –指定要删除的变量。

返回

是否成功删除任何项。

返回类型

bool

返回值

- **true** –成功删除任何项。
- **false** –未删除任何项。

引发

OPTVErrCode.NOT_IN_MODEL –成功删除与指定变量有关的项。

示例

```
expr = 2*x + 3*y + 1
expr.Remove(y)
print(expr) # 2*x + 1
```

SameAs()

判断两个线性表达式是否为同一个对象。

OPTVLinExpr.**SameAs**(*e2*)

参数

e2 (OPTVLinExpr) –需要与当前线性表达式对比的线性表达式。

返回

线性表达式是否相同。

返回类型

bool

返回值

- **true** –线性表达式相同。
- **false** –线性表达式不同。

示例

```
expr1 = 2*x + 3*y + 1
expr2 = x + y
expr3 = 2*x + 3*y + 1
print(expr1.SameAs(expr3)) # True
print(expr1.SameAs(expr2)) # False
```

Size()

获得该表达式的规模。

OPTVLinExpr.**Size**()

该函数返回的数值对应线性表达式中的变量个数，不论变量是否重复出现。例如表达式 x+y 对应 Size = 2，而 x+y+x 对应 Size = 3。

返回
线性表达式大小。

返回类型
int

示例

```
expr = 2*x + 3*y + 1
print(expr.Size) # 2
```

线性矩阵表达式

class OPTVMLinExpr

线性矩阵表达式对象。线性矩阵表达式对象是对*OPTVMVar* 对象进行线性计算后得到的。用户可以使用重载运算符来构建线性矩阵表达式，例如 A @ x 或 x + 1（x 为*OPTVMVar* 对象）。用户也可以使用*OPTVMLinExpr.Zeros()* 来构建一个所有表达式均为零的 OPTVMLinExpr 对象。

Copy()

深度复制一个 OPTVMLinExpr 对象。

OPTVMLinExpr.**Copy**()

返回
原 OPTVMLinExpr 对象的拷贝。

返回类型
OPTVMLinExpr

示例

```
expr = model.AddMVar(3, name="x") + 1
expr_copy = expr.Copy()
```

Clear()

将 OPTVMLinExpr 对象中的所有表达式重置为 0。

OPTVMLinExpr.**Clear**()

返回
无。

返回类型
None

示例

```
expr = model.AddMVar(3, name="x") + 1
expr.Clear()
# All three entries are set to the constant value of 0.0.
print(expr) # ['0.0' '0.0' '0.0']
```

GetValue()

获取 OPTVMLinExpr 对象中线性表达式的值。

OPTVMLinExpr.**GetValue**()

返回
一个 NumPy 数组，其维度和形状与 OPTVMLinExpr 对象相同，其元素为对应表达式的值。

返回类型
numpy.ndarray

示例

```
expr = model.AddMVar(3, name="x") + 1
model.AddConstrs((expr[i].Item() <= 1 for i in range(3)), name='c')
model.Optimize()
values = expr.GetValue()
```

Item()

若是 OPTVMLinExpr 中只有一个 OPTVLinExpr 对象，返回该 OPTVLinExpr 对象的拷贝。若是在 OPTVMLinExpr 有多个线性表达式时调用该方法，会产生 ValueError。

OPTVMLinExpr.**Item**()

返回
OPTVMLinExpr 的唯一元素 OPTVLinExpr 的拷贝。

返回类型
OPTVLinExpr

引发
ValueError –OPTVMLinExpr 中有多个元素。

示例

```
expr = model.AddMVar(3, name="x") + 1
expr_sub = expr[0, 1]       # A 0-D OPTVMLinExpr that includes one␣
↪OPTVLinExpr object within it
expr_le = expr[0, 1].Item() # A copy of the resident OPTVLinExpr␣
↪object
```

Zeros()

创建一个元素全为零的 OPTVMLinExpr 对象。

OPTVMLinExpr.**Zeros**(*shape*)

参数
shape (*int | tuple[int, ...]*) –表示创建的 OPTVMLinExpr 对象的形状。

返回
元素全为零的 OPTVMLinExpr 对象。

返回类型
OPTVMLinExpr

示例

```
expr_zeros = OPTVMLinExpr.Zeros((2, 2)) # all elements in this␣
↪OPTVMLinExpr object are zero
```

Sum()

对 OPTVMLinExpr 中的元素进行求和。

OPTVMLinExpr.**Sum**(*axis=None*)

参数
axis (*int | None*) –若是整数，对给定轴进行求和；若为 None，对所有元素进行求和。

返回
按照给定轴求和后的 OPTVMLinExpr 对象。

返回类型
OPTVMLinExpr

示例

```
expr = model.AddMVar((2, 3), name='x') + 1
sum_row = expr.Sum(axis=0) # sum the OPTVLinExpr for each row
sum_col = expr.Sum(axis=1) # sum the OPTVLinExpr for each column
sum_all = expr.Sum()       # sum all OPTVLinExpr objects, return a 0-
↪D OPTVMLinExpr object
```

Ndim 维度

获取 OPTVMLinExpr 对象的维度。

OPTVMLinExpr.**Ndim**

返回
OPTVMLinExpr 对象的维度数目。

返回类型
int

示例

```
expr1 = model.AddMVar((3,), name="x") + 1
print(expr1.Ndim) # 1
expr2 = model.AddMVar((1, 3), name='y') + 1
print(expr2.Ndim) # 2
```

Shape

获取 OPTVMLinExpr 对象的形状。

OPTVMLinExpr.**Shape**

返回
一个表示 OPTVMLinExpr 对象形状的整数元组。

返回类型
tuple[int, ···]

示例

```
expr1 = model.AddMVar((3,), name="x") + 1
print(expr1.Shape) # (3,)
expr2 = model.AddMVar((1, 3), name='y') + 1
print(expr2.Shape) # (1, 3)
```

Size

获取 OPTVMLinExpr 对象元素的个数。

OPTVMLinExpr.**Size**

返回
OPTVMLinExpr 对象元素的个数。

返回类型
int

示例

```
expr1 = model.AddMVar((3,), name="x") + 1
print(expr1.Size) # 3
expr2 = model.AddMVar((2, 3), name='y') + 1
print(expr2.Size) # 6
```

线性约束

class OPTVConstr

天筹求解器约束对象，用于从模型访问约束。注意约束不可由其构造函数创建，而需要通过模型的*OPTVModel.AddConstr()*或*OPTVModel.AddConstrs()*接口添加约束至模型。此外，二次约束的相应接口为*OPTVModel.AddQConstr()*.

Get()

查询约束属性。

可用属性请参考*天筹属性*。

OPTVConstr.**Get**(*attr*)

参数
attr (OPTVIntAttr / OPTVDblAttr / OPTVStrAttr) –需要查询的约束属性。

返回
属性值。

返回类型
int|float|str

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
c0 = model.AddConstr(x+y+z, 0, 3, "c0")
model.Update()
print(c0.Get(OPTVIntAttr.PROHIBITED)) # 0
```

Set()

设置约束属性，只有调用*OPTVModel.Update()*后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVConstr.**Set**(*attr*, *value*)

参数

- **attr** (OPTVIntAttr / OPTVDblAttr / OPTVStrAttr) –需要设置的约束属性，包括整数型、双精度型和字符串型。
- **value** (*int* / *float* / *str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
c0 = model.AddConstr(x+y+z, 0, 3, "c0")

c0.Set(OPTVDblAttr.UB, 999)
print(c0.Get(OPTVDblAttr.UB)) # 3
model.Update()

print(c0.Get(OPTVDblAttr.UB)) # 999
```

Index()

获得约束在模型中的索引值。

OPTVConstr.**Index**()

返回
约束索引。

返回类型
int

返回值

- **-1** –该约束在模型中不存在。
- **-2** –该约束已从模型中删除。
- **>=0** –该约束在模型中的索引/下标。

示例

```
c0 = model.AddConstr(x+y+z, 0, 3, "c0")
c1 = model.AddConstr(2*x+z, 2, 6, "c1")

print(c0.Index()) # 0
print(c1.Index()) # 1
```

SameAs()

查询两约束是否相同。

OPTVConstr.**SameAs**(*c2*)

参数
c2 (OPTVConstr) –需要与当前约束对比的约束。

返回
约束是否相同。

返回类型
bool

返回值

- **true** –约束相同。
- **false** –约束不同。

示例

```
c0 = model.AddConstr(x+y+z, 0, 3, "c0")
c1 = model.AddConstr(2*x+z, 2, 6, "c1")
copy_c0 = c0

print(c0.SameAs(copy_c0)) # True
print(c1.SameAs(c1))      # False
```

线性约束矩阵

class OPTVMConstr

线性约束矩阵对象。线性约束矩阵对象在数据结构上等效于存储线性约束（*OPTVConstr*）的 numpy.ndarray 数组。推荐用户使用*OPTVModel.AddMConstr()* 添加线性约束矩阵对象。

FromList()

根据约束列表生成一个 OPTVMConstr 对象。新生成的 OPTVMConstr 对象的形状由输入列表决定。一个*OPTVConstr* 对象的列表将生成一维的 OPTVMConstr 对象；一个包含*OPTVConstr* 对象列表的列表将生成二维的 OPTVMConstr 对象；以此类推。

OPTVMConstr.**FromList**(*constr_list*)

参数

constr_list (*list*) –list[OPTVConstr]

返回

新的 OPTVMConstr 对象，其维度取决于*OPTVConstr* 列表的维度。

返回类型

OPTVMConstr

示例

```
constrs_list = model.GetConstrs()
matrix_constrs = OPTVMConstr.FromList(constrs_list)
```

Get()

查询 OPTVMConstr 对象中每条约束的属性。

OPTVMConstr.**Get**(*attrname*)

参数

attr (OPTVIntAttr/OPTVDblAttr/OPTVCharAttr/OPTVStrAttr) –需要查询的约束属性。

返回

一个 NumPy 数组，维度与 OPTVMConstr 的维度相同，其元素为对应约束相关属性的取值。

返回类型

numpy.ndarray

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
x = model.AddMVar(3, ub=1, vtype=OPTV_BINARY, name=['x1', 'y1', 'z1'])
constrs = model.AddMConstr(np.array([[1, 2, 3], [1, 1, 0]]), x, [-
↪OPTV_INF, 1], [4, OPTV_INF], name='c')
result = constrs.Get(OPTVDblAttr.UB)
```

Set()

设置 OPTVMConstr 对象中所有约束的属性，只有调用*OPTVModel.Update()* 后，新设置才可生效。

可用属性请参考天筹属性。

OPTVMConstr.**Set**(*attrname*, *newvalue*)

参数

- **attrname** (OPTVIntAttr / OPTVDblAttr / OPTVCharAttr / OPTVStrAttr) –需要设置的约束属性。
- **newvalue** (*int* / *float* / *str*) –需要设置的属性的值，其形状必须与 OPTVMConstr 对象的形状相同。也可以是一个标量，它将自动扩展为正确的形状。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
x = model.AddMVar(3, ub=1, vtype=OPTV_BINARY, name=['x1', 'y1', 'z1'])
constrs =  model.AddMConstr(np.array([[1, 2, 3], [1, 1, 0]]), x, [-
↪OPTV_INF, 1], [4, OPTV_INF], name='c')
constrs.Set(OPTVDblAttr.LB, -10)
```

ToList()

将 OPTVMConstr 对象转换为包含其所有约束的列表（*OPTVConstr*）。

OPTVMConstr.**ToList**()

返回
包含 OPTVMConstr 所有约束的列表。

返回类型
list[*OPTVConstr*]

示例

```
x = model.AddMVar(3, ub=1, vtype=OPTV_BINARY, name=['x1', 'y1', 'z1'])
constrs =  model.AddMConstr(np.array([[1, 2, 3], [1, 1, 0]]), x, [-
↪OPTV_INF, 1], [4, OPTV_INF], name='c')
constrs_list = constrs.ToList()
```

二次表达式

class OPTVQuadExpr

二次表达式类*OPTVQuadExpr* 用于构建二次的目标函数或约束。二次表达式对象由变量和常量组成，通常包含三部分：二次项、线性项和常数项。其中，二次项依照第一个变量、第二个变量和系数的顺序保存在容器中，而线性部分以前一章节所述的线性表达式的形式保存。二次表达式的索引仅限于获取二次项，线性项部分的需单独获取。

如下例所示的二次表达式 `1*x*x + 2*x*y + 3*x*y + 4*y*y + 5*x + 6*y + 7*x` 的描述如下：

索引	Coeff	变量 1	变量 2
0	1	x	x
1	2	x	y
2	3	x	y
3	4	y	y

它的线性项部分需要通过*OPTVQuadExpr.GetLinExpr()* 接口访问，使用方法可参考*线性表达式* 。

索引	系数	变量
0	5	x
1	6	y
2	7	x

AddConstant()

向二次表达式增加常数项。

OPTVQuadExpr.**AddConstant**(*c*)

此函数向当前二次表达式添加常数项，但不是覆盖。

参数

c (*float*) –要添加的常数。

返回

无。

返回类型

None

示例

```
quad_expr = x*y + x
quad_expr.AddConstant(3)
print(quad_expr) # x*y + x + 3
```

AddTerm()

在二次表达式的末尾添加一个线性项。

OPTVQuadExpr.**AddTerm**(*coeff*, *var*)

此函数会将 coeff * var 项添加到表达式末尾。

参数

- **coeff** (*float*) –该函数向当前二次表达式后添加 coeff * var 项。
- **var** (OPTVVar) –变量对象。

返回

无。

返回类型

None

引发

OPTVErrCode.INVALID_ARGUMENT –需添加的变量 var 为空。

示例

```
quad_expr = x*y + x
quad_expr.AddTerm(3, z)
print(quad_expr) # x*y + x + 3*z
```

向当前二次表达式后添加一个二次性项。

OPTVQuadExpr.**AddTerm**(*coeff*, *var1*, *var2*)

该函数向当前二次表达式后添加 coeff * var1 * var2 项。

参数

- **coeff** (*float*) –该函数向当前二次表达式后添加 coeff * var 项。
- **var1** (OPTVVar) –该项的第一个变量。
- **var2** (OPTVVar) –该项的第二个变量。

返回

无。

返回类型

None

引发

OPTVErrCode.INVALID_ARGUMENT –需添加的项中变量 var1 或 var2 为空。

示例

```
quad_expr = x*y + x
quad_expr.AddTerm(3, z, x)
print(quad_expr) # x*y + x + 3*z*x
```

AddTerms()

向当前二次表达式后添加多个线性项。

OPTVQuadExpr.**AddTerms**(*coeff*, *var*, *cnt*)

该函数向当前二次表达式后添加 $\text{coeff}_i * \text{var}_i, \forall i \in [0, \text{cnt}]$.

参数

- **coeff** (*list[float]*) –系数值列表。
- **var** (*list[OPTVVar]*) –新添加项的变量系数。
- **cnt** (*int*) –新添加项的变量列表。

返回

无。

返回类型

None

引发

OPTVErrCode.INVALID_ARGUMENT –变量 var 或 coeff 为空，或 cnt 取负值。

示例

```
quad_expr = y*y
quad_expr.AddTerms([1, 2], [x, z], 2)
print(quad_expr) # y*y + x + 2*z
```

向当前二次表达式后添加多个二次性项。

OPTVQuadExpr.**AddTerms**(*coeff*, *var1*, *var2*, *cnt*)

该函数向当前二次表达式后添加 $\text{coeff}_i * \text{var1}_i * \text{var2}_i, \forall i \in [0, \text{cnt}]$.

参数

- **coeff** (*list[float]*) –系数值列表。
- **var1** (*list[OPTVVar]*) –第一变量列表。
- **var2** (*list[OPTVVar]*) –第二变量列表。
- **cnt** (*int*) –新添加项的变量列表。

返回

无。

返回类型

None

引发

OPTVErrCode.INVALID_ARGUMENT –变量 var1，var2 或 coeff 为空，或 cnt 取负值。

示例

```
quad_expr = y*y
quad_expr.AddTerms([1, 2], [x, z], [x, z], 2)
print(quad_expr) # y*y + x*x + 2*z*z
```

Clear()

清空该表达式。

OPTVQuadExpr.**Clear**()

该函数清空该表达式涉及的所有存储容器，*OPTVQuadExpr.Size()* 将返回 0。同时，该表达式也清空相应的线性表达式部分，重置常数项为 0。

返回
无。

返回类型
None

示例

```
quad_expr = x*y + z
quad_expr.Clear()
print(quad_expr.Size()) # 0
```

GetCoeff()

据指定索引值获得表达式中对应二次项的系数。

OPTVQuadExpr.**GetCoeff**(*i*)

参数
i (*int*) –表达式中该术语的索引。

返回
对应的二次项系数。

返回类型
float

引发
OPTVErrCode.DATA_NOT_AVAILABLE –若 i $\notin [0, \texttt{this->Size()}]$。

示例

```
quad_expr = x*y - 2*y*z
coeff1 = quad_expr.GetCoeff(0)
coeff2 = quad_expr.GetCoeff(1)
print(coeff1) # 1
print(coeff2) # -2
```

GetConstant()

获得当前表达式中的常数项。

OPTVQuadExpr.**GetConstant**()

返回
表达式中的常量数值。

返回类型
float

示例

```
quad_expr = x*y - 2*y*z + 3
constant = quad_expr.GetConstant()
print(constant) # 3
```

GetLinExpr()

获取二次表达式的线性部分。

OPTVQuadExpr.**GetLinExpr**()

返回
二次表达式的线性部分。

返回类型
OPTVLinExpr

示例

```
quad_expr = x*y - 2*z + 3
lin_expr = quad_expr.GetLinExpr()
print(lin_expr) # -2*z
```

GetValue()

获得该二次表达式的取值。

OPTVQuadExpr.**GetValue**()

返回
对应原始解的表达式的取值，即该表达式的变量值与系数的乘积之和。

返回类型
float

引发
OPTVErrCode.DATA_NOT_AVAILABLE –当前变量取值不存在。

示例

```
model.Optimize()
result = model.GetQObjective()
print(result.GetValue())
```

GetVar1()

根据指定索引值获得表达式中的对应二次项的第一个变量。

OPTVQuadExpr.**GetVar1**(*i*)

该函数返回表达式中的对应二次项的第一个变量，如表达式 `x*x + x*y + y*y` 的描述如下：

索引	变量 1	变量 2
0	x	x
1	x	y
2	y	y

参数
i (*int*) –指定的二次项对应的索引值。

返回
对应二次项的第一个变量。

返回类型
OPTVVar

引发
OPTVErrCode.DATA_NOT_AVAILABLE –若 i ∉ [0, this->Size()]。

示例

```
quad_expr = x1*z + *x2*y
print(quad_expr.GetVar1(1)) # x2
print(quad_expr.GetVar2(1)) # y
```

GetVar2()

根据指定索引值获得表达式中的对应二次项的第二个变量。请参考*OPTVQuadExpr.GetVar1()* 和示例。

OPTVQuadExpr.**GetVar2**(*i*)

该函数返回表达式中的对应二次项的第二个变量。

参数
i (*int*) –指定的二次项对应的索引值。

返回
对应二次项的第二个变量。

返回类型
OPTVVar

引发
OPTVErrCode.DATA_NOT_AVAILABLE –若 i ∉ [0, this->Size()]。

示例

```
quad_expr = x1*z + *x2*y
print(quad_expr.GetVar1(1)) # x2
print(quad_expr.GetVar2(1)) # y
```

Remove()

从二次表达式中删除项。

OPTVQuadExpr.**Remove**(*i*)

根据指定索引值从二次表达式中删除二次项，但不可通过此函数删除线性项的任何部分。

参数

i (*int*) –指定的索引值。

返回

无。

返回类型

None

引发

OPTVErrCode.DATA_NOT_AVAILABLE –若 i ∉ [0, this->Size()]。

示例

```
quad_expr = x*x + x*y + y*y
quad_expr.Remove(1)
print(quad_expr) # x*x + y*y
```

从表达式中删除变量。

OPTVQuadExpr.**Remove**(*v*)

从二次表达式中（包括线性部分）删除指定变量涉及的所有项。

参数

v (OPTVVar) –指定要删除的变量。

返回

无。

返回类型

None

引发

OPTVErrCode.NOT_IN_MODEL –成功删除与指定变量有关的项。

示例

```
quad_expr = x*x + x + y*y
quad_expr.Remove(x)
print(quad_expr) # y*y
```

SameAs()

查询两二次表达式是否相同。

OPTVQuadExpr.**SameAs**(*e2*)

参数

e2 (OPTVQuadExpr) –需要与当前二次表达式对比的线性表达式。

返回
二次表达式是否相同。

返回类型
bool

返回值

- **true** –二次表达式相同。
- **false** –二次表达式不同。

示例

```
quad_expr = x*x + x + y*y
quad_expr1 = x*z + y + 1
quad_expr2 = quad_expr
print(quad_expr.SameAs(quad_expr1)) # False
print(quad_expr.SameAs(quad_expr2)) # True
```

Size()

获得该表达式的规模。

OPTVQuadExpr.**Size**()

该数值对应当前二次表达式中的 二**次**项的个数，不论重复与否。例如，表达式 `x*x + y*y + y` 对应 `Size = 2`，而表达式 `x*x + x*x + y*y + y` 对应 `Size = 3`。

返回
该二次表达式的规模。

返回类型
int

示例

```
quad_expr = x*x + x + y*y
print(quad_expr.Size()) # 2
```

二次约束

class OPTVQConstr

二次约束类用于访问模型中的二次约束。注意二次约束也不可以由其构造函数直接创建，而是通过接口*OPTVModel.AddQConstr()* 将其添加到模型。

警告

当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的，否则会抛出*OPTVErrCode.MATRIX_NOT_PSD* 异常。

Get()

查询二次约束属性。

可用属性请参考天筹属性。

OPTVQConstr.Get(attr) const

参数
attr (OPTVIntAttr/OPTVCharAttr/OPTVDblAttr/OPTVStrAttr) –需要查询的二次约束属性。

返回
属性值。

返回类型
intlfloatlstr

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
c0 = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "c0")
model.Update()
print(c0.Get(OPTVIntAttr.PROHIBITED)) # 0
```

Set()

设置二次约束属性，只有调用*OPTVModel.Update()* 后，新设置才可生效。

可用属性请参考天筹属性。

OPTVQConstr.**Set**(*attr*, *value*)

参数

- **attr** (OPTVIntAttr/OPTVCharAttr/OPTVDblAttr/OPTVStrAttr) –需要设置的二次约束属性。
- **value** (*int*/*float*/*str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
c0 = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "c0")
c0.Set(OPTVStrAttr.NAME, "dummy")
model.Update()
print(c0.Get(OPTVStrAttr.NAME)) # dummy
```

Index()

获取二次约束在模型中的索引值。

OPTVQConstr.**Index**()

返回
二次约束索引。

返回类型
int

返回值

- **-1** –该二次约束在模型中不存在。
- **-2** –该二次约束已从模型中删除。
- **>=0** –该二次约束在模型中的索引/下标。

示例

```
c0 = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "c0")
c1 = model.AddQConstr(3*y*y + z, -OPTV_INF, 6, "c1")
print(c0.Index()) # 0
print(c1.Index()) # 1
```

SameAs()

查询两二次约束是否相同。

OPTVQConstr.**SameAs**(*c2*)

参数
c2 (OPTVQConstr) –需要与当前约束对比的约束。

返回
约束是否相同。

返回类型
bool

返回值

- **true** –约束相同。
- **false** –约束不同。

示例

```
c0 = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "c0")
c1 = c0
invalid = model.AddQConstr(2*x*x + y*y + z, -OPTV_INF, 3, "Invalid")

c0.SameAs(c1)       # True
c0.SameAs(invalid)  # False
```

仿射表达式

class OPTVAffineExpr

OPTVAffineExpr 类用于封装仿射变换 $Fx + g$，是构建锥约束的基础。一个*OPTVAffineExpr* 对象包含以下内容：

- **线性系数**：稀疏矩阵 F ，采用坐标（COO）格式存储非零系数及其对应的变量索引。通过逐行添加线性表达式来构建仿射表达式（COO 存储为内部实现，用户通过 SDK 对象提供数据）。
- **模型变量**：稀疏向量 x 。用户可以通过添加线性表达式 ，提供所需的变量。
- **常数项**：用于平移线性组合的偏移向量 g。

在创建*OPTVCone* 对象时，*OPTVAffineExpr* 提供了仿射变换的数学结构，锥类型参数则指定了该表达式必须满足的几何约束 C 。这种设计将几何约束与数学表达式分离，使得用户能够基于简单的仿射单元，灵活地组合构建复杂的锥约束。

有关锥的详细介绍可见锥约束。

AddRow()

向仿射表达式中添加单行数据。

OPTVAffineExpr.**AddRow**(*expr*)

参数
expr (OPTVLinExpr) –添加到仿射表达式中的线性表达式对象。

返回
无。

返回类型
None

AddRows()

向仿射表达式中添加多行数据。

OPTVAffineExpr.**AddRows**(*expr*)

参数
expr (*list[*OPTVLinExpr*]*) –添加到仿射表达式中的线性表达式列表。每个线性表达式对应仿射表达式中的一行。

返回
无。

返回类型
None

GetColumnIndices()

该函数会返回一个向量。向量中的每个值对应仿射表达式中的一个非零元素的列索引。仿射表达式在内部以COO（坐标）格式存储，其中每个非零元素由其行索引、列索引及其值共同标识。

OPTVAffineExpr.**GetColumnIndices**()

返回
一个整数向量，其每个元素对应一个非零元素的列索引。该向量中元素的顺序与*OPTVAffineExpr.GetRowIndices()* 以及*OPTVAffineExpr.GetValues()* 返回结果的顺序保持一致。

GetOffset()

OPTVAffineExpr.**GetOffset**(*index*)

参数
index(*int*) –查询偏移量的行索引。

返回
一个浮点数，表示索引为 index 的行的偏移量。

返回类型
float

引发
IndexError –index 的值大于仿射表达式中的行数。

GetOffsets()

OPTVAffineExpr.**GetOffsets**()

返回
一个元素为浮点数的列表，每个元素对应仿射表达式中某一行的偏移量。

返回类型
list[float]

GetRowIndices()

该函数返回一个向量。向量中的每个元素对应一个非零元素的行索引。仿射表达式在内部以COO（坐标）格式存储，其中每个非零元素由其行索引、列索引及其值共同标识。

OPTVAffineExpr.**GetRowIndices**()

返回
一个整数数组，其每个元素代表一个非零元素的行索引。向量中的元素顺序与*OPTVAffineExpr.GetColumnIndices()* 及其*OPTVAffineExpr.GetValues()* 的返回结果的顺序一致。

返回类型
list[int]

GetValues()

该函数返回一个向量。向量中的每个元素对应仿射表达式中一个非零元素的值。非零元素的位置由*OPTVAffineExpr.GetRowIndices()* 与*OPTVAffineExpr.GetColumnIndices()* 返回的行索引以及列索引确定。

OPTVAffineExpr.**GetValues**()

返回
一个双精度浮点数向量，其每个元素对应一个非零元素的值。向量中元素的位置与*OPTVAffineExpr.GetRowIndices()* 以及*OPTVAffineExpr.GetColumnIndices()* 返回结果的顺序一致。

返回类型
list[float]

SetOffset()

设置指定行的偏移量。

OPTVAffineExpr.**SetOffset**(*index*, *value*)

参数

- **index** (*int*) –需要设置偏移量的行的索引。
- **value** (*float*) –偏移量的值。

返回
无。

返回类型
None

引发
IndexError –index 的值大于仿射表达式中的行数。

Size()

查询仿射表达式中元素的数量。

OPTVAffineExpr.**Size**()

函数的返回值为仿射表达式中元素的个数，重复的元素会被多次统计。例如，若同一线性表达式被多次添加，仿射表达式中的元素数量将随之增加。

返回
仿射表达式中的元素数量。

返回类型
int

锥约束

class OPTVCone

天筹数学规划求解器的锥约束对象。用于访问模型中锥约束的相关信息。推荐使用*OPTVModel.AddCone()*接口创建锥约束对象，并向模型中添加锥约束，不建议使用构造函数直接创建锥约束对象。

锥约束在数学上定义为：

$$Fx + g \in C$$

其中：

- F 是一个 $m \times n$ 的系数矩阵，用于对优化变量进行线性转换，
- x 是一个 n 维的优化变量向量，
- g 是一个 m 维的常数偏移向量（即右端项），
- C 是约束仿射表达式取值范围的锥。
- 表达式 $Fx + g$ 生成一个属于锥 C 的 m 维向量。

矩阵 F 采用坐标（COO）稀疏格式存储，以提高内存效率，是一个仅包含非零元素的行索引、列索引、值的三元组。用户可以按行添加线性表达式，将表达式 $Fx + g$ 构建为仿射表达式 。

天筹求解器（OptVerse）支持以下锥约束类型：

- *Second-order cone*
- *Rotated second-order cone*

仿射表达式以及锥类型的详细信息请见仿射表达式 与锥类型 。

Get()

查询锥约束某一属性的值。

OPTVCone.**Get**(*attr*)

参数
attr (OPTVStrAttr) –字符串型属性。

返回
属性值。

返回类型
str

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

GetType()

获取锥约束的类型。

有关锥约束类型的详细信息请见*OPTVConeType* 。

OPTVCone.**GetType**()

返回
锥约束的类型。

返回类型
OPTVConeType

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –锥约束已从模型中移除。

Index()

返回锥线性约束在模型中的索引。

OPTVCone.**Index**()

返回
锥约束的索引。

返回类型
int

返回值

- **-1** –锥约束不在任何模型中。
- **-2** –锥约束已从模型中移除。
- **>=0** –锥线性约束在模型中的索引。

SameAs()

判断两个锥约束是否为同一个对象。

OPTVCone.**SameAs**(*c2*)

参数
c2 (OPTVCone) –用于比对是否相同的锥约束。

返回
锥约束是否相同。

返回类型
bool

返回值

- **true** –锥约束相同
- **false** –锥约束不同

Set()

设置锥约束属性的值。在调用*OPTVModel.Update()*之前，该设置不会在模型内部生效。

可用属性请参考*天筹属性*。

OPTVCone.**Set**(*attr*, *value*)

参数

- **attr** (OPTVStrAttr) –字符串型属性。
- **value** (*str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

SOS 约束

class OPTVSOS

天筹求解器 SOS 约束对象，用以从模型中获取 SOS 约束数据。这里，用户需要通过*OPTVModel.AddSOS()*接口向模型中添加 SOS 约束，而非借助于构造函数。

特殊顺序集，special ordered set (SOS)，是指一组有序集合里，至多有一个非零值（SOS1 型）或至多有两个非零值（SOS2 型）。SOS 约束是对一组变量的取值进行限制的特殊约束，可以额外地指定模型中的整数条件。这类约束单独列出来，可以加快线性规划的求解速度。

具体地，天筹求解器的支持下列两种 SOS 约束：

- SOS 类型 1 要求指定的一组变量至多有一个变量可取非零值。
- SOS 类型 2 指定的一组变量至多有两个变量可取非零值，且取非零值的变量顺序要求相邻。

Get()

查询 SOS 约束的属性。

可用属性请参考*天筹属性*。

OPTVSOS.**Get**(*attr*)

参数
attr (OPTVIntAttr / OPTVStrAttr) –需要查询的 SOS 约束属性。

返回
属性值。

返回类型
int|str

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
sos = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos")
# query the type of the SOS constraint
result = sos.Get(OPTVIntAttr.SOS_TYPE)
```

Set()

设置 SOS 约束属性，只有调用*OPTVModel.Update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVSOS.**Set**(*attr*, *value*)

参数

- **attr** (OPTVIntAttr / OPTVStrAttr) –需要设置的 SOS 约束属性。
- **value** (*int* / *str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
sos = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos")
sos.Set(OPTVStrAttr.NAME, "dummy")
model.Update()
print(sos.Get(OPTVStrAttr.NAME)) # dummy
```

Index()

获取 SOS 约束在模型中的索引值。

OPTVSOS.**Index**()

返回
SOS 约束索引。

返回类型
int

返回值

- **-1** –该 SOS 约束在模型中不存在。
- **-2** –该 SOS 约束已从模型中删除。
- **>=0** –该 SOS 约束在模型中的索引/下标。

示例

```
sos = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos")
result = sos.Index()
```

SameAs()

查询两 SOS 约束是否相同。

OPTVSOS.**SameAs**(*c2*)

参数
c2 (OPTVSOS) –需要与当前 SOS 约束对比的 SOS 约束。

返回
SOS 约束是否相同。

返回类型
bool

返回值

- **true** –SOS 约束相同。
- **false** –SOS 约束不同。

示例

```
sos = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos")
sos1 = sos
invalid = OPTVSOS()

sos.SameAs(sos1)     # True
sos.SameAs(invalid)  # False
```

一般约束

class OPTVGenConstr

天筹求解器一般约束，用于从模型中获取一般约束数据。特别地。一般约束需要通过 OPTVModel.AddGenConstrXxx 向模型中添加约束而构造，而不是借助于任何构造函数。

表 15: 天筹求解器支持的一般约束类型

约束	类型	接口	描述
And	*OPTV_GENCONSTR_A*	*OPTVModel.AddGenConstrAnd()*	只有所有变量非零时，结果才非零。
Or	*OPTV_GENCONSTR_O*	*OPTVModel.AddGenConstrOr()*	只要有一个变量非零，结果就非零。
Indicator	*OPTV_GENCONSTR_I*	*OPTVModel.AddGenConstrIndicator*	返回值为布尔型，当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0。

Get()

查询一般约束属性。

可用属性请参考*天筹属性*。

OPTVGenConstr.**Get**(*attr*)

参数
attr (OPTVIntAttr / OPTVDblAttr / OPTVStrAttr) –需要查询的约束属性。

返回
属性值。

返回类型
intlfloatlstr

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "and")
# query the general constraint type
print(c0.Get(OPTVIntAttr.GENCONSTR_TYPE))
```

Set()

设置一般约束属性，只有调用*OPTVModel.Update()* 后，新设置才可生效。

可用属性请参考*天筹属性*。

OPTVGenConstr.**set**(*attr*, *value*)

参数

- **attr** (OPTVDblAttr / OPTVStrAttr) –需要设置的约束属性。
- **value** (*float* / *str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知约束属性。

示例

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "and")
c0.Set(OPTVStrAttr.NAME, "dummy")
model.Update()
print(c0.Get(OPTVStrAttr.NAME)) # dummy
```

Index()

获得一般约束在模型中的索引值。

OPTVGenConstr.**Index**()

返回
约束索引。

返回类型
int

返回值

- **-1** –该约束在模型中不存在。
- **-2** –该约束已从模型中删除。
- **>=0** –该约束在模型中的索引/下标。

示例

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "and")
c0.Index()
```

SameAs()

查询两个一般约束是否为同一对象。

OPTVGenConstr.**SameAs**(*c2*)

参数
c2 (OPTVGenConstr) –需要与当前约束对比的约束。

返回
约束是否相同。

返回类型
bool

返回值

- **true** –约束相同。
- **false** –约束不同。

示例

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "and")
c1 = c0
invalid = model.AddGenConstrOr(x3, [x1, x2], "Or")

c0.SameAs(c1)       # True
c0.SameAs(invalid)  # False
```

模型

class OPTVModel

OptVerse 优化模型对象。

`OPTVModel` 类是 OptVerse 中用于构建和求解优化模型的核心对象。它存储了所有变量、约束条件、目标函数及求解器设置。

模型具有 ** 惰性更新 ** 行为：对模型的更改（添加变量、约束、修改属性等）不会立即反映在底层求解器中。在优化之前调用*OPTVModel.Update()* 以使模型状态与求解器同步。

OPTVModel()

OPTVModel.**OPTVModel**()

创建一个具有自动管理环境的空模型。

OPTVModel.**OPTVModel**(*env*)

创建一个与给定环境关联的空模型。

参数

env (OPTVEnv) – 天箸模型环境变量。

引发

TypeError – env 不是一个*OPTVEnv*。

OPTVModel.**OPTVModel**(*filename*)

通过读取文件，使用自动管理的环境来创建模型。

参数

filename (*str*) – 模型文件的路径（MPS、LP 等）。

引发

- **TypeError** – filename 并非 str。
- ***OPTVErrCode.ERROR_FILE_READ*** – 无法从文件中读取模型。

OPTVModel.**OPTVModel**(*env*, *filename*)

通过读取文件来创建模型。

参数

- **env** (OPTVEnv) – 天箸模型环境变量。
- **filename** (*str*) – 模型文件的路径（MPS、LP 等）。

引发

- **TypeError** – 参数类型不匹配 (OPTVEnv, str)。
- ***OPTVErrCode.ERROR_FILE_READ*** – 无法从文件中读取模型。

OPTVModel.**OPTVModel**(*env*, *content*, *format*)

通过字符串读取来创建一个模型。

参数

- **env** (OPTVEnv) –天筹模型环境变量。
- **content** (*str*) –将模型内容表示为字符串。
- **format** (*str*) –模型格式（例如：”*mps*”、”*lp*”）。

引发

- **TypeError** –参数类型与 (OPTVEnv, str, str) 不匹配。
- ***OPTVErrCode.ERROR_FILE_READ*** –从字符串读取模型失败。

AddCone()

向模型中添加单个锥约束。

OPTVModel.**AddCone**(*affine*, *type*, *name=''*)

该函数将向模型中添加一条锥约束。

参数

- **affine** (OPTVAffineExpr) –锥约束的仿射表达式 $Fx + g$。
- **type** (OPTVConeType) –锥约束类型（如，二阶锥、旋转二阶锥）。
- **name** (*str*) –锥约束名称。

返回

添加到模型中的锥约束对象。

返回类型

OPTVCone

引发

OPTVErrCode.INVALID_ARGUMENT –type 不是*OPTVConeType.SOC* 或者*OPTVConeType.RSOC* 。仿射表达式为空。type 为*OPTVConeType.RSOC*，且仿射表达式中的元素个数小于 *3* 。

AddConstr()

向模型中添加单个线性约束。

OPTVModel.**AddConstr**(*expr*, *lhs*, *rhs*, *name=''*)

该函数向模型中添加一个线性约束。当需要一次性添加大量约束时，建议使用批量模式（*OPTVModel.AddConstrs()*），虽然大多数情况下，效率差别不大。

参数

- **expr** (OPTVLinExpr) –约束的线性表达式部分。
- **lhs** (*float*) –约束的下界。
- **rhs** (*float*) –约束的上界。
- **name** (*str*) –约束名称。

返回
添加到模型中的约束对象。

返回类型
OPTVConstr

引发
OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
# add a single linexpr constraint: 1 <= x+y <= 10
model.AddConstr(x+y, 1, 10, "c0")
```

OPTVModel.**AddConstr**(*constr*, *name=''*)

该方法接收一个*OPTVTempConstr* 对象作为第一个参数，第二个参数为约束名称（可选）。推荐在表达式对象之间或者表达式与数值之间使用比较运算符（如 <=、>=、==）来构建*OPTVTempConstr* 对象。可以通过该方法添加线性约束与一般约束。

参数

- **constr** (OPTVTempConstr) –*OPTVTempConstr* 对象。
- **name** (*str*) –**[optional]** 约束名称。当未提供名称时，将使用默认的名称格式: C_num，其中 name 是模型中当前约束的数量。

返回
添加到模型中的约束对象。*OPTVConstr* 对象或者*OPTVGenConstr* 对象。

返回类型
OPTVConstr | *OPTVGenConstr*

引发
OPTVErrCode.INVALID_ARGUMENT –数值为 NaN，或者模型中已存在名为 name 的约束。

示例

```
# add a single linexpr constraint
model.AddConstr(x + y <= 1, "c0")

# add a single general constraint
model.AddConstr(z == And_([x, y, w]), name="c1")
```

AddConstrs()

向模型中添加多个线性约束。

OPTVModel.**AddConstrs**(*expr*, *lhs*, *rhs*, *names*, *count*)

该函数一次性向模型中添加多个线性约束，尤其适用于添加大量约束的情形。

参数

- **expr** (*list[OPTVLinExpr]*) –约束的线性表达式序列。
- **lhs** (*list[float]*) –约束的左端值序列。
- **rhs** (*list[float]*) –约束的右端值序列。

- **names** (*list[str]*) –约束的名字序列。
- **count** (*int*) –**可选**, 添加约束的个数，默认值为参数序列的长度。v1.2.6 前为必须参数，其后用户可省略此参数，该选项可以通过其他参数序列推断获取。

返回
添加到模型中的约束对象列表。

返回类型
list[*OPTVConstr*]

引发
OPTVErrCode.INVALID_ARGUMENT –name 为空，name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
# add multiple linear constraints
model.AddConstrs([x+y, x+z, 2*x], [1, 2, 3], [6, 7, 8], ["c0", "c1",
↪"c2"])
```

OPTVModel.**AddConstrs**(*constrs*, *name=''*)

该方法通过 Python 生成器表达式向模型批量添加约束。

生成器表达式在每次迭代中生成一个约束，并将其添加至模型中。

本方法支持添加线性约束和一般约束。

若为约束指定名称，生成器表达式必须使用括号（）包裹。这是 Python 语言的语法要求，旨在确保表达式的正确解析。

参数

- **constrs** (*generator*) –生成器表达式。
- **name** (*str*) –约束名称。当未提供名称时，将使用默认的名称格式: C_num ，其中 name 是模型中当前约束的数量。

返回
添加到模型中的约束对象列表。

返回类型
list[*OPTVConstr*] | list[*OPTVGenConstr*]

引发
OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
model.AddConstrs((x[i,j] == 0 for i in range(4)
                          for j in range(4)
                          if i != j), name='c')

model.AddConstrs(z[i] == max_([x[i], y[i]]) for i in range(5))
```

AddQConstr()

向模型中添加单个二次约束。

OPTVModel.**AddQConstr**(*expr*, *lhs*, *rhs*, *name=''*)

该函数向模型中添加一个二次约束。

参数

- **expr** (OPTVQuadExpr) –约束的二次表达式部分。
- **lhs** (*float*) –约束的下界。
- **rhs** (*float*) –约束的上界。
- **name** (*str*) –约束名称。

返回

添加到模型中的约束对象。

返回类型

OPTVQConstr

引发

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
# add a single quadratic constraint: x*x + 2*y +1 <= 100
model.AddQConstr(x*x+2*y+1, -OPTV_INF, 100, name="qconstr")
```

OPTVModel.**AddQConstr**(*constr*, *name=''*)

该方法以一个*OPTVTempConstr* 对象作为主要参数，并可选择性地接收一个字符串作为约束的名称。*OPTVTempConstr* 对象推荐在表达式或者数值之间使用比较操作符（如 <=、==、>=）来构造。

参数

- **constr** (OPTVTempConstr) –*OPTVTempConstr* 对象。
- **name** (*str*) –约束名称。当未提供名称时，将使用默认的名称格式: C_num ，其中 name 是模型中当前约束的数量。

返回

添加到模型中的约束对象。

返回类型

OPTVQConstr

引发

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
# add a single quadratic constraint: x*x + 2*y +1 <= 100
model.AddQConstr(x*x + 2*y + 1 <= 100, name="qconstr")
```

AddGenConstrAnd()

向模型中添加单个 AND 一般约束。

OPTVModel.**AddGenConstrAnd**(*resultant*, *conVars*, *name=''*)

该函数向模型中添加单个 AND 一般约束：只有所有变量非零时，结果才非零。

参数

- **resultant** (OPTVVar) –约束的布尔型返回值。
- **conVars** (*list[OPTVVar]*) –所有参与检验的变量序列。
- **name** (*str*) –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

引发

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
model.AddGenConstrAnd(x3, [x1, x2], "and")
```

AddGenConstrOr()

向模型中添加单个 OR 一般约束。

OPTVModel.**AddGenConstrOr**(*resultant*, *conVars*, *name=''*)

该函数向模型中添加单个 OR 一般约束：只要有一个变量非零，结果就非零。

参数

- **resultant** (OPTVVar) –约束的布尔型返回值。
- **conVars** (*list[OPTVVar]*) –所有参与检验的变量的序列。
- **name** (*str*) –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

引发

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
model.AddGenConstrOr(x3, [x1, x2], "or")
```

AddGenConstrIndicator()

向模型中添加单个 INDICATOR 一般约束。

OPTVModel.**AddGenConstrIndicator**(*binVar*, *binVal*, *expr*, *sense*, *rhs*, *name=''*)

该函数向模型中添加单个 INDICATOR 一般约束：当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0.

参数

- **binVar** (OPTVVar) –二进制指示变量。
- **binVal** (*float*) –要求线性约束必须满足的二进制指示变量的值 (0 or 1).
- **expr** (OPTVLinExpr) –参与检验的线性约束表达式。
- **sense** (*str*) –线性约束的含义，如*OPTV_SENSE_LESS*, *OPTV_SENSE_EQUAL*, 或*OPTV_SENSE_GREATER*。
- **rhs** (*float*) –参与检验的线性约束右端值。
- **name** (*str*) –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

引发

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
# x3 = 1 -> x2 + x1 + 1 <= 10
model.AddGenConstrIndicator(x3, 1, x2+x1+1, OPTV_SENSE_LESS, 10, name=
↪"indicator")
```

AddGenConstrAbs()

向模型中添加单个 ABS 一般约束：$r = \text{abs}(x)$。此约束条件保证因变量 r 的值为自变量 x 的绝对值，即 $r = |x|$, 其中 $r \geq 0$。

形如 $r = \text{abs}(x)$ 的绝对值约束要求变量 r 等于变量 x 的绝对值。该约束施加了关系 $r = |x|$，其中 $r \geq 0$。

OPTVModel.**AddGenConstrAbs**(*res_var*, *input_var*, *name=''*)

该变量的绝对值将被另一变量采用。

参数

- **res_var** (OPTVVar) –该变量的绝对值将被结果变量采用。
- **input_var** (OPTVVar) –添加到模型的一般约束对象。
- **name** (*str*) –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

引发

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
model.AddGenConstrAbs(x1, x2, "abs")
```

AddGenConstrNorm()

向模型中添加一个 NORM 约束。

向 “NORM“ 约束定义为：$r = \text{norm}\{x_1, \ldots, x_n\}$，其中因变量 r 表示向量 $x_1, \ldots, x_n$ 的范数。

OPTVModel.**AddGenConstrNorm** (*res_var*, *vars_list*, *norm_type*, *name=”*)

该函数向模型中添加单个 NORM 一般约束。

参数

- **res_var** (*OPTVVar*) –结果变量，其值为参数向量的范数。
- **vars_list** (*list[OPTVVar]*) –用于计算向量范数的变量。
- **norm_type** (*int*) –范数类型：-1 为 L^∞ 范数，0 为 L^0 范数，1 为 L^1 范数。
- **name** (*str*) –约束名称。

返回

name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型

OPTVGenConstr

引发

OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
model.AddGenConstrNorm(x1, [x2, x3, x4], 0, "norm")
```

AddGenConstrMax()

向模型中添加单个 MAX 约束：$r = \max\{x_1, \ldots, x_n\}$。

形式为 $r = \max\{x_1, \ldots, x_n\}$ 的 MAX 约束确保变量 r 等于参数向量 $x_1, \ldots, x_n$ 中的最大值。

OPTVModel.**AddGenConstrMax** (*resultant*, *var_list*, *name=”*, *constant=-OPTV_INF*)

该函数向模型中添加单个 MAX 约束。

参数

- **resultant** (*OPTVVar*) –从中取最大值的一组变量。
- **var_list** (*list[OPTVVar]*) –向模型中添加单个 MAX 约束：$r = \max\{x_1, \ldots, x_n\}$。
- **name** (*str*) –约束名称。
- **constant** (*float*) –[**可选**] 参与 MAX 操作的常量。

返回
name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型
OPTVGenConstr

引发
OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
# x1 = max([x2, x3, x4])
model.AddGenConstrMax(x1, [x2, x3, x4] name="max")
```

AddGenConstrMin()

向模型中添加单个 MIN 约束：$r = \min\{x_1, \ldots, x_n\}$ 。

形式为 $r = \min\{x_1, \ldots, x_n\}$ 的 MIN 约束确保变量 r 等于参数向量 $x_1, \ldots, x_n$ 中的最小值。

OPTVModel.**AddGenConstrMin**(*resultant*, *var_list*, *name=''*, *constant=OPTV_INF*)

该函数向模型中添加单个 MIN 约束。

参数

- **resultant** (OPTVVar) –从中取最小值的一组变量。
- **var_list** (*list[OPTVVar]*) –变量序列，该向量的最小值会被结果变量采用。
- **name** (*str*) –约束名称。
- **constant** (*float*) –[**可选**] 参与 MIN 操作的常量。

返回
name 为空，数值参数中存在 NaN 或者 name 与模型中的其他约束重名。

返回类型
OPTVGenConstr

引发
OPTVErrCode.INVALID_ARGUMENT –name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
# x1 = min([x2, x3, x4])
model.AddGenConstrMin(x1, [x2, x3, x4] name="min")
```

AddSOS()

向模型中添加单个 SOS 约束。

OPTVModel.**AddSOS**(*vars*, *weights*, *type*, *name=''*)

参数

- **vars** (*list[OPTVVar]*) –SOS 约束所涉及变量的序列。
- **weights** (*list[float]*) –SOS 的变量权重的序列。

- **type** (*int*) –SOS 类型，取值为 1 或 2。
- **name** (*str*) –SOS 约束的名字。

返回
添加到模型的 SOS 约束对象。

返回类型
OPTVSOS

引发

- ***OPTVErrCode.INVALID_ARGUMENT*** –type 取值不是 1 或 2。
- ***OPTVErrCode.NOT_IN_MODEL*** –vars 序列中的某个变量不属于当前模型。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –当前模型存在无效下标，比如模型关联的环境发生溢出时，执行本操作有可能抛出此错误。

示例

```
# At most one variable can be non-zero
model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos1")
# At most two adjacent variables can be non-zero
model.AddSOS([x, y, z], [1, 2, 3], 2, name="sos2")
```

AddVar()

向模型中添加单个变量。

OPTVModel.**AddVar**(*lb*, *ub*, *obj*, *vtype*, *name=''*)

该函数向模型中添加一个变量。当需要一次性添加大量变量时，建议使用批量模式（*OPTVModel.AddVars()* ），虽然大多数情况下，效率差别不大。

参数

- **lb** (*float*) –变量的下界值。
- **ub** (*float*) –变量的上界值。
- **obj** (*float*) –变量在目标函数中的（线性）系数。
- **vtype** (*str*) – 变量类型，取值包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **name** (*str*) –变量名称。

返回
添加到模型中的变量对象。

返回类型
OPTVVar

引发
OPTVErrCode.INVALID_ARGUMENT –未知变量类型，name 为空，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
# add a binary variable
model.AddVar(0, 1, 0, OPTV_BINARY, name="x")
```

OPTVModel.**AddVar**(*lb=0.0*, *ub=OPTV_INF*, *obj=0.0*, *type=OPTV_CONTINUOUS*, *col=None*, *name=''*)

该函数向模型中添加一个变量，并可以设置现有约束中的相应系数。col 是可选参数，若是提供一个*OPTVColumn* 对象，可以指定变量参与的约束及其在约束当中的系数；若是采用默认值，则变量不与任何约束关联。当需要一次性添加大量变量时，建议使用批量模式 (*OPTVModel.AddVars()*)，虽然大多数情况下，效率差别不大。

参数

- **lb** (*float*) –[**可选**] 变量的下界值。
- **ub** (*float*) –[**可选**] 变量的上界值。
- **obj** (*float*) –[**可选**] 变量在目标函数中的（线性）系数。
- **type** (*str*) –[**可选**] 变量类型，取值包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*。
- **col** (OPTVColumn) –[**可选**] 设置约束系数的列对象。
- **name** (*str*) –[**可选**] 变量名称。当未提供名称时，将使用默认的名称格式: v_num, 其中 num 是模型中变量的数量。

返回

添加到模型中的变量对象。

返回类型

OPTVVar

引发

OPTVErrCode.INVALID_ARGUMENT –未知变量类型，数值参数中存在 NaN 或者 name 与模型中的其他组件重名。

示例

```
model.AddVar()                                          # var name is "v_0"
model.AddVar(vtype=OPTV_BINARY, obj=1.0, name="y") # var name is "y"

# add a binary variable and set its coefficient for pre-existing␣
↪constraint
col = OPTVColumn()
col.AddTerms(3, c0)
model.AddVar(0, 1, 0, OPTV_BINARY, col, name="x")
```

AddMVar()

向模型中添加一个 OPTVMVar 对象。

OPTVModel.**AddMVar**(*shape*, *lb=0.0*, *ub=OPTV_INF*, *obj=0.0*, *vtype=OPTV_CONTINUOUS*, *name=''*)

参数

- **shape** –OPTVMVar 对象的形状，可以是一个整数或者元组。
- **lb** –可选参数。变量下界。
- **ub** –可选参数。变量上界。
- **obj** –可选参数。变量在目标函数中的系数。

- **vtype** –可选参数。变量类型取值包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*。
- **name** –可选参数。变量名称。

变量 lb、ub、obj 以及 vtype 可以是一个标量、列表或者 Numpy 数组。它们的形状与 OPTVMVar 的形状相同，或者可以通过 NumPy 的广播机制形成给定的形状。

变量 name 可以是一个字符串，不同变量的名称会根据其索引在 name 后面添加后缀，或者是一个与 OPTVMVar 对象形状相同的 NumPy 数组。

返回
一个新的 OPTVMVar 对象。

返回类型
OPTVMVar

示例

```
# Add a 2-by-3 OPTVMVar
model.AddMVar((2, 3), name="x")
# Add a vector of three variables with non-default lower bounds
model.AddMVar((3), lb=[1, 2, 3], name="y")
```

AddVars()

向模型中添加多个变量。

OPTVModel.**AddVars**(*lb*, *ub*, *obj*, *vtype*, *names*, *count=None*)

该函数一次性向模型中添加多个变量，尤其适用于添加大量变量的情形。

参数

- **lb** (*list[float]*) –变量的下界列表。
- **ub** (*list[float]*) –变量的上界列表。
- **obj** (*list[float]*) –变量对应的目标系数列表。
- **vtype** (*list[str]*) –变量类型，包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **names** (*list[str]*) –变量名称列表。
- **count** (*int*) –**可选**, 添加变量的个数，默认值为参数序列的长度。v1.2.6 前为必须参数，其后用户可省略此参数，该选项可以通过其他参数序列推断获取。

返回
添加到模型的变量列表。

返回类型
list[*OPTVVar*]

引发
OPTVErrCode.INVALID_ARGUMENT –type 包含未知变量类型，names 为空，数值参数中存在 NaN 或者 name 包含的变量名与模型中的其他组件重名。

示例

```
# add multiple variables
model.AddVars([0]*3, [1]*3, [0]*3, [OPTV_BINARY]*3, [col_x, col_y,␣
↪col_z], name=["x", "y", "z"])
```

OPTVModel.**AddVars**(*lb*, *ub*, *obj*, *vtype*, *col*, *names*, *count=None*)

该函数向模型中添加多个变量，并可以设置现有约束中的相应系数。当需要一次性添加大量变量时，建议使用此功能，虽然大多数情况下，效率差别不大。

参数

- **lb** (*list[float]*) –变量的下界列表。
- **ub** (*list[float]*) –变量的上界列表。
- **obj** (*list[float]*) –变量对应的目标系数列表。
- **type** (*list[str]*) – 变量类型，包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*.
- **col** (*list[OPTVColumn]*) –设置约束系数的列对象序列。
- **names** (*list[str]*) –变量名称列表。
- **count** (*int*) –添加变量的个数。

返回

添加到模型的变量列表。

返回类型

list[*OPTVVar*]

引发

OPTVErrCode.INVALID_ARGUMENT –type 包含未知变量类型，name 为空，数值参数中存在 NaN 或者 name 包含的变量名与模型中的其他组件重名。

示例

```
# add multiple variables and set coefficients for pre-existing␣
↪constraints
col_x = OPTVColumn()
col_x.addTerm(1, c0)
col_y = OPTVColumn()
col_y.addTerm(1, c0)
col_z = OPTVColumn()
col_z.addTerm(1, c0)
model.AddVars([0]*3, [1]*3, [0]*3, [OPTV_BINARY]*3, [col_x, col_y,␣
↪col_z], name=["x", "y", "z"])
```

OPTVModel.**AddVars**(**indices*, *lb=0.0*, *ub=OPTV_INF*, *obj=0.0*, *vtype=OPTV_CONTINUOUS*, *col=None*, *names=''*, *count=None*)

该函数用于向模型批量添加变量，并可以设置现有约束中的相应系数。

函数返回一个 OPTVTupleDict 对象，用于存储所创建的变量。该对象的键（keys）对应于输入参数 indices 中的值。参数 indices，用于指定变量的索引，这些索引将作为 OPTVTupleDict 对象的键，其支持多种形式，具体详见下文说明。

第一个参数指定作为键的索引，这些索引将用于访问返回的 OPTVTupleDict 中的变量。indices 的基础形式为传入一个或者多个整数，函数内部会创建一个多维变量数组。例如，x = model.AddVars(2, 2, name="x"), 将创建 4 个变量，名称分别为：x_0_0、x_0_1、x_1_0、x_1_1。

当传入多个列表时，函数内部会将各个列表的元素进行组合，得到变量索引。例如，x = model.AddVars([3, 7], ['a', 'b', 'c'], name="x"), 将创建 6 个变量，名称分别为：x_3_a, x_7_a 等。

也可以只传入一个列表，这时列表的元素就是变量的索引。例如，`x = model.addVars([(3,'a'), (3,'b'), (7,'b'), (7,'c')], name="x")` 同样会创建 6 个变量，名称为 `x_3_a, x_7_a` 等。

虽然参数 `indices` 可以是多个列表，也可以表示为一个元组列表，但是变量的每个索引元素必须是标量（如整数、浮点数或者字符串）。该函数不支持将符合类型（如嵌套元组）作为变量索引的单个组成部分。使用下面的例子创建变量，将会报错:

```
x = model.AddVars([((1, 3),'a'), (7,'a')], name="x")

y = model.AddVars([(1, 3), 7], ['a'], name="y")
```

关键字参数（如 `lb` 或 `obj`），如果它们是标量，其值将会应用于所有创建的变量。例如，设置 `lb=1.0`，那么所有创建的变量的下界均为 1.0。参数 `name` 与其他关键字参数不同：当该参数为标量时（如 `name="x"` ），函数内部会将其作为变量名称的前缀，并将变量索引作为名称后缀，自动生成变量的名称。如果参数 `indices` 是一个列表，则可以为参数 `name` 提供一个长度相同的字符串列表。该列表中的每个字符串将作为对应变量的显式名称。例如，`name=["varA", "varB", "varC"]` 会按顺序将这些名称分别分配给对应的变量。

参数

- **indices** –新创建变量的索引。
- **lb** –[**可选**] 变量下界。
- **ub** –[**可选**] 变量上界。
- **obj** –[**可选**] 变量对应的目标系数。
- **vtype** –变量类型。包括*OPTV_CONTINUOUS*, *OPTV_INTEGER*, *OPTV_BINARY*。
- **col** –[**可选**] 设置约束系数的列（或列集）对象。
- **name** –一个或多个变量名称。
- **count** –[**可选**] 添加变量的个数。

返回

一个 `OPTVTupleDict` 对象，字典的值为添加的变量，键为提供的索引。

返回类型

OPTVTupleDict

引发

OPTVErrCode.INVALID_ARGUMENT –`type` 包含未知变量类型，`name` 为空，数值参数中存在 `NaN` 或者 `name` 包含的变量名与模型中的其他组件重名。

示例

```
# 3-D array of binary variables
x = model.AddVars(3, 4, 5, vtype=OPTV_BINARY)

# variables index by tuplelist
l = OPTVTupleList([(1, 2), (1, 3), (2, 3)])
y = model.addVars(l, ub=[1, 2, 3])

# variables with predetermined names
z = model.AddVars(3, name=["a","b","c"])
```

AddMConstr()

通过矩阵表达方式向模型中添加约束。添加约束的形式为 `lhs <= Ax <= rhs`. 约束中的 A 为 `numpy.ndarray`.

`OPTVModel.`**`AddMConstr`**(*A*, *x*, *lhs*, *rhs*, *name=''*)

参数

- **A** (*numpy.ndarray*) –一个二维的 `numpy.ndarray`.
- **x** (*OPTVMVar | list[OPTVVar] | None*) –该参数可以是*OPTVMVar* 对象, 或者是元素为*OPTVVar* 的列表, 也可以为 None(这时会用模型中的*OPTVVar* 填充该参数). 变量个数与参数 A 中的列数相同。
- **lhs** (*numpy.ndarray*) –约束的左端项向量, 一个一维的 `numpy.ndarray`。向量中元素个数与参数 A 的行数相同。
- **rhs** (*numpy.ndarray*) –约束的右端项向量, 一个一维的 `numpy.ndarray`。向量中元素个数与参数 A 的行数相同。
- **name** (*numpy.ndarray | list[str] | str*) –新增约束的名称。若是该参数为字符串标量, 会使用约束在矩阵中的下标填充该标量；若是该参数为一个字符串列表或者一维的 `numpy.ndarray` ，那么元素个数一定要与参数 A 的行数相同。

返回

一个新的 OPTVMConstr 对象。

返回类型

OPTVMConstr

引发

- **TypeError** –x 不是列表、*OPTVMVar* 或者 None。
- **IndexError** –x 的维度不符合要求。
- **ValueError** –x 的形状不符合要求。

示例

```
vars = model.AddMVar(3, vtype=OPTV_BINARY, name=['x', 'y', 'z'])
model.Update()
obj = vars @ np.array([1, 1, 2])
A = np.array([[1, 2, 3], [1, 1, 0]])
constrs = model.AddMConstr(A, vars, [-OPTV_INF, 1], [4, OPTV_INF],␣
↪name='c')
```

CheckSolution()

检验指定的解是否可行。

`OPTVModel.`**`CheckSolution`**(*solution*, *tol=1e-8*)

该函数供用户检验指定的解是否满足当前模型的可行性要求，但 **不可以**检验内部存储的解。

参数

- **solution** (*list[float]*) –(列) 解向量。
- **tol** (*float*) –约束可行性整数性检查精度（**可选**，默认值为 10^{-8}）。

返回
解对模型是否可行。

返回类型
bool

返回值

- **false** –解对模型不可行。
- **true** –解对模型是可行的。

示例

```
result = model.CheckSolution(solution)
```

ComputeIIS()

计算不可约不一致子系统（IIS）。

有关 IIS 的细节请参考*IIS 快速入门*。

OPTVModel.**ComputeIIS**()

返回
IIS 是否计算成功。

返回类型
bool

返回值

- **false** –IIS 计算不成功。
- **true** –成功计算 IIS。

示例

```
if model.ComputeIIS():
    model.WriteIIS("iis.ilp")
else:
    print("IIS computation failed")
```

FeasRelax()

进行不可行问题的诊断及分析。

更多细节请参考*LP 不可行诊断及修复快速入门*。

OPTVModel.**FeasRelax**()
该函数进行不可行问题的诊断及分析，目前仅支持 L^1 范数。

返回
不可行问题修复的状态。

返回类型
int

返回值

- **0** –当前问题是可行的。

- **-1** –不可行问题修复失败。
- **>0** –Phase 1 最小违背量的目标函数值。

示例

```
result = model.FeasRelax()
```

GetCoef()

查询给定约束中指定变量的系数。

OPTVModel.**GetCoef**(*con*, *var*)

参数

- **con** (OPTVConstr) –需要查询的约束。
- **var** (OPTVVar) –需要查询的变量。

返回
查询变量在该约束中的系数。

返回类型
float

引发

- ***OPTVErrCode.INVALID_ARGUMENT*** –约束或变量不存在。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –数据不存在，或约束的线性表达式部分不可用。

示例

```
c0 = model.AddConstr(x+2*y, 1, 10, "c0")
# get the coefficient of y in c0
result = model.GetCoef(c0, y)
```

GetColumn()

获取指定变量对应的列对象。

OPTVModel.**GetColumn**(*v*)

参数
v –需要查询的变量。

返回
指定变量对应的列对象。

返回类型
OPTVColumn

引发

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –该变量不存在于当前模型。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –变量索引值*Index* 超过了模型的变量总数。若当前模型需要执行更新，或者当前变量属于其它模型时，该错会被抛出。

示例

```
result = model.GetColumn(x)
```

GetCone()

获取指定位置的锥约束。

OPTVModel.**GetCone**(*i*)

参数
i –锥约束的索引。

返回
索引为 i 的锥约束。

返回类型
OPTVCone

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –索引值大于模型中锥约束的数量。

GetCones()

获取模型所有的锥约束。

OPTVModel.**GetCones**()

返回
包含模型所有锥约束的列表。

返回类型
list[*OPTVCone*]

GetConstrByName()

根据指定索引值获得模型中对应的约束。

OPTVModel.**GetConstrByName**(*name*)

参数
name (*str*) –需要访问的约束名称。

返回
名称匹配 name 的约束。

返回类型
OPTVConstr

引发
OPTVErrCode.DATA_NOT_AVAILABLE –给定的名称为空，或模型不存在命名为 name 的约束。

示例

```
# get a constraint with the name c0
result = model.GetConstrByName("c0")
```

GetConstr()

根据指定索引值获得模型中对应的约束。

OPTVModel.**GetConstr**(*i*)

参数
i (*int*) –线性约束在模型中的索引。

返回
索引为 i 的约束。

返回类型
OPTVConstr

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –指定的索引值超出了模型的约束总数。

示例

```
result = model.GetConstr(1)
```

GetConstrs()

获得模型中的所有线性约束。

OPTVModel.**GetConstrs**()

返回
包含模型中所有约束的列表。

返回类型
list[*OPTVConstr*]

示例

```
result = model.GetConstrs()
c0 = result[0]
```

GetGenConstr()

获取指定类型的单个通用约束。

OPTVModel.**GetGenConstr**(*gcType*, *index*)

参数

- **gcType** (*int*) – 通用约束类型（例如*OPTV_GENCONSTR_AND*、*OPTV_GENCONSTR_OR*）。
- **index** (*int*) –要检索的通用约束的索引（从 0 开始）。

返回
所查询的通用约束。

返回类型
OPTVGenConstr

引发

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –模型已释放。
- ***OPTVErrCode.INVALID_ARGUMENT*** –无效的约束类型。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –index 超出范围，或者约束尚未添加到模型中。

示例

```
c0 = model.AddGenConstrAnd(x1, [x2, x3, x4], "and0")
model.Update()
gen_constr = model.GetGenConstr(OPTV_GENCONSTR_AND, 0)
```

GetGenConstrs()

获取指定类型的所有常规约束。

OPTVModel.**GetGenConstrs**(*gcType*)

参数
gcType(*int*) –通用约束类型（例如*OPTV_GENCONSTR_AND*、*OPTV_GENCONSTR_OR*）。

返回
指定类型的所有通用约束列表。

返回类型
list[*OPTVGenConstr*]

引发

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –模型已释放。
- ***OPTVErrCode.INVALID_ARGUMENT*** –无效的约束类型。

示例

```
model.AddGenConstrAnd(x1, [x2, x3], "and0")
model.AddGenConstrAnd(x4, [x5, x6], "and1")
model.Update()
and_constrs = model.GetGenConstrs(OPTV_GENCONSTR_AND) # [and0, and1]
```

GetGenConstrAnd()

获得模型中特定的 AND 一般约束数据。

OPTVModel.**GetGenConstrAnd**(*genConstr*)

参数
genConstr(OPTVGenConstr) –需要查询的一般约束。

返回
元组，索引值 0 对应该一般约束对应的结果变量，索引值 1 对应参与检验该一般约束的所有变量的列表。

返回类型
(*OPTVVar*, list[*OPTVVar*])

引发
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

示例

```
c0 = model.AddGenConstrAnd(x1, [x2, x3, x4], "and")
model.Update()
resvar, vars_list = model.GetGenConstrAnd(c0)
```

GetGenConstrOr()

获得模型中特定的 OR 一般约束数据。

OPTVModel.**GetGenConstrOr**(*genConstr*)

参数

genConstr(OPTVGenConstr) –需要查询的一般约束。

返回

元组，索引值 0 对应该一般约束对应的结果变量，索引值 1 对应参与检验该一般约束的所有变量的列表。

返回类型

(*OPTVVar*, list[*OPTVVar*])

引发

OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

示例

```
c0 = model.AddGenConstrAnd(x3, [x1, x2], "or")
model.Update()
resvar, vars_list = mdoel.GetGenConstrOr(c0)
```

GetGenConstrIndicator()

获得模型中特定的 INDICATOR 一般约束数据。

OPTVModel.**GetGenConstrIndicator**(*genConstr*)

参数

genConstr(OPTVGenConstr) –需要查询的一般约束。

返回

元组，索引值 0 对应二进制指示变量, 索引值 1 对应要求线性约束必须满足的二进制指示变量的值, 索引值 2 对应参与检验的线性约束表达式, 索引值 3 对应线性约束的含义, 索引值 4 对应参与检验的线性约束右端值。

返回类型

(*OPTVVar*, float, *OPTVLinExpr*, str, float)

引发

OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

示例

```
# x3 = 1 -> x2+x1+1 <= 10
c0 = model.AddGenConstrIndicator(x3, 1, x2+x1+1, OPTV_SENSE_LESS, 10,␣
↪name="indicator")
model.Update()
binVar, binVal, expr, sense, rhs = mdoel.GetGenConstrIndicator(c0)
```

GetGenConstrAbs()

获取模型中指定的 ABS 一般约束数据。

OPTVModel.**GetGenConstrAbs**(*genConstr*)

参数
genConstr (OPTVGenConstr) –需要查询的一般约束。

返回
一个元组，第一个元素为结果变量，第二个元素为参数变量。

返回类型
(*OPTVVar*, *OPTVVar*)

引发
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

示例

```
c0 = model.AddGenConstrAbs(x1, x2, "abs")
model.Update()
resvar, consvar = model.GetGenConstrAbs(c0)
```

GetGenConstrNorm()

获得模型中指定的 NORM 一般约束数据。

OPTVModel.**GetGenConstrNorm**(*genConstr*)

参数
genConstr (OPTVGenConstr) –需要查询的一般约束。

返回
一个元组，第一个元素为结果变量，第二个元素为用于计算范数的变量列表，第三个元素为范数类型（-1 为 L^{∞} 范数，0 为 L^{0} 范数，1 为 L^{1} 范数）。

返回类型
(*OPTVVar*, list[*OPTVVar*], int)

引发
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

示例

```
c0 = model.AddGenConstrNorm(x1, [x2, x3, x4], 1, "norm")
model.Update()
resvar, consvars, norm_type = model.GetGenConstrNorm(c0)
```

GetGenConstrMin()

获得模型中指定的 MIN 一般约束数据。

OPTVModel.**GetGenConstrMin**(*genConstr*)

参数
genConstr (OPTVGenConstr) –需要查询的一般约束。

返回
一个元组，第一个元素为结果变量，第二个元素为参与检验该约束的所有变量的列表，第三个元素为参与该约束的常量操作数。

返回类型
(*OPTVVar*, list[*OPTVVar*], float)

引发
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

示例

```
c0 = model.AddGenConstrMin(x1, [x2, x3, x4], "min")
model.Update()
resvar, vars_list, constant_oprend = model.GetGenConstrMin(c0)
```

GetGenConstrMax()

获取模型中指定的 MAX 一般约束数据。

OPTVModel.**GetGenConstrMax**(*genConstr*)

参数
genConstr (OPTVGenConstr) –需要查询的一般约束。

返回
一个元组，第一个元素为结果变量，第二个元素为参与检验该约束的所有变量的列表，第三个元素为参与该约束的常量操作数。

返回类型
(*OPTVVar*, list[*OPTVVar*], float)

引发
OPTVErrCode.NOT_IN_MODEL –模型中不存在该一般约束，当该一般约束已从模型中被删除或者是由默认的构造函数创建时，会抛出此错误。

示例

```
c0 = model.AddGenConstrMax(x1, [x2, x3, x4], "max")
model.Update()
resvar, vars_list, constant_oprend = model.GetGenConstrMax(c0)
```

GetLinMatrix()

获得模型线性约束的矩阵。

OPTVModel.**GetLinMatrix**()

该函数用于获得模型线性约束的 CSR 形式的矩阵。

返回
模型线性约束的 CSR 形式的矩阵。

返回类型
scipy.sparse.csr_matrix

示例

```
result = mdoel.GetLinMatrix()
```

GetObjective() 获取线性目标函数

获取线性目标函数。

OPTVModel.**GetObjective**()

返回
目标函数的线性表达式。

返回类型
OPTVLinExpr

示例

```
result = model.GetObjective()
```

GetObjectiveN()

在多目标优化中，获取指定目标函数的线性表达式。

OPTVModel.**GetObjectiveN**(*objId*)

参数
objId(*int*) –目标函数的索引。

返回
表示指定的目标函数的线性表达式对象。

返回类型
OPTVLinExpr

引发
OPTVErrCode.VALUE_OUT_OF_RANGE –多目标优化的目标函数的索引值大于模型中目标函数的数量。

示例

```
result = model.GetObjectiveN(0)
```

GetQConstr()

根据指定索引值获得模型中对应的二次约束。

OPTVModel.**GetQConstr**(*i*)

参数
i (*int*) –线性约束在模型中的索引。

返回
索引为 i 的约束。

返回类型
OPTVQConstr

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –指定的索引值超出了模型的约束总数。

示例

```
result = model.GetQConstr(1)
```

对应的二次约束。

GetQConstrs()

获得模型中的所有二次约束。

OPTVModel.**GetQConstrs**()

返回
包含模型中所有二次约束的列表。

返回类型
list[*OPTVQConstr*]

示例

```
result = model.GetQConstrs()
qc0 = result[0]
```

GetSOS()

根据索引值获取指定类型的 SOS 约束。

OPTVModel.**GetSOS**(*i*, *type*)

参数

- **i** (*int*) –指定 SOS 约束在模型中的索引值。
- **type** (*int*) –SOS 类型，取值为 1 或 2。

返回
对应索引值 i 的指定类型的 SOS 约束。

返回类型
OPTVSOS

引发

- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –索引值 `i` 超出了模型的该类型的 SOS 约束总数。
- ***OPTVErrCode.INVALID_ARGUMENT*** –`type` 取值不是 `1` 或 `2`。

示例

```
c0 = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos1")
result = model.GetSOS(0, 1)
```

这里，参数 `i` 特指某一类型 SOS 约束集中的索引值，例如，如果用户先后添加了类型为 `1` 和 `2` 的两个 SOS 约束，那么两者的索引值都是 `0`.

GetSOSData()

OPTVModel.**GetSOSData**(*sos*, *sosVars*, *weights*)

获取 SOS 约束的内部数据。

参数

- **sos** (OPTVSOS) –当前访问的 SOS 约束对象。
- **sosVars** (*list[OPTVVar]*) –变量序列的空向量。如果非空，该容器在本次函数调用中会被清空。
- **weights** (*list[float]*) –变量权重序列的空向量。如果非空，该容器在本次函数调用中会被清空。

返回

SOS 涉及的变量及权重的元组。

返回类型

Tuple[*OPTVVar*, float]

引发

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –SOS 约束是由默认构造函数生成的。
- ***OPTVErrCode.NOT_IN_MODEL*** –SOS 约束属于另外一个模型，或已经从本模型中被删除。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –约束下表越界、超出容器范围，可能需要对当前模型执行更新。

示例

```
c0 = model.AddSOS([x, y, z], [1, 2, 3], 1, name="sos1")
(var, weight) = model.GetSOSData(c0, [x, y, z], [1, 2, 3])[0]
```

GetSOSs()

获取指定类型的所有 SOS 约束。

OPTVModel.**GetSOSs**(*type*)

参数
type (*int*) –SOS 类型，取值为 1 或 2。

返回
包含模型中所有类型为 type 的 SOS 约束的列表。

返回类型
list[*OPTVSOS*]

引发

- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –SOS 约束的索引值超出了模型的该类型的 SOS 约束总数，可能需要对当前模型执行更新。
- ***OPTVErrCode.INVALID_ARGUMENT*** –type 取值不是 1 或 2。

示例

```
result = model.GetSOSs(1)
```

GetQObjective()

获取二次目标函数。

OPTVModel.**GetQObjective**()

返回
目标函数的二次表达式。

返回类型
OPTVQuadExpr

示例

```
result = model.GetQObjective()
```

GetRow()

获取指定约束对应的线性函数表达式。

OPTVModel.**GetRow**(*c*)

参数
c (OPTVConstr) –指定的约束对象。

返回
该约束对应的线性表达式。

返回类型
OPTVLinExpr

引发

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –该约束不存在于当前模型。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –*Index* of the constraint is larger than the number of constraints in model. This may occur if the constraint needs to be flushed to the model. This exception may also be thrown if the constraint is from another model.

示例

```
constrs = model.GetConstrs()
linexpr = model.GetRow(constrs[0])
```

Get()

参数

获取求解器的参数值。

可用参数请参考数学规划求解器参数。

OPTVModel.**Get**(*param*)

参数
param(OPTVIntParam/OPTVDblParam/OPTVStrParam)–需要查询的参数，包括整数型、双精度型、字符串型。

返回
参数取值。

返回类型
int|float|str

引发
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

示例

```
result1 = model.Get(OPTVIntParam.CUTS)
result2 = model.Get(OPTVDblParam.GAP)
result3 = model.Get(OPTVStrParam.LOG_FILE)
```

属性

获取属性的取值。

可用属性请参考天筹属性。

OPTVModel.**Get**(*attr*)

参数
attr(OPTVIntAttr/OPTVDblAttr/OPTVLongAttr/OPTVStrAttr)–需要查询的属性，包括整数型、双精度型。

返回
属性取值。

返回类型
int|float|str

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

示例

```
result1 = model.Get(OPTVIntAttr.NUM_CONSTRS)
result2 = model.Get(OPTVDblAttr.MIP_GAP)
result3 = model.Get(OPTVLongAttr.FINGERPRINT)
result4 = model.Get(OPTVStrAttr.OBJ_N_NAME)
```

GetVarByName()

通过名称获取变量。

OPTVModel.**GetVarByName**(*name*)

参数
name (*str*) –需要访问的变量名称。

返回
名称匹配 name 的变量。

返回类型
OPTVVar

引发
OPTVErrCode.DATA_NOT_AVAILABLE –给定的名称为空，或模型不存在命名为 name 的变量。

示例

```
result = model.GetVarByName("x")
```

GetVar()

根据指定索引值获得模型中对应的变量。

OPTVModel.**GetVar**(*i*)

参数
i (*int*) –变量在模型中的索引。

返回
在模型中索引 i 的变量。

返回类型
OPTVVar

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –指定的索引值超出了模型的变量总数。

示例

```
result = model.GetVa(0)
```

GetVars()

获得模型中的所有变量。

OPTVModel.**GetVars**()

返回
包含模型中所有变量的列表。

返回类型
list[*OPTVVar*]

示例

```
result = model.GetVars()
x = result[0]
```

Optimize()

对模型进行求解。

OPTVModel.**Optimize**()

返回
无。

返回类型
None

引发
OPTVErrCode.MATRIX_NOT_PSD–模型中包含二次目标函数或者二次约束，而该表达式的矩阵不满足正半定性质。当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的。

示例

```
model.Optimize()
```

求解器采用哪种优化算法取决于求解器参数配置和模型属性，例如变量类型被用以自动识别 LP, QP, MIP 及网络流问题。

Presolve()

对模型进行预处理。

OPTVModel.**Presolve**()

该函数会应用预处理流程，返回一个新的*OPTVModel* 对象。原模型的状态被修改为 被预处理.

返回
预处理后的模型。

返回类型
OPTVModel

示例

```
result = mdoel.Presolve()
```

Read()

从文件中读取读取模型和基。

OPTVModel.**Read**(*fileName*)

该函数供读取基信息，也可以读取模型文件，然而建议由构造函数读取文件中的模型，参考文件格式。

表 16: 支持读入的文件格式

.mps	.lp	.bas

表 17: 支持读入的压缩格式

.gz	.z	.Z

参数

fileName (*str*) –C++ 字符串文件名。

返回

无。

返回类型

None

引发

OPTVErrCode.ERROR_FILE_READ –文件名或后缀为空，文件存在格式错误。

示例

```
model.Read("input.mps")
model.Read("input.gz")
```

Relax()

进行线性规划 (LP) 松弛。

OPTVModel.**Relax**()

该函数将原模型中的所有整数类型变量替换为*OPTV_CONTINUOUS* 变量，生成并返回一个新的*OPTVModel* 对象，对原模型不做任何修改。

返回

模型的 LP 松弛副本。

返回类型

OPTVModel

示例

```
r = model.Relax()
```

Remove()

从模型中删除变量。

OPTVModel.**Remove**(*var*)

该函数将变量从模型中删除，对应的索引会被相应地调整。

例如，若下列模型 model 构建时包含以下变量：

变量	索引
x0	0
x1	1
x2	2
x3	3

调用 model.Remove(x0) 后，相应变化为：

变量	索引
x1	0
x2	1
x3	2

参数

var (OPTVVar) –指定要删除的变量。

返回

无。

返回类型

None

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –模型不包含此变量。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –该变量属于其他模型，而不属于当前模型。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –变量的下标超出了模型的变量总数，可能需要执行模型更新。

OPTVModel.**Remove**(*constr*)

该函数将约束从模型中删除，对应的索引会被相应地调整。

例如，若模型 model 创建时包含如下约束：

约束	索引
c0	0
c1	1
c2	2
c3	3

调用 `model.Remove(c0)` 后，相应变化为：

约束	索引
c1	0
c2	1
c3	2

参数
constr (OPTVConstr) –指定要删除的约束。

返回
无。

返回类型
None

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –模型不包含此约束。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –该约束属于其他模型，而不属于当前模型。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –约束的下标超出了模型的变量总数，可能需要执行模型更新。

OPTVModel.**Remove**(*genConstr*)

从模型中删除单个一般约束。执行后对应的相同类型的一般约束索引会被相应地调整。

例如，若下列模型 `model` 构建时包含以下一般约束：

约束	类型	索引
and0	AND	0
and1	AND	1
or0	OR	0
or1	OR	1

调用 `model.Remove(and0)` 后，相应变化为：

约束	类型	索引
and1	AND	0
or0	OR	0
or1	OR	1

参数
genConstr (OPTVGenConstr) –指定要删除的一般约束。

返回
无。

返回类型
None

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –模型不包含此约束。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –该约束属于其他模型，而不属于当前模型。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –该一般约束的下标超出了模型的变量总数，可能需要执行模型更新。

OPTVModel.**Remove**(*sos*)

从模型中删除单个 SOS 约束。与其他约束类型不同的是，删除该 SOS 约束后，相同类型的约束的索引**不会**改变。

例如，若模型 `model` 创建时包含如下约束：

约束	类型	索引
sos1_1	1	0
sos1_2	1	1
sos1_3	1	2
sos2_1	2	0

调用 `model.Remove(sos1_2)` 后，相应变化为：

约束	类型	索引
sos1_1	1	0
sos1_3	1	2
sos2_1	2	0

参数

sos (OPTVSOS) –指定要删除的 SOS 约束。

返回

无。

返回类型

None

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –模型不包含此约束。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –该约束属于其他模型，而不属于当前模型。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –SOS 约束的索引值超出了模型的该类型的 SOS 约束总数，可能需要对当前模型执行更新。

OPTVModel.**Remove**(*cone*)

该函数从模型中移除一个锥约束。移除操作将导致后续锥约束的索引发生前移。

约束	类型	索引
c0	SOC	0
c1	RSOC	1
c2	SOC	2
c3	RSOC	3

调用 `model.Remove(c0)` 后，相应变化为：

约束	类型	索引
c1	RSOC	0
c2	SOC	1
c3	RSOC	2

参数

cone (OPTVCone) –需要移除的锥约束对象。

返回

无。

返回类型

None

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –锥约束不在任何模型中。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –锥约束属于另一个模型，不在当前模型中。
- ***OPTVErrCode.INDEX_OUT_OF_RANGE*** –锥约束的索引大于模型中锥约束的数量。添加锥约束后，没有将其更新到模型中，则可能发生此情况。

Reset()

将模型重置为 *未求解成功*状态。

OPTVModel.**Reset**(*clearAll=false*)

该函数会清除内部解的信息，默认情况下，包括解的值、LP 基和模型状态。`clearAll` 标志是否清除额外的模型属性值（包括 MIP 热启动值，MIP 分支优先度，LP 的禁用属性状态）。

参数

clearAll (*bool*) –清除额外信息（默认为 `false`）。

返回

无。

返回类型

None

示例

```
model.Reset()
```

GetObjSA()

目标函数敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**GetObjSA**(*down*, *up*, *list*)

参数

- **down** (*list[float]*) –敏感度区间左端点。
- **up** (*list[float]*) –敏感度区间右端点。
- **list** (*list[int]*) –需要进行目标函数敏感度分析的变量索引集合。

返回

敏感度分析是否成功。

返回类型

bool

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

示例

```
# Objective coefficient sensitivity info, within which the current␣
↪optimal basis would remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.Optimize()
cols = [0, 1, 2]
objLower = []
objUpper = []
result = model.GetObjSA(objLower, objUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, objLower[i], objUpper[i]))
```

GetVarLbSA()

变量下界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**GetVarLbSA**(*down*, *up*, *list*)

参数

- **down** (*list[float]*) –下界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** (*list[float]*) –下界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** (*list[int]*) –需要进行变量下界敏感度分析的约束索引集合。

返回
敏感度分析是否成功。

返回类型
bool

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

示例

```
# Lower bound sensitivity info, within which the current optimal␣
↪basis would remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.Optimize()
cols = [0, 1, 2]
lbLower = []
lbUpper = []
result = model.GetVarLbSA(lbLower, lbUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, lbLower[i], lbUpper[i]))
```

GetVarUbSA()

变量上界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**GetVarUbSA**(*down*, *up*, *list*)

参数

- **down** (*list[float]*) –上界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** (*list[float]*) –上界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** (*list[int]*) –需要进行变量上界敏感度分析的约束索引集合。

返回
敏感度分析是否成功。

返回类型
bool

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

示例

```
# Upper bound sensitivity info, within which the current optimal␣
↪basis would remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
```

(下页继续)

(续上页)

```
model.Optimize()
cols = [0, 1, 2]
ubLower = []
ubUpper = []
result = model.GetVarUbSA(ubLower, ubUpper, cols)
for i in cols:
    print("x%s : [%s, %s]" % (i+1, ubLower[i], ubUpper[i]))
```

GetConstrLbSA()

约束左端值/下界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**GetConstrLbSA**(*down*, *up*, *list*)

参数

- **down** (*list[float]*) –下界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** (*list[float]*) –下界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** (*list[int]*) –需要进行左端值敏感度分析的约束索引集合。

返回

敏感度分析是否成功。

返回类型

bool

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

示例

```
# LHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.AddConstr(-x1 + x2 + x3, -OPTV_INF, 20, "c1")
model.AddConstr(x1 - 3*x2 + x3, -OPTV_INF, 30, "c2")
model.Optimize()
rows = [0, 1]
lhsLower = []
lhsUpper = []
result = model.GetConstrLbSA(lhsLower, lhsUpper, rows)
for i in rows:
    print("c%s : [%s, %s]" % (i+1, lhsLower[i], lhsUpper[i]))
```

GetConstrUbSA()

约束右端值/上界敏感度分析。

请参考*LP 敏感度分析快速入门*。

OPTVModel.**GetConstrUbSA**(*down*, *up*, *list*)

参数

- **down** (*list[float]*) –上界可以改变的最小值度使得当前最优基仍然是最优的。
- **up** (*list[float]*) –上界可以改变的最大幅度使得当前最优基仍然是最优的。
- **list** (*list[int]*) –需要进行右端值敏感度分析的约束索引集合。

返回
敏感度分析是否成功。

返回类型
bool

返回值

- **false** –敏感度分析失败。
- **true** –敏感度分析成功。

示例

```
# RHS sensitivity info, within which the current optimal basis would␣
↪remain optimal
x1 = model.AddVar(-OPTV_INF, 40, -1, OPTV_CONTINUOUS, "x1")
x2 = model.AddVar(0, OPTV_INF, -2, OPTV_CONTINUOUS, "x2")
x3 = model.AddVar(0, OPTV_INF, -3, OPTV_CONTINUOUS, "x3")
model.AddConstr(-x1 + x2 + x3, -OPTV_INF, 20, "c1")
model.AddConstr(x1 - 3*x2 + x3, -OPTV_INF, 30, "c2")
model.Optimize()
rows = [0, 1]
rhsLower = []
rhsUpper = []
result = model.GetConstrUbSA(rhsLower, rhsUpper, rows)
for i in rows:
    print("c%s : [%s, %s]" % (i+1, rhsLower[i], rhsUpper[i]))
```

SetCoef()

设置给定约束中指定变量的系数。

OPTVModel.**SetCoef**(*con*, *var*, *newValue*)

参数

- **con** (OPTVConstr) –需要修改的约束。
- **var** (OPTVVar) –需要修改系数的变量。
- **newValue** (*float*) –新的系数值。

返回
无。

返回类型
None

引发

- ***OPTVErrCode.INVALID_ARGUMENT*** –约束或变量不存在，或数值参数中存在 NaN。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –数据不存在，或约束的线性表达式部分不可用。

示例

```
c0 = mdoel.AddConstr(x+y, 1, 10, "c0")
# c0: 1 <= x+2*y <= 10
model.SetCoef(c0, y, 2)
```

SetObjective()

设置线性目标函数。

OPTVModel.**SetObjective**(*expr*, *sense=OPTVSense.MINIMIZE*)

该函数清除当前目标函数（相对于在其基础上进行叠加）。若未指定 sense 参数，求解器会采用默认的（*OPTVSense.MINIMIZE*）。

参数

- **expr** (OPTVLinExpr / OPTVQuadExpr) –需要设定的目标函数表达式。
- **sense** (OPTVSense) –**可选**，最小化或最大化，默认值为*OPTVSense.MINIMIZE*，参考*OPTVSense*。

返回
无。

返回类型
None

示例

```
model.SetObjective(x+y+z, OPTVSense.MINIMIZE)
model.SetObjective(x*y+z, OPTVSense.MAXIMIZE)
```

SetObjectiveN()

在多目标优化中，设置线性目标函数。

OPTVModel.**SetObjectiveN**(*objId*, *expr*, *name*, *priority*, *weight*, *absTolerance*, *relTolerance*)

当模型中有多个目标函数时，该函数会替换当前具有相同 Id 的目标函数（而不是进行累加），否则会向模型中添加一个新的目标函数。目标函数的优化方向由参数 weight 的符号决定。

参数

- **objId** (*int*) –待修改目标函数的索引。
- **expr** (OPTVLinExpr) –需要设定的目标函数表达式。
- **name** (*str*) –目标函数的名称。
- **priority** (*int*) –目标函数的优先级（高优先级的目标函数会被优先求解）。

- **weight** (*float*) –[**可选**, 默认值 1.0] 目标函数的权重（用于确定目标函数的优化方向，以及处理多个目标函数拥有相同优先级的情况）。
- **absTolerance** (*float*) –[**可选**, 默认值 0.0] 目标函数的绝对容差（决定在后续的优化过程中该目标函数允许的最大绝对偏差）。
- **relTolerance** (*float*) –[**可选**, 默认值 0.0] 目标函数的相对容差（决定在后续优化过程中该目标函数允许的最大相对偏差）。

返回
无。

返回类型
None

示例

```
model.SetObjectiveN(0, x+y+0.3, "obj1", 1, 1.0, 0.0, 0.0)
model.SetObjectiveN(1, x+y+z, "obj2", 2, 1.5, 1.0, 0.5)
```

Set()

参数

设置求解器的参数取值，一旦模型被创建，该接口仅提供控制此模型专用参数的接口，直到调用*OPTVModel.Update()* 方法后，新修改才会在内部生效。

可用参数请参考*数学规划求解器参数*。

OPTVModel.**Set** (*param*, *value*)

参数

- **param** (OPTVIntParam / OPTVDblParam / OPTVStrParam) –需要设置的参数。
- **value** (*int* / *float* / *str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

示例

```
model.Set(OPTVIntParam.PRESOLVE_LEVEL, 0)
model.Set(OPTVStrParam.RESULT_FILE, sol_file)
model.Set("resultFile", "example2.sol")
```

属性

设置求解器的属性值，同样，直到调用*OPTVModel.Update()* 方法后，新修改才会在内部生效。

可用属性请参考*天筹属性*。

OPTVModel.**Set**(*attr*, *value*)

参数

- **attr** (OPTVIntAttr / OPTVDblAttr / OPTVStrAttr) –需要设置的属性。
- **value** (*int* / *float* / *str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_ATTRIBUTE –未知属性。

示例

```
model.Set(OPTVIntAttr.OBJ_SENSE, OPTVSense.MAXIMIZE)
model.Set(OPTVDblAttr.OBJ_OFFSET, 2.5)
model.Set(OPTVStrAttr.OBJ_N_NAME, "obj2")
```

Update()

执行模型更新。

OPTVModel.**Update**()

该函数会进行模型更新，或 lazy 构建（如必要）。只有调用该函数 **后**，模型的更新才会生效。SDK 构建的模型需要调用此函数来保证内部模型的即时性。例如，用户调用*OPTVModel.Optimize()* 前 **不需要**调用此函数，因为该函数在内部已被默认执行。

返回
无。

返回类型
None

示例

```
model.Update()
```

WriteIIS()

将不可约不一致子系统（IIS）写入文件（.ilp 格式）。

有关 IIS 的细节请参考*IIS 快速入门*。

OPTVModel.**WriteIIS**(*fileName*)

参数
fileName (*str*) –写出的文件名，必须包含 .ilp 后缀（例如 iis.ilp）.

返回
无。

返回类型
None

引发
OPTVErrCode.ERROR_FILE_WRITE –不支持的文件格式，或者未计算出 IIS.

注解

此函数 **不会**计算 IIS，若 IIS 未被计算，本函数将抛出异常。

示例

```
if model.ComputeIIS():
    model.WriteIIS("iis.ilp")
else:
    print("IIS computation failed")
```

Write()

将模型或基写出到文件。

OPTVModel.**Write**(*fileName*)
支持的文件格式请参考文件格式。

表 18: 支持写出的文件格式

.mps	.lp	.bas	.ilp

表 19: 支持写出的压缩格式

.gz	.z	.Z

参数
fileName (*str*) –C++ 字符串文件名。

返回
无。

返回类型
None

引发
OPTVErrCode.ERROR_FILE_WRITE –文件名或后缀为空或错误。

示例

```
model.Write("out.mps")
mdoel.Write("out.gz")
```

临时约束

class OPTVTempConstr

天筹求解器的临时约束对象。OPTVTempConstr 类用于表示通过重载运算符（==、<=、>=）构建的约束表达式的中间结果，起到临时占位的作用。该类不提供任何成员函数。当对变量或者表达式（*OPTVVar*、*OPTVLinExpr*、*OPTVQuadExpr*）执行比较操作（==、<=、>=）时，返回的结果即为 OPTVTempConstr 对象。这类对象通常会被添加约束的方法立即使用，不建议用户将其保存在自定义的变量中。

OPTVTempConstr 允许用户创建多种不同类型的约束:

- **线性约束**：线性约束的形式为 Expr1 sense Expr2，其中 Expr1 与 Expr2 可以是*OPTVVar*、*OPTVLinExpr*，或者常量，而 sense 则是以下比较操作符的其中之一：==、<=、>=。例如，x + y <= 1 + z 与 x + y == 5 都是有效的线性约束。
- **二次约束**：二次约束的形式为 Expr1 sense Expr2，其中 Expr1 与 Expr2 可以是*OPTVVar*、*OPTVLinExpr*、*OPTVQuadExpr*，或者常量，而 sense 则是以下比较操作符的其中之一：==、<=、>=。例如，x*y <= 1 + z 与 x*x + y*y <= z*z 都是有效的二次约束。需要注意的是 Expr1 与 Expr2 至少有一个是*OPTVQuadExpr*，否则该约束为线性约束。
- **绝对值约束**：该约束的形式为 x == Abs_(y)，其中 x 与 y 都是*OPTVVar*对象。
- **逻辑约束**：该约束的形式为 x == And_(y) 或者 x == Or_(y)。其中 x 是一个 0-1 类型的*OPTVVar*对象，y 是一个元素为*OPTVVar*对象的列表。
- **最大或最小约束**：该约束的形式为 x == Max_(y) 或者 x == Min_(y)。其中 x 是一个*OPTVVar*对象，y 是一个元素为*OPTVVar*对象的列表。

可以参考以下示例:

```
model.AddConstr(x + y >= 1, name="c0")

model.AddConstr(x*x + y*y <= z, name="c1")

model.AddConstr(x == Abs_(y), name="c2")

model.AddConstr(z == And_([x, y, w]), name="c3")

model.AddConstr(z == Max_([x, y, w]), name="c4")
```

异常

class OPTVException

异常类。

此类代表天筹求解器抛出的异常。它提供对错误代码（*OPTVErrCode*）和描述性消息的访问。

使用*OPTVException.GetErrorCode()* 确定具体的错误类型，并使用*OPTVException.GetMessage()* 获取错误信息。

OPTVException.**GetMessage**()

获取异常的错误信息。

返回
异常的错误信息。

返回类型
str

OPTVException.**GetErrorCode**()

获取异常的错误码。

返回
异常的错误码。

返回类型
OPTVErrCode

关于*OPTVErrCode* 更多细节请参考*OptVerse Error Code*.

错误码

OPTVErrCode 用来产生特定的*OPTVException* 异常对象，并给出相应的详细信息。

class OPTVErrCode

天筹错误码。

ERROR

错误：错误信息，一般从求解器内部调用抛出。

WARNING

警告：警告信息，一般从求解器内部调用抛出。

UNKNOWN

未知：遇到未知错误。

INVALID_ARGUMENT

无效参数：传入函数的参数不满足条件。

例如在设置变量类型*OPTVCharAttr.VAR_TYPE* 时，传入未定义的属性值，或者用 0 除线性表达式。

UNKNOWN_ATTRIBUTE

未知属性：访问或者设置未定义的属性。

例如*OPTVConstr.Get(OPTVIntAttr.NUM_VARS)* 将抛出此错误码，因为*OPTVIntAttr.NUM_VARS* 不是模型的属性。

DATA_NOT_AVAILABLE

数据不可用：访问的属性值不可用。

例如在调用*OPTVModel.Optimize()* 之前，试图获取解的信息*OPTVVar.Get(OPTVDblAttr.X)* 将会抛出此错误。

INDEX_OUT_OF_RANGE

下标越界：用于获取对象的索引值超出了允许范围。

例如，当前模型仅有 2 个变量，调用*model.GetVar(10)* 会抛出此异常。

UNKNOWN_PARAMETER

未知参数：访问未定义的参数。

VALUE_OUT_OF_RANGE

数值越界：参数值超出了允许范围，例如尝试获取不存在的*OPTVSense*.

ERROR_FILE_READ

文件读取失败：读取文件时发生错误。

ERROR_FILE_WRITE

文件写出失败：写出文件时发生错误。

LICENSE_CHECK_FAILED

许可检查失败：未发现有效许可。

注：许可文件 OPTV_LICENSE_FILE 需要通过环境变量设置，细节请参考安装说明。

INVALID_MODEL_INPUT

模型输入无效：试图使用无效的模型输入信息。

NOT_IMPLEMENTED

功能不可用：试图使用的功能暂未开放。

NOT_IN_MODEL

模型组件不存在：访问的变量或约束在模型中不存在。

MATRIX_NOT_PSD

非正半定矩阵：二次表达式的矩阵不满足正半定性质。

当求解的模型 **包含**二次约束时，QCQP 求解器要求模型的二次目标函数或二次约束的矩阵都是正半定的。

优化方向

class OPTVSense

目标含义，默认值为 1，表征最小化问题。

MINIMIZE

最小化问题 (+1)

类型
int

取值
1

MAXIMIZE

最大化问题 (-1)

类型
int

取值
-1

锥类型

class OPTVConeType

锥类型。

锥类型用于指定锥约束中的仿射表达式 必须满足几何约束 C。

SOC

二阶锥

二阶锥 C_s 的定义如下：

$$C_s^n = \left\{ x \in \mathbb{R}^n \mid x_1 \geq \sqrt{x_2^2 + x_3^2 + \cdots + x_n^2} \right\}.$$

RSOC

旋转二阶锥。

旋转二阶锥 C_r 的定义如下：

$$C_r^n = \left\{x \in \mathbb{R}^n \mid 2x_1x_2 \geq x_3^2 + \cdots + x_n^2,\ x_1, x_2 \geq 0\right\}.$$

有关锥的详细介绍可见锥约束。

调参工具

该类用来为测例构造最佳参数组合。

class OPTVTune

OPTVTune()

构造函数。

OPTVTune(*targets:* OPTVModel | *List[*OPTVModel*]*, *definedSpaces: List[*OPTVTuneSpace*]*, *maxTime: int*, *timeLimit: int*, *numSuggestion: int*, *indicator: str*, *bkptFile: str*, *bkptDir: str*, *logDir: str*, *resultDir: str*)

参数

- **targets** –需要进行调参的模型
- **definedSpaces** –参数搜索空间
- **maxTime** –调参时限
- **timeLimit** –模型单次求解时间上限
- **numSuggestion** –每轮尝试的最多参数组合个数
- **inidcator** –调参指标，包括 - `time`: 求解时间 - `gap`: 相对偏差（仅适用于 MIP 问题）- `solve`: 可解性（批量调参任务）
- **bkptFile** –断点文件读取路径，用以继续暂停的调参任务；若为空，则开始全新的调参任务。
- **bkptDir** –断点文件保存路径，保存当前调参进程的断点；若为空，则不保存任何断点。
- **logDir** –调参日志保存路径；若为空，则不保存任何日志。
- **resultDir** –调参结果保存路径；若为空，则不保存任何信息。

返回

无。

SetFixedParams()

设置调参任务的固定参数。

OPTVTune.**SetFixedParams**(*self*, *fixedParams: Dict*)

参数
fixedParams (*Dict*) –指定固定的参数

返回
无。

Run()

执行调参任务。

OPTVTune.**Run**()

返回
无。

GetBestParams()

获取最佳参数组合。

OPTVTune.GetBestParams(self):

可获取最佳参数组合信息，包括

nRound: int: 获取最佳参数组合所在的调参轮数

paramID: int: 获取最佳参数组合的索引

bestParams: Dict: 获取最佳参数组合值

OPTVTuneSpace 调参空间

调参任务的预设参数搜索空间

class OPTVTuneSpace

- PRESOLVE: 预处理参数的搜索空间。
- SEARCH: Branch 方法参数的搜索空间。
- SEPARATOR: Separation 方法参数的搜索空间。
- HEURISTIC: 启发式方法参数的搜索空间。
- CONFLICT: 冲突分析方法参数的搜索空间。
- PROPAGATION: 邻域传播方法参数的搜索空间。
- LP: LP 算法参数的搜索空间。

常量

OPTV_INF

表示一个无穷大的值。

类型
float

取值
`1.0e+100`

OPTV_UNDF

未定义的数值。

类型
float

取值
`1.0e+101`

变量类型

OPTV_BINARY

0-1 整数变量类型。

类型
str

取值
`'B'`

OPTV_INTEGER

整数变量类型。

类型
str

取值
`'I'`

OPTV_CONTINUOUS

连续变量类型。

类型
str

取值
`'C'`

模型求解状态

OPTV_UNKNOWN

未知模型求解状态（默认）。

类型
int

取值
0

OPTV_OPTIMAL

模型找到最优解，可以查询到最优解。

类型
int

取值
1

OPTV_INFEASIBLE

模型被证明无解。

类型
int

取值
2

OPTV_INF_OR_UNBD

模型被证明无解或者无界。

类型
int

取值
3

OPTV_UNBOUNDED

模型被证明无界。

类型
int

取值
4

OPTV_TIME_LIMIT

模型在指定时间*OPTVDblParam.TIME_LIMIT* 内没有完成求解。

类型
int

取值
5

OPTV_MEM_LIMIT

模型求解使用内存超过预设值*OPTVIntParam.MEM_LIMIT*.

类型
int

取值
6

OPTV_INTERRUPTED

用户终止模型求解（如通过 Ctrl+C）.

类型
int

取值
7

变量和约束的基状态

OPTV_BASIC

变量或约束为基。

类型
int

取值
0

OPTV_NONBASIC_LOWER

变量或约束非基，取下界值。

类型
int

取值
-1

OPTV_NONBASIC_UPPER

变量或约束非基，取上界值。

类型
int

取值
-2

OPTV_SUPERBASIC
变量或约束为超基。

类型
int

取值
-3

天筹一般约束类型

OPTV_GENCONSTR_AND
只有所有变量非零时，结果才非零。

类型
int

取值
0

OPTV_GENCONSTR_OR
只要有一个变量非零，结果就非零。

类型
int

取值
1

OPTV_GENCONSTR_INDICATOR
返回值为布尔型，当且仅当相应的逻辑条件满足，取值为 1，否则取值为 0。

类型
int

取值
2

OPTV_GENCONSTR_ABS
绝对值约束：`x == Abs(y)`。

类型
int

取值
3

OPTV_GENCONSTR_NORM
欧几里得范数约束：`x == Norm(y1, y2, ...)`。

类型
int

取值
4

OPTV_GENCONSTR_MIN

最小约束：`x == Min(y1, y2, ...)`。

类型

int

取值

5

OPTV_GENCONSTR_MAX

最大约束：`x == Max(y1, y2, ...)`。

类型

int

取值

6

约束含义

OPTV_SENSE_LESS

小于或等于。

类型

str

取值

<

OPTV_SENSE_EQUAL

等于。

类型

str

取值

=

OPTV_SENSE_GREATER

大于或等于。

类型

str

取值

>

CP Infinity

OPTV_CP_INF

Infinity value for CP solver.

类型
int

取值
int64_t max

回调

class OPTVCallback

天筹求解器的回调对象。

该类为抽象基类，用户可以从中派生出自己的回调子类。在求解器的求解过程中，*OPTVCallback.RunCallback()* 函数将会被定期调用。根据回调类型的不同，可以在求解过程中查询不同的数据。

用户可以通过 OPTVModel.SetCallback() 函数将回调提供给指定的模型。可以设置多个回调。

下面的示例中，向模型提供了两个自定义的回调。一个回调用于查询解的信息，另一个则用于查询 MIP 的节点信息。

```
class SDKMipSolutionCallback(OPTVCallback):
    def __init__(self, callback, x):
        super().__init__(callback)
        self.varToWatch = x

    def RunCallback(self):
        print("Callback 1: ")
        print("Time: ", self.GetInfo(OPTVCBDblInfo.RUN_TIME), end=", ")
        print("dual bound: " , self.GetInfo(OPTVCBDblInfo.MIP_DUAL_BOUND), end=", ")
        print("new incumbent: ", self.GetInfo(OPTVCBDblInfo.MIP_INCUMBENT_OBJ), end=",
↪ ")
        print("new incumbent sol @ i: ", self.GetSolution(self.varToWatch), end= ", ")
        print("type: ", self.GetType())

class SDKMipNodeCallback(OPTVCallback):
    def __init__(self, callback, x):
            super().__init__(callback)
            self.varToWatch = x

    def RunCallback(self):
        print("Callback 2: ")
        print("n solution: ", self.GetInfo(OPTVCBIntInfo::MIP_NUM_SOLUTION), end=", ")
        print("Time: ", self.GetInfo(OPTVCBDblInfo::RUN_TIME), end=", ")
        print(self.SetSolution(self.varToWatch, 0))
        print("type: ", self.GetType())

        self.Abort()

if __name__ == "__main__":

    env = OPTVEnv()
    model = OPTVModel(env, "cb_demo.mps")
```

(下页继续)

(续上页)

```
    model.Update()
    x1 = model.GetVarByName("x1")

    sdkMipSolutionCallback = SDKMipSolutionCallback(OPTVCBType.MIP_SOLUTION, x1)
    sdkMipNodeCallback = SDKMipNodeCallback(OPTVCBType.MIP_NODE, x1)

    model.SetCallback(sdkMipSolutionCallback)
    model.SetCallback(sdkMipNodeCallback)

    model.Optimize()
```

Abort()

中断求解进程。

OPTVCallback.**Abort**()

返回
无。

返回类型
None

GetInfo()

OPTVCallback.**GetInfo**(*info*)

参数
info (OPTVCBDblInfo/OPTVCBIntInfo/OPTVCBLongInfo/OPTVCBStrInfo) – 需要查询的回调信息。

返回
查询到的数据。

返回类型
floatlintlstr

引发

- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –当前的回调类型不支持查询该类型数据的信息。
- ***OPTVErrCode.INVALID_ARGUMENT*** –查询未知的回调数据类型。

GetNodeRel()

OPTVCallback.**GetNodeRel**(*var*)

仅在回调类型为*OPTVCBType.MIP_NODE*时可以调用。

参数
var (OPTVVar) –期望获取松弛解的变量。

返回
变量 var 在当前节点松弛解的值。

返回类型
float

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –变量 var 不在当前模型中。如果变量 var 被删除，或者数据未刷新，可能会抛出该错误。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –无法获取变量的松弛解, 或者回调类型不是*OPTVCBType.MIP_NODE*。

OPTVCallback.**GetNodeRel**()

仅在回调类型为*OPTVCBType.MIP_NODE*时可以调用。

返回
包含当前节点所有变量松弛解的列表。

返回类型
list[float]

引发
OPTVErrCode.DATA_NOT_AVAILABLE –无法获取变量的松弛解, 或者回调类型不是*OPTVCBType.MIP_NODE*。

GetSolution()

OPTVCallback.**GetSolution**(*var*)

仅当回调类型为*OPTVCBType::MIP_SOLUTION*时可调用。

参数
var (OPTVVar) –用于获取在当前候选解中的值的变量。

返回
变量 var 在当前候选解中的值。

返回类型
float

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –变量 var 不在当前模型中。如果变量 var 被删除，或者数据未刷新，可能会抛出该错误。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –无法获取变量的值, 或者回调类型不是*OPTVCBType.MIP_SOLUTION*。

OPTVCallback.**GetSolution**()

仅当回调类型为*OPTVCBType::MIP_SOLUTION* 时可调用。

返回
包含当前阶段候选解的列表。

返回类型
list[float]

引发
OPTVErrCode.DATA_NOT_AVAILABLE – 无法获取变量的值，或者回调类型不是*OPTVCBType.MIP_SOLUTION*。

GetType()

返回回调类型。详情请见*OPTVCBType*.

OPTVCallback.**GetType**()

返回
回调类型。

返回类型
OPTVCBType

RunCallback()

这是一个虚函数，在创建用户自定义的回调类时需要重写。该函数将在回调触发时执行，以便用户获取回调信息并执行其他自定义逻辑。

OPTVCallback.**RunCallback**()

返回
无。

返回类型
None

SetSolution()

OPTVCallback.**SetSolution**(*var*, *value*)

仅在回调类型为*OPTVCBType.MIP_NODE* 时可以调用。

参数

- **var** (OPTVVar) –希望设置其值的变量。
- **value** –需要设置的值。

返回
无。

返回类型
None

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –变量 var 不在当前模型中。如果变量 var 被删除，或者数据未刷新，可能会抛出该错误。
- ***OPTVErrCode.DATA_NOT_AVAILABLE*** –无法设置变量的值，或者回调类型不是*OPTVCBType.MIP_NODE*。

OPTVCallback.**SetSolution**(*values*)

仅在回调类型为*OPTVCBType.MIP_NODE* 时可以调用。

参数
values(*list[float]*)–包含需要设置的可行解值的列表。

返回
无。

返回类型
None

引发
OPTVErrCode.DATA_NOT_AVAILABLE –无法设置变量的值，或者回调类型不是*OPTVCBType.MIP_NODE*。

UseSolution()

OPTVCallback.**UseSolution**()

通过*OPTVCallback.SetSolution()* 设置一个可行解后，用户可以在类型为*OPTVCBType::MIP_NODE* 的回调类中使用 UseSolution，这样求解器可以在此基础上，调用启发式算法生成一个新的解。

返回
新候选解目标值，如果给定的变量取值不可行或者未找到新的候选解为*OPTV_INF*。

返回类型
float

引发
OPTVErrCode.DATA_NOT_AVAILABLE –回调类型不是*OPTVCBType.MIP_NODE*。

回调信息

OPTVCBDblInfo

class OPTVCBDblInfo

表 20: 双精度型回调信息

名称	描述
MIP_DUAL_BOUND	当前求解阶段的下界。
MIP_INCUMBENT_OBJ	新的可行解。
MIP_PRIMAL_BOUND	当前求解阶段的上界。
RUN_TIME	求解时间。
SIMPLEX_OBJ	单纯形目标值。

MIP_DUAL_BOUND

MIP_DUAL_BOUND
当前求解阶段的下界。可以在除了*OPTVCBType.MESSAGE*之外的其他类型的回调类中使用。

MIP_INCUMBENT_OBJ

MIP_INCUMBENT_OBJ
新可行解的目标函数值。可以在除了*OPTVCBType.MIP_SOLUTION*之外的其他类型的回调类中使用。

MIP_PRIMAL_BOUND

MIP_PRIMAL_BOUND
当前求解阶段的上界。可以在除了*OPTVCBType.MESSAGE*之外的其他类型的回调类中使用。

RUN_TIME

RUN_TIME
求解时间。可以在任何类型的回调类中使用。

SIMPLEX_OBJ

SIMPLEX_OBJ
单纯形目标值。仅在类型为*OPTVCBType.SIMPLEX*的回调中可用。

OPTVCBIntInfo

class OPTVCBIntInfo

表 21: 整型回调信息

名称	描述
MIP_NODE_STATUS	节点 Lp 模型的状态。
MIP_NUM_SOLUTION	发现的可行解的数量。
SIMPLEX_NUM_ITERS	单纯形迭代次数。

MIP_NODE_STATUS

MIP_NODE_STATUS
当前节点的 LP 模型的状态。只能在类型为*OPTVCBType.MIP_NODE*的回调类中使用。

MIP_NUM_SOLUTION

MIP_NUM_SOLUTION

发现的可行解数量。可以在除了*OPTVCBType.MESSAGE*之外其他类型的回调类中使用。

SIMPLEX_NUM_ITERS

SIMPLEX_NUM_ITERS

单纯形迭代次数。仅在类型为*OPTVCBType.SIMPLEX*的回调函数中可用。

OPTVCBLongInfo

class OPTVCBLongInfo

表 22: 长整型回调信息

名称	描述
MIP_NUM_NODE	搜索的节点数量。
MIP_NUM_OPENNODE	待处理的开放节点数量。

MIP_NUM_NODE

MIP_NUM_NODE

已探索的节点数量。可以在除了*OPTVCBType.MESSAGE*之外的其他类型的回调类中使用。

MIP_NUM_OPENNODE

MIP_NUM_OPENNODE

待处理的开放节点数量只能在类型为*OPTVCBType.MIP*的回调类中使用。

OPTVCBStrInfo

class OPTVCBStrInfo

表 23: 字符串回调信息

名称	描述
MESSAGE	下一条日志信息。

MESSAGE

MESSAGE

将要打印的下一条日志信息。只能在类型为*OPTVCBType.MESSAGE* 的回调类中使用。

回调类型

天筹求解器通过 OPTVCBType 来区分不同类型的回调功能。不同的回调类型在对应的上下文中被触发。回调类型在实例化回调类时指定，且之后不可更改。

class OPTVCBType

MESSAGE

在打印日志信息之前触发。

MIP

在 MIP 求解过程中的各个阶段触发。

MIP_NODE

在对节点进行 LP 求解后触发。

MIP_SOLUTION

在找到新候选解后触发。

SIMPLEX

在单纯形优化过程中触发。

通用约束辅助函数

天筹求解器的通用约束辅助函数。这些辅助函数旨在帮助用户在模型中快速构建通用约束。他们通常与重载运算符以及*OPTVModel.AddConstr()* 或*OPTVModel.AddConstrs()* 等方法结合使用，使用户能够高效的构建复杂的约束表达式。

And_()

用于将一个 0-1 变量设置为等于一组其他 0-1 变量的逻辑“与”结果。

And_(*vars_list*)

参数
vars_list(*list[OPTVVar]*) –所有参与检验的变量列表。

返回
一个通用表达式对象。

返回类型
OPTVGenExpr

引发
TypeError –变量列表中存在无效类型。

示例

```
c0 = model.AddConstr(x3 == And_([x1, x2]), "and")
```

Or_()

用于将一个 0-1 变量设置为等于一组其他 0-1 变量的逻辑“或”结果。

Or_(*vars_list*)

参数
vars_list (*list[OPTVVar]*) –所有参与检验的变量序列。

返回
一个通用表达式对象。

返回类型
OPTVGenExpr

引发
TypeError –变量列表中存在无效类型。

示例

```
c0 = model.AddConstr(x3 == Or_([x1, x2]), "or")
```

Abs_()

用于将变量的绝对值转化为另一个决策变量的值。

Abs_(*arg_var*)

参数
arg_var (OPTVVar) –添加到模型的一般约束对象。

返回
一个通用表达式对象。

返回类型
OPTVGenExpr

示例

```
c0 = model.AddConstr(x1 == Abs_(x2), "abs")
```

Norm_()

用于将变量向量的范数赋值给其他决策变量。

Norm_(*vars_list*, *which*)

参数

- **vars_list** (*list[OPTVVar]*) –用于计算范数的变量向量。
- **which** (*int*) –范数类型：-1 为 L^∞ 范数，0 为 L^0 范数，1 为 L^1 范数。

返回
一个通用表达式对象。

返回类型
OPTVGenExpr

引发
TypeError –变量列表中存在无效类型。

示例

```
c0 = model.AddConstr(x3 == Norm_([x1, x2]), "norm")
```

Min_()

用于通过一个新的决策变量表示最小值函数，该表达式中也可以包含一个常数项。

Min_(*vars_list*, *constant=OPTV_INF*)

参数

- **vars_list** (*list[OPTVVar]*) –变量序列，该向量的最小值会被结果变量采用。
- **constant** (*float*) –[**可选**] 参与 MIN 操作的常量。

返回
一个通用表达式对象。

返回类型
OPTVGenExpr

引发
TypeError –变量列表中存在无效类型。

示例

```
c0 = model.AddConstr(x3 == Min_([x1, x2]), "min")
```

Max_()

用于通过一个新的决策变量表示最大值函数，该表达式中也可以包含一个常数项。

Max_(*vars_list*, *constant=-OPTV_INF*)

参数

- **vars_list** (*list[OPTVVar]*) –向模型中添加单个 MAX 约束：$r = \max\{x_1, \ldots, x_n\}$ 。
- **constant** (*float*) –[**可选**] 参与 MAX 操作的常量。

返回
一个通用表达式对象。

返回类型
OPTVGenExpr

引发
TypeError –变量列表中存在无效类型。

示例

```
c0 = model.AddConstr(x3 == Max_([x1, x2]), "min")
```

全局函数

天筹求解器的全局函数。这些函数可以直接调用，无需预先创建实例对象。

OPTVMultiDict()

该函数会将单个字典拆分为多个字典。输入的字典中的每个键对应的值为一个包含 n 个元素的列表。该函数的返回值为一个列表：第一个元素是共享键的列表，随后为 n 个单独的字典，每个字典对应列表中的一个位置。

OPTVMultiDict(*data*)

参数
data (*dict*) –一个值为列表的字典。

返回
一个列表：第一个元素为共享键的列表，随后为多个字典。

返回类型
list

示例

```
keys, dict1, dict2 = OPTVMultiDict( {
    "key1": [1, 2],
    "key2": [1, 3],
    "key3": [1, 4] } )
```

OPTVQuickSum()

该函数在构建大型表达式（例如*OPTVLinExpr* 或者*OPTVQuadExpr*）时，比 Python 内置的 sum 函数更高效。该函数接受任意包含表达式项的可迭代对象。

尽管该函数比 Python 内置的 sum 函数快得多，但它并非构建大型表达式的最高效的方法。如需获得最佳性能，推荐使用*AddTerms*。

OPTVQuickSum(*data*)

参数
data (*Iterable*) –构建表达式元素的可迭代对象。构建表达式的元素可以是：常量、*OPTVVar* 对象、*OPTVLinExpr* 对象、*OPTVQuadExpr* 对象。

返回
一个表示输入参数各项之和的表达式。

返回类型
OPTVLinExpr | *OPTVQuadExpr*

示例

```
expr = OPTVQuickSum([2*x, 3*y+1, 4*z*z])
expr = OPTVQuickSum(model.GetVars())
```

8.3 枚举类

8.3.1 C++ 枚举类

参数枚举类

参数对应求解器内部的使用设置。参数既可以在创建模型之前通过*OPTVEnv* 对象设置，也可以在创建模型之后通过*OPTVModel* 对象设置。

若是需要查看所有参数及其用途，请看参数说明 节。

enum class **OPTVIntParam**

该枚举用于设置或者获取整数类型的参数。

enum class **OPTVDblParam**

该枚举用于设置或者获取双精度浮点数类型的参数。

enum class **OPTVStrParam**

该枚举用于设置或者获取字符串类型的参数。

属性枚举类

这些枚举类用来设置或者获取求解器的属性值。

可以通过*OPTVModel::Get* 或*OPTVModel::Set* 等方法查询或者设置指定属性的值。

若是需要查看所有属性及其用途，请看属性介绍 章节。

enum class **OPTVIntAttr**

该枚举用于获取或设置整型属性。

enum class **OPTVDblAttr**

该枚举用于获取或设置双精度浮点数类型属性。

enum class **OPTVStrAttr**

该枚举用于获取或设置字符串型属性。

enum class **OPTVCharAttr**

该枚举用于获取或设置字符型属性。

enum class **OPTVLongAttr**

该枚举用于获取或设置长整型属性。

enum class **OPTVBoolAttr**

该枚举用于获取或设置布尔型属性。

8.3.2 Java 枚举类

参数枚举类

参数对应求解器内部的使用设置。参数既可以在创建模型之前通过*OPTVEnv* 对象设置，也可以在创建模型之后通过*OPTVModel* 对象设置。

若是需要查看所有参数及其用途，请看参数说明 节。

class OPTVIntParam
　该类用于设置或者获取整数类型的参数。

class OPTVDblParam
　该类用于设置或者获取双精度浮点数类型的参数。

class OPTVStrParam
　该类用于设置或者获取字符串类型的参数。

属性枚举类

这些枚举类用来设置或者获取求解器的属性值。

可以通过*OPTVModel.get* 或*OPTVModel.set* 等方法查询或者设置指定属性的值。

若是需要查看所有属性及其用途，请看属性介绍 章节。

class OPTVIntAttr
　该类用于获取或设置整型属性。

class OPTVDblAttr
　该类用于获取或设置双精度浮点数类型属性。

class OPTVStrAttr
　该类用于获取或设置字符串型属性。

class OPTVCharAttr
　该类用于获取或设置字符型属性。

class OPTVLongAttr
　该类用于获取或设置长整型属性。

class OPTVBoolAttr
　该类用于获取或设置布尔型属性。

8.3.3 Python 枚举类

参数枚举类

参数对应求解器内部的使用设置。参数既可以在创建模型之前通过*OPTVEnv* 对象设置，也可以在创建模型之后通过*OPTVModel* 对象设置。

若是需要查看所有参数及其用途，请看参数说明 节。

class OPTVIntParam
　该类用于设置或者获取整数类型的参数。

class OPTVDblParam
该类用于设置或者获取双精度浮点数类型的参数。

class OPTVStrParam
该类用于设置或者获取字符串类型的参数。

属性枚举类

这些枚举类用来设置或者获取求解器的属性值。

可以通过*OPTVModel.Get* 或*OPTVModel.Set* 等方法来查询或者设置指定属性的值。

若是需要查看所有属性及其用途，请看*属性介绍* 章节。

class OPTVIntAttr
该类用于获取或设置整型属性。

class OPTVDblAttr
该类用于获取或设置双精度浮点数类型属性。

class OPTVStrAttr
该类用于获取或设置字符串型属性。

class OPTVCharAttr
该类用于获取或设置字符型属性。

class OPTVLongAttr
该类用于获取或设置长整型属性。

class OPTVBoolAttr
该类用于获取或设置布尔型属性。

8.4 约束规划求解器 API 参考手册

8.4.1 C++ API 参考手册

模型

class **OPTVCPModel**

天筹约束规划求解器的模型类。该类为构建约束规划问题提供了一个基础框架。

通过使用模型类，用户可以：

- 定义决策变量
- 添加约束
- 指定优化目标。

构建完模型后，可以使用*OPTVCPSolver* 求解模型，从而在复杂的约束条件下获得最优决策。

NewIntVar()

向模型中添加一个整数变量。

OPTVCPModel::**NewIntVar**(int64_t lb, int64_t ub, const std::string &name)

该函数会向模型中添加一个整数变量。

参数

- **lb** –变量的下界值。
- **ub** –变量的上界值。
- **name** –变量名称。

返回

添加到模型中的变量对象。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –参数 name 为空，或者存在为 NaN 的数值。

NewIntVarFromDomain()

向模型中添加一个整数型变量，并指定其取值区间。

我们使用向量表示该取值区间，向量中每两个连续的元素表示一个闭区间 $[a, b]$ （其中 $a \leq b$ ）。例如，domain $= \{1, 2, 5, 9\}$，该向量表示两个区间：domain $= [1, 2] \cup [5, 9]$。因此，变量的取值可能为：$\{1, 2, 5, 6, 7, 8, 9\}$。

注解

表示取值区间的向量必须满足以下条件：

- 元素个数必须为偶数（每一对元素定义一个区间）。
- 区间必须严格不重叠且不相邻（端点不能相同或连续）。

OPTVCPModel::**NewIntVarFromDomain**(const std::vector<int64_t> &domain, const std::string &name)

该函数会向模型中添加一个整数型变量，并指定其取值区间。

参数

- **domain** –变量的有效取值区间。
- **name** –变量名称。

返回

添加到模型中的变量对象。

抛出

OPTVErrCode::*INVALID_ARGUMENT* –参数 domain 为空，或参数 name 为空，或参数 domain 中存在空值。

NewBoolVar()

向模型中添加一个布尔变量。布尔变量的值只能为 0 或 1。

OPTVCPModel::**NewBoolVar**(const std::string &name)

该函数会向模型中添加一个布尔变量。

参数

name –变量名称。

返回

添加到模型中的变量对象。

NewConstant()

向模型中添加一个表示常数值的变量。

OPTVCPModel::**NewConstant**(int64_t value)

该函数会向模型中添加一个表示常数值的变量。

参数

value –变量的值。

返回

添加到模型中的变量对象。

AddLinearConstraint()

向模型中添加一条线性约束。

OPTVCPModel::**AddLinearConstraint**(const *OPTVCPLinExpr* &expr, int64_t lhs, int64_t rhs)

该函数会向模型中添加一条线性约束。

参数

- **expr** –约束的线性表达式部分。
- **lhs** –约束的下界。
- **rhs** –约束的上界。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –lhs 与 rhs 的值不合法。

OPTVCPModel::**AddLinearConstraint**(const *OPTVCPTempConstr* &constr)

该方法接收一个*OPTVCPTempConstr* 对象作为参数。推荐在表达式对象之间或者表达式与数值之间使用比较运算符（如 <=、>=、==）来构建*OPTVCPTempConstr* 对象。

参数

constr –*OPTVCPTempConstr* 对象。

返回

添加到模型中的约束对象。

警告

一条约束只能包含一个比较运算符。尽管 `1 <= x + y <= 2` 看起来像是一条有效的约束，但 `AddLinearConstraint` 目前并不接受这种写法。

AddLinearConstraintInDomain()

向模型添加一个约束，确保给定线性表达式的结果必须属于指定的区间。

我们使用向量表示线性表达式的取值区间，向量中每两个连续的元素表示一个闭区间 $[a, b]$ （其中 $a \leq b$ ）。例如，domain $= \{1, 2, 5, 9\}$, 该向量表示两个区间：domain $= [1, 2] \cup [5, 9]$ 。因此，线性表达式的取值可能为：$\{1, 2, 5, 6, 7, 8, 9\}$ 。

注解

表示取值区间的向量必须满足以下条件：

- 元素个数必须为偶数（每一对元素定义一个区间）。
- 区间必须严格不重叠且不相邻（端点不能相同或连续）。

OPTVCPModel`::`**`AddLinearConstraintInDomain`**(const *OPTVCPLinExpr* &expr, const std::vector<int64_t> &domain)

该函数将向模型添加一个线性约束，以确保表达式的结果被限制在指定的区间内。

参数

- **`expr`** –约束的线性表达式部分。
- **`domain`** –线性表达式的有效取值范围。

返回

添加到模型中的约束对象。

抛出

- *OPTVErrCode*`::`*INVALID_MODEL_INPUT* –提供的向量的元素个数为奇数。
- *OPTVErrCode*`::`*INVALID_ARGUMENT* –提供的向量存在空值。

AddBoolOr()

向模型中添加一个 OR 约束。

OR 约束的形式如下：

$$x_1 \vee x_2 \vee \cdots \vee x_n = 1$$

确保布尔变量 $x_1, x_2, \ldots, x_n$ 中至少有一个等于 1。

OPTVCPModel`::`**`AddBoolOr`**(const std::vector<*OPTVCPVar*> &literals)

该函数会向模型中添加一个 OR 约束。

参数

`literals` –一个布尔变量的向量。

返回
添加到模型中的约束对象。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* – 提供的某些变量不是布尔变量。

AddBoolAnd()

向模型中添加一条 AND 约束。

AND 约束的形式如下：

$$x_1 \wedge x_2 \wedge \cdots \wedge x_n = 1$$

确保所有的布尔变量 $x_1, x_2, \ldots, x_n$ 都等于 1。

OPTVCPModel::**AddBoolAnd**(const std::vector<*OPTVCPVar*> &literals)
该函数会向模型中添加一条 AND 约束。

参数
literals – 一个布尔变量的向量。

返回
添加到模型中的约束对象。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* – 提供的某些变量不是布尔变量。

AddBoolXor()

向模型中添加一条布尔 XOR 约束。

XOR 约束的形式如下：

$$x_1 \oplus x_2 \oplus \cdots \oplus x_n = 1$$

确保布尔变量 $x_1, x_2, \ldots, x_n$ 中有奇数个变量的值为 1 。

OPTVCPModel::**AddBoolXor**(const std::vector<*OPTVCPVar*> &literals)
该函数会向模型中添加一条布尔 XOR 约束。

参数
literals – 一个布尔变量的向量。

返回
添加到模型中的约束对象。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* – 提供的某些变量不是布尔变量。

AddAtLeastOne()

该函数等价于*OPTVCPModel::AddBoolOr()*。

OPTVCPModel::**AddAtLeastOne**(const std::vector<*OPTVCPVar*> &literals)

该函数会向模型中添加一条约束，确保至少有一个变量为非零值。

参数

literals –一个布尔变量的向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些变量不是布尔变量。

AddAtMostOne()

向模型添加一条线性约束，确保提供的变量中至多有一个变量被赋值为 1。

OPTVCPModel::**AddAtMostOne**(const std::vector<*OPTVCPVar*> &literals)

该函数会向模型添加一条约束，确保提供的变量中至多有一个变量取 1。

参数

literals –一个布尔变量的向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些变量不是布尔变量。

AddExactlyOne()

向模型添加一条约束，确保在给定的布尔变量中，恰好有一个变量被赋值为 1，其余所有变量均被赋值为 0。

OPTVCPModel::**AddExactlyOne**(const std::vector<*OPTVCPVar*> &literals)

该函数会向模型添加一条约束，确保在给定的布尔变量中，恰好有一个变量被赋值为 `1`，其余所有变量均被赋值为 `0`。

参数

literals –一个布尔变量的向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些变量不是布尔变量。

AddImplication()

该约束表示一种条件关系 $x \to y$。这意味着如果变量 x 为真，那么变量 y 必须为真。但当变量 x 为假时，变量 y 可以为真，也可以为假。

OPTVCPModel::**AddImplication**(const std::vector<*OPTVCPVar*> &literals)

该函数向模型添加一条蕴含关系约束。

参数

literals –一个布尔变量的向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些变量不是二进制变量，或者参数长度不等于 2。

AddAllowedAssignments()

向模型中添加一条表约束。输入包含表达式（*exprs*）以及值的元组（*tuples*）。每个元组的大小应与提供的表达式数量相等。表约束强制所有变量的值必须赋值为这些值元组中的一个。

以表约束 `AddAllowedAssignments({x1, x2}, {{0, 0}, {1, 0}, {1, 1}})` 为例，约束中的变量 *x1* 与 *x2* 为布尔变量，变量有效取值如下所示：

x1	x2
0	0
1	0
1	1

OPTVCPModel::**AddAllowedAssignments**(const std::vector<*OPTVCPLinExpr*> &exprs, const std::vector<std::vector<int64_t» &tuples)

该函数会向模型中添加一条表约束。

参数

- **exprs** –约束表达式的向量。
- **tuples** –可能值的元组向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –表达式数量为零，或其中一个表达式非仿射，或某一元组的大小与各表达式的大小不一致。

AddForbiddenAssignments()

向模型中添加一条负向表约束。该方法的输入包括表达式（$exprs$）以及值的元组（$tuples$）。每个元组的大小应与提供的表达式个数相等。负向表约束强制要求所有变量的赋值不能与任意一个元组完全匹配。

以负向表约束 `AddForbiddenAssignments({x1, x2}, {{0, 1}})` 为例，其中 *x1* 和 *x2* 为布尔变量，其取值可表示如下：

x1	x2
0	1

OPTVCPModel::**AddForbiddenAssignments**(const std::vector<*OPTVCPLinExpr*> &exprs, const std::vector<std::vector<int64_t» &tuples)

该函数会向模型中添加一条负向表约束。

参数

- **exprs** –约束表达式的向量。
- **tuples** –不可能的元组值向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –表达式数量为零，或其中一个表达式非仿射，或某一元组的大小与各表达式的大小不一致。

AddInterval()

向模型添加单个区间约束。区间约束强制满足 $start + size = end$。区间约束是其他约束的基础，例如无重叠、二维无重叠 或累积。

OPTVCPModel::**AddInterval**(const *OPTVCPLinExpr* &start, const *OPTVCPLinExpr* &size, const *OPTVCPLinExpr* &end)

该函数会向模型中添加一条区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **start** –表示区间起始位置的仿射表达式。
- **size** –表示区间长度的仿射表达式。
- **end** –表示区间结束位置的仿射表达式。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddFixedSizeInterval()

向模型中添加单条固定长度区间约束。固定长度的区间约束本身并无实际意义，但可作为其他约束（例如*No Overlap*、*No Overlap 2D* 或*Cumulative*）的基础。当区间的长度已知时，该函数可以替代*OPTVCPModel::AddInterval()* 函数。

OPTVCPModel::**AddFixedSizeInterval**(const *OPTVCPLinExpr* &start, int64_t size)

该函数会向模型中添加一条可激活的固定长度区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **start** –表示区间起始位置的仿射表达式。
- **size** –表示区间长度的常量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的表达式中包含多个变量。

AddOptionalInterval()

向模型添加一个可选区间约束。可选区间约束强制执行：$isPresent \rightarrow start + size = end$。区间约束是其他约束（如*No Overlap*、*No Overlap 2D* 或*Cumulative*）的基础。当需要强制字面量（布尔变量或其否定）时，此函数是*OPTVCPModel::AddInterval()* 函数的替代方案。仅当强制字面量值为真时，该约束才会生效。

OPTVCPModel::**AddOptionalInterval**(const *OPTVCPLinExpr* &start, const *OPTVCPLinExpr* &size, const *OPTVCPLinExpr* &end, const *OPTVCPVar* &isPresent)

该函数会向模型中添加一条区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **start** –表示区间起始位置的仿射表达式。
- **size** –表示区间长度的仿射表达式。
- **end** –表示区间结束位置的仿射表达式。
- **isPresent** –控制变量（布尔变量或其取反）。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddOptionalFixedSizeInterval()

向模型添加一个可选的固定大小区间约束。固定大小区间约束本身没有意义，但它们可以作为其他约束的基础，例如*No Overlap*、*No Overlap 2D*或*Cumulative*。此函数是*OPTVCPModel::AddInterval()*函数的替代方案，适用于区间大小已知且需要使用强制字面量（布尔变量或其否定形式）的情况。仅当强制字面量的值为真时，该约束才会生效。

OPTVCPModel::**AddOptionalFixedSizeInterval**(const *OPTVCPLinExpr* &start, int64_t size, const *OPTVCPVar* &isPresent)

该函数会向模型中添加一条可激活的固定长度区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **start** –表示区间起始位置的仿射表达式。
- **size** –表示区间长度的常量。
- **isPresent** –控制变量（布尔变量或其取反）。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的表达式中包含多个变量。

AddIntervalSeries()

向模型添加一系列区间约束。区间约束强制 $start+size=end$。区间约束是其他约束的基础，如*No Overlap*、*No Overlap 2D*或*Cumulative*。

OPTVCPModel::**AddIntervalSeries**(const std::vector<*OPTVCPLinExpr*> &starts, const std::vector<*OPTVCPLinExpr*> &sizes, const std::vector<*OPTVCPLinExpr*> &ends)

该函数会向模型中添加多个区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **starts** –表示区间起始位置的仿射表达式向量。
- **sizes** –表示区间大小的仿射表达式向量。
- **ends** –表示区间结束位置的仿射表达式向量。

返回

添加到模型中的约束对象向量。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些表达式包含多个变量，或提供的向量长度不同。

AddFixedSizeIntervalSeries()

向模型中添加一系列固定长度的区间约束。固定长度的区间约束本身没有实际意义，但可作为其他约束（例如*No Overlap*、*No Overlap 2D* 或*Cumulative*）的基础。当区间的长度已知时，该函数可替代*OPTVCPModel::AddIntervalSeries()*。

OPTVCPModel::**AddFixedSizeIntervalSeries**(const std::vector<*OPTVCPLinExpr*> &starts, const std::vector<int64_t> &sizes)

该函数会向模型中添加多条固定长度的区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **starts** –表示区间起始位置的仿射表达式向量。
- **sizes** –表示区间长度的常量向量。

返回

添加到模型中的约束对象向量。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些表达式包含多个变量，或提供的向量长度不同。

AddOptionalIntervalSeries()

区间约束是其他约束的基础，例如*无重叠* 、*二维无重叠* 或*累积* 。当需要使用强制字面量（布尔变量或其否定形式）时，此函数是*OPTVCPModel::AddIntervalSeries()* 函数的一种替代方案。这些约束仅在对应的强制字面量评估为真时才会生效。

OPTVCPModel::**AddOptionalIntervalSeries**(const std::vector<*OPTVCPLinExpr*> &starts, const std::vector<*OPTVCPLinExpr*> &sizes, const std::vector<*OPTVCPLinExpr*> &ends, const std::vector<*OPTVCPVar*> &arePresent)

该函数会向模型中添加多个区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **starts** –表示区间起始位置的仿射表达式向量。
- **sizes** –表示区间大小的仿射表达式向量。
- **ends** –表示区间结束位置的仿射表达式向量。
- **arePresent** –控制变量（布尔变量或其取反）向量。

返回

添加到模型中的约束对象向量。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些表达式包含多个变量，或提供的向量长度不同。

AddOptionalFixedSizeIntervalSeries()

向模型添加一系列可选的固定大小区间约束。固定大小区间约束本身没有意义，但可以作为其他约束的基础，例如无重叠 、二维无重叠 或累积约束。此函数是*OPTVCPModel::AddIntervalSeries()* 函数的替代方案，适用于区间大小已知且需要强制字面量（布尔变量或其否定形式）的场景。约束仅在对应的强制字面量为真时才生效。

OPTVCPModel::**AddOptionalFixedSizeIntervalSeries**(const std::vector<*OPTVCPLinExpr*> &starts, const std::vector<int64_t> &sizes, const std::vector<*OPTVCPVar*> &arePresent)

该函数会向模型中添加多条固定长度的区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **starts** –表示区间起始位置的仿射表达式向量。
- **sizes** –表示区间长度的常量向量。
- **arePresent** –控制变量（布尔变量或其取反）向量。

返回

添加到模型中的约束对象向量。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些表达式包含多个变量，或提供的向量长度不同。

AddNoOverlap()

向模型中添加一条无重叠约束。无重叠约束会保证该约束中包含的区间约束的起止范围不会重叠。

OPTVCPModel::**AddNoOverlap**(const std::vector<*OPTVCPConstr*> &intervals)

该函数会向模型中添加一条无重叠约束。

参数

intervals –区间约束向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些约束不是*interval* 。

AddNoOverlap2D()

向模型添加一个单一的二维无重叠约束。无重叠二维约束强制该约束中包含的所有矩形互不重叠。

OPTVCPModel::**AddNoOverlap2D**(const std::vector<*OPTVCPConstr*> &xIntervals, const std::vector<*OPTVCPConstr*> &yIntervals)

向模型中添加一个二维无重叠约束。

参数

- **xIntervals** –X 轴上的 `intervals` 向量。
- **yIntervals** –Y 轴上的 `intervals` 向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的一些约束并非模型中的*interval*约束，或者提供的向量大小不同。

AddCumulative()

向模型中添加一条累积约束。累积约束要求：在任意时间点上，所有重叠的区间的需求量之和不得超过其容量。

OPTVCPModel::**AddCumulative**(const std::vector<*OPTVCPConstr*> &intervals, const std::vector<*OPTVCPLinExpr*> &demands, *OPTVCPLinExpr* capacity)

该函数会向模型中添加一条累积约束。

参数

- **intervals** –约束向量。
- **demands** –线性表达式向量。
- **capacity** –约束的容量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些约束类型不是*interval*，或提供的向量大小不同，或提供的表达式中包含多个变量。

AddAllDifferent()

向模型中添加一条互异约束。互异约束保证给定的所有表达式会被赋予不同的值。

OPTVCPModel::**AddAllDifferent**(const std::vector<*OPTVCPLinExpr*> &expressions)

该函数会向模型中添加一条互异约束。

参数

expressions –仿射表达式向量，其中每个表达式最多只能包含一个变量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

OPTVCPModel::**AddAllDifferent**(const std::vector<*OPTVCPVar*> &vars)

该函数会向模型中添加一条互异约束。

参数

vars –一个变量向量。

返回

添加到模型中的约束对象。

AddElement()

向模型中添加一条元素约束。元素约束强制满足以下条件：target = $\text{expressions}_{\text{index}}$。

OPTVCPModel::**AddElement**(const *OPTVCPLinExpr* &index, const std::vector<*OPTVCPLinExpr*> &expressions, const *OPTVCPLinExpr* &target)

该函数将向模型添加一个单元素约束。

参数

- **index** –表达式的索引，其值等于 “target“。
- **expressions** –线性表达式向量。
- **target** –与 `expressions[index]` 相等的表达式。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddCircuit()

向模型中添加一条回路约束。回路约束强制要求输入图中包含一个哈密顿回路。

OPTVCPModel::**AddCircuit**(const std::vector<std::tuple<int, int, *OPTVCPVar*» &arcs)

该函数会向模型中添加一条回路约束。

参数

arcs –图的弧向量。一条弧由尾节点、头节点和布尔文字组成的三元组表示。若文字为真，则该弧在回路中被选中。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些变量不是布尔变量。

AddMultipleCircuit()

向模型中添加一条多回路约束。一个多回路约束应满足以下性质：

- 除节点 0 外，每个节点必须有且仅有一条出边；
- 除节点 0 外，每个节点必须有且仅有一条入边；
- 节点 0 的出边数量必须等于入边数量；
- 没有重复的弧；
- 节点 0 不允许自环；
- 图中所有回路都须包含节点 0。

OPTVCPModel::**AddMultipleCircuit**(const std::vector<std::tuple<int, int, *OPTVCPVar*» &arcs)

该函数将向模型添加单个多回路约束。

参数

arcs –图的弧向量。一条弧由尾节点、头节点和布尔文字组成的三元组表示。若文字为真，则该弧在回路中被选中。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –提供的某些变量不是布尔变量。

AddMaxEquality()

向模型中添加一条最大等值约束。

最大等值约束强制满足以下条件：target = max(expressions)。

OPTVCPModel::**AddMaxEquality**(const *OPTVCPLinExpr* &target, const std::vector<*OPTVCPLinExpr*> &expressions)

该函数会向模型中添加一条最大等值约束。

参数

- **target** –线性表达式，该表达式的值为向量 *expressions* 中的最大值。
- **expressions** –线性表达式向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddMinEquality()

向模型中添加一条最小等值约束。最小等值约束强制满足以下条件：target = min(expressions)。

OPTVCPModel::**AddMinEquality**(const *OPTVCPLinExpr* &target, const std::vector<*OPTVCPLinExpr*> &expressions)

该函数会向模型中添加一条最小等值约束。

参数

- **target** –线性表达式，其值为向量 *expressions* 中的最小值。
- **expressions** –线性表达式向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddAbsEquality()

向模型中添加一个绝对等式约束。绝对等式约束强制满足以下条件：target = |expressions|。

OPTVCPModel::**AddAbsEquality**(const *OPTVCPLinExpr* &target, const *OPTVCPLinExpr* &expr)

此函数将向模型中添加一个绝对等式约束。

参数

- **target** –等于 `expr` 绝对值的表达式。
- **expr** –表达式，其绝对值等于 `target`。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddMultiplicationEquality()

向模型中添加一条乘法等式约束。乘法等式约束强制满足以下条件：target = $\prod_{i=0}^{n}$ expressions$_i$。

OPTVCPModel::**AddMultiplicationEquality**(const *OPTVCPLinExpr* &target, const std::vector<*OPTVCPLinExpr*> &expressions)

该函数会向模型中添加一条乘法等式约束。

参数

- **target** –与所有 *expressions* 的乘积相等的表达式。
- **expressions** –线性表达式向量。

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddDivisionEquality()

向模型中添加一条除法等值约束。除法等值约束强制满足以下条件：`target = num / denom`。

OPTVCPModel::**AddDivisionEquality**(const *OPTVCPLinExpr* &target, const *OPTVCPLinExpr* &num, const *OPTVCPLinExpr* &denom)

该函数会向模型中添加一条除法等值约束。

参数

- **target** –表达式，其值为 *num* 除以 *denom* 的结果。
- **num** –分子表达式
- **denom** –分母表达式

返回

添加到模型中的约束对象。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddModuloEquality()

向模型添加一个模等值约束。模等值约束强制满足：`target = expr % mod`。

OPTVCPModel::**AddModuloEquality**(const *OPTVCPLinExpr* &target, const *OPTVCPLinExpr* &expr, const *OPTVCPLinExpr* &mod)

此函数将为模型添加一个模等值约束条件。

参数

- **target** –表达式为 *expr* 除以 *mod* 后的余数。
- **expr** –被除数表达式
- **mod** –除数表达式

返回
添加到模型中的约束对象。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* –某些表达式包含多个变量。

AddInverse()

向模型中添加一个单一互互逆约束。逆约束确保：`variables[i] == j 且 inverseVariables[j] == i`。

OPTVCPModel::**AddInverse**(const std::vector<*OPTVCPVar*> &variables, const std::vector<*OPTVCPVar*> &inverseVariables)

该函数将向模型添加一个单一的互逆约束。

参数

- **variables** –变量数组
- **inverseVariables** –逆变量数组

返回
添加到模型中的约束对象。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* –部分提供的数组为空，或数组大小不匹配。

AddDecisionStrategy()

向模型中添加一种搜索策略。

OPTVCPModel::**AddDecisionStrategy**(const std::vector<*OPTVCPVar*> &vars, *OPTVCPVariableSelectionStrategy* varStrategy, *OPTVCPDomainReductionStrategy* domainStrategy)

该函数会向模型中添加一种搜索策略。

参数

- **vars** –搜索策略作用的变量向量。
- **varStrategy** –用于选择下一个变量的启发式策略。
- **domainStrategy** –用于缩减所选变量定义域的启发式策略。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –提供的某些变量索引为负数或不在模型中。

Maximize()

将模型的目标设置为最大化一个线性表达式的值。

注解

不设置目标函数或设置一个常数目标函数将导致一个可行性问题。

当未指定目标函数时，目标值被视为 0。如果使用常数作为目标函数，则目标值将等于该常数。

OPTVCPModel::**Maximize**(const *OPTVCPLinExpr* &expr)

此函数将模型的目标方向设置为最大化，并指定目标函数。

参数
expr –需要设定的目标函数表达式。

返回
无。

Minimize()

将模型的目标设置为最小化一个线性表达式的值。

注解

不设置目标函数或设置一个常数目标函数将导致一个可行性问题。

当未指定目标函数时，目标值被视为 0。如果使用常数作为目标函数，则目标值将等于该常数。

OPTVCPModel::**Minimize**(const *OPTVCPLinExpr* &expr)

此函数将模型的目标方向设置为最小化，并指定目标函数。

参数
expr –需要设定的目标函数表达式。

返回
无。

HasObjective()

检查模型是否设置了目标函数。

OPTVCPModel::**HasObjective**()

返回
模型是否设置目标函数的标志。

返回值

- **true** –模型设置了目标函数。

- **false** –模型没有设置目标函数。

ClearObjective()

清除模型中设置的目标函数。

OPTVCPModel::**ClearObjective**()

返回
无。

IsMaximize()

检查模型是否最大化目标函数值。

OPTVCPModel::**IsMaximize**()

返回
模型是否最大化目标函数值的标志。

返回值

- **true** –模型最大化目标函数值。
- **false** –模型会最小化目标函数值或者没有设置目标函数。

GetVar()

获取指定的变量。

OPTVCPModel::**GetVar**(int i) const

参数
i –变量在模型中的索引。

返回
在模型中索引 i 的变量。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –索引值为负数或超过了模型中的变量数量。

GetVars()

获取模型中的所有变量。

OPTVCPModel::**GetVars**() const

返回
包含模型中所有变量的向量。

GetConstr()

获取模型中指定位置的约束。

OPTVCPModel::**GetConstr**(int i) const

参数
i –线性约束在模型中的索引。

返回
索引为 i 的约束。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –索引值为负数或大于模型中约束的数量。

GetConstrs()

获取模型中的所有约束。

OPTVCPModel::**GetConstrs**() const

返回
包含模型中所有约束的向量。

GetVarDomain()

查询指定变量的取值区间。

取值区间使用向量表示，向量中每两个连续的元素表示一个闭区间 $[a, b]$ （其中 $a \leq b$ ），且该向量中的元素个数为偶数，其表示的区间不重叠且不相邻。

例如，domain $= \{1, 2, 5, 9\}$ ，该向量表示两个区间：domain $= [1, 2] \cup [5, 9]$ 。因此，变量的取值可能为：$\{1, 2, 5, 6, 7, 8, 9\}$ 。

OPTVCPModel::**GetVarDomain**(int i) const

参数
i –变量在模型中的索引。

返回
一个包含变量取值区间的向量。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –索引值为负数或超过了模型中的变量数量。

GetLinearConstraintVars()

查询指定约束中的所有变量。

OPTVCPModel::**GetLinearConstraintVars**(int i) const

参数
i –线性约束在模型中的索引。

返回
一个包含指定约束所有变量索引的向量。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* –查询的约束不是线性约束。

GetLinearConstraintCoeffs()

查询指定线性约束中所有变量的系数。

OPTVCPModel::**GetLinearConstraintCoeffs**(int i) const

参数
i –线性约束在模型中的索引。

返回
一个包含指定约束中所有变量系数的向量。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* –查询的约束不是线性约束。

GetLinearConstraintDomain()

获取描述指定约束的表达式的取值区间。

取值区间使用向量表示，向量中每两个连续的元素表示一个闭区间 $[a, b]$ （其中 $a \leq b$ ），且该向量中的元素个数为偶数，其表示的区间不重叠且不相邻。

例如，domain $= \{1, 2, 5, 9\}$, 该向量表示两个区间：domain $= [1, 2] \cup [5, 9]$ 。因此，线性表达式的取值可能为：$\{1, 2, 5, 6, 7, 8, 9\}$ 。

OPTVCPModel::**GetLinearConstraintDomain**(int i) const

参数
i –线性约束在模型中的索引。

返回
一个表示线性表达式取值区间的向量。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* –查询的约束不是线性约束。

GetEnforcementLiterals()

若是只有某些变量取值为真时，另一些变量或者约束才会被 激活，这样的变量称为 控制变量。例如：

$$(a \wedge b \wedge c) \rightarrow \text{constraint} \tag{8.1}$$

$$x \rightarrow y \tag{8.2}$$

公式 1 表示只有变量 a, b, c 都为真时，约束 constraint 才会被激活，参考*OPTVCPConstr::OnlyEnforceIf()* 。

公式 2 表示变量 x 为真时，变量 y 必须为真，参考*OPTVCPModel::AddImplication()* 。

其中，变量 a, b, c, x 就是控制变量。

该函数会返回指定约束的控制变量的索引。

OPTVCPModel::**GetEnforcementLiterals**(int i) const

参数
i –线性约束在模型中的索引。

返回
一个元素为控制变量索引的向量。

GetConstraintLiterals()

获取指定布尔约束中所有变量的索引。

OPTVCPModel::**GetConstraintLiterals**(int i) const

参数
i –线性约束在模型中的索引。

返回
一个元素为变量索引的向量。

抛出
OPTVErrCode::*INVALID_MODEL_INPUT* –查询的约束不是布尔约束。

GetConstraintName()

获取指定约束的名称。

OPTVCPModel::**GetConstraintName**(int i) const

参数
i –线性约束在模型中的索引。

返回
索引为 i 的约束名称。

Set()

参数

为模型设置一个求解参数。创建模型后，该接口为模型提供了控制参数的能力。

OPTVCPModel::**Set**(*OPTVIntParam* param, int value)

参数

- **param** –整数型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVCPModel::**Set**(*OPTVDblParam* param, double value)

参数

- **param** –双精度型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

OPTVCPModel::**Set**(const std::string &paramName, const std::string &value)

参数

- **param** –字符串型参数。
- **value** –要设置的值。

返回
无。

抛出
OPTVErrCode::*UNKNOWN_PARAMETER* –传入了未知参数。

Read()

从文件中读取读取模型和基。

OPTVCPModel::**Read**(const std::string &fileName)

此函数用于读取模型数据，也可以读取模型文件，然而建议由构造函数读取文件中的模型，参考文件格式。

表 24: 支持读入的文件格式

.mps	.lp	.fzn

表 25: 支持读入的压缩格式

.gz	.z	.Z

参数
fileName –C++ 字符串文件名。

返回
无。

抛出
OPTVErrCode::*ERROR_FILE_READ* –文件名或后缀为空，文件存在格式错误。

Write()

将模型写出到文件。

OPTVCPModel::**Write**(const std::string &fileName)

支持的文件格式请参考文件格式。

表 26: 支持写出的文件格式

.fzn

表 27: 支持写出的压缩格式

.gz	.z	.Z

参数

fileName –C++ 字符串文件名。

返回

无。

抛出

OPTVErrCode::*ERROR_FILE_WRITE* –文件名或后缀为空或错误。

求解器类

class **OPTVCPSolver**

天筹约束规划求解器的求解器类。该类用于求解约束规划问题。

Solve()

求解给定的约束规划模型。

OPTVCPSolver::**Solve**(*OPTVCPModel* &model)

该函数会求解给定的约束规划模型并返回求解状态。

参数

model –约束规划模型。

返回

求解状态。

GetObjectiveValue()

返回当前求解模型的目标函数值。

OPTVCPSolver::**GetObjectiveValue**()

该函数会返回当前求解模型的目标函数值。

返回

当前求解模型的目标函数值。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –模型未求解。

GetBestObjectiveBound()

对于最小化问题，返回目标函数的最佳下界；对于最大化问题，返回目标函数的最佳上界。

OPTVCPSolver::**GetBestObjectiveBound**()

该函数返回当前求解模型的最佳目标界限。

返回

当前求解模型的最佳目标界限。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –模型未求解。

GetValue()

返回当前求解模型中指定对象的求解结果。

OPTVCPSolver::**GetValue**(const *OPTVCPVar* &var)

该函数会返回当前求解模型中指定变量的求解结果。

参数

var –需要查询求解结果的变量。

返回

变量的求解结果。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –模型未求解或变量不在当前模型中。

OPTVCPSolver::**GetValue**(const *OPTVCPLinExpr* &expr)

该函数会返回当前求解模型中指定表达式的求解结果。

参数

expr –需要查询求解结果的表达式。

返回

表达式的求解结果。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –模型未求解或表达式中的某些变量不在当前模型中。

GetValues()

批量查询当前求解模型中变量的求解结果。

OPTVCPSolver::**GetValues**(const std::vector<*OPTVCPVar*> &vars)

该函数会返回当前求解模型中多个变量的求解结果。

参数

vars –需要查询求解结果的变量向量。

返回

一个包含给定变量求解结果的向量。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –模型未求解或提供的变量不在当前求解的模型中。

GetBooleanValue()

获取当前求解模型中指定布尔变量的值。

OPTVCPSolver::**GetBooleanValue**(const *OPTVCPVar* &var)

该函数会返回当前求解模型中指定的布尔变量的值。

参数

var –需要查询求解结果的布尔变量。

返回

当前求解模型中该布尔变量的值。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –模型未求解，或变量不是布尔变量，或变量不在当前模型中。

GetBooleanValues()

批量获取当前求解模型中布尔变量的值。

OPTVCPSolver::**GetBooleanValues**(const std::vector<*OPTVCPVar*> &vars)

该函数会返回当前求解模型中多个布尔变量的值。

参数

vars –布尔变量向量。

返回

一个包含布尔变量求解结果的向量。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –模型未求解，或提供的某些变量不是布尔变量，或某些变量不在当前模型中。

GetSolution()

返回当前求解模型中所有变量的求解结果。

OPTVCPSolver::**GetSolution**()

该函数会返回当前求解模型中所有变量的值。

返回

一个包含当前求解模型中所有变量值的向量。

抛出

OPTVErrCode::*DATA_NOT_AVAILABLE* –模型未求解。

变量

class **OPTVCPVar**

天筹求解器的变量类。用于从模型中访问有关变量的数据。不推荐使用构造函数创建变量对象。用户可以使用*OPTVCPModel::NewIntVar()* 、*OPTVCPModel::NewIntVarFromDomain()* 或者*OPTVCPModel::NewBoolVar()* 等接口来创建变量。

Index()

获得变量在模型中的索引值。

OPTVCPVar::**Index**() const

返回

变量在模型中的索引。

返回值

- **-1** –变量不在模型中。
- **>=0** –变量在模型中的索引。

GetName()

返回变量名称。

OPTVCPVar::**GetName**() const

返回

变量名称。

抛出

OPTVErrCode::*NOT_IN_MODEL* –变量不在模型中。

GetDomain()

返回变量的取值区间。

OPTVCPVar::**GetDomain**() const

返回
变量的取值区间。

SameAs()

判断两个变量是否为同一个对象。

OPTVCPVar::**SameAs**(const *OPTVCPVar* &v2) const

参数
v2 –需要与当前变量对比的变量。

返回
变量是否相同。

返回值

- **true** –变量相同。
- **false** –变量不同。

IsBinary()

检查该变量是否为布尔变量。

OPTVCPVar::**IsBinary**() const

返回
变量是否为布尔变量。

抛出
OPTVErrCode::*NOT_IN_MODEL* –变量不在模型中。

约束

class **OPTVCPConstr**

天筹约束规划求解器的约束对象。用于访问模型中与约束有关的信息。注意约束不可由其构造函数创建，而需要通过*OPTVCPModel* 的相关接口向模型中添加约束。

OnlyEnforceIf()

为约束添加控制变量（布尔变量或其取反）。只有控制变量全部为真时，该约束才会生效。

OPTVCPConstr::**OnlyEnforceIf**(const std::vector<*OPTVCPVar*> &literals)

参数
literals –控制变量（布尔变量或其取反）向量。

抛出

- *OPTVErrCode*::*NOT_IN_MODEL* –约束不在任何模型中。
- *OPTVErrCode*::*INVALID_MODEL_INPUT* –提供的某些控制变量不是布尔变量。

WithName()

设置约束的名称。

OPTVCPConstr::**WithName**(const std::string &name)

参数
name –约束的名称。

抛出
OPTVErrCode::*NOT_IN_MODEL* –约束不在任何模型中。

Index()

获取线性约束在模型中的索引。

OPTVCPConstr::**Index**() const

返回
约束索引。

返回值

- **-1** –约束不在任何模型中。
- **>=0** –约束在模型中的索引值。

GetName()

获取约束名称。

OPTVCPConstr::**GetName**() const

返回
约束的名称。

抛出
OPTVErrCode::*NOT_IN_MODEL* –约束不在任何模型中。

GetType()

获取约束类型。

OPTVCPConstr::**GetType**() const

返回
约束类型。

抛出
OPTVErrCode::*NOT_IN_MODEL* –约束不在任何模型中。

SameAs()

查询两约束是否相同。

OPTVCPConstr::**SameAs**(const *OPTVCPConstr* &c2) const

参数
c2 –需要与当前约束对比的约束。

返回
约束是否相同。

返回值

- **true** –约束相同。
- **false** –约束不同。

线性表达式

class **OPTVCPLinExpr**

天筹约束规划求解器的线性表达式类。*OPTVCPLinExpr* 类用于构建线性表达式与目标函数。该类的对象由变量和常数项组成。线性表达式按其构造形式进行存储。例如，表达式 `1 * x + 2 * y + 3 * x` 的存储形式如下：

索引	系数	变量
0	1	x
1	2	y
2	3	x

AddTerms()

该功能向当前表达式的尾部添加一组新的项。

OPTVCPLinExpr::**AddTerms**(const std::vector<*OPTVCPVar*> &vars, const std::vector<double> &coeffs)

此函数将 $\text{coeffs}_i * \text{vars}_i, \forall i \in [0, \text{vars.size()}]$ 添加到当前线性表达式后。

参数

- **vars** –变量向量。
- **coeffs** –系数向量。

返回

无。

抛出

OPTVErrCode::*INVALID_MODEL_INPUT* –参数 vars 与 coeffs 的长度不一致。

Clear()

清空该表达式，*OPTVCPLinExpr::Size()* 将返回 0。同时，该表达式从数值意义上将等同于 0。

OPTVCPLinExpr::**Clear**()

返回

无。

GetCoef()

获取线性表达式中指定变量的系数。

OPTVCPLinExpr::**GetCoef**(int i) const

参数

i –变量在线性表达式中的索引。

返回

对应的变量系数。

抛出

OPTVErrCode::*INDEX_OUT_OF_RANGE* –若 i $\notin [0,$ this->Size()$]$。

GetVar()

返回线性表达式中指定位置上的变量。

OPTVCPLinExpr::**GetVar**(int i) const

参数

i –变量在线性表达式中的索引。

返回

指定位置上的变量。

抛出

OPTVErrCode::*INDEX_OUT_OF_RANGE* –若 i $\notin [0,$ this->Size()$]$。

GetConstant()

获得线性表达式中的常数项。

OPTVCPLinExpr::**GetConstant**() const

返回
表达式中的常量数值。

SameAs()

判断两个线性表达式是否为同一个对象。

OPTVCPLinExpr::**SameAs**(const *OPTVCPLinExpr* &e2) const

参数
e2 –需要与当前线性表达式对比的线性表达式。

返回
线性表达式是否相同。

返回值

- **true** –线性表达式相同。
- **false** –线性表达式不同。

Size()

获得该表达式的规模。

OPTVCPLinExpr::**Size**() const
该函数返回的数值对应线性表达式中的变量个数，不论变量是否重复出现。例如表达式 `x+y` 对应 `Size = 2`，而 `x+y+x` 对应 `Size = 3`。

返回
线性表达式大小。

Remove()

从表达式中删除项。

OPTVCPLinExpr::**Remove**(int i)
删除表达式中指定的项。

参数
i –指定的索引值。

返回
无。

抛出
OPTVErrCode::*INDEX_OUT_OF_RANGE* –若 `i` $\notin [0, \texttt{this->Size()}]$。

OPTVCPLinExpr::**Remove**(const *OPTVCPVar* &v)

从线性表达式中删除指定变量涉及的所有项。

参数

v –指定要删除的变量。

返回

是否成功删除任何项。

返回值

- **true** –成功删除任何项。
- **false** –未删除任何项。

抛出

OPTVErrCode::*NOT_IN_MODEL* –变量不在模型中。

临时约束

class **OPTVCPTempConstr**

天筹约束规划求解器临时约束对象。OPTVCPTempConstr 类在使用重载运算符构建的约束表达式中，用于临时存放中间结果。该类不包含任何成员函数。当对表达式或者变量（*OPTVCPLinExpr* 、*OPTVCPVar*）执行比较操作（==, !=, <=, >=）时，会返回临时约束对象。临时约束对象通常会被添加约束的方法立即使用，因此不建议用户将其保存到自定义的变量中。

可以使用临时约束对象构建线性约束：

- **线性约束**：线性约束的形式为 Expr1 sense Expr2，其中 Expr1 与 Expr2 可以是*OPTVCPVar* 对象、*OPTVCPLinExpr* 对象、常量，sense 是比较操作符：==、!=、<=、>= 的其中之一。例如 x + y <= 1 + z 与 x + y == 5 都是有效的线性约束。

以下示例说明了如何使用临时约束添加线性约束：

```
model.AddLinearConstraint(x + y >= 1)
```

变量选择策略

天筹约束规划求解器的变量选择策略。

enum class **OPTVCPVariableSelectionStrategy**

enumerator **CHOOSE_FIRST**

从变量列表中选择第一个变量。

enumerator **CHOOSE_LOWEST_MIN**

选择下界最小的变量。

enumerator **CHOOSE_HIGHEST_MAX**

选择上界最大的变量。

enumerator **CHOOSE_MIN_DOMAIN_SIZE**

选择域范围最小的变量。

enumerator **CHOOSE_MAX_DOMAIN_SIZE**

选择域范围最大的变量。

域缩减策略

天筹约束规划求解器的域缩减策略。

enum class **OPTVCPDomainReductionStrategy**

enumerator **SELECT_MIN_VALUE**
选择下界。

enumerator **SELECT_MAX_VALUE**
选择上界。

enumerator **SELECT_LOWER_HALF**
选择由 `bound = lb + (ub - lb) / 2` 计算得到的边界值。

enumerator **SELECT_UPPER_HALF**
选择由 `bound = ub - (ub - lb) / 2` 计算得到的边界值。

enumerator **SELECT_MEDIAN_VALUE**
选择变量的中位值。

enumerator **SELECT_RANDOM_HALF**
随机选择一半。

求解状态

CP 模型状态码。

OPTV_CP_UNKNOWN

取值

`0`

描述

求解器未能找到解。

OPTV_CP_MODEL_INVALID

取值

`1`

描述

模型无效。

OPTV_CP_FEASIBLE

取值

`2`

描述

求解器找到了可行解。

OPTV_CP_INFEASIBLE

取值

3

描述

求解器证明模型不可行。

OPTV_CP_OPTIMAL

取值

4

描述

求解器找到了最优解。

OPTV_CP_USER_INTERRUPT

取值

5

描述

求解器被用户中断。

8.4.2 Python API 参考手册

模型

class OPTVCPModel

天筹约束规划求解器的模型类。该类为构建约束规划问题提供了一个基础框架。

通过使用模型类，用户可以：

- 定义决策变量
- 添加约束
- 指定优化目标。

构建完模型后，可以使用*OPTVCPSolver* 求解模型，从而在复杂的约束条件下获得最优决策。

NewIntVar()

向模型中添加一个整数变量。

OPTVCPModel.**NewIntVar**(*lb=0*, *ub=OPTV_INF*, *name=''*)

该函数会向模型中添加一个整数变量。

参数

- **lb** (*int*) –变量的下界值。
- **ub** (*int*) –变量的上界值。
- **name** (*str*) –变量名称。

返回

添加到模型中的变量对象。

返回类型

OPTVCPVar

引发

OPTVErrCode.INVALID_ARGUMENT –参数 name 为空，或者存在为 NaN 的数值。

示例

```
x = model.NewIntVar(2, 5, "x")
```

NewIntVarFromDomain()

向模型中添加一个整数型变量，并指定其取值区间。

我们使用向量表示该取值区间，向量中每两个连续的元素表示一个闭区间 $[a, b]$ （其中 $a \leq b$ ）。例如，domain $= \{1, 2, 5, 9\}$，该向量表示两个区间：domain $= [1, 2] \cup [5, 9]$。因此，变量的取值可能为：$\{1, 2, 5, 6, 7, 8, 9\}$ 。

注解

表示取值区间的向量必须满足以下条件：

- 元素个数必须为偶数（每一对元素定义一个区间）。
- 区间必须严格不重叠且不相邻（端点不能相同或连续）。

OPTVCPModel.**NewIntVarFromDomain**(**domain*, *name=''*)

该函数会向模型中添加一个整数型变量，并指定其取值区间。

参数 domain 可以通过以下任一方式提供：

- 传入多个值：NewIntVarFromDomain(a, b, c)
- 传入一个可迭代对象：NewIntVarFromDomain([a, b, c]) 或 NewIntVarFromDomain((a, b, c))

参数

- **domain** –变量的有效取值区间。
- **name** (*str*) –变量名称。

返回
添加到模型中的变量对象。

返回类型
OPTVCPVar

引发
OPTVErrCode.INVALID_ARGUMENT –参数 domain 为空, 或参数 name 为空，或参数 domain 中存在空值。

示例

```
x = model.NewIntVarFromDomain(a, b, c, name="x")

y = model.NewIntVarFromDomain([a, b, c], name="y")
```

NewBoolVar()

向模型中添加一个布尔变量。布尔变量的值只能为 0 或 1。

OPTVCPModel.**NewBoolVar**(*name*)

该函数会向模型中添加一个布尔变量。

参数
name (*str*) –变量名称。

返回
添加到模型中的变量对象。

返回类型
OPTVCPVar

示例

```
model = OPTVCPModel()

x = model.NewBoolVar("x")
```

NewConstant()

向模型中添加一个表示常数值的变量。

OPTVCPModel.**NewConstant**(*value*)

该函数会向模型中添加一个表示常数值的变量。

参数
value (*int*) –变量的值。

返回
添加到模型中的变量对象。

返回类型
OPTVCPVar

示例

```
model = OPTVCPModel()

x = model.NewConstant(5)
```

AddLinearConstraint()

向模型中添加一条线性约束。

OPTVCPModel.**AddLinearConstraint**(*expr*, *lhs=0*, *rhs=OPTV_INF*)

该函数会向模型中添加一条线性约束。

参数

- **expr** (OPTVCPVar / OPTVCPLinExpr) – 约束的线性表达式部分。
- **lhs** (*float*) – 约束的下界。
- **rhs** (*float*) – 约束的上界。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT – lhs 与 rhs 的值不合法。

示例

```
model.AddLinearConstraint(2 * x - y, lhs = 4, rhs = 15)
```

OPTVCPModel.**AddLinearConstraint**(*constr*)

该方法接收一个*OPTVCPTempConstr* 对象作为参数。推荐在表达式对象之间或者表达式与数值之间使用比较运算符（如 <=、>=、==）来构建*OPTVCPTempConstr* 对象。

参数

constr (OPTVCPTempConstr) – *OPTVCPTempConstr* 对象。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

示例

```
model.AddLinearConstraint(2 * x - y <= 15)
```

警告

一条约束只能包含一个比较运算符。尽管 `1 <= x + y <= 2` 看起来像是一条有效的约束，但 AddLinearConstraint 目前并不接受这种写法。

AddLinearConstraintInDomain()

向模型添加一个约束，确保给定线性表达式的结果必须属于指定的区间。

我们使用向量表示线性表达式的取值区间，向量中每两个连续的元素表示一个闭区间 $[a, b]$ （其中 $a \leq b$ ）。例如，domain $= \{1, 2, 5, 9\}$, 该向量表示两个区间：domain $= [1, 2] \cup [5, 9]$ 。因此，线性表达式的取值可能为：$\{1, 2, 5, 6, 7, 8, 9\}$ 。

注解

表示取值区间的向量必须满足以下条件：

- 元素个数必须为偶数（每一对元素定义一个区间）。
- 区间必须严格不重叠且不相邻（端点不能相同或连续）。

OPTVCPModel.**AddLinearConstraintInDomain**(*expr*, **domain*)

该函数将向模型添加一个线性约束，以确保表达式的结果被限制在指定的区间内。

参数 domain 可以通过以下任一方式提供：

- 传入多个值：`AddLinearConstraintInDomain(expr, a, b, c)`
- 传入一个可迭代对象：`AddLinearConstraintInDomain(expr, [a, b, c])` 或 `AddLinearConstraintInDomain(expr, (a, b, c))`

参数

- **expr** (OPTVCPVar / OPTVCPLinExpr) –约束的线性表达式部分。
- **domain** –线性表达式的有效取值范围。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

- ***OPTVErrCode.INVALID_MODEL_INPUT*** –提供的取值区间的元素个数为奇数。
- ***OPTVErrCode.INVALID_ARGUMENT*** –提供的取值区间包含空元素。

示例

```
model.AddLinearConstraintInDomain(3 * x - y + z, [3, 4, 6, 10, 12,␣
↪14])
model.AddLinearConstraintInDomain(3 * x - y + z, 3, 4, 6, 10, 12, 14)
```

AddBoolOr()

向模型中添加一个 OR 约束。

OR 约束的形式如下：

$$x_1 \vee x_2 \vee \cdots \vee x_n = 1$$

确保布尔变量 $x_1, x_2, \ldots, x_n$ 中至少有一个等于 1。

OPTVCPModel.**AddBoolOr**(**literals*)

该函数会向模型中添加一个 OR 约束。

literals 参数可以通过以下任一方式提供：

- 传入多个值：AddBoolOr(a, b, c)
- 传入一个可迭代对象：AddBoolOr([a, b, c]) 或 AddBoolOr((a, b, c))

参数
literals –一个或多个布尔变量。

返回
添加到模型中的约束对象。

返回类型
OPTVCPConstr

引发
OPTVErrCode.INVALID_MODEL_INPUT –提供的某些变量不是布尔变量。

示例

```
model.AddBoolOr(a, b, c)

model.AddBoolOr([a, b, c])
```

AddBoolAnd()

向模型中添加一条 AND 约束。

AND 约束的形式如下：

$$x_1 \wedge x_2 \wedge \cdots \wedge x_n = 1$$

确保所有的布尔变量 $x_1, x_2, \ldots, x_n$ 都等于 1。

OPTVCPModel.**AddBoolAnd**(**literals*)

该函数会向模型中添加一条 AND 约束。

literals 参数可以通过以下任一方式提供：

- 传入多个值：AddBoolAnd(a, b, c)
- 传入一个可迭代对象：AddBoolAnd([a, b, c]) 或 AddBoolAnd((a, b, c))

参数
literals –一个或多个布尔变量。

返回
添加到模型中的约束对象。

返回类型
OPTVCPConstr

引发
OPTVErrCode.INVALID_MODEL_INPUT –提供的某些变量不是布尔变量。

示例

```
model.AddBoolAnd(a, b, c)

model.AddBoolAnd([a, b, c])
```

AddBoolXor()

向模型中添加一条布尔 XOR 约束。

XOR 约束的形式如下：

$$x_1 \oplus x_2 \oplus \cdots \oplus x_n = 1$$

确保布尔变量 $x_1, x_2, \ldots, x_n$ 中有奇数个变量的值为 1 。

OPTVCPModel.**AddBoolXor**(**literals*)

该函数会向模型中添加一条布尔 XOR 约束。

literals 参数可以通过以下任一方式提供：

- 传入多个值：AddBoolXor(a, b, c)
- 传入一个可迭代对象：AddBoolXor([a, b, c]) 或 AddBoolXor((a, b, c))

参数
literals –一个或多个布尔变量。

返回
添加到模型中的约束对象。

返回类型
OPTVCPConstr

引发
OPTVErrCode.INVALID_MODEL_INPUT –提供的某些变量不是布尔变量。

示例

```
model.AddBoolXor(a, b, c)

model.AddBoolXor([a, b, c])
```

AddAtLeastOne()

该函数等价于*OPTVCPModel.AddBoolOr()*。

OPTVCPModel.**AddAtLeastOne**(**literals*)

该函数会向模型中添加一条约束，确保至少有一个变量为非零值。

literals 参数可以通过以下任一方式提供：

- 传入多个值：AddAtLeastOne(a, b, c)
- 传入一个可迭代对象：AddAtLeastOne([a, b, c]) or AddAtLeastOne((a, b, c))

参数
literals –一个或多个布尔变量。

返回
添加到模型中的约束对象。

返回类型
OPTVCPConstr

引发
OPTVErrCode.INVALID_MODEL_INPUT –提供的某些变量不是布尔变量。

示例

```
model.AddAtLeastOne(a, b, c)

model.AddAtLeastOne([a, b, c])
```

AddAtMostOne()

向模型添加一条线性约束，确保提供的变量中至多有一个变量被赋值为 1。

OPTVCPModel.**AddAtMostOne**(**literals*)

该函数会向模型添加一条约束，确保提供的变量中至多有一个变量取 1。

literals 参数可以通过以下任一方式提供：

- 传入多个值：AddAtMostOne(a, b, c)
- 传入一个可迭代对象：AddAtMostOne([a, b, c]) 或 AddAtMostOne((a, b, c))

参数
literals –一个或多个布尔变量。

返回
添加到模型中的约束对象。

返回类型
OPTVCPConstr

引发
OPTVErrCode.INVALID_MODEL_INPUT –提供的某些变量不是布尔变量。

示例

```
model.AddAtMostOne(a, b, c)

model.AddAtMostOne([a, b, c])
```

AddExactlyOne()

向模型添加一条约束，确保在给定的布尔变量中，恰好有一个变量被赋值为 1，其余所有变量均被赋值为 0。

OPTVCPModel.**AddExactlyOne**(**literals*)

该函数会向模型添加一条约束，确保在给定的布尔变量中，恰好有一个变量被赋值为 1，其余所有变量均被赋值为 0。

literals 参数可以通过以下任一方式提供：

- 传入多个值：AddExactlyOne(a, b, c)
- 传入一个可迭代对象：AddExactlyOne([a, b, c]) 或 AddExactlyOne((a, b, c))

参数
literals –一个或多个布尔变量。

返回
添加到模型中的约束对象。

返回类型
OPTVCPConstr

引发
OPTVErrCode.INVALID_MODEL_INPUT –提供的某些变量不是布尔变量。

示例

```
model.AddExactlyOne(a, b, c)

model.AddExactlyOne([a, b, c])
```

AddImplication()

该约束表示一种条件关系 $x \to y$。这意味着如果变量 x 为真，那么变量 y 必须为真。但当变量 x 为假时，变量 y 可以为真，也可以为假。

OPTVCPModel.**AddImplication**(**literals*)

该函数向模型添加一条蕴含关系约束。

literals 参数可以通过以下任一方式提供：

- 传入多个值：AddBoolOr(a, b)
- 传入一个可迭代对象：AddBoolOr([a, b]) 或 AddBoolOr((a, b))

参数
literals –一个或多个布尔变量。

返回
添加到模型中的约束对象。

返回类型
OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –提供的某些变量不是二进制变量，或者参数长度不等于 2。

示例

```
model.AddImplication(a, b)

model.AddImplication([a, b])
```

AddAllowedAssignments()

向模型添加一个表约束。输入包含表达式（*exprs*）和值元组（*tuples*）。每个元组的大小应与表达式的数量相等。表约束强制所有变量必须被赋值为值元组中的某一个值元组。

例如，对于表约束 `AddAllowedAssignments([x1, x2], [[0, 0], [1, 0], [1, 1]])`，其中 *x1* 和 *x2* 是布尔型，其值可以表示如下：

x1	x2
0	0
1	0
1	1

`OPTVCPModel.`**`AddAllowedAssignments`**(*exprs*, *tuples*)

该函数会向模型中添加一条表约束。

参数

- **exprs** (*iterable*) –表达式的一个可迭代对象。
- **tuples** (*iterable*) –一个整数元组的可迭代对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –表达式数量为零，或其中一个表达式非仿射，或某一元组的大小与各表达式的大小不一致。

示例

```
model.AddAllowedAssignments([x1, b2], [[1, 2], [3, 4]])
```

AddForbiddenAssignments()

为模型添加一个单一的负向表约束。输入包含表达式（*exprs*）和值元组（*tuples*）。每个元组的大小应与表达式的大小相等。负向表约束强制要求所有变量不得被分配到任何元组中。

例如，对于负向表约束 `AddForbiddenAssignments([x1, x2], [[0, 1]])`，其中 *x1* 和 *x2* 为二元变量，数值可表示如下：

x1	x2
0	1

OPTVCPModel.**AddForbiddenAssignments**(*exprs*, *tuples*)

该函数会向模型中添加一条负向表约束。

参数

- **exprs** (*iterable*) –表达式的一个可迭代对象。
- **tuples** (*iterable*) –一个整数元组的可迭代对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –表达式数量为零，或其中一个表达式非仿射，或某一元组的大小与各表达式的大小不一致。

示例

```
model.AddForbiddenAssignments([x1, b2], [[1, 2], [3, 4]])
```

AddInterval()

向模型中添加一条区间约束。区间约束要求满足 $start + size = end$ 。区间约束是其他约束 (*No Overlap* or *Cumulative*) 的基础。

OPTVCPModel.**AddInterval**(*start*, *size*, *end*)

该函数会向模型中添加一条区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **start** (*number* | OPTVCPVar | OPTVCPLinExpr) –表示区间起始位置的仿射表达式。
- **size** (*number* | OPTVCPVar | OPTVCPLinExpr) –表示区间长度的仿射表达式。
- **end** (*number* | OPTVCPVar | OPTVCPLinExpr) –表示区间结束位置的仿射表达式。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
model.AddInterval(2 * x, 2 * y + 1, 1)
```

AddFixedSizeInterval()

向模型中添加单条固定长度区间约束。固定长度的区间约束本身并无实际意义，但可作为其他约束（*No Overlap* or *Cumulative* ）的基础。当区间的长度已知时，该函数可以替代*OPTVCPModel.AddInterval()* 。

OPTVCPModel.**AddFixedSizeInterval**(*start*, *size*)

该函数会向模型中添加一条可激活的固定长度区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **start** (*number* / OPTVCPVar / OPTVCPLinExpr) –表示区间起始位置的仿射表达式。
- **size** (*int*) –表示区间长度的常量。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –提供的表达式中包含多个变量。

示例

```
model.AddFixedSizeInterval(2 * x2, 7)

model.AddFixedSizeInterval(x2, 7)
```

AddOptionalInterval()

向模型中添加一条可激活的区间约束。可激活的区间约束形式为 $isPresent \rightarrow start + size = end$ 。区间约束是其他约束（例如*No Overlap* 或*Cumulative* ）的基础。当需要控制变量（布尔变量或其取反）时，此函数可以替代*OPTVCPModel.AddInterval* 。只有当控制变量为真时，该约束才会生效。

OPTVCPModel.**AddOptionalInterval**(*start*, *size*, *end*, *isPresent*)

该函数会向模型中添加一条区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **start** (OPTVCPVar / OPTVCPLinExpr) –表示区间起始位置的仿射表达式。
- **size** (OPTVCPVar / OPTVCPLinExpr) –表示区间长度的仿射表达式。
- **end** (OPTVCPVar / OPTVCPLinExpr) –表示区间结束位置的仿射表达式。
- **isPresent** (OPTVCPVar) –控制变量（布尔变量或其取反）。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
model.AddOptionalInterval(5 * x + 6, 3, 4 * y + 2, z)
```

AddOptionalFixedSizeInterval()

向模型中添加一条可激活的固定长度区间约束。固定长度区间约束本身没有实际意义，但是可以作为其他约束的基础，例如*No Overlap* 或*Cumulative* 。当区间的长度已知，且需要一个控制变量（布尔变量或其否定）来控制其是否生效时，该函数可以替代*OPTVCPModel.AddInterval* 。仅当控制变量为真时，该约束才会被激活并参与求解。

OPTVCPModel.**AddOptionalFixedSizeInterval**(*start*, *size*, *isPresent*)

该函数会向模型中添加一条可激活的固定长度区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **start** (OPTVCPVar / OPTVCPLinExpr) –表示区间起始位置的仿射表达式。
- **size** (*int*) –表示区间长度的常量。
- **isPresent** (OPTVCPVar) –控制变量（布尔变量或其取反）。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –提供的表达式中包含多个变量。

示例

```
model.AddOptionalFixedSizeIntervalSeries([x, 4 * y], [6, 10], [b1,␣
↪b2])
```

AddIntervalSeries()

向模型中一次性添加多条区间约束。区间约束要求满足 $start + size = end$ 。区间约束是其他约束 (*No Overlap* 或*Cumulative*) 的基础。

OPTVCPModel.**AddIntervalSeries**(*starts*, *sizes*, *ends*)

该函数会向模型中添加多条区间约束。若是向该函数中传入表达式，则表达式中至多只有一个变量。

参数

- **starts** (*iterable*) –一个元素类型为*OPTVCPVar* 或*OPTVCPLinExpr* 的可迭代对象，表示区间的起始位置。
- **sizes** (*iterable*) –一个元素类型为*OPTVCPVar* 或*OPTVCPLinExpr* 的可迭代对象，表示区间的长度。
- **ends** (*iterable*) –一个元素类型为*OPTVCPVar* 或*OPTVCPLinExpr* 的可迭代对象，表示区间的结束位置。

返回

添加到模型中的约束对象列表。

返回类型
list[*OPTVCPConstr*]

引发
OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
model.AddIntervalSeries([2 * x, 2 * y], [2 * y + 1, 3], [13, z])
```

AddFixedSizeIntervalSeries()

向模型中一次性添加多条固定长度的区间约束。固定长度的区间约束本身并无实际意义，但可作为其他约束（*No Overlap* 或 *Cumulative* ）的基础。当区间的大小已知时，该函数可以替代*OPTVCPModel.AddIntervalSeries()* 。

OPTVCPModel.**AddFixedSizeIntervalSeries**(*starts*, *sizes*)

该函数会向模型中添加多条固定长度的区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **starts** (*iterable*) – 一个可迭代对象，其元素类型可以为*OPTVCPVar* 或*OPTVCPLinExpr*, 用于表示各区间的开始时间。
- **sizes** (*iterable*) –一个元素全为数值的可迭代对象，用于表示各区间的大小。

返回
添加到模型中的约束对象列表。

返回类型
list[*OPTVCPConstr*]

引发
OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
start1, start2 = x + 1, 15 * y

model.AddFixedSizeIntervalSeries([start1, start2], [3, 5])
```

AddOptionalIntervalSeries()

向模型中添加多条可激活的区间约束。可激活的区间约束形式为 $isPresent \to start + size = end$ 。区间约束是其他约束（例如*No Overlap* 或 *Cumulative* ）的基础。当需要控制变量（布尔变量或其取反）时，此函数可以替代*OPTVCPModel.AddIntervalSeries()* 。只有当控制变量为真时，这些约束才会生效。

OPTVCPModel.**AddOptionalIntervalSeries**(*starts*, *sizes*, *ends*, *arePresent*)

该函数会向模型中添加多个区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **starts** (*iterable*) –一个元素为数值、*OPTVCPVar* 或*OPTVCPLinExpr* 的可迭代对象，用于表示区间的起始位置。
- **sizes** (*iterable*) –一个元素为数值、*OPTVCPVar* 或*OPTVCPLinExpr* 的可迭代对象，用于表示区间的大小。

- **ends** (*iterable*) –一个元素为数值、*OPTVCPVar* 或*OPTVCPLinExpr* 的可迭代对象，用于表示区间的结束位置。
- **arePresent** (*iterable*) –一个元素为控制变量（布尔变量或其取反）的可迭代对象。

返回
添加到模型中的约束对象列表。

返回类型
list[*OPTVCPConstr*]

引发
OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
tmp3 = model.AddOptionalIntervalSeries([2 * x, 2 * y], [x, 3], [8, 3␣
↪* y], [b1, b2])
```

AddOptionalFixedSizeIntervalSeries()

向模型中添加多个可激活的固定大小区间约束。固定大小区间约束本身并无实际意义，但可作为其他约束（例如*No Overlap* 或*Cumulative* ）的基础。当区间的长度已知且需要控制变量（布尔变量或其否定）时，该函数可以替代*OPTVCPModel.AddIntervalSeries()* 。只有控制变量为真时，该约束才会生效。

OPTVCPModel.**AddOptionalFixedSizeIntervalSeries**(*starts*, *sizes*, *arePresent*)

该函数会向模型中添加多条固定长度的区间约束。向该函数传入的表达式中至多只能有一个变量。

参数

- **starts** (*iterable*) –一个元素类型为数值、*OPTVCPVar* 或*OPTVCPLinExpr* 的可迭代对象，用于表示区间的起始位置。
- **sizes** (*iterable*) –一个元素类型为整数的可迭代对象，用于表示区间的大小。
- **arePresent** (*iterable*) –一个元素为控制变量（布尔变量或其取反）的可迭代对象。

返回
添加到模型中的约束对象列表。

返回类型
list[*OPTVCPConstr*]

引发
OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
model.AddOptionalFixedSizeIntervalSeries([x, 4 * y], [6, 10], [b1,␣
↪b2])
```

AddNoOverlap()

向模型中添加一条无重叠约束。无重叠约束会保证该约束中包含的区间约束的起止范围不会重叠。

OPTVCPModel.**AddNoOverlap**(*intervals*)

该函数会向模型中添加一条无重叠约束。

参数

intervals(*iterable*) –一个元素类型为*OPTVCPConstr*的可迭代对象，表示不会重叠的区间约束。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –提供的某些约束类型不是*interval*。

示例

```
tmp1 = model.AddIntervalSeries([2 * x, 2 * y], [2 * y + 1, 3], [13,␣
↪z])
tmp2 = model.AddFixedSizeIntervalSeries([y + 1, 2 * z], [3, 5])
model.AddNoOverlap([tmp1, tmp2])
```

AddCumulative()

向模型中添加一条累积约束。累积约束要求：在任意时间点上，所有重叠的区间的需求量之和不得超过其容量。

OPTVCPModel.**AddCumulative**(*intervals*, *demands*, *capacity*)

该函数会向模型中添加一条累积约束。

参数

- **intervals**(*iterable*) –一个元素类型为*OPTVCPConstr*的可迭代对象。
- **demands**(*iterable*) –一个元素类型为*OPTVCPLinExpr*的可迭代对象。
- **capacity**(*number*/OPTVCPVar/OPTVCPLinExpr) –约束的容量。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –提供的约束类型不全是*interval*，参数capacity中包含多个变量。

示例

```
start1, size1, end1 = 2 * x2, 2 * x1 + 1, 9

start2, size2, end2 = 5 * b3 + 6, 3, 4 * x1 + 2
```

(下页继续)

(续上页)

```
start3 size3, end3 = 10, 5, 15

demand1, demand2, demand3, capacity = 4 * x1, 8, 9, 20

interval1 = model.AddInterval(start1, size1, end1)
interval2 = model.AddInterval(start2, size2, end2)
interval3 = model.AddInterval(start3, size3, end3)

cumulative = model.AddCumulative([interval1, interval2, interval3],␣
↪[demand1, demand2, demand3], capacity)
```

AddAllDifferent()

向模型中添加一条互异约束。互异约束保证给定的所有表达式会被赋予不同的值。

OPTVCPModel.**AddAllDifferent**(*expressions*)

该函数会向模型中添加一条互异约束。参数 expressions 中的元素可以是数字、*OPTVCPVar* 、*OPTVCPLinExpr*，但是该参数中的元素最多只能包含一个变量。

参数

expressions (*list*) –仿射表达式列表，其中每个表达式最多只能包含一个变量。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
model.AddAllDifferent([x+1, y, z])
```

OPTVCPModel.**AddAllDifferent**(*vars*)

该函数会向模型中添加一条互异约束。

参数

vars (*list*) –向量列表。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

示例

```
model.AddAllDifferent([y, z])
```

AddElement()

向模型中添加一条元素约束。元素约束强制满足以下条件：target = $\text{expressions}_{\text{index}}$。

OPTVCPModel.**AddElement**(*index*, *expressions*, *target*)

该函数将向模型添加一个单元素约束。

参数

- **index** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – 数组中所选表达式的索引。该索引必须为仿射表达式（a * var + b）。
- **expressions** (*Iterable[int | float | OPTVCPVar | OPTVCPLinExpr]*) – 仿射表达式的可迭代对象。
- **target** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – 被约束为等于所选表达式的仿射表达式（a * var + b）。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT – 某些表达式包含多个变量。

示例

```
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain([0, 3, 5, 7, 15, 15], name="x2")
x3 = model.NewBoolVar("x3")
element = model.AddElement(x3, [2 * x2, 4 * x2 + 3], x1 + 4)
```

AddMaxEquality()

向模型中添加一条最大等值约束。

最大等值约束强制满足以下条件：target = max(expressions)。

OPTVCPModel.**AddMaxEquality**(*target*, *expressions*)

该函数会向模型中添加一条最大等值约束。

参数

- **target** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – 等于 expressions 中最大值的表达式。
- **expressions** (*Iterable[int | float | OPTVCPVar | OPTVCPLinExpr]*) – 数值、变量或线性表达式的可迭代对象。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT – 某些表达式包含多个变量。

示例

```
model.AddMaxEquality(3 * x2 + 7, [2 * x1 + 4, 3 * x1 - 1, 7 * x2 - 6])
```

AddMinEquality()

向模型中添加一条最小等值约束。最小等值约束强制满足以下条件：target = min(expressions)。

OPTVCPModel.**AddMinEquality**(*target*, *expressions*)

该函数会向模型中添加一条最小等值约束。

参数

- **target** (*int | float | OPTVCPVar | OPTVCPLinExpr*) –线性表达式，其值为向量 expressions 中的最小值。
- **expressions** (*Iterable[int | float | OPTVCPVar | OPTVCPLinExpr]*) –数值、变量或线性表达式的可迭代对象。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
model.AddMinEquality(3 * x2 + 7, [2 * x1 + 4, 3 * x1 - 1, 7 * x2 - 6])
```

AddAbsEquality()

向模型中添加一个绝对等式约束。绝对等式约束强制满足以下条件：target = |expressions|。

OPTVCPModel.**AddAbsEquality**(*target*, *expr*)

此函数将向模型中添加一个绝对等式约束。

参数

- **target** (OPTVCPVar | OPTVCPLinExpr) –等于 expr 绝对值的表达式。
- **expr** (OPTVCPVar | OPTVCPLinExpr) –表示 target 绝对值的变量或线性表达式。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT –某些表达式包含多个变量。

示例

```
model.AddAbsEquality(x + 1, y) # x + 1 = Abs(y)
```

AddMultiplicationEquality()

向模型中添加一条乘法等式约束。乘法等式约束强制满足以下条件：target $= \prod_{i=0}^{n}$ expressions$_i$。

OPTVCPModel.**AddMultiplicationEquality**(*target*, **expressions*)

该函数会向模型中添加一条乘法等式约束。

参数

- **target** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – 与所有 expressions 的乘积相等的表达式。
- ***expressions** (*int* | *float* | OPTVCPVar | OPTVCPLinExpr) – 一个或多个数值、变量或线性表达式。

返回

添加到模型中的约束对象。

返回类型

OPTVCPConstr

引发

OPTVErrCode.INVALID_MODEL_INPUT – 某些表达式包含多个变量。

示例

```
multEquality = model.AddMultiplicationEquality(4 * x1, [2 * x1, x2 +␣
↪1])
```

AddDecisionStrategy()

向模型中添加一种搜索策略。

OPTVCPModel.**AddDecisionStrategy**(*vars*, *varStrategy*, *domainStrategy*)

该函数会向模型中添加一种搜索策略。

参数

- **vars** (*Iterable[OPTVCPVar]*) – 该搜索策略将要赋值的变量可迭代对象。
- **varStrategy** (OPTVCPVariableSelectionStrategy) – 用于选择下一个变量的启发式策略。
- **domainStrategy** (OPTVCPDomainReductionStrategy) – 用于缩减所选变量定义域的启发式策略。

返回

无。

返回类型

None

引发

OPTVErrCode.INDEX_OUT_OF_RANGE – 提供的某些变量索引为负数或不在模型中。

示例

```
model.AddDecisionStrategy([x, y, z], OPTVCPVariableSelectionStrategy.
↪CHOOSE_MIN_DOMAIN_SIZE, OPTVCPDomainReductionStrategy.SELECT_MIN_
↪VALUE)
```

Maximize()

将模型的目标设置为最大化一个线性表达式的值。

注解

不设置目标函数或设置一个常数目标函数将导致一个可行性问题。

当未指定目标函数时，目标值被视为 0。如果使用常数作为目标函数，则目标值将等于该常数。

OPTVCPModel.**Maximize**(*expr*)

此函数将模型的目标方向设置为最大化，并指定目标函数。

参数

expr (*number* | OPTVCPVar | OPTVCPLinExpr) –需要设定的目标函数表达式。

返回

无。

返回类型

None

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVar(1, 4, "x2")

model.Maximize(x1 + x2)
```

Minimize()

将模型的目标设置为最小化一个线性表达式的值。

注解

不设置目标函数或设置一个常数目标函数将导致一个可行性问题。

当未指定目标函数时，目标值被视为 0。如果使用常数作为目标函数，则目标值将等于该常数。

OPTVCPModel.**Minimize**(*expr*)

此函数将模型的目标方向设置为最小化，并指定目标函数。

参数

expr (*number* | OPTVCPVar | OPTVCPLinExpr) –需要设定的目标函数表达式。

返回

无。

返回类型

None

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVar(1, 4, "x2")

model.Minimize(x1 + x2)
```

HasObjective()

检查模型是否设置了目标函数。

OPTVCPModel.**HasObjective**()

返回
模型是否设置目标函数的标志。

返回类型
bool

返回值

- **True** –模型设置了目标函数。
- **False** –模型没有设置目标函数。

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVar(1, 4, "x2")
model.HasObjective() # False

model.Minimize(x1 + x2)
model.HasObjective() # True

model.ClearObjective()
model.HasObjective() # False
```

ClearObjective()

清除模型中设置的目标函数。

OPTVCPModel.**ClearObjective**()

返回
无。

返回类型
None

示例

```
model.ClearObjective()
```

IsMaximize()

检查模型是否最大化目标函数值。

OPTVCPModel.**IsMaximize**()

返回
模型是否最大化目标函数值的标志。

返回类型
bool

返回值

- **True** –模型最大化目标函数值。
- **False** –模型会最小化目标函数值或者没有设置目标函数。

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVar(1, 4, "x2")
model.IsMaximize() # False

model.Maximize(x1 + x2)
model.IsMaximize() # True

model.ClearObjective()
model.IsMaximize() # False
```

GetVar()

获取指定的变量。

OPTVCPModel.**GetVar**(*i*)

参数
i (*int*) –变量在模型中的索引。

返回
在模型中索引 i 的变量。

返回类型
OPTVCPVar

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –索引值为负数或超过了模型中的变量数量。

示例

```
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.GetVar(0) # b1
model.GetVar(1) # b2
```

(下页继续)

(续上页)

```
model.GetVar(2)  # b3
model.GetVar(3)  # b4
```

GetVars()

获取模型中的所有变量。

OPTVCPModel.**GetVars**()

返回
一个包含模型中所有变量对象的列表。

返回类型
list[*OPTVCPVar*]

示例

```
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.GetVars() # [b1, b2, b3, b4]
```

GetConstr()

获取模型中指定位置的约束。

OPTVCPModel.**GetConstr**(*i*)

参数
i (*int*) – 线性约束在模型中的索引。

返回
索引为 i 的约束。

返回类型
OPTVCPConstr

引发
OPTVErrCode.INDEX_OUT_OF_RANGE – 索引值为负数或大于模型中约束的数量。

示例

```
constr = model.GetConstr(0) # Get the constraint at index 0.
```

GetConstrs()

获取模型中的所有约束。

OPTVCPModel.**GetConstrs**()

返回
包含模型中所有约束的列表。

返回类型
list[*OPTVCPConstr*]

示例

```
model.GetConstrs()
```

GetVarDomain()

查询指定变量的取值区间。

取值区间使用向量表示，向量中每两个连续的元素表示一个闭区间 $[a, b]$ （其中 $a \leq b$ ），且该向量中的元素个数为偶数，其表示的区间不重叠且不相邻。

例如，domain $= \{1, 2, 5, 9\}$ ，该向量表示两个区间：domain $= [1, 2] \cup [5, 9]$ 。因此，变量的取值可能为：$\{1, 2, 5, 6, 7, 8, 9\}$ 。

OPTVCPModel.**GetVarDomain**(*i*)

参数
i (*int*) –变量在模型中的索引。

返回
一个包含变量取值区间的列表。

返回类型
list[float]

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –索引值为负数或超过了模型中的变量数量。

示例

```
b1 = model.NewBoolVar("b1")
model.GetVarDomain(b1.Index()) # [0, 1]

x2 = model.NewIntVarFromDomain([1, 2, 5, 6, 7, 15], name="x2")
model.GetVarDomain(x2.Index()) # [1, 2, 5, 6, 7, 15]
```

GetLinearConstraintVars()

查询指定约束中的所有变量。

OPTVCPModel.**GetLinearConstraintVars**(*i*)

参数
i (*int*) –线性约束在模型中的索引。

返回
一个包含指定约束所有变量索引的列表。

返回类型
list[int]

引发
OPTVErrCode.INVALID_MODEL_INPUT –查询的约束不是线性约束。

示例

```
constr = model.AddLinearConstraint(2 * x - y < 15)
model.GetLinearConstraintVars(constr.Index()) == [x.Index(), y.
↪Index()] # True
```

GetLinearConstraintCoeffs()

查询指定线性约束中所有变量的系数。

OPTVCPModel.**GetLinearConstraintCoeffs**(*i*)

参数
i (*int*) –线性约束在模型中的索引。

返回
一个包含指定约束中所有变量系数的列表。

返回类型
list[float]

引发
OPTVErrCode.INVALID_MODEL_INPUT –查询的约束不是线性约束。

示例

```
constr = model.AddLinearConstraint(2 * x - y < 15)
model.GetLinearConstraintCoeffs(constr.Index()) # [2, -1]
```

GetLinearConstraintDomain()

获取描述指定约束的表达式的取值区间。

取值区间使用向量表示，向量中每两个连续的元素表示一个闭区间 $[a, b]$ （其中 $a \leq b$ ），且该向量中的元素个数为偶数，其表示的区间不重叠且不相邻。

例如，domain $= \{1, 2, 5, 9\}$, 该向量表示两个区间：domain $= [1, 2] \cup [5, 9]$ 。因此，线性表达式的取值可能为：$\{1, 2, 5, 6, 7, 8, 9\}$ 。

OPTVCPModel.**GetLinearConstraintDomain**(*i*)

参数
i (*int*) –线性约束在模型中的索引。

返回
一个表示线性表达式取值区间的列表。

返回类型
list[float]

引发
OPTVErrCode.INVALID_MODEL_INPUT –查询的约束不是线性约束。

示例

```
constr = model.AddLinearConstraint(2 * x - y < 15)
model.GetLinearConstraintDomain(lin1.Index()) # [-OPTV_INF, 15]
```

GetEnforcementLiterals()

若是只有某些变量取值为真时，另一些变量或者约束才会被 激活，这样的变量称为 控制变量。例如:

$$(a \wedge b \wedge c) \rightarrow \text{constraint} \tag{8.3}$$

$$x \rightarrow y \tag{8.4}$$

公式 1 表示只有变量 a, b, c 都为真时，约束 constraint 才会被激活，参考*OPTVCPConstr.OnlyEnforceIf()* 。

公式 2 表示变量 x 为真时，变量 y 必须为真，参考*OPTVCPModel.AddImplication()* 。

其中，变量 a, b, c, x 就是控制变量。

该函数会返回指定约束的控制变量的索引。

OPTVCPModel.**GetEnforcementLiterals**(*i*)

参数
i (*int*) –线性约束在模型中的索引。

返回
一个元素为控制变量索引的列表。

返回类型
list[int]

示例

```
boolOr = model.AddBoolOr(b1, b2, b4)
model.GetEnforcementLiterals(boolOr.Index()) == []
boolOr.OnlyEnforceIf([b3, b4])
model.GetEnforcementLiterals(boolOr.Index()) == [b3.Index(), b4.
↪Index()]
```

GetConstraintLiterals()

获取指定布尔约束中所有变量的索引。

OPTVCPModel.**GetConstraintLiterals**(*i*)

参数
i (*int*) –线性约束在模型中的索引。

返回
指定约束所有变量下标的列表。

返回类型
list[int]

引发
OPTVErrCode.INVALID_MODEL_INPUT –查询的约束不是布尔约束。

示例

```
boolOr = model.AddBoolOr(b1, b2, b4)
model.GetConstraintLiterals(boolOr.Index()) == [b1.Index(), b2.
↪Index(), b4.Index()]
```

GetConstraintName()

获取指定约束的名称。

OPTVCPModel.**GetConstraintName**(*i*)

参数
i (*int*) –线性约束在模型中的索引。

返回
索引为 i 的约束名称。

返回类型
str

示例

```
lin_constr = model.AddLinearConstraint(-x1 + 4 * x2 + 11 - 3, lhs = 2,
↪ rhs = 15)
lin_constr.WithName("c0")
model.GetConstraintName(lin_constr.Index()) # c0
```

Set()

参数

为模型设置一个求解参数。创建模型后，该接口为模型提供了控制参数的能力。

OPTVCPModel.**Set**(*param*, *value*)

参数

- **param** (OPTVIntParam/OPTVDblParam/*str*) –需要设置的参数。

- **value** (*int | float | str*) –要设置的值。

返回
无。

返回类型
None

引发
OPTVErrCode.UNKNOWN_PARAMETER –传入了未知参数。

示例

```
model.Set(OPTVIntParam.CP_THREADS, 3)
model.Set("CP_GAP", "0.01")
```

求解器类

class OPTVCPSolver

天筹约束规划求解器的求解器类。该类用于求解约束规划问题。

Solve()

求解给定的约束规划模型。

OPTVCPSolver.**Solve**(*model*)

该函数会求解给定的约束规划模型并返回求解状态。

参数
model (OPTVCPModel) –约束规划模型。

返回
求解状态。

返回类型
int

示例

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
c1 = model.NewConstant(11)

lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_INF,␣
↪15)
lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, [3, 4, 6,␣
↪10, 12, 14])

model.Minimize(b1 + b2 + b3 + b4 + 2 * x1 + 2 * x2)
status = solver.Solve(model)
```

GetObjectiveValue()

返回当前求解模型的目标函数值。

OPTVCPSolver.**GetObjectiveValue**()

该函数会返回当前求解模型的目标函数值。

返回

当前求解模型的目标函数值。

返回类型

float

引发

OPTVErrCode.DATA_NOT_AVAILABLE –模型未求解。

示例

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
objVal = solver.GetObjectiveValue()
```

GetBestObjectiveBound()

对于最小化问题，返回目标函数的最佳下界；对于最大化问题，返回目标函数的最佳上界。

OPTVCPSolver.**GetBestObjectiveBound**()

该函数返回当前求解模型的最佳目标界限。

返回

当前求解模型的最佳目标界限。

返回类型

float

引发

OPTVErrCode.DATA_NOT_AVAILABLE –模型未求解。

示例

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
objBnd = solver.GetBestObjectiveBound()
```

GetValue()

返回当前求解模型中指定对象的求解结果。

OPTVCPSolver.**GetValue**(*expr*)

该函数会返回当前求解模型中指定变量或表达式的求解结果。

参数

expr (OPTVCPVar / OPTVCPLinExpr) –需要查询求解结果的变量（*OPTVVar*）或表达式（*OPTVLinExpr*）。

返回

变量或表达式的求解结果。

返回类型

float

引发

OPTVErrCode.DATA_NOT_AVAILABLE –模型未求解或提供的变量不在当前求解的模型中。

示例

```
env = OPTVEnv()
model = OPTVCPModel(env)
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
c1 = model.NewConstant(11)

lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_INF,␣
↪15)
lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, [3, 4, 6,␣
↪10, 12, 14])

model.Minimize(b1 + b2 + b3 + b4 + 2 * x1 + 2 * x2)
status = solver.Solve(model)

resB1 = solver.GetValue(b1)
```

GetValues()

批量查询当前求解模型中变量的求解结果。

OPTVCPSolver.**GetValues**(*vars*)

该函数会返回当前求解模型中多个变量的求解结果。

参数

vars (*iterable*) –一个或多个变量。

返回

一个给定变量求解结果的列表。

返回类型

list[float]

引发

OPTVErrCode.DATA_NOT_AVAILABLE –模型未求解或提供的变量不在当前求解的模型中。

示例

```
env = OPTVEnv()
model = OPTVCPModel(env)
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain([1, 2, 5, 7, 15, 15], name="x2")
c1 = model.NewConstant(11)

lin0 = model.AddLinearConstraint(-x1 + 4 * x2 + c1 - 3, -OPTV_CP_INF,␣
↪15)
lin1 = model.AddLinearConstraintInDomain(3 * b1 - b2 + x1, [3, 4, 6,␣
↪10, 12, 14])

model.Minimize(b1 + b2 + b3 + b4 + 2 * x1 + 2 * x2)
status = solver.Solve(model)

result = solver.GetValues(b1, b2, b4)
```

GetBooleanValue()

获取当前求解模型中指定布尔变量的值。

OPTVCPSolver::GetBooleanValue(var)

该函数会返回当前求解模型中指定的布尔变量的值。

参数

var (OPTVCPVar) –需要查询求解结果的布尔变量。

返回

当前求解模型中该布尔变量的值。

返回类型

float

引发

OPTVErrCode.DATA_NOT_AVAILABLE –模型未求解，或变量不是布尔变量，或变量不在当前模型中。

示例

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
```

(下页继续)

(续上页)

```
solver.Solve(model)
result = solver.GetBooleanValue(b1)
```

GetBooleanValues()

批量获取当前求解模型中布尔变量的值。

OPTVCPSolver.**GetBooleanValues**(**vars*)

该函数会返回当前求解模型中多个布尔变量的值。

参数

vars –来自最后求解模型的多个布尔变量。

返回

一个包含给定变量求解结果的列表。

返回类型

list[float]

引发

OPTVErrCode.DATA_NOT_AVAILABLE –模型未求解，或提供的某些变量不是布尔变量，或某些变量不在当前模型中。

示例

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
result = solver.GetBooleanValues(b1, b2, b3, b4)
```

GetSolution()

返回当前求解模型中所有变量的求解结果。

OPTVCPSolver.**GetSolution**()

该函数会返回当前求解模型中所有变量的值。

返回

一个包含当前求解模型中所有变量值的列表。

返回类型

list[int]

引发

OPTVErrCode.DATA_NOT_AVAILABLE –模型未求解。

示例

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

model.Minimize(b1 + b2 + b3 + b4)

solver = OPTVCPSolver()
solver.Solve(model)
result = solver.GetSolution()
```

变量

class OPTVCPVar

天筹求解器的变量类。用于从模型中访问有关变量的数据。不推荐使用构造函数创建变量对象。用户可以使用*OPTVCPModel.NewIntVar* 、*OPTVCPModel.NewIntVarFromDomain* 或者*OPTVCPModel.NewBoolVar* 等接口来创建变量。

Index()

获得变量在模型中的索引值。

OPTVCPVar.**Index**()

返回
变量在模型中的索引。

返回类型
int

返回值

- **-1** –变量不在模型中。
- **>=0** –变量在模型中的索引。

示例

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")

print(x2.Index())  # 1
print(x1.Index())  # 0
```

GetName()

返回变量名称。

OPTVCPVar.**GetName**()

返回
变量名称。

返回类型
str

引发
OPTVErrCode.NOT_IN_MODEL –变量不在模型中。

示例

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")

print(x2.GetName())  # "x1"
print(x1.GetName())  # "x2"
```

GetDomain()

返回变量的取值区间。

OPTVCPVar.**GetDomain**()

返回
变量的取值区间。

返回类型
list[int]

示例

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")

print(x2.GetDomain())  # [0, 3, 5, 7, 15, 15]
print(x1.GetDomain())  # [2, 5]
```

SameAs()

判断两个变量是否为同一个对象。

OPTVCPVar.**SameAs**(*v2*)

参数
v2 (OPTVCPVar) –需要与当前变量对比的变量。

返回
变量是否相同。

返回类型
bool

返回值

- **True** –变量相同。
- **False** –变量不同。

示例

```
model = OPTVCPModel(env)

b1 = model.NewBoolVar("b1")
x1 = model.NewIntVar(2, 5, "x1")

print(b1.SameAs(x1)) # False
```

IsBinary()

检查该变量是否为布尔变量。

OPTVCPVar.**IsBinary**()

返回
变量是否为布尔变量。

返回类型
bool

引发
OPTVErrCode.NOT_IN_MODEL –变量不在模型中。

示例

```
model = OPTVCPModel(env)

b1 = model.NewBoolVar("b1")
x1 = model.NewIntVar(2, 5, "x1")

print(b1.IsBinary()) # True
print(x1.IsBinary()) # False
```

线性表达式

class OPTVCPLinExpr

天筹约束规划求解器的线性表达式类。*OPTVCPLinExpr* 类用于构建线性表达式与目标函数。该类的对象由变量和常数项组成。线性表达式按其构造形式进行存储。例如，表达式 `1 * x + 2 * y + 3 * x` 的存储形式如下：

索引	系数	变量
0	1	x
1	2	y
2	3	x

AddTerms()

该功能向当前表达式的尾部添加一组新的项。

OPTVCPLinExpr.**AddTerms**(*coeffs*, *vars*)

此函数将 $\text{coeffs}_i * \text{vars}_i$, $\forall i \in [0, \text{vars.size()}]$ 添加到当前线性表达式后。

参数

- **coeffs** (*float | iterable*) –一个或多个变量系数。
- **vars** (*iterable*) –一个或多个变量。

返回

无。

返回类型

None

引发

OPTVErrCode.INVALID_MODEL_INPUT –参数 `vars` 与 `coeffs` 的长度不一致。

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")
x3 = model.NewIntVar(0, 5, "x3")
x4 = model.NewIntVar(0, 3, "x4")

expr = x1 + 1 + x2
expr.AddTerms([2, 3], [x3, x4]) # x1 + x2 + 2*x3 + 3*x4 + 1
```

Clear()

清空该表达式，*OPTVCPLinExpr.Size()* 将返回 0。同时，该表达式从数值意义上将等同于 0。

OPTVCPLinExpr.**Clear**()

返回
无。

返回类型
None

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")

expr = x1 + 1 + x2

expr.Clear()
```

GetCoef()

获取线性表达式中指定变量的系数。

OPTVCPLinExpr.**GetCoef**(*i*)

参数
i (*int*) –变量在线性表达式中的索引。

返回
对应的变量系数。

返回类型
float

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0, $ this->Size() $]$。

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")

expr = 3*x1 + 1 - x2

print(expr.GetCoef(0)) # 3
print(expr.GetCoef(1)) # -1
```

GetVar()

返回线性表达式中指定位置上的变量。

OPTVCPLinExpr.**GetVar**(*i*)

参数
i (*int*) –变量在线性表达式中的索引。

返回
指定位置上的变量。

返回类型
OPTVCPVar

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i $\notin [0, \texttt{this->Size()}]$。

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")

expr = 3*x2 + 1 - x1

print(expr.GetVar(0).SameAs(x2)) # True
print(expr.GetVar(1).SameAs(x1)) # True
```

GetConstant()

获得线性表达式中的常数项。

OPTVCPLinExpr.**GetConstant**()

返回
表达式中的常量数值。

返回类型
float

示例

```
model = OPTVCPModel()

x1 = model.NewIntVar(0, 10, "x1")
x2 = model.NewIntVar(0, 15, "x2")

expr1 = 3*x1 + 1 - x2
expr2 = x1 + x2

print(expr1.GetConstant()) # 1
print(expr2.GetConstant()) # 0
```

SameAs()

判断两个线性表达式是否为同一个对象。

OPTVCPLinExpr.**SameAs**(*e2*)

参数
e2 (OPTVCPLinExpr) –需要与当前线性表达式对比的线性表达式。

返回
线性表达式是否相同。

返回类型
bool

返回值

- **True** –线性表达式相同。
- **False** –线性表达式不同。

示例

```
expr1 = x1 + x2
expr2 = x1 + 1
expr3 = expr1 + expr2

expr3.SameAs(x1 + x2 + x1 + 1) # True
```

Size()

获得该表达式的规模。

OPTVCPLinExpr.**Size**()

该函数返回的数值对应线性表达式中的变量个数，不论变量是否重复出现。例如表达式 x+y 对应 Size = 2，而 x+y+x 对应 Size = 3。

返回
线性表达式大小。

返回类型
int

示例

```
expr1 = x1 + x2
expr2 = x1 + 1
expr3 = x1 + x2 + x1 + 1

print(expr1.Size()) # 2
print(expr2.Size()) # 1
print(expr3.Size()) # 3
```

Remove()

从表达式中删除项。

OPTVCPLinExpr.**Remove**(*i*)

删除表达式中指定的项。

参数
i (*int*) –指定的索引值。

返回
无。

返回类型
None

引发
OPTVErrCode.INDEX_OUT_OF_RANGE –若 i ∉ [0, this->Size()]。

示例

```
expr = 3*x2 + 1 - x1

expr.Remove(1)
print(expr) # 3*x2 + 1
```

OPTVCPLinExpr.**Remove**(*v*)

从线性表达式中删除指定变量涉及的所有项。

参数
v (OPTVCPVar) –指定要删除的变量。

返回
是否成功删除任何项。

返回类型
bool

返回值

- **True** –成功删除任何项。
- **False** –未删除任何项。

引发
OPTVErrCode.NOT_IN_MODEL –成功删除与指定变量有关的项。

示例

```
expr = 3*x2 + 1 - x1

expr.Remove(x2)
print(expr) # 1 - x1
```

约束

class OPTVCPConstr

天筹约束规划求解器的约束对象。用于访问模型中与约束有关的信息。注意约束不可由其构造函数创建，而需要通过*OPTVCPModel* 的相关接口向模型中添加约束。

OnlyEnforceIf()

为约束添加控制变量（布尔变量或其取反）。只有控制变量全部为真时，该约束才会生效。

OPTVCPConstr.**OnlyEnforceIf**(*literals*)

参数
literals (*OPTVCPVar | iterable*) –一个或多个控制变量（布尔变量或其取反）。

返回
无。

返回类型
None

引发

- ***OPTVErrCode.NOT_IN_MODEL*** –约束不在任何模型中。
- ***OPTVErrCode.INVALID_MODEL_INPUT*** –提供的某些控制变量不是布尔变量。

示例

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")ref-man/EN/source/manual_text/optverse_api/
↪cp-sat/python_api/OPTVCPModel/get_enforcement_literals.rst
b4 = model.NewBoolVar("b4")

boolAnd = model.AddBoolAnd(b1, ~b2)
boolAnd.OnlyEnforceIf(b3, b4)

model.GetEnforcementLiterals(boolAnd) == [b3.Index(), b4.Index()]
```

WithName()

设置约束的名称。

OPTVCPConstr.**WithName**(*name*)

参数
name (*str*) –约束的名称。

返回
无。

返回类型
None

引发
OPTVErrCode.NOT_IN_MODEL –约束不在任何模型中。

示例

```
model = OPTVCPModel()
domain = [1, 2, 5, 6, 7, 15]
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domain, name="x2")

constr = model.AddLinearConstraint(-x1 + 4 * x2 + 11 - 3, lhs = 2,␣
↪rhs = 15)

constr.WithName("c0")

print(constr.GetName()) # c0
```

Index()

获取线性约束在模型中的索引。

OPTVCPConstr.**Index**()

返回
约束索引。

返回类型
int

返回值

- **-1** –约束不在任何模型中。
- **>=0** –约束在模型中的索引值。

示例

```
model = OPTVCPModel()
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")

boolAnd = model.AddBoolAnd(b1, ~b2)
boolOr = model.AddBoolOr(b1, b2)

print(boolAnd.Index()) # 0
print(boolOr.Index())  # 1
```

GetName()

获取约束名称。

OPTVCPConstr.**GetName**()

返回
约束的名称。

返回类型
str

引发

OPTVErrCode.NOT_IN_MODEL –约束不在任何模型中。

示例

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")

table = model.AddAllowedAssignments([x1, x2], [[1, 2], [3, 4]])
table.WithName("table")

print(table.GetName()) # table
```

GetType()

获取约束类型。

OPTVCPConstr.**GetType**()

返回

约束类型。

返回类型

OPTVCBType

引发

OPTVErrCode.NOT_IN_MODEL –约束不在任何模型中。

示例

```
model = OPTVCPModel()

domainX2 = [0, 3, 5, 7, 15, 15]

x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domainX2, name="x2")
b1 = model.NewBoolVar("b1")
b2 = model.NewBoolVar("b2")
b3 = model.NewBoolVar("b3")
b4 = model.NewBoolVar("b4")

table = model.AddAllowedAssignments([x1, x2], [[1, 2], [3, 4]])
interval1 = model.AddInterval(2 * x2, 2 * x1 + 1, 9)
boolXor = model.AddBoolXor(b1, b2, b3, b4)
AMO = model.AddAtMostOne(b1, b2, b3)
EO = model.AddExactlyOne(b1, b2, b3)
allDiff = model.AddAllDifferent(2 * x1 + 3, 11 * x2 + 12, 15, b2)

table.GetType() == OPTVCPConstraintType.TABLE
interval1.GetType() == OPTVCPConstraintType.INTERVAL
boolXor.GetType() == OPTVCPConstraintType.BOOL_XOR
AMO.GetType() == OPTVCPConstraintType.AT_MOST_ONE
EO.GetType() == OPTVCPConstraintType.EXACTLY_ONE
allDiff.GetType() == OPTVCPConstraintType.ALL_DIFF
```

SameAs()

查询两约束是否相同。

OPTVCPConstr.**SameAs**(*c2*)

参数
c2 (OPTVCPConstr) –需要与当前约束对比的约束。

返回
约束是否相同。

返回类型
bool

返回值

- **True** –约束相同。
- **False** –约束不同。

示例

```
model = OPTVCPModel()
domain = [1, 2, 5, 6, 7, 15]
x1 = model.NewIntVar(2, 5, "x1")
x2 = model.NewIntVarFromDomain(domain, name="x2")

constr = model.AddLinearConstraint(-x1 + 4 * x2 + 11 - 3, lhs = 2,␣
↪rhs = 15)

model.GetConstr(0).SameAs(constr)
```

临时约束

class OPTVCPTempConstr

天筹约束规划求解器临时约束对象。OPTVCPTempConstr 类在使用重载运算符构建的约束表达式中，用于临时存放中间结果。该类不包含任何成员函数。当对表达式或者变量（*OPTVCPLinExpr* 、*OPTVCPVar*）执行比较操作（==, !=, <=, >=）时，会返回临时约束对象。临时约束对象通常会被添加约束的方法立即使用，因此不建议用户将其保存到自定义的变量中。

可以使用临时约束对象构建线性约束：

- **线性约束**：线性约束的形式为 `Expr1 sense Expr2`，其中 `Expr1` 与 `Expr2` 可以是*OPTVCPVar* 对象、*OPTVCPLinExpr* 对象、常量，`sense` 是比较操作符：==、!=、<=、>= 的其中之一。例如 `x + y <= 1 + z` 与 `x + y == 5` 都是有效的线性约束。

以下示例说明了如何使用临时约束添加线性约束：

```
model.AddLinearConstraint(x + y >= 1)
```

变量选择策略

天筹约束规划求解器的变量选择策略。

class OPTVCPVariableSelectionStrategy

CHOOSE_FIRST

类型： int

从变量列表中选择第一个变量。

CHOOSE_LOWEST_MIN

类型： int

选择下界最小的变量。

CHOOSE_HIGHEST_MAX

类型： int

选择上界最大的变量。

CHOOSE_MIN_DOMAIN_SIZE

类型： int

选择域范围最小的变量。

CHOOSE_MAX_DOMAIN_SIZE

类型： int

选择域范围最大的变量。

域缩减策略

天筹约束规划求解器的域缩减策略。

class OPTVCPDomainReductionStrategy

SELECT_MIN_VALUE

类型： int

选择下界。

SELECT_MAX_VALUE

类型： int

选择上界。

SELECT_LOWER_HALF

类型： int

选择由 `bound = lb + (ub - lb) / 2` 计算得到的边界值。

SELECT_UPPER_HALF

类型： int

选择由 `bound = ub - (ub - lb) / 2` 计算得到的边界值。

SELECT_MEDIAN_VALUE

类型： int

选择变量的中位值。

SELECT_RANDOM_HALF

类型： int

随机选择一半。

求解状态

CP 模型状态码。

OPTV_CP_UNKNOWN

求解器未能找到解。

类型

int

取值

0

OPTV_CP_MODEL_INVALID

模型无效。

类型

int

取值

1

OPTV_CP_FEASIBLE

求解器找到了可行解。

类型

int

取值

2

OPTV_CP_INFEASIBLE

求解器证明模型不可行。

类型

int

取值

3

OPTV_CP_OPTIMAL

求解器找到了最优解。

类型

int

取值

4

OPTV_CP_USER_INTERRUPT

求解器被用户中断。

类型

int

取值

5

CHAPTER 9

SDK 回放功能

数学规划求解器的 SDK 功能支持创建”回放”文件，用户可以将一些列的 SDK 命令写入该文件。”回放”文件会压缩所有必须的输入文件（例如 .mps 格式数据），因而用户仅需上述”回放”文件即可”回放”所有相应的 API 调用。

使用这些文件和可执行文件 optverse-replay，用户即可实现”回放”。

使用该功能，用户不仅可以轻松测试部署场景，还可以为开发者提供重要的技术支持。

9.1 Record 记录

只有设置*OPTVStrParam::REPLAY_FILE* 参数为非空时，相应的”回放”文件才会被创建；否则，不会创建任何”回放”文件（默认）.

与其他参数不同的是，该参数必须通过环境对象在*OPTVModel* 创建前设置，而 **不可以**通过*OPTVModel* 对象设置。

```
/**
 * This script illustrates how to generate a recording file.
 * This is accomplished through the `OPTVStrParam::REPLAY_FILE` parameter in the␣
↪OPTVEnv object.
 *
 * The recording file produced by this script can be rerun using optverse-replay.
 */
#include "optv_c++.h"
#include <iostream>

using namespace std;

int main()
{
    try {
        OPTVEnv env("lp1.log");
        env.Set(OPTVStrParam::REPLAY_FILE, "optv.rec");
```

(下页继续)

(续上页)

```
        OPTVModel model(env);

        // add variables to the model, all variables are continuous
        OPTVVar x = model.AddVar(0, 1, -1, OPTV_CONTINUOUS, "x");
        OPTVVar y = model.AddVar(0, OPTV_INF, -14, OPTV_CONTINUOUS, "y");
        OPTVVar z = model.AddVar(0, 3, -6, OPTV_CONTINUOUS, "z");

        // add constraint to the model
        OPTVConstr c0 = model.AddConstr(2 * x + y + z, -OPTV_INF, 3, "c0");
        OPTVConstr c1 = model.AddConstr(3 * y + z, -OPTV_INF, 6, "c1");

        model.Optimize();
        model.Write("lp1.lp");

        if (model.Get(OPTVIntAttr::SOL_COUNT) > 0) {
            cout << "Best solution: " << model.Get(OPTVDblAttr::OBJ_VAL) << endl;

            cout << "x = " << x.Get(OPTVDblAttr::X) << endl;
            cout << "y = " << y.Get(OPTVDblAttr::X) << endl;
            cout << "z = " << z.Get(OPTVDblAttr::X) << endl;
        } else {
            cout << "No feasible solution available!" << endl;
        }

    } catch (const OPTVException& e) {
        cout << e.GetErrorCode() << ": " << e.GetMessage() << endl;
    } catch (...) {
        cerr << "Unknown exception" << endl;
    }
    return 0;
}
```

注解

当前数学规划求解器仅支持单个环境关联单个”回放”应用。

9.2 Replay 回放

使用数学规划求解器的”回放”功能时，只需将”回放”文件的路径提供给可执行文件 `optverse-replay`，相应的 SDK 命令即被顺序执行。

注解

当前数学规划求解器仅支持单个环境关联单个”回放”应用。

CHAPTER 10

网络流规划问题

网络流规划问题 (network programming) 是一类特殊的线性规划问题，本章将对这类问题进行简要说明。数学规划求解器可以自动识别网络流规划问题结构，从而使用更高效的算法（网络流单纯形法，network simplex method）进行求解。

10.1 问题定义

一个满足以下条件的线性规划问题被称为 **网络流规划问题**:

$$\begin{aligned} \min \quad & c^T x \\ \text{s.t.} \quad & Ax = b, \\ & l \le x \le u, \end{aligned}$$

这里

$$\begin{aligned} c &\in \mathbb{R}^m \\ b &\in \mathbb{R}^n \\ A &\in \{-1, 0, 1\}^{n \times m} \end{aligned}$$

而且问题满足下列条件:

- 系数矩阵 A 的每列只有一个 1 和 -1 (即其余系数都为零)。
- $\sum_i b_i = 0, i \in \{1, \ldots, n\}$

这里, n 为约束数量, m 为变量数量。

10.2 网络流规划样例

- 目标函数为最小化:

$$x_1 + x_2 + x_3 + x_4$$

- 约束条件:

$$\begin{bmatrix} 1 & 0 & 1 & 1 \\ -1 & 1 & 0 & 0 \\ 0 & -1 & -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 5 \\ 0 \\ -5 \end{bmatrix}$$

- 变量边界:

$$\begin{aligned} 0 \leq x_1 \leq 4, \\ 0 \leq x_2 \leq 2, \\ 0 \leq x_3 \leq 4, \\ 0 \leq x_4 \leq 10 \end{aligned}$$